

PrimeSim™ Continuum User Guide: Elements

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	13
Related Products, Publications, and Trademarks	13
Conventions	14
Customer Support	14
Statement on Inclusivity and Diversity	15

1. Elements	16
Passive Elements: Resistors, Capacitors, and Inductors	16
Resistors	17
Linear Resistor	17
Behavioral Resistor	17
Resistor Temperature Equations	18
Examples	19
Frequency-dependent Resistors	20
Skin Effect Resistors	20
Capacitors	21
Linear Capacitor	21
Capacitance Temperature Equation	23
Examples	23
Frequency-dependent Capacitors	24
Behavioral Capacitors in the PrimeSim Continuum Solution	25
DC Block Capacitors	26
Charge-conserved Capacitors	27
Inductors	28
Linear Inductor	29
Inductor Temperature Equation	30
Examples	31
Frequency-dependent Inductors	31
AC Choke Inductors	32
Mutual Inductors	33
Reluctors	35
Ideal Transformer	38
Multi-terminal Linear Elements	40
S-element (Scattering Parameter Data)	41
S-element Syntax	41
S-element Example	47

W-element (Distributed Transmission Lines)	48
W-element Syntax	48
Five W-element Models	51
W-element Examples	53
T-element (Ideal Transmission Lines)	54
T-element Syntax	54
T-element Examples	55
U-element (Lumped Transmission Lines)	56
U-element Syntax	56
U-element Examples	57
Port Element	58
Port Element Syntax	58
PAM3_MAPPING	66
Port Element Example	66
Removing and Recovering DC Offset Level in AMI_GetWave Flow	67
Using the Port Element for Mixed-Mode Measurement	67
Mixed Mode Port Element Syntax	68
Using the Five Terminal Port Element	69
Active Elements	69
Diode Element	70
Syntax	70
Examples	71
Bipolar Junction Transistor (BJT) Element	72
Syntax	72
Examples	73
JFETs and MESFETs	74
Syntax	74
Examples	75
MOSFETs	76
Syntax	76
Examples	78
Extended MOSFET Element Support Using the .OPTION MACMOD	
Command	79
Example 1	80
Example 2	81
Example 3	81
MACMOD Option Limitations	82
Direct X-Element Mapping to a MOSFET Model Card	82
TMI Dummy Device	83
 2. Independent Sources and Stimuli	 84

Contents

Independent Source Elements	84
Source Element Conventions	85
Independent Source Element Syntax	85
Independent Source Element Examples	87
DC Sources	90
Syntax	90
Examples	90
AC Sources	91
Syntax	91
Examples	91
Transient Sources	91
Transient Source Functions	92
Trapezoidal Pulse Source	92
Syntax	93
Effect of Jitter on the PULSE Source	94
Examples	95
Sinusoidal Source Function	97
Syntax	97
Example	98
Repeat Option in Sinusoidal Sources	100
Exponential Source Function	100
Syntax	100
Example	102
Repeat Option in Exponential Sources	104
PWL Source	104
PrimeSim Continuum PWL Form	104
MSINC and ASPEC PWL Form	105
Example	106
Data-driven PWL Source	107
File-driven PWL Source	108
High Impedance State PWL (PWLZ)	110
Syntax	110
Example	111
Single-frequency FM Source	111
Syntax	111
Example	112
Single-frequency AM Source	113
Syntax	113
Example	113
Pattern Source	115
Syntax	116

Contents

Examples	118
Nested Structure Pattern Source	119
Pattern-command Driven Pattern Source	120
Defining Pattern Sources with more than 1024 Characters	121
Using String Parameters to Define Pattern Sources	123
High-impedance Z-state in Pattern Sources	123
Pseudo Random Bit Generator Source	124
Syntax	124
Examples	127
Linear Feedback Shift Register	129
Conventions for Feedback Tap Specification	130
Example: Noise Generator Used for a Pulse or DC Level	130
Mixed Sources	132
Example	132
Power Sources	132
Independent Power Sources	133
Using the Keyword POWER	133
Examples	134
Calculation for Total Dissipated Power and Voltage Source Power	135
Subcircuit Power Calculation	136
PrimeSim Continuum Advanced Analog Analyses Sources and Stimuli	137
Steady-state Voltage and Current Sources	137
I and V Element Syntax	137
Examples	139
Steady-state HB Sources	140
Examples	141
Phase Differences Between HB and SIN Sources	142
Behavioral Noise Sources	143
Using Noise Analysis Results as Input Noise Sources	144
Syntax Rules for Behavioral Noise Source Files	145
Power Supply Current and Voltage Noise Sources	147
Function Approximations for Distributed Devices	148
Foster Pole-Residue Form for Transconductance or Gain	149
Advantages of Foster Form Modeling	149
G and E-element Syntax	149
Complex Signal Sources and Stimuli	150
Vector-modulated RF (VMRF) Source	150
Voltage and Current Source Elements	152
SWEEPBLOCK in Sweep Analyses	157
Input Syntax	158
Using SWEEPBLOCK in a DC Parameter Sweep	159

Contents

Using in Parameter Sweeps in TRAN, AC, and HB Analyses	159
Using in Parameter Sweeps in HBAC Analyses	160
Limitations	160
Clock Source with Random Jitter	160
Syntax of SIN, COS, and Pulse Sources	160
Specifying a Digital Vector File and Mixed Mode Stimuli	163
Vector Patterns	164
Defining Tabular Data	164
Input Stimuli	165
Expected Output	166
Verilog Value Format	167
Periodic Tabular Data	168
Accumulated Time	168
Waveform Characteristics	168
Modifying Waveform Characteristics	169
Using the Context-Based Control Option (CBC)	170
Comment Lines and Line Continuations	171
Parameter Usage	171
First Group	171
Second Group	172
Third Group	172
Digital Vector File Example	173
Using the VCD Control File	174
Defining Bus Syntax – #format	174
Defining Vector Types – #input, #output, and #bidirectional	175
Defining Mapping Information – #alias	177
Defining Attributes for Vectors	178
Defining the Hierarchical Scope – #scope	180
3. Behavioral Modeling and Controlled Sources	182
Voltage and Current Controlled Elements	182
Voltage-Dependent Voltage Sources — E-elements	184
Voltage-Controlled Voltage Source (VCVS)	185
Linear	185
Polynomial (POLY)	185
PWL	186
Multi-Input Gates	186
Delay Element	186
Laplace Transform	186

Contents

Pole-Zero Function	187
Frequency Response Table	188
Foster Pole-Residue Form	189
Behavioral Voltage Source	189
Behavioral Time Derivative Voltage Source	190
Behavioral Voltage Noise Source	190
Ideal Op-Amp	191
Ideal Transformer	191
Z Transform Function Call	194
E-element Parameters	194
E-element Examples	197
Ideal OpAmp	198
Voltage Summer	198
Polynomial Function	199
Zero-Delay Inverter Gate	199
Delayed and Inverted Signal	199
Differential Amplifiers and Opamp Signals	200
Ideal Transformer	200
Voltage-Controlled Oscillator	200
Switching between Two Voltage Sources Connected to the Same Node	200
VCCS as the Controlling Element for a CCCS	201
Using the E-element for AC Analysis	201
Example	202
Voltage-dependent Current Sources — G-elements	202
Voltage-controlled Current Source (VCCS)	203
Linear	203
Polynomial (POLY)	203
PWL	204
Multi-Input Gate	204
Delay Element	204
Z Transform Function Call	204
Laplace Transform	205
Pole-Zero Function	206
Frequency Response Table	206
Foster Pole-Residue Form	207
Behavioral Current Source	208
Behavioral Time Derivative Current Source	208
Behavioral Current Noise Source	209
Voltage-controlled Resistor (VCR)	211
Voltage-Controlled Capacitor (VCCAP)	212
Voltage-controlled Resistance Switch	215
Magnetic Tunnel Junction MRES0 Mode	217
G-element Parameters	217

Contents

G-element Examples	221
Modeling Switches	222
Switch-level MOSFET	223
Runtime Current Source with Equation Containing Output Variable	223
Voltage-controlled Capacitor	224
Zero-delay Gate	224
Delay Element	224
Diode Equation	224
Diode Breakdown	225
Diode Lookup Table (versus Model)	225
Triodes	225
Behavioral Noise Model	225
Turning off Smoothing	226
Using Dependent Sources to Convert I to V and V to I	226
Additional Full Demonstration Netlists Using G-Element	226
Current-dependent Current Sources - F-elements	227
Current-controlled Current Source (CCCS)	227
Linear	227
Polynomial (POLY)	228
PWL	228
Multi-Input Gates	228
Delay Element	228
F-element Parameters	228
F-element Examples	230
Current-dependent Voltage Sources - H-elements	231
Current-controlled Voltage Source (CCVS)—H-Element	232
Linear	232
Polynomial (POLY)	232
PWL	232
Multi-Input Gate	232
Delay Element	233
H-element Parameters	233
H-element Examples	234
Polynomial and Piecewise Linear Functions	236
Polynomial Functions	236
One-Dimensional Function	237
Two-Dimensional Function	238
Three-Dimensional Function	239
N-Dimensional Function	240
Piecewise Linear Function	241
Exponential Piecewise Linear Function	241

4. Modeling Input/Output Buffers Using IBIS Files	243
IBIS Syntax Conventions for I/O Buffers	243
Buffer Types	246
Input Buffer	247
Output Buffer	249
Tristate Buffer	252
Input/Output Buffer	255
Open Drain, Open Sink, Open Source Buffers	257
I/O Open Drain, I/O Open Sink, I/O Open Source Buffers	258
Input ECL Buffer	258
Output ECL Buffer	259
Tristate ECL Buffer	260
Input-Output ECL Buffer	261
Terminator Buffer	263
Series Buffer	265
Series Switch Buffer	265
Multilingual Model Support	267
Examples	270
Example 1	270
Example 2	270
Example 3	270
Example 4	271
Specifying Required and Optional Common Keywords	271
file	272
Syntax	272
Example	272
model	272
Syntax	272
Example	273
buffer	273
Syntax	273
typ	274
Syntax	274
hsp_ver	275
Syntax	275
power	276
Syntax	276
interpol	276

Contents

Syntax	276
xv_pu xv_pd	276
Syntax	276
ramp_fwf ramp_rwf	277
Syntax	277
Invalid Ramp Warning	278
fwf_tune rwf_tune	279
Syntax	279
rwf_pd_dly fwf_pu_dly	280
Syntax	280
pd_scal pu_scal pc_scal gc_scal rwf_scal fwf_scal	281
Syntax	281
dt_scale	282
Syntax	282
ss_state	282
Syntax	282
rm_dly_rwf rm_dly_fwf rm_tail_rwf rm_tail_fwf	282
Syntax	282
nowarn	284
Syntax	284
c_com_pu c_com_pd c_com_pc c_com_gc	284
Syntax	284
detect_oti_mid	285
Syntax	285
time_control	286
Syntax	286
.OPTION D_IBIS	286

A. Lumped Transmission Line — U-Model	287
Using the U-element for Lossy Transmission Lines	287
Syntax	288
U Model for Transmission Lines	289
Selecting U Models	290
Lossy U Model Parameters for Planar Geometries	292
Common Planar Model Parameters	292
Physical Parameters	293
Loss Parameters	294
Geometric Parameter Recommended Ranges	295
Reference Planes and the PrimeSim Continuum Ground with LLEV	295

Contents

Estimating the Skin Effect Frequency	296
Number of Lumped-Parameter Sections	297
Ringing	297
Geometric Parameters (ELEV=1)	298
Lossy U Model Parameters for Geometric Coax (PLEV=2, ELEV=1)	302
Lossy U Model Parameters Geometric Twinlead (PLEV=3, ELEV=1)	303
Precomputed Model Parameters (ELEV=2)	305
Conductor Width Relative to Reference Plane Width	308
Alternative Multiconductor Capacitance/Conductance Definitions	308
Measured Parameters (ELEV=3)	310
U-element Examples, Models, and Applications	313
Three Coupled Lines, Stripline Configuration	313
Three Coupled Lines, Sea of Dielectric Configuration	318
Simulation Output	322
IcWire Output Section	322
Capacitance and Inductance Matrices	324
Five Coupled Lines, Stripline Configuration	327
U Model Applications	327
Data Entry Examples	328
Printed Circuit Board Models	328
Via Modeling for PCBs in the PrimeSim Continuum Solution	329
Coax Models	330
Twinlead Models	331
Two Coupled Microstrips	331
Solving Ringing Problems with U-elements	334
Oscillations Due to Simulation Errors	334
Timestep Control Error	334
Incorrect Number of Element Lumps	335
Default Computation	335
Using a Multi-Stage RC Filter to Prevent Ringing	336
Signal Reflections Due to Impedance Mismatch	338

About the PrimeSim Continuum User Guide: Elements

The *PrimeSim Continuum User Guide: Elements* describes the syntax for the basic elements of a circuit netlist in the PrimeSim Continuum solution. It provides descriptions for each of the element keywords and examples of common usage for each element.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the PrimeSim Continuum solution Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the PrimeSim Continuum simulators, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

Synopsys PrimeSim™ XA™

PrimeSim™ HSPICE®

PrimeSim™

Synopsys SolvNetPlus support site

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Elements

Describes the syntax for the basic elements of a circuit netlist in the PrimeSim Continuum solution.

Elements are local and sometimes customized instances of a device model that you specified in your design netlist. Elements (instances) use standard device models as their bases. For descriptions of the standard device models, see the *PrimeSim™ Continuum Reference Manual: Device Models* and the *PrimeSim™ Continuum Reference Manual: MOSFET Models*. For signal integrity applications see the *PrimeSim™ HSPICE® User Guide: Signal Integrity Modeling and Analysis*.

Element names can be up to 1024 characters.

The following sections discuss these topics:

- [Passive Elements: Resistors, Capacitors, and Inductors](#)
- [Multi-terminal Linear Elements](#)
- [Active Elements](#)
- [TMI Dummy Device](#)

Passive Elements: Resistors, Capacitors, and Inductors

This section describes the passive elements: resistors, capacitors, and inductors. See [Multi-terminal Linear Elements](#) for discussion of the S-, W-, T-, and U-elements.

The content of this section includes:

- [Resistors](#)
- [Capacitors](#)
- [Inductors](#)

Resistors

The content of this section includes:

- [Linear Resistor](#)
- [Behavioral Resistor](#)
- [Resistor Temperature Equations](#)
- [Examples](#)
- [Frequency-dependent Resistors](#)
- [Skin Effect Resistors](#)

Linear Resistor

```
Rxxx n1 n2 [mname] [R=]resistance [TC1 TC2] [SCALE=val]  
+ [M=val]  
+ [AC=val] [DTEMP=val] [L=val] [W=val] [C=val]  
+ [NOISE=val]
```

```
Rxxx n1 n2 [mname] [R=]resistance [TC1=val]  
+ [TC2=val] [SCALE=val] [M=val]  
+ [AC=val] [DTEMP=val] [L=val] [W=val]  
+ [C=val] [NOISE=val]
```

Behavioral Resistor

```
Rxxx n1 n2 [R=]'equation' ...
```

Parameter	Description
Rxxx	Resistor element name. Must begin with R, followed by up to 1023 alphanumeric and special characters as defined in Special Characters in <i>PrimeSim HSPICE</i> ® <i>User Guide: Basic Simulation and Analysis</i> .
n1	Positive terminal node name.
n2	Negative terminal node name.
mname	Resistor model name. Use this name in elements, to reference a resistor model. See Passive Device Models in <i>PrimeSim HSPICE</i> ® <i>Reference Manual: Device Models</i> for resistor model information.
TC	TC1 alias. The current definition overrides the previous definition.
TC1	First-order temperature coefficient for the resistor.
TC2	Second-order temperature coefficient for the resistor.

Parameter	Description
SCALE	Element scale factor; scales resistance and capacitance by its value. Default=1.0.
R= <i>resistance</i>	Resistance value at room temperature. This can be: - a numeric value in ohms - a parameter in ohms
M	Multiplier to simulate parallel resistors. For example, for two parallel instances of a resistor, set M=2, to multiply the number of resistors by 2. Default=1.0.
AC	Resistance for AC analysis. Default=Reff.
DTEMP	Temperature difference between the element and the circuit, in degrees Celsius. Default=0.0. To modify the temperature for a particular element, use the DTEMP parameter in an instance line.
L	Resistor length in meters. Default=0.0, if you did not specify L in a resistor model.
W	Resistor width. Default=0.0, if you did not specify W in the model.
C	Capacitance connected from node n1 to bulk and node n2 to bulk. Default=0.0, if you did not specify C in a resistor model.
NOISE	- NOISE=0, do not evaluate resistor noise. - NOISE=1, evaluate resistor noise (default).
R= <i>'equation'</i>	- a function of any node voltages - a function of branch currents - any independent variables such as <code>time</code>, <code>hertz</code>, and <code>temper</code>

Resistance can be a value (in units of ohms) or an equation. Required parameters are the two nodes, and the resistance or model name. If you specify other parameters, the node and model name must precede those parameters. Other parameters can follow in any order. If you specify a resistor model (see the Passive Device Models chapter in the *PrimeSim HSPICE® Reference Manual: Device Models*), the resistance value is optional.

For output templates for resistor models, see the *Resistor (R-element)* table in Element Template Listings in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

Resistor Temperature Equations

Resistor values can vary as a function of temperature and temperature coefficients TC1 and TC2 as follows:

$$R(t) = R \cdot (1.0 + TC1 \cdot \Delta t + TC2 \cdot \Delta t^2)$$

$$RAC(t) = RAC \cdot (1.0 + TC1 \cdot \Delta t + TC2 \cdot \Delta t^2)$$

Parameter	Description
Δt	$t - t_{nom}$
t	Element temperature in °K: $t = \text{circuit temp} + DTEMP + 273.15$
t_{nom}	Nominal temperature in °K: $t_{nom} = 273.15 + TNOM$

Examples

In the following example, the R1 resistor connects from the Rnode1 node to the Rnode2 node, with a resistance of 100 ohms.

```
R1 Rnode1 Rnode2 100
```

The RC1 resistor connects from node 12 to node 17, with a resistance of 1 kilohm, and temperature coefficients of 0.001 and 0.

```
RC1 12 17 R=1k TC1=0.001 TC2=0
```

The Rterm resistor connects from the input node to ground, with a resistance determined by the square root of the analysis frequency (non-zero for AC analysis only).

```
Rterm input gnd R='sqrt(HERTZ)'
```

The Rxxx resistor connects from node 989999999 to node 87654321 with a resistance of 1 ohm for DC and time-domain analyses, and 10 gigohms for AC analyses.

```
Rxxx 989999999 87654321 1 AC=1e10
```

Behavioral Resistor Examples

Some basic behavioral resistor examples include:

- R1 is a resistor whose resistance follows the voltage at node c.

```
R1 1 0 'v(c)'
```
- R2 is a resistor whose resistance is the sum of the absolute values of nodes c and d.

```
R2 1 0 'abs(v(c)) + abs(v(d))'
```
- R3 is a resistor whose resistance is the sum of the rconst parameter, and 100 times tx1 for a total of 1100 ohms.

```
.PARAM rconst=100 tx1=10  
R3 4 5 'rconst + tx1 * 100'
```
- R1 A B R='V(A) + I(VDD)'

Frequency-dependent Resistors

Rxxx n1 n2 [R=] 'equation' [CONVOLUTION=[0|1|2]]
+ [FBASE=value] [FMAX=value]

Parameter	Description
CONVOLUTION	Indicates which method to use (at the instance level only). • 0: Acts the same as the conventional method. This is the default. • 1: Applies recursive convolution, and if the rational function is not accurate enough, it switches to linear convolution. • 2: Applies linear convolution.
FBASE	Specifies the lower bound of the transient analysis frequency. For CONVOLUTION=1 mode, the PrimeSim Continuum solution starts sampling at this frequency. For CONVOLUTION=2 mode, the PrimeSim Continuum solution uses this value as the base frequency point for Inverse Fourier Transformation. For recursive convolution, the default value is 0Hz, and for linear convolution, the PrimeSim Continuum solution uses the reciprocal of the transient period.
FMAX	Specifies the possible maximum frequency of interest. The default value is the frequency point where the function reaches close enough to infinity value. This assumes that frequency point reaches 10THz and the monotonous function is approaching the infinity value. The equation should be a function of HERTZ. If CONVOLUTION turns on when a HERTZ keyword is not in the equation, it is automatically turns off to let the resistor behave conventionally. The equation can be a function of temperature, but it cannot be node voltage or branch current and time.

The equation can only be a function of time-independent variables such as HERTZ, and temperature.

You can model a frequency-dependent resistor and use it in the time domain only when CONVOLUTION=1.

Example

R1 1 2 r='1.0 + 1e-5*sqrt(HERTZ)' CONVOLUTION=1

Skin Effect Resistors

Rxxx n1 n2 R=value Rs=value

The R_s indicates the skin effect coefficient of the resistor.

Equation 1 shows the expression of the complex impedance of the resistor:

Equation 1

$$R(f) = R_o + (1 + j) \cdot R_s \cdot \sqrt{f}$$

Where, R_0 is the DC resistance, j is the imaginary term and f is the frequency. The imaginary part of the equation represents the frequency-dependent inductance.

In the time domain, the PrimeSim Continuum solution uses the following rational function to represent [Equation 1](#):

Equation 2

$$H(\omega) \cong \sum_k \frac{\alpha_k}{\omega_k + j\omega}$$

Recursive convolution evaluates the rational function skin effect resistor. An advantage of using the skin effect resistor is that it automatically guarantees the causality.

Capacitors

The content of this section includes:

- [Linear Capacitor](#)
- [Capacitance Temperature Equation](#)
- [Examples](#)
- [Frequency-dependent Capacitors](#)
- [Behavioral Capacitors in the PrimeSim Continuum Solution](#)
- [DC Block Capacitors](#)
- [Charge-conserved Capacitors](#)

Linear Capacitor

Linear form:

```
Cxxx n1 n2 [mname] [C=]capacitance [TC1=val]
+ [TC2=val] [SCALE=val] [IC=val] [M=val]
+ [W=val] [L=val] [DTEMP=val] [SHRINK=val]
```

Polynomial form:

```
Cxxx n1 n2 POLY c0 c1... [IC=val] [M=val]
```

Parameter	Description
Cxxx	Capacitor element name. Must begin with C, followed by up to 1023 alphanumeric and special characters as defined in Special Characters in <i>PrimeSim HSPICE® User Guide: Basic Simulation and Analysis</i> .

Parameter	Description
n1	Positive terminal node name.
n2	Negative terminal node name.
mname	Model name; mname must refer to a capacitor model defined in the netlist using a .MODEL construct. See Passive Device Models in <i>PrimeSim™ Continuum® Reference Manual: Device Models</i> for capacitor model information.
C= <i>capacitance</i>	Capacitance at room temperature—a numeric value or a parameter in farads.
TC1	First-order temperature coefficient for the capacitor.
TC2	Second-order temperature coefficient for the capacitor.
SCALE	Element scale parameter, scales capacitance by its value. Default=1.0.
IC	The initial voltage across the capacitor, in volts when the UIC parameter in the .TRAN statement is used. When the UIC parameter is used, the PrimeSim Continuum solution does not calculate the initial DC operating point and the voltage will be across the capacitor at transient simulation <code>time=0</code> and will be released at <code>time>0</code> to allow the capacitor voltage to vary with circuit operation. If you use an .IC statement to set an initial voltage across the capacitor, then the .IC statement overrides the IC value set on the instance.
M	Multiplier, used to simulate multiple parallel capacitors. Default=1.0
W	Capacitor width, in meters. Default=0.0, if you did not specify W in a capacitor model.
L	Capacitor length, in meters. Default=0.0, if you did not specify L in a capacitor model.
DTEMP	Element temperature difference from the circuit temperature, in degrees Celsius. Default=0.0.
SHRINK	Local shrink factor; a value of shrink=1 disables shrinking. For more information, see .OPTION SHRINK.
CTYPE	Determines capacitance charge calculation for elements with capacitance equations. If the C capacitance is a function of V(n1[,n2]), set CTYPE=0. Use this setting correctly, to ensure proper capacitance calculations, and correct simulation results. Default=0.
POLY	Keyword, to specify capacitance as a non-linear polynomial.
c0 c1...	Coefficients of a polynomial, described as a function of the voltage across the capacitor. c0 represents the magnitude of the 0th order term, c1 represents the magnitude of the 1st order term, and so on.

You can specify capacitance as a numeric value, in units of farads, as an equation, or as a polynomial of the voltage. The only required fields are the two nodes, and the capacitance or model name.

- If you use the parameter labels, the nodes and model name must precede the labels. Other arguments can follow in any order.
- If you specify a capacitor model (see the Passive Device Models chapter in *PrimeSim™ Continuum® Reference Manual: Device Models*), the capacitance value is optional.

If you use an equation to specify capacitance, the `CTYPE` parameter determines how the PrimeSim Continuum solution calculates the capacitance charge. The calculation is different, depending on whether the equation uses a self-referential voltage (that is, the voltage across the capacitor, where the equation determines the capacitance).

To avoid syntax conflicts, if a capacitor model has the same name as a capacitance parameter, the PrimeSim Continuum solution uses the model name.

For output templates for capacitor models, see the *Capacitor (C-element)* table in Element Template Listings in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

Capacitance Temperature Equation

Capacitor values can vary as a function of temperature and temperature coefficients TC1 and TC2 as follows:

$$C(t) = C \cdot (1.0 + TC1 \cdot \Delta t + TC2 \cdot \Delta t^2)$$

Parameter	Description
Δt	$t - t_{nom}$
t	Element temperature in °K: $t = \text{circuit temp} + DTEMP + 273.15$
t_{nom}	Nominal temperature in °K: $t_{nom} = 273.15 + TNOM$

Examples

Example 1

In the following example, C1 assumes its capacitance value from the model, not the parameter.

```
.PARAMETER CAPXX=1
C1 1 2 CAPXX
.MODEL CAPXX C CAP=1
```

Example 2

In the following example, the C1 capacitors connect from node 1 to node 2, with a capacitance of 20 picofarads:

```
C1 1 2 20p
```

In this next example, Cshunt refers to three capacitors in parallel, connected from the node output to ground, each with a capacitance of 100 femtofarads.

```
Cshunt output gnd C=100f M=3
```

The Cload capacitor connects from the driver node to the output node. The voltage on the capcontrol node, times 1E-6, determines the capacitance. The initial voltage across the capacitor is 0 volts.

```
Cload driver output C='1u*v(capcontrol)' CTYPE=1 IC=0v
```

The C99 capacitor connects from the in node to the out node. The polynomial $C=c_0 + c_1*v + c_2*v^2$, where v is the voltage across the capacitor, determines the capacitance.

```
C99 in out POLY 2.0 0.5 0.01
```

Example 3

```
Cbypass 1 0 10PF  
C1 2 3 CBX  
.MODEL CBX C  
CB B 0 10P IC=4V  
CP X1.XA.1 0 0.1P
```

In this example:

- Cbypass is a straightforward, 10-picofarad (PF) capacitor.
- C1, which calls the CBX model, does not have a constant capacitance.
- CB is a 10 PF capacitor, with an initial voltage of 4V across it.
- CP is a 0.1 PF capacitor.

For a full demonstration example of voltage variable capacitance, see the path to the `calg2.sp` netlist in Behavioral Application Examples in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

Frequency-dependent Capacitors

You can specify frequency-dependent capacitors by using the `C='equation'` with the `HERTZ` keyword. The `HERTZ` keyword represents the operating frequency. In time domain analyses, an expression with the `HERTZ` keyword behaves differently according to the value assigned to the `CONVOLUTION` keyword.

Syntax

```
Cxxx n1 n2 C='equation' [CONVOLUTION=[0|1|2]]  
+ [FBASE=val] [FMAX=val]
```


Parameter	Description
<i>n1 n2</i>	Names or numbers of connecting nodes.
<i>equation</i>	Expressed as a function of HERTZ. If CONVOLUTION=1 or 2 and HERTZ is not in the equation, CONVOLUTION turns off and the capacitor behaves conventionally. The equation can be a function of temperature, but it does not support variables of node voltage, branch current, or time. If these variables exist in the expression and CONVOLUTION=1 or 2, then the simulation considers only their values at the operating point in calculation.
CONVOLUTION	Specifies the method used. 0 (default): HERTZ=0 in time domain analysis. 1 or 2: performs Inverse Fast Fourier Transformation (IFFT) linear convolution.
FBASE	Base frequency to use for transient analysis. This value becomes the base frequency point for Inverse Fast Fourier Transformation (IFFT) when CONVOLUTION=1 or 2. If you do not set this value, the base frequency is a reciprocal value of the transient period.
FMAX	Maximum frequency to use for transient analysis. Used as the maximum frequency point for Inverse Fourier Transformation. If you do not set this value, the simulation takes the reciprocal value of RISETIME.

Example

```
C1 1 2 C='1e-6 - HERTZ/1e16' CONVOLUTION=1 fbase=10
+ fmax=30meg
```

Behavioral Capacitors in the PrimeSim Continuum Solution

```
Cxxxxn1 n2 . . . C='equation' CTYPE=0 or 1
```

Parameter	Description
CTYPE	Determines the calculation mode for elements that use capacitance equations. Set this parameter carefully, to ensure correct simulation results. The PrimeSim Continuum solution advanced analog analyses extends the definition and values of CTYPE, relative to the PrimeSim Continuum solution: - CTYPE=0, if C depends only on its own terminal voltages—that is, a function of $V(n1[, n2])$. - CTYPE=1, if C depends only on outside voltages or currents. - CTYPE=2, if C depends on both its own terminal and outside voltages. This is the default for the PrimeSim Continuum solution advanced analog analyses. The PrimeSim Continuum solution does not support CTYPE=2.

You can specify the capacitor value as a function of any node voltage or branch current, and any independent variables such as `time`, `hertz`, and `temper`.

Example

```
C1 1 0 C='1e-9*V(10)' CTYPE=1  
V10 10 0 PWL(0,1v t1,1v t2,4v)
```

DC Block Capacitors

```
Cxxx node1 node2 [C=] INFINITY [IC=val]
```

When the capacitance of a capacitor is infinity, this element takes the name of “DC block.” In the PrimeSim Continuum solution, you specify an `INFINITY` value for such capacitors.

The PrimeSim Continuum solution does not support any other capacitor parameters for DC block elements because the PrimeSim Continuum solution assumes that an infinite capacitor value is independent of any scaling factors.

The DC block acts as an open circuit for all DC analyses. The PrimeSim Continuum solution calculates the DC voltage across the nodes of the circuit. In all other (non-DC) analyses, a DC voltage source of this value represents the DC block—the PrimeSim Continuum solution does not allow dv/dt variations.

DC Block and Choke Elements

In the PrimeSim Continuum solution advanced analog analyses, you can specify an `INFINITY` value for capacitors and inductors to model ideal DC block and choke elements. The following input syntax is for the DC block (ideal infinite capacitor):

Syntax

```
Cxxx node1 node2 [C=] INFINITY [IC=val]
```

The PrimeSim Continuum advanced analog analyses does not support any other capacitor parameters for DC block elements, because the PrimeSim Continuum advanced analog analyses assumes that the infinite capacitor value is independent of temperature and scaling factors. The DC block acts as an open circuit for all DC analyses. The PrimeSim Continuum advanced analog analyses calculates the DC voltage across the circuit's nodes. In all other (non-DC) analyses, a DC voltage source of this value represents the DC block (that is, the PrimeSim Continuum advanced analog analyses does not then allow dv/dt variations).

The following input syntax is for the Choke (ideal infinite inductor):

Syntax

```
Lxxx node1 node2 [L=] INFINITY [IC=val]
```

The PrimeSim Continuum advanced analog analyses does not support any other inductor parameters, because the PrimeSim Continuum advanced analog analyses assumes that the infinite inductance value is independent of temperature and scaling factors. The choke

acts as a short circuit for all DC analyses. The PrimeSim Continuum advanced analog analyses calculates the DC current through the inductor. In all other (non-DC) analyses, a DC current source of this value represents the choke (that is, the PrimeSim Continuum advanced analog analyses does not then allow di/dt variations).

Charge-conserved Capacitors

```
Cxxx node1 node2 q='expression'
```

The PrimeSim Continuum solution supports AC, DC, TRAN, and PZ analyses for charge-conserved capacitors.

The `expression` supports the following parameters and variables:

- Parameters
 - node voltages
 - branch currents
- Variables
 - `time`
 - `temper`
 - `hertz`

Note:

Transient analyses do not support the `hertz` variable.

Note:

- You must use parameters directly in an equation. The PrimeSim Continuum solution does not support parameters that represent an equation containing variables.
- If you use an unsupported parameter in an expression, the PrimeSim Continuum solution issues an error message and aborts the simulation. The PrimeSim Continuum solution ignores unsupported analysis types and then issues warning a message.

Limitations

The following syntax does not support charge-conserving capacitors:

```
Cxxx node1 node2 C='expression'
```

The PrimeSim Continuum solution does not implicitly convert capacitor equations to charge equations.

Example 1: Capacitance-based Capacitor

```
C1 a b C='Co*(1+alpha*V(a,b))' ctype=0
```

You can obtain Q by integrating 'C' with respect to V(a,b)

Example 2: Charge-based Capacitor

```
C1 a b Q='Co*V(a,b) (1+0.5*alpha*V(a,b))'
```

Example 3: Capacitance-based Capacitor

```
.option list node post
r1 1 2 100
r2 3 0 200
Vin 1 0 pulse(0 5v 1ns 2ns 2ns 10ns 20ns)
C1 2 3 c='cos(v(2,3)) + v(1,2)' ctype=2
.tran 1ns 100ns
.print tran i(c1)
.end
```

Example 4: Charge-based Capacitor

```
.option list node post
r1 1 2 100
r2 3 0 200
Vin 1 0 pulse(0 5v 1ns 2ns 2ns 10ns 20ns)
C1 2 3 q='sin(v(2,3)) + v(2,3)*v(1,2)'
.tran 1ns 100ns
.print tran i(c1)
.end
```

Inductors

The content of this section includes:

- [Linear Inductor](#)
- [Inductor Temperature Equation](#)
- [Examples](#)
- [Frequency-dependent Inductors](#)
- [AC Choke Inductors](#)
- [Mutual Inductors](#)
- [Reluctors](#)
- [Ideal Transformer](#)

Linear Inductor

Linear form:

```
Lxxx n1 n2 [L=]inductance [TC1=val]
+ [TC2=val] [SCALE=val] [IC=val] [M=val]
+ [DTEMP=val] [R=val]

Lxxx n1 n2 L='equation' [LTYPE=val] [above_options...]
```

Polynomial form:

```
Lxxx n1 n2 POLY c0 c1... [above_options...]
```

Inductor with magnetic winding:

```
Lxxx n1 n2NT=turns [above_options...]
```

Parameter	Description
Lxxx	Inductor element name. Must begin with L, followed by up to 1023 alphanumeric and special characters as defined in Special Characters in <i>PrimeSim HSPICE® User Guide: Basic Simulation and Analysis</i> .
n1	Positive terminal node name.
n2	Negative terminal node name.
TC1	First-order temperature coefficient for the inductor.
TC2	Second-order temperature coefficient for the inductor.
SCALE	Element scale parameter; scales inductance by its value. Default=1.0.
IC	The current forced through the inductor for the duration of the DC operating point calculation, in amperes. The PrimeSim Continuum solution uses this value as the DC operating point current only. If the .TRAN statement uses UIC, then the PrimeSim Continuum solution does not calculate the DC operating point. For this case, the current flow through the inductor at transient simulation time=0 and be released at time>0 to allow the inductor current to vary with circuit operation.
L=inductance	Inductance value. This can be: • a numeric value, in henries • a parameter in henries
M	Multiplier, used to simulate parallel inductors. Default=1.0.
DTEMP	Temperature difference between the element and the circuit, in degrees Celsius. Default=0.0.
R	Resistance of the inductor, in ohms. Default=0.0.

Parameter	Description
<code>L= 'equation'</code>	Inductance at room temperature, specified as: - a function of any node voltages - a function of branch currents - any independent variables such as <code>time</code>, <code>hertz</code>, and <code>temper</code>
<code>LTYPE</code>	Calculates inductance flux for elements, using inductance equations. If the L inductance is a function of I(Lxxx), then set LTYPE=0. Otherwise, specify LTYPE=1. Use this setting correctly, to ensure proper inductance calculations, and correct simulation results. Default=0.
<code>POLY</code>	Keyword that specifies the inductance, calculated by a polynomial.
<code>c0 c1...</code>	Coefficients of a polynomial in the current, describing the inductor value. c0 is the magnitude of the 0th order term, c1 is the magnitude of the 1st order term, and so on.
<code>NT=turns</code>	Number of turns of an inductive magnetic winding.

In this syntax, the inductance can be either a value (in units of henries), an equation, a polynomial of the current, or a magnetic winding. Required fields are the two nodes, and the inductance or model name.

- If you specify parameters, the nodes and model name must be first. Other parameters can be in any order.
- If you specify an inductor model (see the Passive Device Models chapter in the *PrimeSim™ Continuum® Reference Manual: Device Models*), the inductance value is optional.

For a listing of output templates for inductor (L-element) models, see the *Inductor (L-element)* table in Element Template Listings in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

Inductor Temperature Equation

The following equation provides the effective inductance as a function of temperature:

$$L(t) = L \cdot (1.0 + TC1 \cdot \Delta t + TC2 \cdot \Delta t^2)$$

Parameter	Description
Δt	t - tnom
t	Element temperature in degrees Kelvin. t=circuit_temp + DTEMP + 273.15
tnom	Nominal temperature in degrees Kelvin. tnom=273.15 + TNOM

Examples

Example 1

In the following example, the L1 inductor connects from the coilin node to the coilout node, with an inductance of 100 nanohenries.

```
L1 coilin coilout 100n
```

Example 2

The Lloop inductor connects from node 12 to node 17. Its inductance is 1 microhenry, and its temperature coefficients are 0.001 and 0.

```
Lloop 12 17 L=1u TC1=0.001 TC2=0
```

Example 3

The Lcoil inductor connects from the input node to ground. The product of the current through the inductor, and 1E-6, determines its inductance.

```
Lcoil input gnd L='1u*i(input)' LTYPE=0
```

Example 4

The L99 inductor connects from the in node to the out node. The polynomial $L=c_0 + c_1*i + c_2*i^2$, where i is the current through the inductor, determines its inductance. The inductor also has a specified DC resistance of 10 ohms.

```
L99 in out POLY 4.0 0.35 0.01 R=10
```

Example 5

The L inductor connects from node 1 to node, as a magnetic winding element, with 10 turns of wire.

```
L 1 2 NT=10
```

Example 6:

```
LX A B 1E-9  
LR 1 0 1u IC=10mA
```

- LX is a 1-nH inductor.
- LR is a 1-uH inductor, with an initial current of 10 mA.

Frequency-dependent Inductors

You can specify frequency-dependent inductors with the `L='equation'` with the `HERTZ` keyword. The `HERTZ` keyword represents the operating frequency. In time domain

analyses, an expression with the `HERTZ` keyword behaves differently according to the value assigned to the `CONVOLUTION` keyword.

Syntax

```
Lxxx n1 n2 L='equation' [CONVOLUTION=[0|1|2] [FBASE=value]
+ [FMAX=value]]
```

Parameter	Description
Lxxx	Inductor element name. Must begin with L, followed by up to 1023 alphanumeric and special characters as defined in Special Characters in <i>PrimeSim HSPICE® User Guide: Basic Simulation and Analysis</i> .
n1 n2	Positive and negative terminal node names.
equation	The equation should be a function of HERTZ. If CONVOLUTION turns on when a HERTZ keyword is not in the equation, CONVOLUTION automatically turns off and the inductor behaves conventionally. The equation can be a function of temperature, but it does not support variables of node voltage, branch current, or time. If these variables exist in the equation with CONVOLUTION on, the calculation considers only their values at the operating point.
CONVOLUTION	Indicates which method to use: • 0 (default): Acts the same as the conventional method. • 1: Applies recursive convolution, and if the rational function is not accurate enough, it switches to linear convolution. • 2: Applies linear convolution.
FBASE	Specifies the lower bound of the transient analysis frequency. • For CONVOLUTION=1 mode, the PrimeSim Continuum solution starts sampling at this frequency. • For CONVOLUTION=2 mode, the PrimeSim Continuum solution uses this value as the base frequency point for Inverse Fourier Transformation. • For recursive convolution, the default value is 0Hz. • For linear convolution, the PrimeSim Continuum solution uses the reciprocal of the transient period.
FMAX	Specifies the possible maximum frequency of interest. The default value is the frequency point where the function reaches close enough to infinity value. This assumes that the frequency reaches 10THz and that the monotonous function is approaching the infinity value.

Example

```
L1 1 2 L='0.5n + 0.5n/(1 + HERTZ/1e8)' CONVOLUTION=1 fbase=10
+ fmax=30meg
```

AC Choke Inductors

Syntax

```
Lxxx node1 node2 [L=] INFINITY [IC=val]
```


When the inductance of an inductor is infinity, this element takes the name “AC choke.” In the PrimeSim Continuum solution , you specify an `INFINITY` value for inductors.

The PrimeSim Continuum solution does not support any other inductor parameters because it assumes that the infinite inductance value is independent of temperature and scaling factors. The AC choke acts as a short circuit for all DC analyses and the PrimeSim Continuum solution calculates the DC current through the inductor. In all other (non-DC) analyses, a DC current source of this value represents the choke — the PrimeSim Continuum solution does not allow di/dt variations.

Mutual Inductors

General form:

```
Kxxx Lyyy Lzzz [K=coupling | coupling]
```

Mutual core form:

```
Kaaa Lbbb [Lccc ... Lddd] mname [MAG=magnetization]
```

Parameter	Description
Kxxx	Mutual inductor element name. Must begin with K, followed by up to 1023 alphanumeric and special characters as defined in Special Characters in <i>PrimeSim HSPICE® User Guide: Basic Simulation and Analysis</i> .
Lyyy	Name of the first of two coupled inductors.
Lzzz	Name of the second of two coupled inductors.
K=coupling	Coefficient of mutual coupling. K is a unitless number, with magnitude > 0. If K is negative, the direction of coupling reverses. This is equivalent to reversing the polarity of either of the coupled inductors. Use the K=coupling syntax when using a parameter value or an equation, and the keyword “k=” can be omitted.
Kaaa	Saturable core element name. Must begin with K, followed by up to 1023 alphanumeric and special characters as defined in Special Characters in <i>PrimeSim HSPICE® User Guide: Basic Simulation and Analysis</i> .
Lbbb, Lccc, Lddd	Names of the windings about the Kaaa core. Requires one winding element, and each winding element must use the magnetic winding syntax. Write all winding elements with the same magnetic core model in one mutual inductor statement in the netlist.
mname	Saturable core model name. (See the Passive Device Models chapter in the <i>PrimeSim™ Continuum® Reference Manual: Device Models</i> for more information.)
MAG=magnetization	Initial magnetization of the saturable core. You can set this to +1, 0, or -1, where +/- 1 refer to positive and negative values of the BS model parameter. (See the Passive Device Models chapter in the <i>PrimeSim™ Continuum® Reference Manual: Device Models</i> for more information.)

In this syntax, *coupling* is a unitless value from zero upward, representing the coupling strength. If you use parameter labels, the nodes and model name must be first. Other arguments can be in any order. If you specify an inductor model (see the Passive Device Models chapter in the *PrimeSim™ Continuum® Reference Manual: Device Models*), the inductance value is optional.

You can determine the coupling coefficient, based on geometric and spatial information. To determine the final coupling inductance, the PrimeSim Continuum solution divides the coupling coefficient by the square-root of the product of the self-inductances.

When using the mutual inductor element to calculate the coupling between more than two inductors, the PrimeSim Continuum solution can automatically calculate an approximate second-order coupling (see Example 3 in this section).

Note:

The automatic inductance calculation is an estimation, and is accurate for a subset of geometries. The second-order coupling coefficient is the product of the two first-order coefficients, which is not correct for many geometries.

For a listing of output templates for mutual inductor (K-element) models, see the *Mutual Inductor (K-element)* table in Element Template Listings in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

To create coupling between inductors, use a separate coupling element.

To specify mutual inductance between two inductors, use the coefficient of coupling, K value. The following equation defines K value:

$$K = \frac{M}{(L_1 \cdot L_2)^{\frac{1}{2}}}$$

Parameter	Description
L_1, L_2	Inductances of the two coupled inductors.
M	Mutual inductance, between the inductors.

The linear branch relation for transient analysis, is:

$$v_1 = L_1 \cdot \frac{d i_1}{d t} + M \cdot \frac{d i_2}{d t}$$

$$v_2 = M \cdot \frac{d i_1}{d t} + L_2 \cdot \frac{d i_2}{d t}$$

The linear branch relation for AC analysis, is:

$$V_1 = (j \cdot \omega \cdot L_1) \cdot I_1 + (j \cdot \omega \cdot M) \cdot I_2$$

$$V_2 = (j \cdot \omega \cdot M) \cdot I_1 + (j \cdot \omega \cdot L_2) \cdot I_2$$

Note:

If you do not use a mutual inductor statement to define an inductor reference, then an error message appears, and simulation terminates.

Checking Parameter Limits

If an inductive element value exceeds 0.1 Henry, the output listing file receives a warning message. This feature helps you identify elements that are missing units or that have incorrect values, particularly if the elements are in automatically-produced netlists.

Example 1

This example couples the Lin and Lout inductors, with a coefficient of 0.9.

```
K1 Lin Lout 0.9
```

Example 2

This example couples the Lhigh and Llow inductors, with a coefficient equal to the value of the COUPLE parameter.

```
Kxfmr Lhigh Llow K=COUPLE
```

- The K1 mutual inductor couples L1 and L2.
- The K2 mutual inductor couples L2 and L3.

Example 3

The coupling coefficients are 0.98 and 0.87. The PrimeSim Continuum solution automatically calculates the mutual inductance between L1 and L3, with a coefficient of $0.98 \cdot 0.87 = 0.853$.

```
K1 L1 L2 0.98  
K2 L2 L3 0.87
```

Reluctors

Syntax

Reluctance inline form:

```
Lxxx n1p n1n ... nNp nNn  
+ RELUCTANCE=(r1, c1, val1, r2, c2, val2, ... , rm, cm, valm)  
+ [SHORTALL=yes | no] [IGNORE_COUPLING=yes | no]
```

Reluctance external file form:

```
Lxxx n1p n1n ... nNp nNn RELUCTANCE
+ FILE="filename1" [FILE="filename2" [...]]
+ [SHORTALL=yes | no] [IGNORE_COUPLING=yes | no]
```

Parameter	Description
Lxxx	Name of a reluctor. Must begin with L, followed by up to 1023 alphanumeric and special characters as defined in Special Characters in <i>PrimeSim HSPICE® User Guide: Basic Simulation and Analysis</i> .
n1p n1n ... nNp nNn	Names of the connecting terminal nodes. The number of terminals must be even. Each pair of ports represents the location of an inductor.
RELUCTANCE	Keyword to specify reluctance (inverse inductance).
r1, c1, val1, r2, c2, val2, ... rm, cm, valm	Reluctance matrix data. In general, K is sparse and only non-zero values appear in the matrix. A triplet (r,c,val) represents each matrix entry. The value r and c are integers referring to a pair of inductors from the list of terminal nodes. If there are 2*N terminal nodes, there are N inductors, and the r and c values must be in the range [1,N]. The val value is a reluctance value for the (r,c) matrix location, and the unit for reluctance is the inverse Henry (H^{-1}). The reluctance_matrix specifies only terms along and above the diagonal. The simulator fills in the lower triangle to ensure symmetry. If you specify lower diagonal terms, the simulator converts that entry to the appropriate upper diagonal term. If you supply multiple entries for the same (r,c) location, it only uses the first one, and ignores the others. A warning advises of the ignored entries. You must assign a positive value all diagonal entries of the reluctance matrix. The reluctance matrix should be positive definite.
FILE=filename1	For the external file format, the data files should contain three columns of data. Each row should contain an (r,c,val) triplet separated by white space. The r, c, and val values may be expressions surrounded by single quotes. You can specify multiple files to spread the reluctance data over several files, if necessary.
SHORTALL	• SHORTALL=yes: Converts all inductors in this model to short circuits, and ignores all reluctance matrix values. • SHORTALL=no (default): Does not convert inductors to short circuits, and does not ignore reluctance matrix values.
IGNORE_COUPLING	• IGNORE_COUPLING=yes: Ignores all off-diagonal terms (that is, re-sets them to zero). • IGNORE_COUPLING=no (default): Does not do off-diagonal terms.

Example

This example has 9 segments (or ports) with 12 nodes and can potentially generate a 9x9 reluctance matrix with 81 elements.

```
L_ThreeNets a 1 1 2 2 a_1 b 4 4 5 5 b_1 c 7 7 8 8 c_1
+ RELUCTANCE= (
```

Chapter 1: Elements

Passive Elements: Resistors, Capacitors, and Inductors

```
+ 1 1 103e9
+ 1 4 -34.7e9
+ 1 7 -9.95e9
+ 4 4 114e9
+ 4 7 -34.7e9
+ 7 7 103e9
+ 2 2 103e9
+ 2 5 -34.7e9
+ 2 8 -9.95e9
+ 5 5 114e9
+ 5 8 -34.7e9
+ 8 8 103e9
+ 3 3 103e9
+ 3 6 -34.7e9
+ 3 9 -9.95e9
+ 6 6 114e9
+ 6 9 -34.7e9
+ 9 9 103e9 )
+ SHORTALL = no IGNORE_COUPLING = no
```

Alternatively, you can specify the same element by using:

```
L_ThreeNets a 1 1 2 2 a_1 b 4 4 5 5 b_1 c 7 7 8 8 c_1 RELUCTANCE
+ FILE="reluctance.dat" SHORTALL = no IGNORE_COUPLING = no
```

Where `reluctance.dat` contains:

```
+ 1 1 103e9
+ 1 4 -34.7e9
+ 1 7 -9.95e9
+ 4 4 114e9
+ 4 7 -34.7e9
+ 7 7 103e9
+ 2 2 103e9
+ 2 5 -34.7e9
+ 2 8 -9.95e9
+ 5 5 114e9
+ 5 8 -34.7e9
+ 8 8 103e9
+ 3 3 103e9
+ 3 6 -34.7e9
+ 3 9 -9.95e9
+ 6 6 114e9
+ 6 9 -34.7e9
+ 9 9 103e9
```

The following shows the mapping between the port numbers and node pairs:

Ports	1	2	3	4	5	6	7	8	9
Node pairs	(a,1)	(1,2)	(2,a_1)	(b,4)	(4,5)	(5,b_1)	(c,7)	(7,8)	(8,c_1)

Ideal Transformer

Kxxx Li Lj [k=IDEAL | IDEAL]

Ideal transformers use the `IDEAL` keyword with the `K` element to designate ideal `K` transformer coupling.

Equation 3 presents multiple coupled inductors. The `IDEAL` keyword activates the equation set for non-DC values. I_j is the current into the first terminal of L_j .

Equation 3

$$\frac{v1}{\sqrt{L1}} = \frac{v2}{\sqrt{L2}} = \frac{v3}{\sqrt{L3}} = \frac{v4}{\sqrt{L4}} = \dots$$

Equation 4

$$0 = (i1 \cdot \sqrt{L1}) + (i2 \cdot \sqrt{L2}) + (i3 \cdot \sqrt{L3}) + (i4 \cdot \sqrt{L4}) + \dots$$

The PrimeSim Continuum solution can solve any I or V in terms of L ratios and treats DC as expected—while it treats inductors as short circuits. The simulation ignores mutual coupling for DC.

You can couple inductors that use the `INFINITY` keyword with `IDEAL` `K` elements. In this situation, all inductors involved must have the `INFINITY` value, and for `K=IDEAL`, the ratio of all L values is unity. Then, for two L values:

```
v2= v1
i2 + i1=0
```

Example 1

This example is a standard 5-pin ideal balun transformer subcircuit. Two pins are `gnd` for standard operation. With all `K` values being `IDEAL`, the absolute L values are not crucial—only their ratios are important.

```
**
**   all K's ideal   -----o out1
**                               Lo1=.25
**   o-----in-     -----o 0
**               Lin=1   Lo2=.25
** 0 o-----         -----o out2
**
.subckt BALUN1  in  out1  out2
Lin   in   gnd   L=1
Lo1   out1 gnd   L=0.25
Lo2   gnd  out2  L=0.25
K12   Lin  Lo1   IDEAL
K13   Lin  Lo2   IDEAL
K23   Lo1  Lo2   IDEAL
.ends
```

Example 2

This example is a 2-pin ideal 4:1 step-up balun transformer subcircuit with shared DC path (no DC isolation). Input and output have a common pin, and both inductors have the same value. Note that $R_{load}=4 \cdot R_{in}$.

```
**
**   all K's ideal
**in o-----o out=in
**               L1=1
**               ----o 0
**               L2=1
**               ----o out2
**
** With all K's ideal, the actual L's values are
** not important -- only their ratio to each other.
.subckt BALUN2 in out2
L1    in    gnd    L=1
L2    gnd    out2   L=1
K12   L1    L2    IDEAL
.ends
```

Example 3

This example is a 3-pin ideal balun transformer with shared DC path (no DC isolation). All inductors have the same value (here set to unity).

```
**
**   all K's ideal   ----o out1
**               Lo2=1
**               ----o 0
**               Lo1=1
**               ----o out2
**   in             Lin=1
** o-----o in
**
.subckt BALUN3 in out1 out2
Lo2    gnd    out1   L=1
Lo1    out2   gnd    L=1
Lin    in     out2   L=1
K12    Lin    Lo1    IDEAL
K13    Lin    Lo2    IDEAL
K23    Lo1    Lo2    IDEAL
.ends
```

Ideal Transformer Format

The ideal transformer format simplifies modeling of baluns. They previously used mutual inductors (K elements) with the `IDEAL` keyword. A given balun model required Multiple L and K elements. The ideal transformer model allows modeling of a balun using a single L element.

In the ideal transformer format, you do not to specify no absolute inductance or reluctance values. Instead, you specify the transformer’s coupling characteristics using inductor number-of-turns values. The behavior of the ideal transformer depends on ratios of the inductors’ number of turns.

Syntax

```
Lxxx n1p n1n ... nNp nNn TRANSFORMER_NT=(nt1, ... , ntN)
```

Parameter	Description
Lxxx	Inductor element name. Must begin with L, followed by up to 1023 alphanumeric and special characters as defined in Special Characters in <i>PrimeSim HSPICE® User Guide: Basic Simulation and Analysis</i> .
n1pn1nnNpnNn ...	Positive and negative terminal node names. The number of terminals must be even. Each pair of reports represents the location of an inductor.
TRANSFORMER_NT	Number of turns values. These parameters must match the number of inductors.

The ideal transformer element obeys the standard ideal transformer equations:

$$\frac{v_1}{nt_1} = \frac{v_2}{nt_2} = \dots = \frac{v_N}{nt_N}$$
$$i_1nt_1 + i_2nt_2 + \dots + i_Nnt_N = 0$$

Example

```
L1 1 0 0 2 3 0 transformer_nt=(1,2,2)
```

Multi-terminal Linear Elements

The following sections discuss:

- [S-element \(Scattering Parameter Data\)](#)
- [W-element \(Distributed Transmission Lines\)](#)
- [T-element \(Ideal Transmission Lines\)](#)
- [U-element \(Lumped Transmission Lines\)](#)
- [Port Element](#)

S-element (Scattering Parameter Data)

The content of this section includes:

- [S-element Syntax](#)
- [S-element Example](#)

S-element Syntax

Use the following S-element syntax to show the connections within a circuit:

```
Sxxx nd1 nd2 ... ndN ndRef
+ [ENFORCE_PASSIVE=0|1]
+ [MNAME=Smodel_name] [FQMODEL=sp_model_name]
+ [TYPE=[s|y]] [Z0=[value| vector_value]]
+ [FBASE = base_frequency] [FMAX=maximum_frequency]
+ [PRECFAC=val] [DELAYHANDLE=[1|2|0|ON|OFF]]
+ [DELAYFREQ=val]
+ [INTERPOLATION=STEP|LINEAR|SPLINE|HYBRID]
+ [INTDATYP=[RI|MA|DBA]] [HIGHPASS=[0|1|2|3|4]]
+ [LOWPASS=[0|1|2|3]] [MIXEDMODE=[0|1]]
+ [DATATYPE=data_string] [XLINELength=val]
+ [NOISE=[1|0]] [NoiPassiveChk=1|0] [DTEMP=val]
+ [PASSIVE=[0|1]] [PASSIVE_TOL=val] [COLSUM_LIMIT=val]
+ [RATIONAL_FUNC=[0|1|2]] [RATIONAL_FUNC_REUSE=[0|1]]
+ [RATIONAL_FUNC_WINDOWING=[0|1]]
+ [RATIONAL_FUNC_FOR_AC=0|1]
+ [STAMP=[S|Y|YSTS|SSTS|DEEMBED|AUTO]] [M=int]
+ [SMOOTH=val] [SMOOTHPTS=val]
+ [RATIONAL_FUNC_CACHE_DIR = dir_str]
+ [CACHE_DIR = path]
+ [CAUSALITY=[0|1]]
+ [CAUSALITY_FMAX_SCALE = val]
+ [WRITE_CAUSALITY_ENFORCED_FILE = [0|1]]
+ [CAUSALITY_ENFORCED_FILE_NAME = file_name]
```

Parameter	Description
nd1 nd2...ndN	Nodes of an S-element (see Figure 1 on page 47) and S-element Example. Three kinds of definitions are present: • With no reference node ndRef, the default reference node is GND. Each node ndi (i=1~N) and GND construct one of the N ports of the S-element. • Defines ndRef with one reference node. Each node ndi (i=1~N) and the ndRef construct one of the N ports of the S-element. • With an N reference node, each port has its own reference node. You can write the node definition in a clearer way as: nd1+ nd1- nd2+ nd2- ... ndN+ ndN- Each pair of the nodes (ndi+ and ndi-, i=1~N) constructs one of the N ports of the S-element.
ndRef	Reference node

Parameter	Description
ENFORCE_PASSIVE	With the ENFORCE_PASSIVE=1 keyword, the S-element statement checks passivity of all the given frequency sampling points. Once the PrimeSim Continuum solution finds passivity violations, the S-element seeks a minimum amount of loss property which restores passivity of all the violated points then adds the loss to all the given frequency points.
MNAME	Name of the S model; (Supports string parameters in calling an MNAME.)
FQMODEL	Frequency behavior of the parameters. .MODEL statement of sp type, which defines the frequency-dependent matrices array
TYPE	Parameter type: • S: (scattering) (default) • Y: (admittance)
Z0 (or Zo)	Characteristic impedance value for the reference line (frequency-independent). For multiple terminals (N>1), the PrimeSim Continuum solution assumes that the characteristic impedance matrix of the reference lines is diagonal, and that you set diagonal values to Z0. Default=50.
FBASE	Base frequency to use for transient analysis. This value becomes the base frequency point for Inverse Fast Fourier Transformation (IFFT). • If you do not set this value, the base frequency is a reciprocal value of the transient period. • If you set a frequency that is smaller than the reciprocal value of the transient, then transient analysis performs circular convolution, and uses the reciprocal value of FBASE as its base period.
FMAX	Maximum frequency use in transient analysis. Used as the maximum frequency point for Inverse Fast Fourier Transformation (IFFT).
PRECFAC	In almost all cases, you do not need to specify a value for this parameter. This parameter specifies the precondition factor keyword used for the precondition process of the S-parameter. A precondition avoids an infinite admittance matrix. The default is 0.75, which is good for most cases.

Parameter	Description
DELAYHANDLE	DELAYHANDLE extracts a system delay before constructing the system impulse response. This may improve transient accuracy when the system does have delay, such as transmission line system. Because S-parameters represent a system which has delay, you should turn DELAYHANDLE on. When DELAYHANDLE is ON (or 1) the S-element extracts propagation delay to simplify transfer functions, then proceeds to approximation. The extracted delay is handled separately in the time domain. You must set the delay handler, if the delay of the model is longer than the base period specified in the FBASE parameter. If DELAYHANDLE is 2, the S-element extracts the propagation delay according to the model property over the whole frequency band, instead of calculating group delay upon single frequency point as that when DELAYHANDLE=1. This option is now only available when the rational_func=1. When RATIONAL_FUNC=1 is used, it is recommended not to specify the DELAYHANDLE keyword. S-element automatically detects and handles the propagation delay. If you set DELAYHANDLE=OFF but DELAYFREQ is not zero, the PrimeSim Continuum solution simulates the S-element in delay mode.
DELAYFREQ	Delay frequency for transmission-line type parameters. The default is FMAX. If the DELAYHANDLE is OFF, but DELAYFREQ is nonzero, the PrimeSim Continuum solution still simulates the S-element in delay mode.
INTERPOLATION	The interpolation method: • STEP: piecewise step • SPLINE: b-spline curve fit • LINEAR: piecewise linear (default) • HYBRID: The PrimeSim Continuum solution combines different interpolation/extrapolation methods, and switches automatically between them to get the best accuracy. If needed, it also does causality correction down to DC. It is most useful for the S-parameters showing local resonances, and provides the proper interpolation and low-frequency extrapolation method for each entry of the S matrix, which shows different behaviors. For best accuracy, provide low frequency examples.
INTDAT TYP	Data type for the linear interpolation of the complex data. • RI: real-imaginary based interpolation • DBA: dB-angle based interpolation • MA: magnitude-angle based interpolation (default)
HIGHPASS	Method to extrapolate higher frequency points. • 0: Use zero in Y dimension (open circuit). • 1: Use the given spectrum for IFFT process. No extrapolation is done. This is only available for S-type-based model. • 2: Use linear extrapolation with the highest two points. • 3: (default) Apply window function. • 4: Estimates average derivatives of the phase and magnitude from highest 10% of sampling points. Extrapolation is performed using the highest sampling point and these derivatives. This option overrides EXTRAPOLATION in .MODEL SP.

Parameter	Description
LOWPASS	Method to extrapolate lower frequency points. • 0: Cut off. • 1: Make use of the S matrix at the magnitude of the lowest given frequency point; Set the magnitude value of each entry as the element of DC matrix. The real part of the extrapolated value at DC point determines the sign of each value (default). • 2: Perform linear extrapolation with the magnitude of the lowest two points. • 3: Perform rational function approximation based on low end frequency extrapolation.
MIXEDMODE	Set to =1 if the parameters are in the mixed mode.
DATATYPE	A string used to determine the order of the indices of the mixed-signal incident or reflected vector. The string must be an array of a letter and a number (Xn) where: • X = D to indicate a differential term = C to indicate a common term = S to indicate a single (grounded) term • n = the port number
XLINLENGTH	The line length of the transmission line system where the S-parameters are extracted. This keyword is required only when the S Model is used in a W-element.
NOISE	Activates thermal noise. • 1 (default): Element generates thermal noise • 0: Considers element noiseless
NoiPassiveChk	Checks S-parameter for passivity in noise analysis (only). • 1 (default): Checks for passivity; if it fails at any frequency, thermal noise is <i>off</i> for the specific frequency point. • 0: Disables the passivity checker; thermal noise is always <i>on</i>.
DTEMP	Temperature difference between the element and the circuit, expressed in $\times C$. The default is 0.0. Calculates element temperature as: $T = \text{Element temperature } (\times K) = 273.15 (\times K) + \text{circuit temperature } (\times C) + DTEMP (\times C)$ Where you specify circuit temperature by using either the .TEMP statement, or by sweeping the global TEMP variable in .DC, .AC, or .TRAN statements. When a you do not use a .TEMP statement or TEMP variable, .OPTION TNOM, sets the circuit temperature, which defaults to 25 $\times C$ unless you use .OPTION SPICE, which raises the default to 27 $\times C$.
PASSIVE	Activates the passive checker to help debug passive models. The default is 0 for the S-element where 0=<i>deactivate</i> and 1=<i>activate</i>. Using the tolerance value specified by PASSIVE_TOL keyword, the eigenvalues of matrix $(I-S^*S')$, $ev[i]$, is checked. • If any frequency point violates $RE(ev[i]) > -(TOL*0.1)$, the PrimeSim Continuum solution issues a warning containing a list of violating frequencies with an "E" flag. Also, the checker verifies potential passivity violations by checking the summation of each S-parameter matrix column. • If $Sum > COLSUM_LIMIT$, the PrimeSim Continuum solution issues a warning containing a list of those violating frequencies with a "C" flag.

Parameter	Description
PASSIVE_TOL	Tolerance for eigenvalue checking activated by PASSIVE keyword. Default is 0.01.
COLSUM_LIMIT	Maximum value allowed for S-parameter matrix column summation. The limiting value is used for passivity checker and passivity enforcement. Default is 1.0
RATIONAL_FUNC	• 0: Performs the same as conventional S-element. FBASE/FMAX-based linear convolution is performed. • 1 (default): Performs rational function approximation then recursive convolution; also handles noncausal S-parameters. • 2: Performs rational function approximation to both time and frequency domain simulation. When RATIONAL_FUNC=1 is used, it is recommended to not specify the DELAYHANDLE keyword. S-element automatically detects and handles the propagation delay. See also .OPTION S_RATIONAL_FUNC in the <i>PrimeSim HSPICE Reference Manual: Commands and Control Options</i>.
RATIONAL_FUNC_REUSE	The S-element rational function approximation process stores the fitting data into a binary file named MODEL_NAME.yrf (DELAYHANDLE=0) or MODEL_NAME.yrfd (DELAYHANDLE=1). The S-element seeks these files and reuse when available, if RATIONAL_FUNC_REUSE=1 (default). Reusing rational function data increases efficiency especially for large systems. • 0: Discard previously extracted rational function data and rerun the rational function approximation. • 1: (default) Reuse rational function data if available.
RATIONAL_FUNC_WINDOWING	When you specify this argument with a value 1, a window function is applied that smoothly rolls off the propagation toward the maximum frequency when generating the rational function. The default maximum frequency is the highest sampling point in the given S-parameter data. When FMAX is specified, the value of FMAX is used for the maximum frequency. When you specify this argument with a value 0 (default), no windowing function is applied. See also .OPTION S_RATIONAL_FUNC_WINDOWING in the <i>PrimeSim Continuum Reference Manual: Commands and Control Options</i>.
RATIONAL_FUNC_FOR_AC	When used in conjunction with RATIONAL_FUNC=1, RATIONAL_FUNC_FOR_AC = 1 causes the S-element to use the extracted rational function in frequency domain evaluations. Default is 0.

Parameter	Description
STAMP	- AUTO (Default): Automatically uses an optimized stamping method for S-Parameter - Y: Conventional admittance based stamp - S: Scattering parameter based stamp (Note 1) - YSTS: Admittance parameter based state space stamp (see Note 2) - SSTS: Scattering parameter based state space stamp (see Note 2) - DEEMBED: Produces negated stamp to de-embed given a S-parameter block from the adjacent DUT connected in series. Note: Use STAMP=SSTS when running Shooting Newton (SN) based analyses using the PrimeSim SPICE tool.
M	S-element multiplier; replicates element <i>int</i> times, in parallel; default is 1. Do not assign a negative value or zero as the M value.
SMOOTH	An integer value to choose one of following methods: 0: no smoothing (default). 1: mean. 2: median. 3: 2nd order polynomial fit. 4: 4th order polynomial fit.
SMOOTHPTS	An integer value to specify width of the smoothing window on each side of the target point. In total, 2*x +1 point is taken at each point calculation.
RATIONAL_FUNC_CACHE_DIR	Specifies the directory to write (for reuse) the generated cache files, such as rational function model generation.
CACHE_DIR	Specifies the location of the IFFT impulse response, causality enforced touchstone file, rational function srf, yrf, and penf files. If not specified the S-parameters file directory is used by default.
CAUSALITY	Turns on or off causality enforcement algorithm for IFFT model: 0 (default): OFF ** 1: ON ** When causality enforcement is turned on, the HIGHPASS and FMAX keyword of S element or model is ignored. - For PrimeSim SPICE: default is 0. - For PrimeSim: default is 1.
CAUSALITY_FMAX_SCALE	A real value <i>val</i>, which is used to manually set $F_{MAX}=val * F_{max_given}$ in causality enforcement algorithm, where <i>Fmax_given</i> is the highest frequency value in the S model. Default is 3. Only valid when CAUSALITY=1.
WRITE_CAUSALITY_ENFORCED_FILE	Turns on or off causality enforced result output to a touchstone format file. 0 (default): do not output 1: output Only valid when CAUSALITY=1

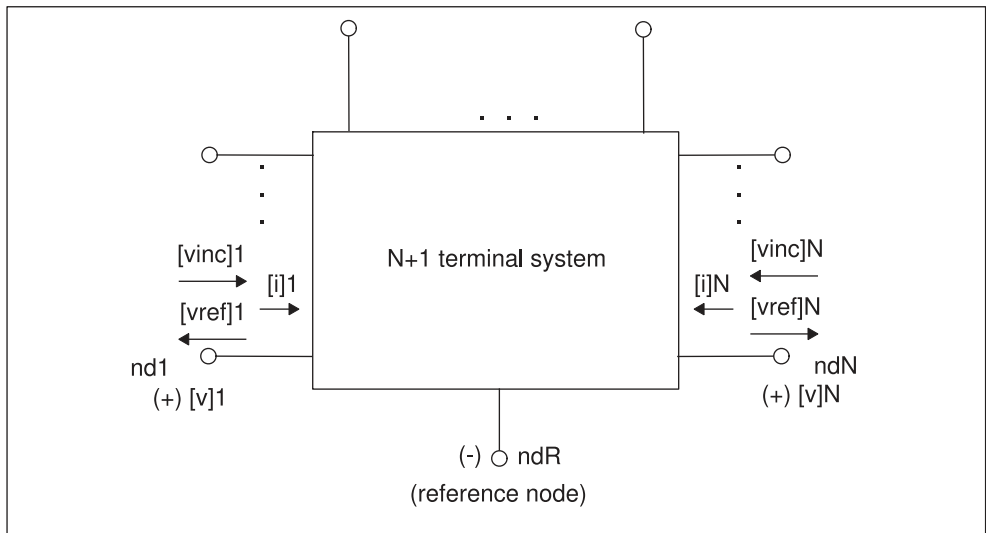
Parameter	Description
CAUSALITY_ENFORCED_FILE_NAME	Touchstone file name for the causality enforced result output. Only valid when CAUSALITY=1 When file name is not specified, while WRITE_CAUSALITY_ENFORCED_FILE=1, the default filename is constructed by S model name, such as model_name_enforced.s#p

The nodes of the S-element must come first. If you do not declare the MNAME, you must specify the FQMODEL. You can specify all the optional parameters in both the S-element and S model statements, except for the MNAME argument.

You can enter the optional arguments in any order, and the parameters specified in the element statement have a higher priority.

S-element Example

Figure 1 Terminal Node Notation



The following example illustrates the nd1 nd2...ndN—no reference, single reference, and multi-reference parameters.

```
**S-parameter example

.opt post
.ac lin 500 1Hz 30MegHz
.tran 0.1ns 10ns

V1 n1 0 ac=1v PULSE 0v 5v 5n 0.5n 0.5n 25n
```

```
* no reference
S_no_ref n1 n2 mname=s_model

* single reference
S_one_ref n1 n3 gnd mname=s_model

*multi-reference
S_multi_ref n1 gnd n4 gnd mname=s_model
Rt1 n2 0 50
Rt2 n3 0 50
Rt3 n4 0 50

* 50 ohm resistor
.MODEL s_model S
+ N=2 FQMODEL=SFQMODEL TYPE=S Z0=50 50
.MODEL SFQMODEL SP N=2 SPACING=POI INTERPOLATION=LINEAR
+ MATRIX=NONSYMMETRIC
+ DATA=1
+ 1.0 0.3333333333 0.0 0.6666666667 0.0 0.6666666667 0.0 0.3333333333 0.0

.end
```

The S-element must have a call to one of the supported S-parameter file formats (Touchstone, Citi or .SC#). The PrimeSim Continuum solution gets the number of ports from the S-parameter file. You can also explicitly specify N=n where 'n' is the number of ports.

- For n terminals, the S-element assumes no reference node.
- For n+1 terminals, the S-element assumes one reference node.
- For 2n terminals, the S-element assumes signal nodes and n reference nodes. Each pair of nodes is a signal and a reference node.

W-element (Distributed Transmission Lines)

The content of this section includes:

- [W-element Syntax](#)
- [Five W-element Models](#)
- [W-element Examples](#)

W-element Syntax

```
Wxxx i1 i2 ... iN iR o1 o2 ... oN oR N=val L=val
+ [RLGCMODEL=name | RLGCFILE=name | UMODEL=name
+ FSMODEL=name | TABLEMODEL=name | SMODEL=name]
+ [RSTYPE=1|2]
+ [INCLUDESIMAG=YES|NO FGD=val ] [DELAYOPT=0|1|2|3]
```



```
+ [INCLUDEGDMAG=YES|NO] [NODEMAP=XiYj] [DCACC=[1|0]]
+ [NOISE=[1|0]] [DTEMP=val]
+ [PRINTZO=frequency_sweep MIXEDMODEZO=0|1]
+ [SCALE_RS=val]
```

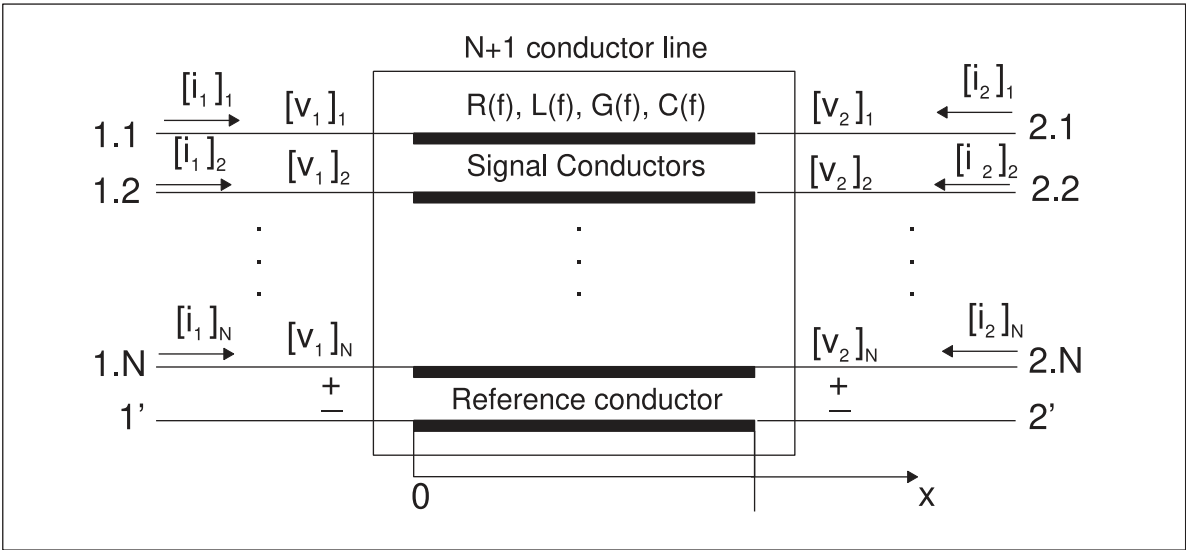
Parameter	Description
<i>N</i>	Number of signal conductors (excluding the reference conductor).
<i>i1...iN</i>	Node names for the near-end signal-conductor terminal (see Figure 2).
<i>iR</i>	Node name for the near-end reference-conductor terminal.
<i>o1...oN</i>	Node names for the far-end signal-conductor terminal (see Figure 2).
<i>oR</i>	Node name for the far-end reference-conductor terminal. Note: The connection of iR and oR should ensure that all the return current flow through the reference plane and there are no peripherals to branch out the return current.
<i>L</i>	Length of the transmission line.
RLGCMODEL	Name of the RLGC model.
RLGCFILE	Name of the external file with RLGC parameters. A RLGC file name must start with an alphabetic letter (not a number). (Input Model 1: W-element, RLGC Model.)
UMODEL	Name of the U model. (See Input Model 2: U-element, RLGC Model.)
FSMODEL	Name of the field solver model. (See Input Model 3: Built-in Field-Solver Model.)
TABLEMODEL	Name of the frequency-dependent tabular model. (See Input Model 4: Frequency-Dependent Tabular Model.)
SMODEL	Name of the S model. (See Input Model 5: S Model.)
RSTYPE	Choose skin effect formulation. (See Frequency-Dependent Matrices.)
INCLUDERSIMAG	Imaginary term of the skin effect to be considered. The default value is YES. (See Frequency-Dependent Matrices.) This keyword activates the complex dielectric loss model and can operate with the DELAYOPT parameter (see Introducing the Complex Dielectric Loss Model).

Parameter	Description
INCLUDEGDMAG	Activates the complex dielectric loss model (see Fitting Procedure Triggered by INCLUDEGDMAG Keyword). Gd: coefficient matrices of the frequency dependency; wp: corresponding frequency value of the polarization time constants. If INCLUDEGDMAG=yes and there is no wp input, the W-element regards the Gd matrix as the conventional model and then automatically extracts constants for the complex dielectric model. The INCLUDEGDMAG keyword operates with the DELAYOPT parameter. To set this parameter as a global option for all W-elements in the PrimeSim Continuum solution netlist, see .OPTION WINCLUDEGDMAG in the <i>PrimeSim HSPICE® Reference Manual: Commands and Control Options</i>.
FGD	Specifies the cut-off frequency of dielectric loss. (See Handling the Dielectric-loss Matrix.)
DELAYOPT	• 0: Deactivates • 1: Activates • 0: Default Deactivates (0), activates (1), determines automatically (2), or high frequency (3). The default is 0. To set this parameter as a global option for all W-elements in the PrimeSim Continuum solution netlist, see .OPTION WDELAYOPT in the <i>PrimeSim HSPICE® Reference Manual: Commands and Control Options</i>.
DCAAC	Deactivates(0), activates(1). The default is 1. An additional step to check the accuracy of low frequency and DC coverage.
NODEMAP	String that assigns each index of the S-parameter matrix to one of the W-element terminals. This string must be an array of pairs that consists of a letter and a number, (for example, Xn), where • X= I, i, N, or n to indicate near end (input side) terminal of the W-element • X= O, o, F, or f to indicate far end (output side) terminal of the W-element. The default value is NODEMAP = I1I2I3...InO1O2O3...On.
NOISE	Activates thermal noise. • 1 (default): element generates thermal noise • 0: element is considered noiseless
DTEMP	Temperature difference between the element and the circuit, expressed in °C. The default is 0.0. Element temperature is calculated as: $T = \text{Element temperature (°K)} = 273.15 (\text{°K}) + \text{circuit temperature (°C)} + \text{DTEMP (°C)}$ Where circuit temperature is specified using either the .TEMP statement, or by sweeping the global TEMP variable in .DC, .AC, or .TRAN statements. When a .TEMP statement or TEMP variable is not used, the circuit temperature is set by .OPTION TNOM, which defaults to 25°C unless you use .OPTION SPICE, which raises the default to 27°C.

Parameter	Description
PRINTZO	Type of frequency sweep to allow checking of the complex characteristic impedance matrix of the system. You can specify any of LIN, DEC, OCT, or POI (see Using the PRINTZO Option). Specify the nsteps, start, and stop values using the following syntax for each type of sweep: • LIN nsteps start stop • DEC nsteps start stop • OCT nsteps start stop • POI nsteps freq_values
MIXEDMODEZO	• 0: Single-ended impedance is printed out (default) • 1: Output characteristic impedance is printed in mixed mode
SCALE_RS	RS matrix scaling factor, W-element instance

Figure 2 shows node numbers for the element syntax.

Figure 2 Terminal Node Numbering for the W-element



Five W-element Models

The W-element supports the following formats to specify the transmission line properties

- Model 1: RLGC-Model specification
 - Internally specified in a .model statement
 - Externally specified in a different file
- Model 2: U-Model specification

- RLGC input for up to five coupled conductors
- Geometric input (planer, coax, twin-lead)
- Measured-parameter input
- Skin effect
- Model 3: Built-in field solver model
 - Standard format (using geometric data with the W-element)
 - Tabular format
- Model 4: Frequency-dependent tabular model
- Model 5: S-parameter Model

The general syntax for a lossy (W-element) transmission line element is:

RLGC input form (Model 1):

```
Wxxx in1 [in2 [...inx]] refin out1 [out2 [...outx]]  
+ refout [RLGCfile=filename | RLGCMODEL=name] N=val L=val
```

U-Model form (Model 2):

```
Wxxx in1 [in2 [...inx]] refin out1 [out2 [...outx]]  
+ refout Umodel=modelname N=val L=val
```

Field solver form (Model 3):

```
Wxxx in1 [in2 [...inx]] refin out1 [out2 [...outx]]  
+ refout FModel=modelname N=val L=val
```

Table Model form (Model 4):

```
Wxxx in1[in2 [...inx]] refin out1 [out2 [...outx]]  
+ refout N=val L=val TABLEMODEL=name
```

S-Model form (Model 5):

```
Wxxx in1 [in2 [...inx]] refin out1 [out2 [...outx]]  
+ refout Smodel=modelname [NODEMAP=XiYj...] N=val L=val
```

The number of ports on a single transmission line is unlimited. You must provide one input and output port, the ground references, a model or file reference, a number of conductors, and a length.

For demo files of the S-Model usage, see Transmission (W-element) Line Examples.

For additional information about the W-element, see W- and T-elements Transmission-Line Modeling in the *PrimeSim HSPICE User Guide: Signal Integrity Modeling and Analysis*.

W-element Examples

Example 1

The W1 lossy transmission line connects the in node to the out node:

```
W1 in gnd out gnd RLGCfile=cable.rlgc N=1 L=5
```

where

- Both signal references are `gnd`
- The RLGC file name: `cable.rlgc`
- The transmission line is 5 meters long.

Example 2

The Wcable element is a two-conductor lossy transmission line:

```
Wcable in1 in2 gnd out1 out2 gnd Umodel=umod_1 N=2  
+ L=10
```

where

- `in1` and `in2` input nodes connect to the `out1` and `out2` output node
- Both signal references are `gnd`.
- `umod_1` references the U-model.
- The transmission line is 10 meters long.

Example 3

The Wnet1 element is a five-conductor lossy transmission line:

```
Wnet1 i1 i2 i3 i4 i5 gnd o1 gnd o3 gnd o5 gnd  
+ FSmodel=board1 N=5 L=1m
```

In the preceding example

- Nodes `i1`, `i2`, `i3`, `i4`, and `i5` are all input nodes.
- The `gnd` after `i5` is the input reference conductor.
- Only the output nodes `o2` and `o4` are grounded.
- The `gnd` after `o5` is the output reference conductor.

Example 4: S Model Example

```
Wnet1 i1 i2 gnd o1 o2 gnd
+ Smodel=smod_1 nodemap=i1i2o1o2
+ N=2 L=10m
```

where

- `in1` and `in2` input nodes connect to the `out1` and `out2` output node.
- Both signal references are `gnd`.
- `smod_1` references the S-Model.
- The transmission line is 10 meters long.

You can specify parameters in the W-element card in any order. You can specify the number of signal conductors, `N`, after the node list. You can also mix nodes and parameters in the W-element card.

You can specify only one of the `RLGCfile`, `FModel`, `Umodel`, or `Smodel` models, in a single W-element card.

T-element (Ideal Transmission Lines)

The content of this section includes:

- [T-element Syntax](#)
- [T-element Examples](#)

T-element Syntax

General form:

```
Txxx in refin out refout Z0=val TD=val [L=val]
+ [IC=v1,i1,v2,i2]
```

```
Txxx in refin out refout Z0=val F=val [NL=val]
+ [IC=v1,i1,v2,i2]
```

U Model form:

```
Txxx in refin out refout mname L=val
```

Parameter	Description
Txxx	Lossless transmission line element name. Must begin with T, followed by up to 1023 alphanumeric characters.
in	Signal input node.

<i>refin</i>	Ground reference for the input signal.
<i>out</i>	Signal output node.
<i>refout</i>	Ground reference for the output signal.
<i>Z0</i>	Characteristic impedance of the transmission line (Ohms).
<i>TD</i>	Propagation time delay of the transmission line (in seconds). If physical length (L) is specified, then units for TD are considered in seconds per meter.
<i>L</i>	Physical length of the transmission line, in units of meters. Default=1.
<i>IC=v1,i1,v2,i2</i>	Initial conditions of the transmission line. Specify the voltage on the input port (v1), current into the input port (i1), voltage on the output port (v2), and the current into the output port (i2).
<i>F</i>	Frequency at which the transmission line has the electrical length specified in NL.
<i>NL</i>	Normalized electrical length of the transmission line (at the frequency specified in the F parameter), in units of wavelengths per line length. Default=0.25, which is a quarter-wavelength.
<i>mname</i>	U-model reference name. A lossy transmission line model, representing the characteristics of the lossless transmission line.

Only one input and output port is allowed.

T-element Examples

Example 1

The T1 transmission line connects the in node to the out node:

```
T1 in gnd out gnd Z0=50 TD=5n L=5
```

- Both signal references are grounded.
- Impedance is 50 ohms.
- The transmission delay is 5 nanoseconds per meter.
- The transmission line is 5 meters long.

Example 2

The Tcable transmission line connects the in1 node to the out1 node:

```
Tcable in1 gnd out1 gnd Z0=100 F=100MEG NL=1
```

- Both signal references are grounded.
- Impedance is 100 ohms.
- The normalized electrical length is 1 wavelength at 100 MHz.

Example 3

The Tnet1 transmission line connects the driver node to the output node:

```
Tnet1 driver gnd output gnd Umodel1 L=1m
```

- Both signal references are grounded.
- Umodel1 references the U-model.
- The transmission line is 1 millimeter long.

U-element (Lumped Transmission Lines)

For detailed information on modeling lossy non-ideal transmission lines in terms of lumped-element equivalent circuits, see [Appendix A, Lumped Transmission Line — U-Model](#).

The content of this section includes:

- [U-element Syntax](#)
- [U-element Examples](#)

U-element Syntax

```
Uxxx in1 [in2 [...in5]] refin out1 [out2 [...out5]]  
+ refout mname L=val [LUMPS=val]
```

Parameter	Description
Uxxx	Lossy (U-element) transmission line-element name. Must begin with U, followed by up to 1023 alphanumeric and special characters as defined in Special Characters in <i>PrimeSim HSPICE® User Guide: Basic Simulation and Analysis</i> .
inx	Requires signal input node for the xth transmission-line (<i>in1</i>).
refin	Ground reference for the input signal.
outx	Signal output node for the xth transmission line (each input port must have a corresponding output port).
refout	Ground reference for the output signal.
mname	Model reference name for the U-model lossy transmission-line.

Parameter	Description
<i>L</i>	Physical length of the transmission line, in units of meters.
<i>LUMPS</i>	Number of lumped-parameter sections used to simulate the element.

In this syntax, there is a limit to the number of ports on a single transmission line—five in and five out. These are the required parameters: one input and output port, the ground references, a model reference, and a length.

U-element Examples

Example 1

The U1 transmission line connects the in node to the out node:

```
U1 in gnd out gnd umodel_RG58 L=5
```

- Both signal references are `gnd`.
- `umodel_RG58` references the U-model.
- The transmission line is 5 meters long.

Example 2

The Ucable transmission line connects the in1 and in2 input nodes to the out1 and out2 output nodes:

```
Ucable in1 in2 gnd out1 out2 gnd twistpr L=10
```

- Both signal references are `gnd`.
- `twistpr` references the U-model.
- The transmission line is 10 meters long.

Example 3

The Unet1 element is a five-conductor lossy transmission line:

```
Unet1 i1 i2 i3 i4 i5 gnd o1 gnd o3 gnd o5 gnd Umodel1 L=1m
```

- The i1, i2, i3, i4, and i5 input nodes connect to the o1, o3, and o5 output nodes.
- The i5 input, and the three outputs (o1, o3, and o5) are all `gnd`.
- `Umodel1` references the U-model.
- The transmission line is 1 millimeter long.

Port Element

The port element (P-element) identifies the ports used in .LIN analysis and in all other analyses behaves as either a noiseless impedance or a voltage source in series with the port impedance (DC, AC, or TRAN). Each port element requires a unique port number. Each port has an associated system impedance, $z0$. If you do not explicitly specify the system impedance, the default is 50 ohms.

- You can use this element as a pure terminating resistance or as a voltage or power source.
- You can use the `RDC`, `RAC`, `RHB`, `RHBAC`, and `RTRAN` values to override the port impedance value for a particular analysis.

The port element accepts transient waveforms AM, EXP, PULSE, PWL, SFFM, SIN, LFSR and, for signal integrity usage, the PAT source.

The mixed-mode port element has an additional reference pin that allows further flexibility in creating, detecting, and separating common-mode and differential-mode signals. It is useful for measuring mixed-mode S-parameters (see Using the P-element for Mixed-Mode Measurement).

The content of this section includes:

- [Port Element Syntax](#)
- [Removing and Recovering DC Offset Level in AMI_GetWave Flow](#)
- [Using the Port Element for Mixed-Mode Measurement](#)
- [Mixed Mode Port Element Syntax](#)
- [Using the Five Terminal Port Element](#)

Port Element Syntax

```
Pxxx node1 node2 [node3 node4 node5] port=portindex
+ [Z0=val]                               $Port Impedance
+ [DC mag] [AC mag phase] [HBAC mag phase] $Voltage/Power Info
+ [HB mag phase harm tone modharm modtone]
+ [transient_waveform]
+ [ENCODE=DW8B10B [RD_INIT=0|1]]
+ [ENCODE=DW64b66b [TXC=txcdata]]
+ [ENCODE=DW128b130b|DW128b132b [HEAD=DATA|CONTROL]]
+ [TRANFORHB=[0|1]] [DCOPEN=[0|1]]
+ [power=[0|1|2|W|dbm]]                 $Power Switch
+ [POWERSIN=[0|1|2|W|dbm]]
+ [POWERHB=[0|1|2|W|dbm]]
+ [POWERHBAC=[0|1|2|W|dbm]]
+ [POWERSNAC=[0|1|2|W|dbm]]
```

```
+ [RDC=val] [RAC=val]           $ Source Impedance Overrides
+ [RHBAC=val] [RHB=val] [RTRAN=val]
+ [Emphasis_Level=val] [Emphasis_Time=val]           $ Emphasis
+ $ **** DFE ****
+ [DFE_tap=n1 n2 ... nN DFE_coeff=val1, val2, ... valN]
+ [DFE_VTH=val]
+ [DFE_CLK=0|1]
+ [DCD=val] [DCD_TYPE=0|1|2|3]           $ Duty Cycle Distortion
+ [PJ=val] [PJ_TYPE=0|1|2|3|4|5|6]           $ Period Jitter
+ [RJ=[val1 ... valN]] [RJ_MEAN=[val1...valN] ] [VN=val] $ Rand Jitter
Stateye
+ [RJPP=val RJPP_BER=val]
+ [VNPP=val VNPP_BER=val]
+ [JITTER_FILE=filename]
+ [IBIS_AMI_FILE=filename IBIS_AMI_MODEL=modelname]
+ [AMI_OBJ=filename] [AMI_PARAM=filename] [AMI_CDR=0|1]$AMI
+ [AMI_CORNER=idx]$AMI
+ [AMI_Remove_DC_Offset = 0|1]
+ [AMI_Recover_DC_Offset = 0|1]
+ [MODULATION=PAM4|PAM4]
+ [PAM3_MAPPING = string]
+ [PAM4_MAPPING=string]
+ [AMI_PARAM_xx = string|quoted string]
+ [IBIS_AMI_VRANGE_MODE = 1|2]
+ [IBIS_AMI_USE_CCOMP = 0|1]
+ [IBIS_AMI_CCOMP_ORDER = 1|2]
+ [IBIS_AMI_USE_RAMP = 0|1]
+ [IBIS_AMI_USE_VRANGE = 0|1]
+ [IBIS_AMI_USE_ZSRC = 0|1]
```

Parameter	Description
<i>node1node2node3node4node5[]</i>	The port element can have either two, or three, or five terminals. The three terminal model can be used for differential port (see Using the Port Element for Mixed-Mode Measurement on page 67). You can also add a five-terminal model (see Using the Five Terminal Port Element on page 69) mainly to take four port S-parameter block for the IBIS-AMI's "Ts4file Analog Buffer Models" (BIRD 158).
<i>port=portindex</i>	The port index. Numbered sequentially beginning with 1 with no shared port indexes.
<i>z0=val (or Zo=val)</i>	Port impedance (Ohms). (Default: 50). Sets port characteristic impedance used for .LIN analysis, sets port termination impedance for other analyses, and also sets source impedance when you use the port element as a signal source.
<i>DC mag</i>	DC voltage or power source value.
<i>AC mag phase</i>	AC voltage or power source value.
<i>HBAC mag phase</i>	(HBAC analysis) HBAC voltage or power source value.

Parameter	Description
HB mag phase harm tone modharm modtone	(HB analysis) HB voltage, current, or power source value. Allows multiple HB specifications with different harm, tone, modharm, and modtone values. • phase is in degrees • harm and tone are indices corresponding to the tones specified in the .HB statement. Indexing starts at 1 (corresponding to the first harmonic of a tone). • modtone and modharm specify sources for multi-tone simulation. A source specifies a tone and a harmonic, and up to 1 offset tone and harmonic (modtone for tones and modharm for harmonics). Then it describes the signal as: $V(\text{or } I) = \text{mag} * \cos(2 * \pi * (\text{harm} * \text{tone} + \text{modharm} * \text{modtone}) * t + \text{phase})$
transient_waveform	(Transient analysis) Voltage or power source waveform. Any one of waveforms: AM, EXP, PULSE, PWL, SFFM, SIN, PAT, or PRBS. Does <i>not</i> allow multiple transient descriptions.
ENCODE=DW8b10b	Keyword to specify 8b/10b encoding.
RD_INIT=0 1	Initial value of Running Disparity. <i>Running Disparity.1</i> is the name of the one-bit memory that recalls the bias of the last unbalanced code word. • 0: Specifies that a Running Disparity value of zero is synonymous with negative Running Disparity (-) • 1: Specifies that a Running Disparity value of one is synonymous with positive Running Disparity (+).
ENCODE=DW64b66b	Keyword to specify 64b/66b encoding. Note: DW64b66b encoding is valid only for the PAT <code>transient_waveform</code> type.
TXC=txcdata	The input 64-bit pattern signal will be divided into 8 blocks with 8 bits each; <i>txcdata</i> is an 8-bit string with only 0 and 1s. It indicates the block type (data block or control block) for each 8-bit block. • 0: specifies that the corresponding input block is a data block. • 1: specifies that the corresponding input block is a control block. For example: TXC=10000000 indicates that the first block in the input pattern signal (bits 1 to 8) is a control block, and blocks 2, 3, 4, 5, 6, 7, and 8 (that is, 9-16, 17-24, 25-32, 33-40, 41-48, 49-56, and 57-64 bits respectively) are data blocks.
ENCODE=DW128b130b DW128b132b	Keyword to specify 128b/130b or 128b/132b encoding. Note: DW128b130b and DW128b132b encoding are valid only for the PAT <code>transient_waveform</code> type.
HEAD = DATA CONTROL	Indicates whether the pattern input signal is a data signal or control signal. Default value of HEAD is DATA. • DATA: specifies that the 128-bit pattern input signal is a DATA signal. • CONTROL: specifies that the 128-bit pattern input signal is a CONTROL signal.

Parameter	Description
TRANFORHB=[0 1]	(HB analysis) 0 (default): Ignores the transient description if you specify an HB value or a DC value. If you do not provide a DC or HB value and TRANFORHB=0, then HB analysis treats the source as a DC source, and the DC source value is the time=0 value. 1: HB analysis uses the transient description if its value is VMRF, SIN, PULSE, PWL, or LFSR. If the type is a non-repeating PWL source, the simulation treats the time=infinity value as a DC analysis source value. For example, the simulation treats the following statement as a DC source with value=1 for HB analysis: <code>v1 1 0 PWL (0 0 1n 1 1u 1) + TRANFORHB=1</code> In contrast, the following statement is a 0V DC source: <code>v1 1 0 PWL (0 0 1n 1 1u 1) + TRANFORHB=0</code> The simulation treats the following statement as a periodic source with a 1u period that uses PWL values: <code>v1 1 0 PWL (0 0 1n 1 0.999u 1 1u 0) R + TRANFORHB=1</code> To override the global TRANFORHB option, explicitly set TRANFORHB for a voltage or current source.
DCOPEN	Switch for open DC connection if you have not set DC mag. 0 (default): P-element behaves as an impedance termination. 1: P-element behaves as an open circuit in DC operating point analysis. .LIN analysis mainly uses DCOPE=1 so the P-element does not affect the self-biasing device under test by opening the termination at the operating point.
RDC=val	(DC analysis) Series resistance (overrides z0).
RAC=val	(AC analysis) Series resistance (overrides z0).
RHBAC=val	(HBAC analysis) Series resistance (overrides z0).
RHB=val	(HB analysis) Series resistance (overrides z0).
RTRAN=val	(Transient analysis) Series resistance (overrides z0).
power=[0 1 2 W dbm]	Power switch. Causes treatment of the signal amplitude quantities specified with the port element to as power levels instead of voltage levels. The port element then becomes a power source, realized as a voltage source in series with the port impedance (Z0). The simulation calculates the voltage source value internally as that is necessary to realize the appropriate RMS available power. For transient analysis, this is only appropriate for DC and SIN sources. 0: (default), power entry disabled. 1: W, power given in units of Watts. 2: dBm, power given in units of dBm (dB relative to 1 mW).

Parameter	Description
<code>powersin=[0 1 2 W dbm]</code>	Sinusoidal source power switch: - When 0 (default), the element is treated as a sinusoidal voltage source. - When 1 or W, the element is treated as a power source, realized as a voltage source with a series impedance. In this case, the source value is interpreted as RMS available power in units of Watts. - When 2 or dbm, the element is treated as a voltage source with a series impedance. The source value is power ratio in the units of dBm (decibels (dB) referenced to one milliwatt (mW)). You can use this parameter for transient analysis if the power source is SIN.
<code>powerhb=[0 1 2 W dbm]</code>	Harmonic Balance power switch: - When 0 (default), the element is treated as a HB voltage source. - When 1 or W, the element is treated as a power source, realized as a voltage source with a series impedance. In this case, the source value is interpreted as RMS available power in units of Watts. - When 2 or dbm, the element is treated as a voltage source with a series impedance. The source value is power ratio in the units of dBm (decibels (dB) referenced to one milliwatt (mW)). You can use this parameter for HB analysis.
<code>powerhbac=[0 1 2 W dbm]</code>	HBAC power switch: - When 0 (default), the element is treated as a HBAC voltage source. - When 1 or W, the element is treated as a power source, realized as a voltage source with a series impedance. In this case, the source value is interpreted as RMS available power in units of Watts. - When 2 or dbm, the element is treated as a voltage source with a series impedance. The source value is power ratio in the units of dBm (decibels (dB) referenced to one milliwatt (mW)). The source value is power ratio in the units of dBm (decibels (dB) referenced to one milliwatt (mW)). You can use this parameter for HBAC analysis.
<code>powersnac=[0 1 2 W dbm]</code>	SNAC power switch: - When 0 (default), the element is treated as a SNAC voltage source. - When 1 or W, the element is treated as a power source, realized as a voltage source with a series impedance. In this case, the source value is interpreted as RMS available power in units of Watts. - When 2 or dbm, the element is treated as a voltage source with a series impedance. The source value is power ratio in the units of dBm (decibels (dB) referenced to one milliwatt (mW)). You can use this parameter for SNAC analysis.
<code>Emphasis_Level</code>	Aids in .STATEYE analysis pre-emphasis and de-emphasis
<code>Emphasis_Time</code>	Aids in .STATEYE analysis pre-emphasis and de-emphasis
<code>DFE_tap</code>	Integer number(s) to specify relative feedback tap location.
<code>DFE_coeff</code>	Real number(s) to specify corresponding feedback coefficient.
Note: If <code>DFE_tap</code> is specified, <code>DFE_coeff</code> is required.	

Parameter	Description
DFE_vth	A real number to specify logical threshold of the system. Default value is 0.
DFE_CLK	Works with <code>CLOCK_PORT</code> and <code>CLOCK_MODE=2</code> keywords in <code>.StatEye</code> command. When <code>DFE_CLK=1</code>, <code>StatEye</code> passes clock ticks at <code>CLOCK_PORT</code> to the built-in DFE. DFE model will use the given clock ticks in slicer operation. The default value of this argument is 0. See also Applying External Clock to the Built-In DFE Model in <i>PrimeSim HSPICE® User Guide: Signal Integrity Modeling and Analysis</i>.
Note: For more information on DFE function, see Built-In DFE Function for Rx Port Element in <i>StatEye Analysis</i> in <i>PrimeSim HSPICE® User Guide: Signal Integrity Modeling and Analysis</i> .	
DCD	Aids in <code>.STATEYE</code> analysis; specifies peak percentage of the duty cycle distortion (DCD). Default value is zero.
DCD_TYPE	Aids in <code>.STATEYE</code> analysis; specifies variation type. Default type for non-zero DCD is 1 (constant). 0: No DCD 1: Constant DCD. Applies fixed duty cycle For example, 40% (high) - 60% (low) 2: Uncorrelated triangular DCD variation. Assumes random DCD change between 0 and specified value with equal probability. 3: Uncorrelated sinusoidal DCD variation. Assumes the probability of the jitter amount that has sinusoidal distribution.
PJ	Aids in the <code>.STATEYE</code> analysis; specifies periodic jitter (voltage) magnitude. Default value is zero.
PJ_TYPE	Aids in <code>.STATEYE</code> analysis. Specifies an array of integer numbers for periodic jitter variation type. Default type for nonzero PJ is 3 (uncorrelated sinusoidal voltage variation): 0: No periodic jitter 1: Constant voltage shift 2: Uncorrelated triangular voltage variation 3: Uncorrelated sinusoidal voltage variation 4: Constant time shift 5: Uncorrelated triangular time variation 6: Uncorrelated sinusoidal time variation Note: - Triangular assumes random change between 0 and specified value with equal probability. - Sinusoidal assumes the probability of the jitter amount has sinusoidal distribution. - Types 4, 5, and 6 are deterministic jitters.
RJ	An array of the real numbers to specify the standard deviation of the Gaussian random jitter. The array must be in the order of the port element index. By default, the simulation does not add random jitter.

Parameter	Description
RJ_MEAN	Specifies an array of the real numbers for the mean values of Gaussian random jitter specified by RJ keyword. Default mean value for nonzero RJ is zero.
RJPP	Peak-to-peak jitter
RJPP_BER	BER for the peak-to-peak jitter specification
VNPP	Peak-to-peak jitter with voltage noise (VN)
VNPP_BER	BER for the peak-to-peak jitter specification with voltage noise (VN)
JITTER_FILE	Specifies custom jitter file name
VN	(Voltage noise); aids in .STATEYE analysis. Specify the standard deviation of the Gaussian random voltage noise. By default, the simulation does not add random voltage noise.
AMI_OBJ	Aids in .STATEYE analysis; specifies an Algorithmic Modeling Interface (AMI) shared object (typically, lib*.so for UNIX, *.dll for Windows)
AMI_PARAM	Aids in .STATEYE analysis; specifies an AMI parameter file (*.ami)
AMI_CDR	Aids in .STATEYE. Switch to use AMI_GetWave's clock data recovery (CDR) output in eye diagram generation. Default value is 0 (disabled state).
AMI_CORNER	Specifies an index number. The IBIS keyword "... (Corner val1 val2 ...)" defines the corner values in the IBIS-AMI file specified by AMI_PARAM. When you use AMI_CORNER, <i>idx</i> is used to chose the <i>idx</i>th value defined by the "... (Corner ...)" keyword. For example, if AMI_CORNER =2 is specified in a port element and the associated AMI parameter file has the following: <pre>(My_AMI_param1 ... (Corner 1 2 3)) (My_AMI_param2 ... (Corner 4 5 6)) (My_AMI_param3 ... (Corner 7 8 9))</pre> Then, 2, 5 and 8 are chosen for My_AMI_param{1,2 and 3} respectively.
AMI_Remove_DC_Offset	Removes DC_Offset from the waveform into the AMI_GetWave function. Default is set to 0. See also, Removing and Recovering DC Offset Level in AMI_GetWave Flow.
AMI_Recover_DC_Offset	Recovers removed DC_Offset to the output waveform from the AMI_GetWave function. Default is set to 1, when the AMI_Remove_DC_Offset function is activated. When the AMI_Remove_DC_Offset function is not activated, the AMI_Recover_DC_Offset keyword does not have any effect. For more information, see Removing and Recovering DC Offset Level in AMI_GetWave Flow.
PMODULATION =PAM3 PAM4	Activates pulse amplitude modulation (PAM) 4 signaling when the source has bit pattern (LFSR/PAT) input. By default, the source element uses the conventional two logic level signaling.

Parameter	Description
<code>PAM3_MAPPING=string</code>	Specifies the PAM3 modulation maps for each three consecutive bits to a pair of symbols which takes three states, low, middle and high. Therefore, the <code>PAM3_MAPPING</code> keyword takes a string argument which specifies the mapping. By default, <code>PAM3_MAPPING</code> = "LL, LM, LH, ML, MH, HL, HM, HH" which represents in the mapping Table 1 .
<code>PAM4_MAPPING=string</code>	Specifies mapping from a pair of bits to four-level symbols. <code>PAM4_Mapping</code> takes a four character string argument, a combination of '0' '1' '2' and '3' which represent pairs of binary symbols "00", "01", "10", and "11" respectively. The first, second, third, and fourth character positions in the argument string correspond to v0 (lowest), v1, v2, and v3 (highest) voltage levels, respectively. For example, <code>PAM4_MAPPING</code> = "0132" maps "00", "01", "11", and "10" to v0, v1, v2 and v3, respectively. Default value for <code>PAM4_MAPPING</code> (when <code>MODULATION=PAM4</code>) is "0132".
<code>IBIS_AMI_FILE=filenameI</code> <code>BIS_AMI_MODEL=modelname</code>	Arguments for direct IBIS (*.ibs) file call. - The PrimeSim Continuum solution retrieves the following information based on the specified pair of file and model names: - AMI object and parameter file names for the appropriate computer platform (Linux or Windows) - Parasitic information such as <code>C_comp</code> - Input source information such as source impedance, rise/fall time, and voltage range <code>AMI_CORNER=n</code> argument which currently is used to specify the corner index of the AMI parameter (*.ami) file. This corner index is applied in retrieving these quantities from the IBIS (*.ibs) file. Port element applies these parasitic and input source properties in all the PrimeSim Continuum solutions. AMI information is used in the StatEye simulations.
<code>AMI_PARAM_xx = string</code> <code> quoted string</code>	Argument for direct AMI parameter control from the PrimeSim Continuum solution netlist. - The PrimeSim Continuum solution extracts "xx" as the target name, then assigns the argument string when the name is found in the AMI parameter (*.ami) file. For example, when <code>AMI_PARAM_Ignore_Bits=1000</code> is specified, the PrimeSim Continuum solution assigns 1000 to <code>Ignore_Bits</code> in the AMI parameter file. - When the argument value is quoted using single quotation character, the PrimeSim Continuum solution first evaluates the string as an expression and assigns the result to the AMI parameter file. - You can have multiple declarations of this argument. - Since the keywords in both the PrimeSim Continuum solution netlist and the AMI parameter file are case insensitive, parameter name matching is carried out in a case-insensitive manner. However, the PrimeSim Continuum solution preserves the case of the given argument in the assignment.

Parameter	Description
IBIS_AMI_USE_CCOMP = 0 1	Specifies whether or not to use C_comp value based on the AMI_CORNER=n index argument. The IBIS_AMI_CCOMP_ORDER = 1 argument picks up C_comp in the order of n=1:typ 2:min 3:max. The IBIS_AMI_CCOMP_ORDER = 2 argument picks up C_comp in the order of n=1:typ 2:slow(max) 3:fast(min). Default is 1.
IBIS_AMI_CCOMP_ORDER = 1 2	Selects the AMI_CORNER indexing scheme for C_comp when C_comp value is selected up from *.ibs file for IBIS-AMI simulation.
IBIS_AMI_VRANGE_MODE = 1 2	Specifies how to use the VRANGE value in the *.ibs file in IBIS-AMI simulation. • 1 (default): Uses VRANGE as differential voltage. The incident source has (-VRANGE to VRANGE) voltage swing. • 2: Uses VRANGE as peak-to-peak voltage. The incident source has (-0.5*VRANGE to 0.5*VRANGE) voltage swing.
IBIS_AMI_USE_RAMP = 0 1	Specifies whether to use the RAMP information in the *.ibs file for incident's rise/fall time in IBIS-AMI simulation. Default is 1.
IBIS_AMI_USE_VRANGE = 0 1	Specifies whether to use the VRANGE information in the *.ibs file for incident's voltage swing magnitude in IBIS-AMI simulation. Default is 1.
IBIS_AMI_USE_ZSRC = 0 1	Specifies whether to use linearized pull-up/pull-down information in the *.ibs file for port element's impedance in IBIS-AMI simulation. Default is 1.

PAM3_MAPPING

Table 1 lists the PAM3_MAPPING = "LL, LM, LH, ML, MH, HL, HM, HH."

Table 1 PAM3_MAPPING

Index	0	1	2	3	4	5	6	7
3 bits (input)	000	001	010	011	100	101	110	111
2 symbols (output)	LL	LM	LH	ML	MH	HL	HM	HH

Port Element Example

The following port element specifications identify a 2-port network with 50-ohm reference impedances between the "in" and "out" nodes.

```
P1 in gnd port=1 z0=50
P2 out gnd port=2 z0=50
```

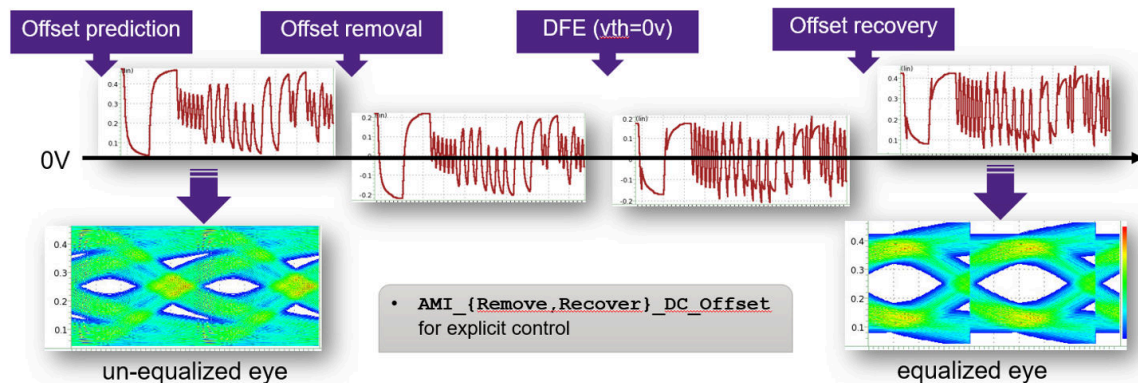
Computing scattering parameters requires z_0 reference impedance values. The order of the *port* parameters (in the P-element) determines the order of the S, Y, and Z parameters. Unlike the `.NET` command, the `.LIN` command does not require you to insert additional sources into the circuit. To calculate the requested transfer parameters, the PrimeSim Continuum solution automatically inserts these sources as needed at the port terminals. You can define an unlimited number of ports.

Removing and Recovering DC Offset Level in AMI_GetWave Flow

In equalizers that require logic detection such as Decision Feedback Equalizer (DFE), the filtering process becomes simpler if the simulator pre-removes the DC offset from the incoming waveform to the `AMI_GetWave` function and recovers it after the `AMI_GetWave` function. This is because the AMI filter can assume 0V as logical threshold. The `AMI_Remove_DC_Offset` and `AMI_Recover_DC_Offset` are the keywords for the port element.

As is shown in Figure 3, when the `AMI_Remove_DC_Offset=1` function is specified, DC offset will be removed and when the `AMI_Recover_DC_Offset=1` function, removed DC offset will be recovered.

Figure 3 Automatic DC Offset Prediction, Removal and Recovery in IBIS-AMI Filtering Flow



The amount of DC offset is automatically determined from channel's pre-characterization (initial transient) and will be displayed as a status message. Optionally, when the `DC_Offset` keyword is given in the `*.ami` file, you can manually specify the offset removal amount using the `AMI_PARAM_DC_Offset = value` keyword in port element instead of automatic determination.

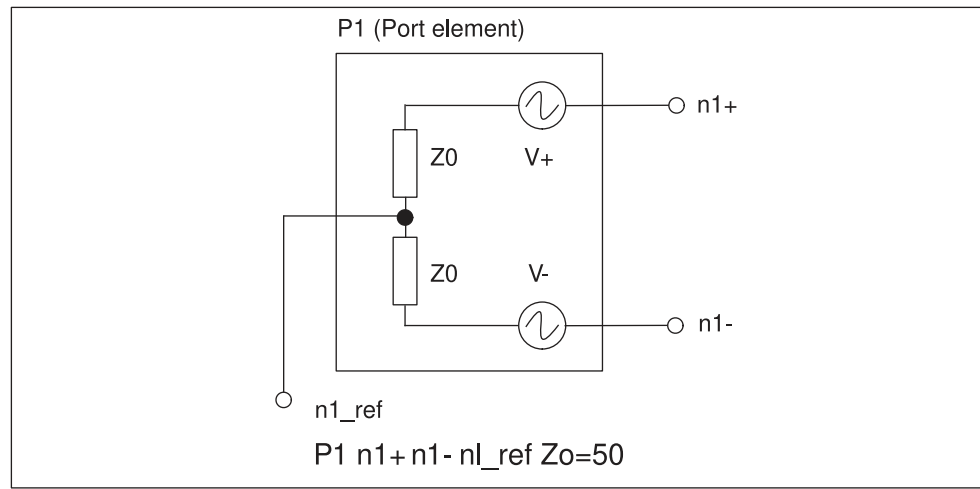
See also [AMI_PARAM_xx](#) to understand AMI parameter control in top level the PrimeSim Continuum solution netlist.

Using the Port Element for Mixed-Mode Measurement

To measure mixed mode S-parameters you can use a port element with three terminals. Except for the number of external terminals, the syntax of the port element remains the

same. The LIN analysis function internally sets the necessary drive mode (common/differential) of these mixed mode port elements. For analyses other than the LIN analysis (such as DC, AC, TRAN, and so on), the mixed-mode P-element acts as a differential driver that drives positive nodes with half of their specified voltage and the negative nodes with a negated half of the specified voltage. [Figure 4 on page 68](#) shows the block diagram of the mixed mode port element.

Figure 4 Mixed Mode Port Element



Mixed Mode Port Element Syntax

```
Pxxx n1 n2 ref port = portnumber
+ [Zo=val]
```

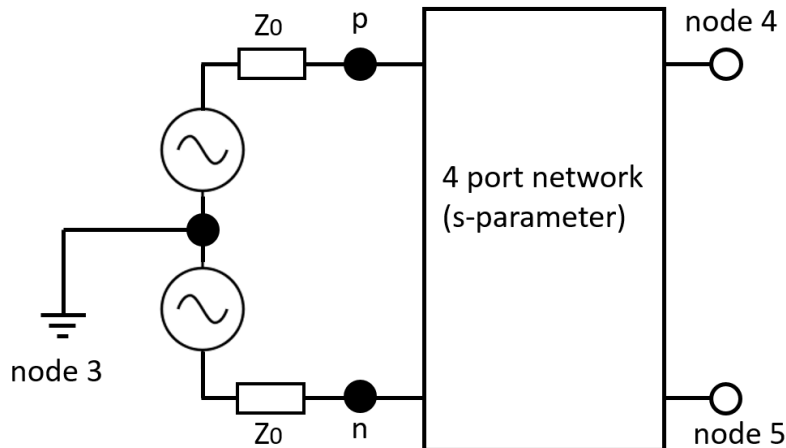
Example

For example, the following port element specifications identify mixed mode network with 50-ohm reference impedances between the differential input and output nodes:

```
P1 inp inn gnd port=1 z0=50
P2 outp outn gnd port=2 z0=50
```

Using the Five Terminal Port Element

Figure 5 Circuit Topology of a Five Terminal Port Element



The five terminal port element can be used for direct import of IBIS-AMI's "Ts4file Analog Buffer Models" (BIRD 158). When a port element is instantiated with five terminals, it represents a circuit as shown in [Figure 5](#). When the port includes an AMI model with Ts4file analog buffer models (AMI model has the Ts4file keyword in the reserved parameter section), the port element imports the specified four port S-parameter to the block in between (p, n) ports and (node 4, node 5) ports. When there is no AMI model or AMI model is specified without the Ts4file keyword, it instantiates a through connection where node p and node 4, and node n and node 5 are shorted.

See also the IBIS specification for the S-parameter port mapping and load impedance definitions.

Active Elements

The content of this section includes:

- [Diode Element](#)
- [Bipolar Junction Transistor \(BJT\) Element](#)
- [JFETs and MESFETs](#)
- [MOSFETs](#)
- [Extended MOSFET Element Support Using the .OPTION MACMOD Command](#)

Diode Element

The content of this section includes:

- [Syntax](#)
- [Examples](#)

Syntax

Non-Geometric (LEVEL=1) form:

```
Dxxx nplus nminus mname [AREA=area] [PJ=val]  
+ [OFF] [IC=vd] [M=val] [DTEMP=val]
```

Geometric (LEVEL=3) form:

```
Dxxx nplus nminus mname [AREA=area] [PJ=val]  
+ [WP=val] [LP=val] [WM=val] [LM=val] [OFF]  
+ [IC=vd] [M=val] [DTEMP=val]  
  
Dxxx nplus nminus mname [W=width] [L=length] [WP=val]  
+ [LP=val] [WM=val] [LM=val] [OFF] [IC=vd] [M=val]  
+ [DTEMP=val]
```

Fowler-Nordheim (LEVEL=2) form:

```
Dxxx nplus nminus mname [W=val] [L=val]  
+ [OFF] [IC=vd] [M=val]
```

Parameter	Description
Dxxx	Diode element name. Must begin with D, followed by up to 1023 alphanumeric and special characters as defined in Special Characters in <i>PrimeSim HSPICE® User Guide: Basic Simulation and Analysis</i> .
nplus	Positive terminal (anode) node name. The series resistor for the equivalent circuit attaches to this terminal.
nminus	Negative terminal (cathode) node name.
mname	References diode model name.
AREA	Area of the diode (unitless for LEVEL=1 diode, and square meters for LEVEL=3 diode). This affects saturation currents, capacitances, and resistances (diode model parameters are IK, IKR, JS, CJO, and RS). The SCALE option does not affect the area factor for the LEVEL=1 diode. Default=1.0. Overrides AREA from the diode model. If you do not specify the AREA, the PrimeSim Continuum solution calculates it from the width and length.
PJ	Periphery of junction (unitless for LEVEL=1 diode, and meters for LEVEL=3 diode). Overrides PJ from the diode model. If you do not specify PJ, the PrimeSim Continuum solution calculates it from the width and length specifications.

Parameter	Description
WP	Width of polysilicon capacitor, in meters (for LEVEL=3 diode only). Overrides WP in the diode model. Default=0.0.
LP	Length of polysilicon capacitor, in meters (for LEVEL=3 diode only). Overrides LP in the diode model. Default=0.0.
WM	Width of metal capacitor, in meters (for LEVEL=3 diode only). Overrides WM in the diode model. Default=0.0.
LM	Length of metal capacitor, in meters (for LEVEL=3 diode only). Overrides LM in the diode model. Default=0.0.
OFF	Sets the initial condition for this element to OFF, in DC analysis. Default=ON.
IC=vd	Initial voltage, across the diode element. Use this value when you specify the UIC option in the .TRAN statement. The .IC statement overrides this value.
M	Multiplier, to simulate multiple diodes in parallel. The M setting affects all currents, capacitances, and resistances. Default=1.
DTEMP	The difference between the element temperature and the circuit temperature, in degrees Celsius. Default=0.0.
W	Width of the diode, in meters (LEVEL=3 diode model only)
L	Length of the diode, in meters (LEVEL=3 diode model only)

You must specify two nodes and a model name. If you specify other parameters, the nodes and model name must be first and the other parameters can appear in any order.

For a listing of output templates for diode (D-element) models, see the *Diode (D-element)* table in Element Template Listings in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

Examples

Example 1

The D1 diode, with anode and cathode, connects to nodes 1 and 2. diode1 specifies the diode model.

```
D1 1 2 diode1
```

Example 2

The Dprot diode, with anode and cathode, connects to both the output node and ground, references the firstd diode model, and specifies an area of 10 (unitless for LEVEL=1 model). The initial condition has the diode OFF.

```
Dprot output gnd firstd 10 OFF
```

Example 3

The Ddrive diode, with anode and cathode, connects to the driver and output nodes. The width and length are 500 microns. This diode references the model_d diode model.

```
Ddrive driver output model_d W=5e-4 L=5e-4 IC=0.2
```

Bipolar Junction Transistor (BJT) Element

The content of this section includes:

- [Syntax](#)
- [Examples](#)

Syntax

```
Qxxx nc nb ne [ns] mname [area] [OFF]  
+ [IC=vbeval,vceval] [M=val] [DTEMP=val]  
  
Qxxx nc nb ne [ns] mname [AREA=area] [AREAB=val]  
+ [AREAC=val] [OFF] [VBE=vbeval] [VCE=vceval]  
+ [M=val] [DTEMP=val]
```

Parameter	Description
Qxxx	BJT element name. Must begin with Q, then up to 1023 alphanumeric and special characters as defined in Special Characters in <i>PrimeSim HSPICE® User Guide: Basic Simulation and Analysis</i> .
nc	Name of collector terminal node.
nb	Name of case terminal node.
ne	Name of emitter terminal node.
ns	Name of substrate terminal node, which is optional. You can also use the BULK parameter to set this name in the BJT model.
mname	BJT model name reference.
area, AREA=area	Emitter area multiplying factor, which affects currents, resistances, and capacitances. Default=1.0.
OFF	Sets initial condition for this element to OFF, in DC analysis. Default=ON.
IC=vbeval, vceval, VBE, VCE	Initial internal base-emitter voltage (vbeval) and collector-emitter voltage (vceval). The PrimeSim Continuum solution uses this value when the .TRAN statement includes UIC. The .IC statement overrides it.
M	Multiplier, to simulate multiple BJTs in parallel. The M setting affects all currents, capacitances, and resistances. Default=1.

Parameter	Description
DTEMP	The difference between the element temperature and the circuit temperature, in degrees Celsius. Default=0.0.
AREAB	Base area multiplying factor, which affects currents, resistances, and capacitances. Default=AREA.
AREAC	Collector area multiplying factor, which affects currents, resistances, and capacitances. Default=AREA.

The only required fields are the collector, base, and emitter nodes, and the model name. The nodes and model name must precede other fields in the netlist.

For a listing of output templates for BJT (Q element) models, see the *BJT (Q-element)* table in Element Template Listings in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

Examples

Example 1

In the Q1 BJT element:

```
Q1 1 2 3 model_1
```

- The collector connects to node 1.
- The base connects to node 2.
- The emitter connects to node 3.
- model_1 references the BJT model.

Example 2

In the following, Qopamp1 BJT element:

```
Qopamp1 c1 b3 e2 s Mod1stagepnp AREA=1.5 AREAB=2.5  
AREAC=3.0
```

- The collector connects to the c1 node.
- The base connects to the b3 node.
- The emitter connects to the e2 node.
- The substrate connects to the s node.
- Mod1stagepnp references the BJT model.
- The AREA area factor is 1.5.

- The `AREAB` area factor is 2.5.
- The `AREAC` area factor is 3.0.

Example 3

In the `Qdrive` BJT element:

```
Qdrive driver in output model_npn 0.1
```

- The collector connects to the `driver` node.
- The base connects to the `in` node.
- The emitter connects to the `output` node.
- `model_npn` references the BJT model.
- The area factor is 0.1.

JFETs and MESFETs

The content of this section includes:

- [Syntax](#)
- [Examples](#)

Syntax

```
Jxxx nd ng ns [nb] mname [[AREA=area | [W=val]  
+ [L=val]] [OFF] [IC=vdsval,vgsval] [M=val]  
+ [DTEMP=val]
```

```
Jxxx nd ng ns [nb] mname [[AREA=area] | [W=val]  
+ [L=val]] [OFF] [VDS=vdsval] [VGS=vgsval]  
+ [M=val] [DTEMP=val]
```

Parameter	Description
Jxxx	JFET or MESFET element name. Must begin with J, followed by up to 1023 alphanumeric and special characters as defined in Special Characters in <i>PrimeSim HSPICE® User Guide: Basic Simulation and Analysis</i> .
nd	Name of drain terminal node
ng	Name of gate terminal node
ns	Name of source terminal node
nb	Name of bulk terminal node, which is optional.

Parameter	Description
<i>mname</i>	JFET or MESFET model name reference
<i>area</i> , <i>AREA=area</i>	Area multiplying factor that affects the BETA, RD, RS, IS, CGS, and CGD model parameters. Default=1.0, in units of square meters.
<i>W</i>	FET gate width in meters
<i>L</i>	FET gate length in meters
<i>OFF</i>	Sets initial condition to OFF for this element, in DC analysis. Default=ON.
<i>IC=vdsval</i> , <i>vg sval</i> , <i>VDS</i> , <i>VGS</i>	Initial internal drain-source voltage (<i>vdsval</i>) and gate-source voltage (<i>vg sval</i>). Use this argument when the .TRAN statement contains UIC. The .IC statement overrides it.
<i>M</i>	Multiplier to simulate multiple JFETs or MESFETs in parallel. The <i>M</i> setting affects all currents, capacitances, and resistances. Default=1.
<i>DTEMP</i>	The difference between the element temperature and the circuit temperature, in degrees Celsius. Default=0.0.

The syntax only requires drain, gate, and source nodes, and model name fields. Node and model names must precede other fields.

For a listing of output templates for JFET (J-element) models, see the *JFET (J-element)* table in Element Template Listings in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

Examples

Example 1

In the J1 JFET element:

```
J1 1 2 3 model_1
```

- The drain connects to node 1.
- The source connects to node 2.
- The gate connects to node 3.
- *model_1* references the JFET model.

Example 2

In the following Jopamp1 JFET element:

```
Jopamp1 d1 g3 s2 b Mod1stage AREA=100u
```

- The drain connects to the d1 node.
- The source connects to the g3 node.
- The gate connects to the s2 node.
- Mod1stage references the JFET model.
- The area is 100 microns.

Example 3

In the Jdrive JFET element:

```
Jdrive driver in output model_jfet W=10u L=10u
```

- The drain connects to the driver node.
- The source connects to the in node.
- The gate connects to the output node.
- model_jfet references the JFET model.
- The width is 10 microns.
- The length is 10 microns.

MOSFETs

The content of this section includes:

- [Syntax](#)
- [Examples](#)

Syntax

```
Mxxx nd ng ns [nb] mname [[L=length] [[W=width]
+ [AD=val] AS=val] [PD=val] [PS=val]
+ [NRD=val] [NRS=val] [RDC=val] [RSC=val] [OFF]
+ [IC=vds,vgs,vbs] [M=val] [DTEMP=val]
+ [GEO=val] [DELVTO=val]
.OPTION WL
```

```
Mxxx nd ng ns [nb] mname [width] [length] [other_options...]
```

Parameter	Description
Mxxx	MOSFET element name. Must begin with M, followed by up to 1023 alphanumeric and special characters as defined in Special Characters in <i>PrimeSim HSPICE® User Guide: Basic Simulation and Analysis</i> .
nd	Name of drain terminal node.
ng	Name of gate terminal node.
ns	Name of source terminal node.
nb	Name of bulk terminal node, which is optional.
mname	MOSFET model name reference or subckt name if you set .OPTION MACMOD.
L	MOSFET channel length, in meters. This parameter overrides .OPTION DEFL, with a maximum value of 0.1m. Default=DEFL.
W	MOSFET channel width, in meters. This parameter overrides .OPTION DEFW. Default=DEFW.
AD	Drain diffusion area. Overrides .OPTION DEFAD. Default=DEFAD, if you set the ACM=0 model parameter.
AS	Source diffusion area. Overrides .OPTION DEFAS. Default=DEFAS, if you set the ACM=0 model parameter.
PD	Perimeter of drain junction, including channel edge. Overrides .OPTION DEFDPD. Default=DEFAD, if you set the ACM=0 or 1 model parameter. Default=0.0, if you set ACM=2 or 3.
PS	Perimeter of source junction, including channel edge. Overrides .OPTION DEFPS. Default=DEFAS, if you set the ACM=0 or 1 model parameter. Default=0.0, if you set ACM=2 or 3.
NRD	NRD (Number of squares of drain diffusion for resistance calculations) overrides .OPTION DEFNRD. For non-CMI models such as BSIM3 etc... Default=DEFNRD, if you set ACM=0 or 1 model parameter. Default=0.0, if you set ACM=2 or 3. For CMI models such as BSIM4 etc... Default=1.0
NRS	NRS (Number of squares of source diffusion for resistance calculations) overrides .OPTION DEFNRS. For non-CMI models such as BSIM3 etc... Default=DEFNRS, if you set ACM=0 or 1 model parameter. Default=0.0, if you set ACM=2 or 3. For CMI models such as BSIM4 etc... Default=1.0
RDC	Additional drain resistance due to contact resistance, in units of ohms. This value overrides the RDC setting in the MOSFET model specification. Default=0.0.
RSC	Additional source resistance due to contact resistance, in units of ohms. This value overrides the RSC setting in the MOSFET model specification. Default=0.0.
OFF	Sets initial condition for this element to OFF, in DC analysis. Default=ON. This command does not work for depletion devices.

Parameter	Description
IC= <i>vds</i> , <i>vgs</i> , <i>vbs</i>	Initial voltage across external drain and source (<i>vds</i>), gate and source (<i>vgs</i>), and bulk and source terminals (<i>vbs</i>). Use these arguments with <code>.TRAN UIC</code> . <code>.IC</code> statements override these values.
M	Multiplier, to simulate multiple MOSFETs in parallel. Affects all channel widths, diode leakages, capacitances, and resistances. Default=1.
DTEMP	The difference between the element temperature and the circuit temperature, in degrees Celsius. Default=0.0.
GEO	Source/drain sharing selector for a MOSFET model parameter value of ACM=3. Default=0.0.
DELVTO	Zero-bias threshold voltage shift. Default=0.0.

The only required fields are the drain, gate, and source nodes, and the model name. The nodes and model name must precede other fields in the netlist. If you did not specify a label, use the second syntax with the `.OPTION WL` statement, to exchange the width and length options.

For a full listing of output templates for MOSFET models, see MOSFET Output Templates in the *PrimeSim™ Continuum® Reference Manual: MOSFET Models*.

Examples

In the following `M1` MOSFET element:

```
M1 1 2 3 model_1
```

- The drain connects to node 1.
- The gate connects to node 2.
- The source connects to node 3.
- `model_1` references the MOSFET model.

In the following `Mopamp1` MOSFET element:

```
Mopamp1 d1 g3 s2 b Mod1stage L=2u W=10u
```

- The drain connects to the `d1` node.
- The gate connects to the `g3` node.
- The source connects to the `s2` node.
- `Mod1stage` references the MOSFET model.

- The length of the gate is 2 microns.
- The width of the gate is 10 microns.

In the following `Mdrive` MOSFET element:

```
Mdrive driver in output bsim3v3 W=3u L=0.25u DTEMP=4.0
```

- The drain connects to the `driver` node.
- The gate connects to the `in` node.
- The source connects to the `output` node.
- `bsim3v3` references the MOSFET model.
- The length of the gate is 3 microns.
- The width of the gate is 0.25 microns.
- The device temperature is 4°Celsius higher than the circuit temperature.

Extended MOSFET Element Support Using the .OPTION MACMOD Command

Use option `MACMOD` to enable the PrimeSim Continuum solution to access a subcircuit definition for MOSFETs, diodes, and BJTs, when no model reference exists. `MACMOD=1` treats subcircuits as primitive models with an “M” instantiation. `.OPTION MACMOD` syntax is:

```
.OPTION MACMOD= [1|2|3|0]
```

When `macmod=1`, the PrimeSim Continuum solution seeks a subcircuit definition for the `M/Q/D***` element if no model reference exists. The desired subcircuit name must match (case insensitive) the `mname` field in the `M/Q/D***` instance statement. In addition, the number of terminals of the subcircuit must match with the `M/Q/D***` element which references it; otherwise the PrimeSim Continuum solution aborts the simulation based on no definition for the `M/Q/D***` element.

The following limitations apply when `macmod=1`:

1. Element template output does not support elements which use subcircuit definitions.
2. This feature does not support an element if a string parameter defines the `mname`.
3. The number of terminals for the PrimeSim Continuum solution element must be within the range of 3-7; any number of terminals that is out of this range causes the simulation to fail.

When `macmod=2`, the PrimeSim Continuum solution seeks a MOSFET/BJT/Diode model definition when it cannot find matching subcircuit or Verilog-A definition for an X-element. The targeted MODEL card could be either the PrimeSim Continuum solution built-in model or CMI model. If the model card that matched with the X-element reference name is not a type of MOSFET/BJT/Diode model, the simulator errors out with message of reference "not found."

The following limitations apply when `macmod=2`:

1. The feature of "string parameter supported in model name" does not apply to X-elements that are mapped to the model cards; that is, reference name of the X-element must be constant string characters.
2. Subcircuit direct port probing command, `isub()` does not support X-elements mapped to the model cards.
3. The PrimeSim Continuum solution MOSRA analysis does not work on the X-elements, even when they directly map to compact model cards.

When `macmod=3`, the PrimeSim Continuum solution enables both of the above features; the PrimeSim Continuum solution seeks a `.subckt` definition for an M/Q/D-element if there is no matching model reference; the PrimeSim Continuum solution seeks a `.model` definition for an X-element if there is no matching `.subckt` or Verilog-A definition. Usage considerations and limitations remain the same for both features, respectively.

When `.OPTION TMIFLAG ≥ 1`, `.OPTION MACMOD` automatically equals 3.

The case of no `.OPTION MACMOD` in the input files or `MACMOD=0` disables the above two features; the PrimeSim Continuum solution ignores the `MACMOD` option if you set any value other than 1|2|3|0. The `MACMOD` option is a global option; if there are multiple `MACMOD` options in one simulation, the PrimeSim Continuum solution uses the value of the last `MACMOD` option.

When `.option macmod=2` or `.option macmod=3`, for `isub(x*.n)`, if `x*` is a MOSFET instance and `n` is 1, 2, 3, 4, d, g, s, or b (and not a valid subcircuit port number), `isub(x*.n)` is equivalent to `i_number(x*)`, as in the following examples:

- `isub(*.1)/isub(*.d) => i1(*)`
- `isub(*.2)/isub(*.g) => i2(*)`
- `isub(*.3)/isub(*.s) => i3(*)`
- `isub(*.4)/isub(*.b) => i4(*)`

Example 1

```
**
.option MACMOD=1
M1 net1 net2 net3 net4 nch l=0.2u w=0.2u p1=1
.model nch nmos level=49 ...
```



```
.subckt nch d g s b w=1 l=1 p1=gp1
.if (p1 > 0)
Mnch d g s b model_1 w=w l=1
.else
Mnch d g s b model_2 w=w l=1
.endif
.ends
```

In Example 1, extended MOSFET is on. However, because the `mname` in a `.MODEL` statement matches the `mname` of the `M1` element, element `M1` uses model `nch` rather than the subcircuit definition. The simulation ignores the extra instance parameter `p1`.

Example 2

```
**
.option MACMOD
.param gp1=1 gp2=2 gp3=3
M1 net1 net2 net3 net4 nch l=0.2u w=0.2u p1=gp1 p2=gp2 p3=gp3
.subckt nch d g s b w=1 l=1 p1=gp1 p2=gp2 p3=gp3
.if (p1 > 0 && p2==1 && p3 ==1)
Mnch d g s b model_1 w=w l=1
.else if ( p1 == 0 && p2 ==1 && p3 ==1)
Mnch d g s b model_2 w=w l=1
.else
Mnch d g s b model_3 w=w l=1
.endif
.ends
.model model_1 nmos level=49 ...
.model model_2 nmos level=53 ...
.model model_3 nmos level=54 ...
```

In Example 2, extended MOSFET element support is on; since there is no matching `.MODEL` statement, `M1` uses subcircuit definition `nch`; after evaluation, `M1` results in a MOSFET element that species MOSFET model `model_3`.

Example 3

```
**
.option MACMOD
.param gp1=1 gp2=2 gp3=3
M1 net1 net2 net3 net4 nch l=0.2u w=0.2u p1=gp1 p2=gp2 p3=gp3
.subckt nch d g s b w=1 l=1 p1=gp1 p2=gp2 p3=gp3
.if (p1 > 0 && p2==1 && p3 ==1)
Mnch d g s b model_1 w=w l=1
.else if ( p1 == 0 && p2 ==1 && p3 ==1)
Mnch d g s b model_2 w=w l=1
.else
Mnch d g s b model_3 w=w l=1
.endif
C1 g 0 1p
.ends
```

Example 3 shows extended MOSFET element support turned on. Instance `M1` uses macro model `nch`, which is a subcircuit definition that consists of one MOSFET device and one capacitor.

MACMOD Option Limitations

- The number of terminals for `M***` must be within the range of 3 to 7. A number of terminals outside of that range cause the simulation to fail.
- A string parameter must not define a MOSFET element's `mname`.
- The MACMOD option only applies to the PrimeSim Continuum solution MOSFET elements.
- Element template output does not support MOSFET elements which use subcircuit definitions.

For example, if Example 3 includes the output command:

```
.PRINT LX8 (M1) LV9 (M1)
```

—then the PrimeSim Continuum solution ignores the above output command. Because `M1` is using a subcircuit definition, it is no longer the PrimeSim Continuum solution primitive MOSFET device.

- The desired subcircuit name must match the `mname` field in the `M***` instance statement. The match of subcircuit name and the `mname` field is case insensitive. The `.MODEL` definition takes precedence over a subcircuit definition even when `.OPTION MACMOD` is on.

Direct X-Element Mapping to a MOSFET Model Card

The PrimeSim Continuum solution seeks a MOSFET model definition when it cannot find a matching `.subckt` or Verilog-A definition for an X-element. This applies mainly to a custom model, which can only become enabled by the `.option CMIFLAG` or `.option TMIFLAG`; Subcircuit or Verilog-A module definitions always take preference over the CMI model card. The PrimeSim Continuum solution considers the X-elements that are mapped to model cards as MOSFET devices with certain. The following sections describe these usage considerations and limitations. For information about the PrimeSim Continuum solution CMI and TMI, contact the Synopsys support team.

Considerations

Syntax check rules of MOSFET devices apply to the X-elements directly mapping to MOSFET model cards, such as:

- The valid instance parameter list of that X-element becomes the valid instance parameter list of the particular MOSFET model in the mapping model card; the PrimeSim Continuum solution ignores invalid instance parameter of that X-element;

default values of the mapping MOSFET model apply to any undefined instance parameter by comparing with the corresponding MOSFET model.

- The `.PARAM` netlist commands cannot override MOSFET instance parameters which are device native parameters; such instance parameter rules apply to X-elements with direct mapping to MOSFET model cards, i.e., instance parameters of such an X-element become native device parameters, thus netlist `.PARAM` commands cannot be override them.
- The number of terminals of the X-element must be in the valid range of its mapping MOSFET model; or simulator exits with an error message.

Limitations

- The feature of “string parameter supported on MOSFET model name” does not apply to X-elements that are mapped to MOSFET model cards. In other words, you must write a reference name of the X-element as constant string characters.
- The subcircuit direct port probing command, `Isub()` does not support X-elements mapped to MOSFET model cards.

TMI Dummy Device

The general syntax of a TMI dummy device is:

```
Nxxx node_1 node_2 ... node_N mname
+ inst_param_1=value_1...[inst_param_M=value_M]
+ keyword_1=value_1 ... [keyword_M=value_M]
```

Parameter	Description
Nxxx	TMI Dummy Device name. Must begin with N, followed by up to 1023 alphanumeric and special characters as defined in Special Characters in <i>PrimeSim HSPICE® User Guide: Basic Simulation and Analysis</i> .
node_1 node_2 ... node_N	Terminal node name.
mname	Model name
inst_param_1... inst_param_M	Name of the instance parameter.

2

Independent Sources and Stimuli

Describes element statements for independent sources, power sources, and digital vector sources.

This chapter also explains each type of element and model statement and provides explicit formulas and examples to show how various combinations of parameters affect the simulation.

The PrimeSim Continuum solution ships over a dozen source examples for your use; see Listing of Demonstration Input Files for paths to demo files and Sources Examples, in *PrimeSim HSPICE® User Guide: Demonstration Netlists*.

The following sections discuss these topics:

- [Independent Source Elements](#)
- [Transient Source Functions](#)
- [Mixed Sources](#)
- [Power Sources](#)
- [PrimeSim Continuum Advanced Analog Analyses Sources and Stimuli](#)
- [Specifying a Digital Vector File and Mixed Mode Stimuli](#)
- [Using the VCD Control File](#)

Independent Source Elements

Use independent source element (current, I-element, or voltage, V-element) statements to specify DC, AC, transient, and mixed independent voltage and current sources. These elements are also the associated analysis sources depending on the analysis you perform.

The following sections discuss these topics:

- [Source Element Conventions](#)
- [Independent Source Element Syntax](#)

- [Independent Source Element Examples](#)
- [DC Sources](#)
- [AC Sources](#)
- [Transient Sources](#)

Source Element Conventions

You do not need to ground voltage sources. The PrimeSim Continuum solution assumes that positive current flows from the positive node, through the source, to the negative node. A positive current source forces current to flow out of the n+ node, through the source, and into the n- node.

You can use parameters as values in independent sources. Do not use any of the following reserved keywords to identify these parameters: AC, ACI, AM, DC, EXP, PAT, PE, PL, PU, PULSE, PWL, R, RD, SFFM, POWER, or SIN.

Independent Source Element Syntax

```
Vxxx n+ n- [[DC=] dcval tranfun [AC=acmag acphase]]  
+ [POWER=[0|1|2|w|dbm]]  
+ [POWERSIN=[0|1|2|W|dbm]]  
+ [POWERHB=[0|1|2|W|dbm]]  
+ [POWERHBAC=[0|1|2|W|dbm]]  
+ [POWERSNAC=[0|1|2|W|dbm]]
```

```
Ixxx n+ n- [[DC=] dcval tranfun [AC=acmag acphase]]  
+ [POWER=[0|1|2|w|dbm] [M=val]]
```

Parameter	Description
Vxxx	Independent voltage-source element name. Must begin with V, followed by up to 1023 alphanumeric and special characters.
Ixxx	Independent current source element name. Must begin with I, followed by up to 1023 alphanumeric and special characters.
n+	Positive node.
n-	Negative node.
DC=dcval	DC source keyword and value in volts. Used for the operating point calculation for all simulations except transient. Transient analysis calculates an additional operating point with the tranfun value at time zero. Default=0.0.
tranfun	Transient source function (one or more of: AM, DC, EXP, PAT, PE, PL, PU, PULSE, PWL, SFFM, SIN). The functions specify the characteristics of a time-varying source. See the individual functions for syntax.

Parameter	Description
AC	AC source keyword for use in AC small-signal analysis.
<i>acmag</i>	Magnitude (RMS) of the AC source, in volts.
<i>acphase</i>	Phase of the AC source, in degrees. Default = 0.0.
M	Multiplier, to simulate multiple parallel current sources. The PrimeSim Continuum solution multiplies source current by M. Default = 1.0.
<i>power=[0 1 2 W dbm]</i>	Power switch: - When 0 (default), the element is treated as a voltage or current source. - When 1 or W, the element is treated as a power source, realized as a voltage source with a series impedance or a current source with a parallel impedance. In this case, the source value is interpreted as RMS available power in units of Watts. - When 2 or dbm, the element is treated as a voltage source with a series impedance or a current source with a parallel impedance. Values are in dbms. You can use this parameter for transient analysis if the power source is either DC or SIN.
<i>powersin=[0 1 2 W dbm]</i>	Sinusoidal source power switch: - When 0 (default), the element is treated as a sinusoidal voltage source. - When 1 or W, the element is treated as a power source, realized as a voltage source with a series impedance. In this case, the source value is interpreted as RMS available power in units of Watts. - When 2 or dbm, the element is treated as a voltage source with a series impedance. The source value is power ratio in the units of dBm (decibels (dB) referenced to one milliwatt (mW)). You can use this parameter for transient analysis if the power source is SIN.
<i>powerhb=[0 1 2 W dbm]</i>	Harmonic Balance power switch: - When 0 (default), the element is treated as a HB voltage source. - When 1 or W, the element is treated as a power source, realized as a voltage source with a series impedance. In this case, the source value is interpreted as RMS available power in units of Watts. - When 2 or dbm, the element is treated as a voltage source with a series impedance. The source value is power ratio in the units of dBm (decibels (dB) referenced to one milliwatt (mW)). You can use this parameter for HB analysis.
<i>powerhb=[0 1 2 W dbm]</i>	HBAC power switch: - When 0 (default), the element is treated as a HBAC voltage source. - When 1 or W, the element is treated as a power source, realized as a voltage source with a series impedance. In this case, the source value is interpreted as RMS available power in units of Watts. - When 2 or dbm, the element is treated as a voltage source with a series impedance. The source value is power ratio in the units of dBm (decibels (dB) referenced to one milliwatt (mW)). The source value is power ratio in the units of dBm (decibels (dB) referenced to one milliwatt (mW)). You can use this parameter for HBAC analysis.

Parameter	Description
<code>powersnac=[0 1 2 W dbm]</code>	SNAC power switch: • When 0 (default), the element is treated as a SNAC voltage source. • When 1 or W, the element is treated as a power source, realized as a voltage source with a series impedance. In this case, the source value is interpreted as RMS available power in units of Watts. • When 2 or dbm, the element is treated as a voltage source with a series impedance. The source value is power ratio in the units of dBm (decibels (dB) referenced to one milliwatt (mW)). You can use this parameter for SNAC analysis.

For a listing of output templates for independent voltage source (V-element) models (PrimeSim HSPICE only), see the *Independent Voltage Source (V-element)* table in Element Template Listings in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

For a listing of output templates for independent current source (I-element) models (PrimeSim HSPICE only), see the *Independent Current Source (I-element)* table in Element Template Listings in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

Independent Source Element Examples

Example 1

`VX 1 0 5V`

where:

- The `VX` voltage source has a 5-V DC bias.
- The positive terminal connects to node 1.
- The negative terminal is the ground.

Example 2

`VB 2 0 DC=VCC`

where:

- The `VCC` parameter specifies the DC bias for the `VB` voltage source.
- The positive terminal connects to node 2.
- The negative terminal is the ground.

Example 3

```
VH 3 6 DC=2 AC=1,90
```

where:

- The `VH` voltage source has a 2-V DC bias, and a 1-V RMS AC bias, with 90-deg phase offset.
- The positive terminal connects to node 3.
- The negative terminal connects to node 6.

Example 4

```
VCC in out DC=0 PWL 0 0 10NS VCC 15NS VCC 20NS 0
```

where:

- The `DC=0` keyword specifies the initial DC bias value for the `VCC` voltage source.
- The piecewise-linear relationship defines the time-varying response for the `VCC` voltage source, which is 0 V at time 0, `VCC` from 10 to 15 ns, and back to 0 volts at 20 nanoseconds.
- The positive terminal connects to the `in` node.
- The negative terminal connects to the `out` node.
- The PrimeSim Continuum solution determines the operating point using the specified DC bias value. If no initial DC bias value is specified, the PrimeSim Continuum solution determines the operating point using the initial transient source value and outputs the following informational message: `DC voltage reset to initial transient source value.`

Example 5

```
Vin1 in 0 DC=0 PULSE(-1 1 10n 1n 1n 20n 40n)
```

where:

- The `DC=0` specifies the DC bias for the `Vin1` voltage source.
- The positive terminal connects to the `in` node.
- The negative terminal connects to the ground (`0`) node.
- The output high voltage is 1 V.
- The output low voltage is -1 V.
- The delay is 10 ns.

- The rise and fall times are each 1 ns.
- The high pulse width is 20 ns.
- The period is 40 ns.
- The PrimeSim Continuum solution determines the operating point using the specified DC bias value. If no initial DC bias value is specified, the PrimeSim Continuum solution determines the operating point using the initial transient source value and output the following informational message: DC voltage reset to initial transient source value.

Example 6

```
VIN 13 2 0.001 AC 1 SIN (0 1 1MEG)
```

where:

- The `VIN` voltage source has a 0.001-volt DC bias, and a 1-volt RMS `AC` bias.
- The sinusoidal time-varying response ranges from 0 to 1 volts, with a frequency of 1 megahertz.
- The positive terminal connects to node 13.
- The negative terminal connects to node 2.

Example 7

```
VMEAS 12 9
```

where:

- The `VMEAS` voltage source has a 0-volt DC bias.
- The positive terminal connects to node 12.
- The negative terminal connects to node 9.

Example 8

```
IG 8 7 PL(1MA OS 5MA 25MS)
```

where:

- The piecewise-linear relationship defines the time-varying response for the `IG` current source, which is 1 MA at time=0, and 5 MA at 25 ms.
- The positive terminal connects to node 8.
- The negative terminal connects to node 7.

Example 9

```
ISRC 23 21 AC 0.333 45.0 SFFM (0 1 10K 5 1K)
```

where:

- The `ISRC` current source has a 1/3-amp RMS `AC` response, with a 45-degree phase offset.
- The frequency-modulated, time-varying response ranges from 0 to 1 volts, with a carrier frequency of 10 kHz, a signal frequency of 1 kHz, and a modulation index of 5.
- The positive terminal connects to node 23.
- The negative terminal connects to node 21.

DC Sources

The following sections discuss these topics:

- [Syntax](#)
- [Examples](#)

Syntax

```
Vxxx n+ n- [DC=] dcval
```

```
Ixxx n+ n- [DC=] dcval [M=val]
```

Examples

Example 1: DC voltage source

```
V1 1 0 DC=5V  
V1 1 0 5V
```

These examples specify a DC voltage source of 5 V, connected between node 1 and ground.

Example 2: DC current source

```
I1 1 0 DC=5mA  
I1 1 0 5mA
```

These examples specify a 5 mA DC current source between node 1 and ground. The direction of current in both sources is from node 1 to ground.

AC Sources

The following sections discuss these topics:

- [Syntax](#)
- [Examples](#)

Syntax

```
Vxxx n+ n- AC=acmag acphase
```

```
Ixxx n+ n- AC=acmag acphase [M=val]
```

AC analysis uses AC current and voltage sources as impulse functions. To specify the magnitude and phase of the impulse, use the `AC` keyword.

Examples

```
V1 1 0 AC=10V,90  
VIN 1 0 AC 10V 90
```

The preceding two examples specify an AC voltage source, with a magnitude of 10 V and a phase of 90 degrees. To specify the frequency sweep range of the AC analysis, use the `.AC` analysis statement.

Transient Sources

The zero time value of the transient source of a calculated transient operating point overrides the value of the DC source.

For transient analysis, you can specify the source as a function of time. The following functions are available:

- Trapezoidal pulse (`PULSE` function)
- Sinusoidal (`SIN` function)
- Exponential (`EXP` function)
- Piecewise linear (`PWL` function)
- Single-frequency FM (`SFFM` function)
- Single-frequency AM (`AM` function)
- Pattern (`PAT` function)
- Pseudo Random-Bit Generator Source (`PRBS` function)

For detailed information, see [Transient Source Functions](#).

Transient Source Functions

The PrimeSim Continuum solution also provides a data-driven version of PWL (not supported in the PrimeSim Continuum solution advanced analog analyses). If you use the data-driven PWL , you can reuse the results of an experiment or of a previous simulation, as one or more input sources for a transient simulation. See Sources Examples in *PrimeSim HSPICE® User Guide: Demonstration Netlists*, for syntax of a data-driven sweep.

Transient source functions allow you to specify several useful analog and digital test vectors for steady-state, time-domain analysis. You can specify both current and voltage waveforms, as exponential, sinusoidal, piecewise linear (PWL), AM, or single-sided FM functions. You can also specify pattern and pseudo random bit streams for use in signal integrity applications.

The PrimeSim Continuum solution offers the following transient source functions:

- [Trapezoidal Pulse Source](#)
- [Sinusoidal Source Function](#)
- [Exponential Source Function](#)
- [PWL Source](#)
- [High Impedance State PWL \(PWLZ\)](#)
- [Single-frequency FM Source](#)
- [Single-frequency AM Source](#)
- [Pattern Source](#)
- [Pseudo Random Bit Generator Source](#)

Trapezoidal Pulse Source

The PrimeSim Continuum solution provides a trapezoidal pulse source function which starts with an initial delay from the beginning of the transient simulation interval to an onset ramp. During the onset ramp, the voltage or current changes linearly from its initial value to the pulse plateau value. After the pulse plateau, the voltage or current moves linearly along a recovery ramp back to its initial value. The entire pulse repeats, with a period named *per*, from onset to onset.

The following sections discuss these topics:

- [Syntax](#)
- [Effect of Jitter on the PULSE Source](#)
- [Examples](#)

Syntax

```
Vxxx n+ n- PU[LSE] [(v1 v2 [td [tr [tf [pw [per]]]])] []]  
+ [RMSJITTER=val PERJITTER=val [SEED=val]]  
+ [EDGETYPE=halfsine]  
  
Ixxx n+ n- PU[LSE] [(v1 v2 [td [tr [tf [pw [per]]]])] []]  
+ [RMSJITTER=val PERJITTER=val [SEED=val]]  
+ [EDGETYPE=halfsine]
```

Parameter	Description
Vxxx, Ixxx	Independent voltage/ current source, which exhibits the pulse response.
PULSE	Keyword for a pulsed time-varying source. The short form is PU.
v1	Initial value of voltage or current before the pulse onset (units: volts/amps).
v2	Pulse plateau value (units of volts or amps).
td	Delay (propagation) time in seconds from the beginning of the transient interval to the first onset ramp. Default=0.0
tr	Duration of the onset ramp (in seconds) from the initial value to the pulse plateau value (reverse transit time). Default = TSTEP
tf	Duration of the recovery ramp (in seconds) from the pulse plateau back to the initial value (forward transit time). Default TSTEP
pw	Pulse width (the width of the plateau portion of the pulse), in seconds. Default =TSTOP.
per	Pulse repetition period, in seconds. Default = TSTEP
rmsjitter	RMS value for random phase jitter, in seconds.
perjitter	RMS value for period jitter, adjusts the magnitude of the random time.
seed	Used to generate random number sequences with different seed value. The default value is 1.
EDGETYPE	Type of the rising and falling edges of the waveform. Supported types are: LINEAR (default) or HALFSINE. The edges rise and fall by linear or by half-sinusoid, respectively.

Table 2 *Time-Value Relationship for a PULSE Source*

Time	Value
0	v1
td	v1
td + tr	v2
td + tr + pw	v2
td + tr + pw + tf	v1
tstop	v1

Linear interpolation determines the intermediate points.

Note:

TSTEP is the printing increment, and TSTOP is the final time.

Effect of Jitter on the PULSE Source

The effect of jitter on the PULSE source results in random shifts of the rise and fall transitions that normally take place at:

RISE edge: $td + n \cdot T_0 \leq t \leq td + n \cdot T_0$

FALL edge: $td + pw + n \cdot T_0 \leq t \leq td + tr + pw + tf + n \cdot T_0$

The jitter effect is equivalent to introducing random shifts in the period OT consistent with the 1st order jitter model based on Period Jitter, also according to the expression for period variations $T_0 \rightarrow T_j = T_0 + \Delta T(t)$. The specified syntax determines the first period of the PULSE source, yet all subsequent pulse periods are random according to $T_0 + \Delta T(t)$.

The PULSE source with jitter still maintains constant rise and fall times. This creates some uncertainty in how the pulse width (pw) varies as the period varies due to jitter.

The PrimeSim Continuum solution advanced analog analyses uses a special calculation that holds the rise and fall times (tr and tf) constant, and also holds the 50% Duty Cycle constant. The halfway points on the rise and fall times determines the 50 percent duty cycle definition. This results in the pulse width (pw) change due to jitter variations as:

Equation 5

$$pw_j = pw \cdot \left(\frac{T_j}{T_0} \right) + \frac{1}{2} \cdot (tr + tf) \cdot \left(\frac{T_j}{T_0} - 1 \right)$$

Also, this sets the minimum period for a PULSE source with jitter to be $tr + tf$, resulting in the extreme case of a sawtooth waveform.

A Gaussian random number generator computes the random $\Delta T(t)$ variations after each leading edge of the clock sources. For flexibility, the `SEED` parameter (integer) generates different random number sequences when you specify different `SEED` integers for initialization.

Examples

Example 1

The following example shows the pulse source connected between node 3 and node 0. In the pulse:

- The output high voltage is 1 V.
- The output low voltage is -1 V.
- The delay is 2 ns.
- The rise and fall time are each 2 ns.
- The high pulse width is 50 ns.
- The period is 100 ns.
- The RMS value for period jitter is 10 ns.
- The seed is 1.

```
VIN 3 0 PULSE (-1 1 2NS 2NS 2NS 50NS 100NS) perjitter=10ns seed=1
```

Example 2

The following example is a pulse source which connects between node 99 and node 0. The syntax shows parameter values for all specifications:

```
V1 99 0 PU lv hv tdlay tris tfall tpw tper
```

Example 3

The following example shows an entire netlist which contains a PULSE voltage source. In the source:

- The initial voltage is 1 V.
- The pulse voltage is 2 V.
- The delay time, rise time, and fall time are each 5 ns.

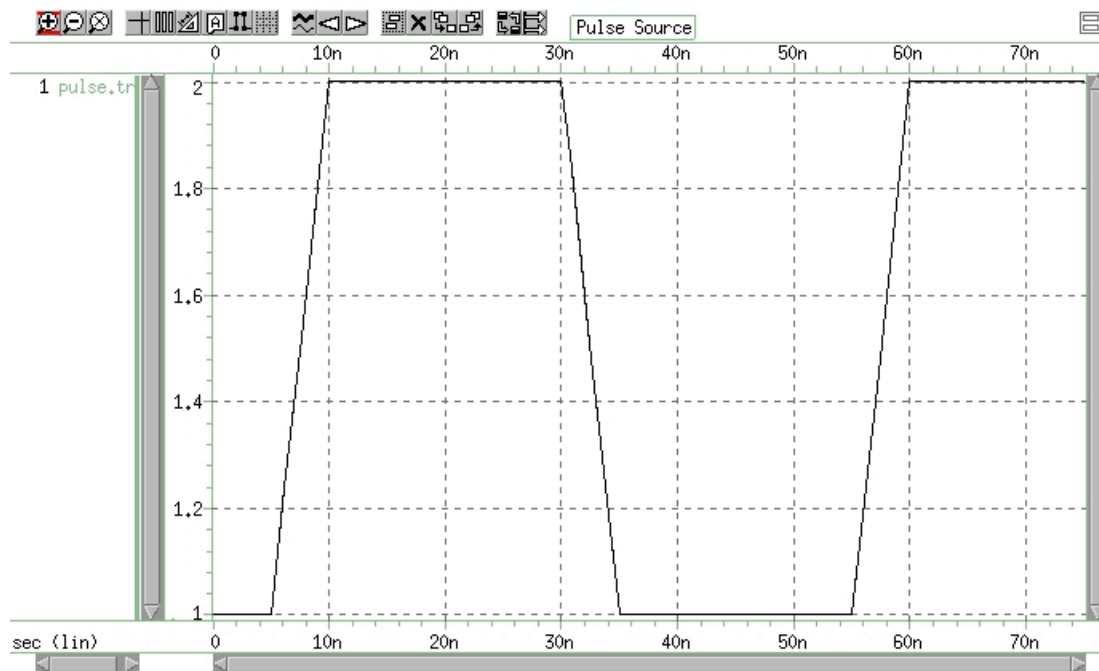
- The pulse width is 20 ns.
- The pulse period is 50 ns.

This example uses demonstration netlist `pulse.sp`, which is available in directory `$installdir/demo/hspice/sources`:

```
file pulse.sp test of pulse
.option post
.tran .5ns 75ns
vpulse 1 0 pulse( v1 v2 td tr tf pw per )
r1 1 0 1
.param v1=1v v2=2v td=5ns tr=5ns tf=5ns pw=20ns per=50ns
.end
```

Figure 6 shows the result of simulating this netlist in the PrimeSim Continuum solution.

Figure 6 Pulse Source Function



Sinusoidal Source Function

The PrimeSim Continuum solution provides a sinusoidal source function that can be either a constant sinusoid or a damped sinusoidal source function that produces a dying exponential with a sine wave:

The following sections discuss these topics:

- [Syntax](#)
- [Example](#)
- [Repeat Option in Sinusoidal Sources](#)

Syntax

```
Vxxx n+ n- SIN [ ( ) vo va [freq [td [q [j]]]] ] [ ]
+ [[RMSJITTER=val PERJITTER=val [SEED=val]]]
```

```
Vxxx n+ n- SIN ( vo va [freq [td [q [j]]]] )
+ [FMMDI=val] [FMFQ=val] [AMMDI=val]
+ [AMFQ=val] [AMOFF=val] [AMPHA=val]
```

```
Ixxx n+ n- SIN [ ( ) vo va [freq [td [q [j]]]] ] [ ]
+ [[RMSJITTER=val PERJITTER=val [SEED=val]]]
```

```
Ixxx n+ n- SIN ( vo va [freq [td [q [j]]]] )
+ [FMMDI=val] [FMFQ=val] [AMMDI=val]
+ [AMFQ=val] [AMOFF=val] [AMPHA=val]
```

Parameter	Description
Vxxx, Ixxx	Independent voltage source that exhibits the sinusoidal response.
SIN	Keyword for a sinusoidal time-varying source.
vo	Voltage or current offset in volts or amps.
va	Voltage or current peak value (vpeak), in volts or amps.
freq	Source frequency in Hz. Default = 1/TSTOP.
td	Time (propagation) delay before beginning the sinusoidal variation, in seconds. Default = 0.0. Response is 0 V or A, until the PrimeSim Continuum solution reaches the delay value, even with a nonzero DC voltage.
q	Damping factor, in units of 1/s. Default t= 0.0.
j	Phase delay, in units of degrees. Default = 0.0.
rmsjitter	RMS value for random phase jitter, in seconds.
perjitter	RMS value for period jitter used to adjust the magnitude of the random time.

Parameter	Description
seed	Used to generate random number sequences with different seed value. The default value is 1.
FMMDI	Frequency modulation index, which determines the magnitude of the deviation from the carrier frequency. Optional. Default=0.0.
FMFQ	Frequency modulation frequency in Hz. Optional. Default=0.0.
AMMDI	Amplitude modulation index, which determines the magnitude of the deviation from the carrier frequency. Optional. Default=1.0.
AMFQ	Amplitude modulation frequency in Hz. Optional. Default=0.0.
AMOFF	Amplitude modulation offset in volts or amps. Optional. Default=1.0.
AMPHA	Amplitude modulation phase in degrees. Optional. Default=0.0.

The following table of expressions defines the waveform shape:

Table 3 Waveform Shape Expressions

Time	Value
0 to td	$vo + va \cdot \sin\left(\frac{2 \cdot \pi \cdot \varphi}{360}\right)$
td to tstop	$vo + va \cdot \exp[-(Time - td) \cdot q] \cdot j$ Where q and j are the damping factor and phase delay in the syntax.

In these expressions, TSTOP is the final time.

Example

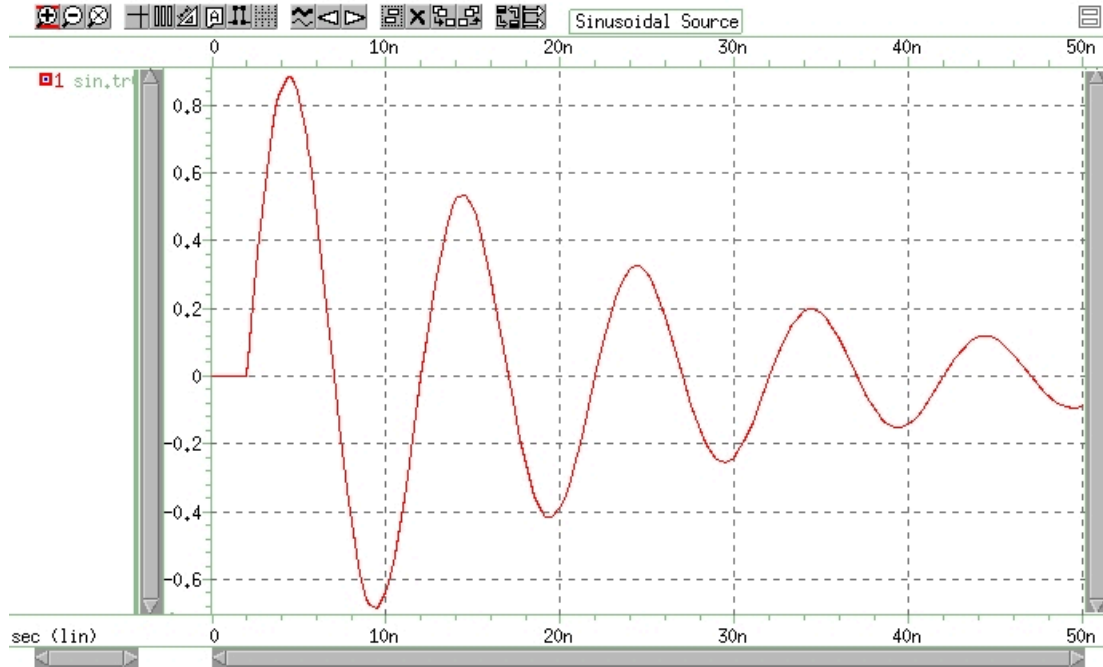
```
VIN 3 0 SIN (0 1 100MEG 1NS 1e10)
```

This damped sinusoidal source connects between nodes 3 and 0. In this waveform:

- Peak value is 1 V.
- Offset is 0 V.
- Frequency is 100 MHz.
- Time delay is 1 ns.
- Damping factor is 1e10.
- Phase delay is 0 deg.

See [Figure 7](#) for a plot of the source output.

Figure 7 Sinusoidal Source Function



This example uses demonstration netlist `sin.sp`, which is available in directory `$installdir/demo/hspice/sources`:

```
*file: sin.sp
sinusoidal source
.options post

.param v0=0 va=1 freq=100meg delay=2n theta=5e7 phase=0
v 1 0 sin(v0 va freq delay theta phase)
r 1 0 1
.tran .05n 50n
.end
```

Table 4 SIN Voltage Source

Parameter	Value
initial voltage	0 volts
pulse voltage	1 volt
delay time	2 nanoseconds
frequency	100 MHz
damping factor	50 MHz

Repeat Option in Sinusoidal Sources

You can use the `R-repeat` option in `SIN` (damping) sources. The syntax is:

```
sin (0 1 300meg 10n 1e8 R=val)
```

Where `val` can be a number specified or a parameter. A syntax of `R=R` is acceptable. For `SIN`, the `R` option means the length of each repeated period. For example:

```
VIN1 in out SIN (1.2 0.11 4G 5.5NS 2e9 180 R=1.2n)
```

This `SIN` function repeats from the beginning of the `SIN` waveform to `R=1.2n`.

Note:

The `SIN R`-option is not fully consistent with `PWL`. For `PWL`, the `R`-option specifies the start point of the waveform to repeat. In the example below, this `pwl` function repeats from `r=2n` to the last defined points (`5n, 0`).

```
v1 1 0 pwl 0 0 0.5n 0.5 2n 0.8 5n 0 r=2n
```

Exponential Source Function

The PrimeSim Continuum solution provides a exponential source function, in an independent voltage or current source.

The following sections discuss these topics:

- [Syntax](#)
- [Example](#)
- [Repeat Option in Exponential Sources](#)

Syntax

```
Vxxx n+ n- EXP [(] v1 v2 [tdr1 tcr1 tdf1 tcf1 tdr2 tcr2 tdf2 tcf2 ...]  
[)] [R=val]
```

```
Ixxx n+ n- EXP [(] v1 v2 [tdr1 tcr1 tdf1 tcf1 tdr2 tcr2 tdf2 tcf2 ...]  
[)] [R=val]
```

Parameter	Description
Vxxx, Ixxx	Independent voltage source, with an exponential response.
EXP	Keyword for an exponential time-varying source.
v1	Initial value of voltage or current, in volts or amps. 0
v2	Pulsed value of voltage or current, in volts or amps.

Parameter	Description
tdr	Rise delay time, in seconds. Default=0.0(first delay) $tdf_{(n-1)} + tcf_{(n-1)} + TSTEP$.
tdf	Fall delay time, in seconds. Default= $tdr_{n-1} + tcr_{n-1} + TSTEP$.
$tcr(\tau)$	Rise time constant, in seconds. default=TSTOP.
$tcf(\tau)$	Fall time constant, in seconds. default=TSTOP.
R	Repeat period for damped exponential.

The waveform shape follows equation:

$$val = v_1 + \sum_{n=1}^N \{(-1)^n \cdot (v_1 - v_2) \cdot [1 - \exp(-\frac{Time - td_n}{\tau_n})]\} \quad \text{while Time} > td_n$$

or

$$val = v_1 + (v_2 - v_1) \cdot \left[1 - \exp\left(-\frac{Time - td_1}{\tau_1}\right)\right] + (v_1 - v_2) \cdot \left[1 - \exp\left(-\frac{Time - td_2}{\tau_2}\right)\right] \\ + \dots + (v_2 - v_1) \cdot \left[1 - \exp\left(-\frac{Time - td_{n-1}}{\tau_{n-1}}\right)\right] + (v_1 - v_2) \\ \cdot \left[1 - \exp\left(-\frac{Time - td_n}{\tau_n}\right)\right]$$

while Time > td_n

Table 5 lists the equation for waveform shape definition.

Table 5 Waveform Shape Definition

Time	Value
0 to td_1	v_1
td_1 to td_2	$v_1 + (v_2 - v_1) \cdot [1 - \exp(-\frac{Time - td_1}{\tau_1})]$
td_2 to td_3	$v_1 + (v_2 - v_1) \cdot \left[1 - \exp\left(-\frac{Time - td_1}{\tau_1}\right)\right] +$ $(v_1 - v_2) \cdot [1 - \exp(-\frac{Time - td_2}{\tau_2})]$
.....	

Table 5 Waveform Shape Definition (Continued)

Time	Value
tdn to tstop	$v1 + (v2 - v1) \cdot \left[1 - \exp\left(-\frac{\text{Time} - td1}{\tau_1}\right) \right] +$ $(v1 - v2) \cdot \left[1 - \exp\left(-\frac{\text{Time} - td2}{\tau_2}\right) \right] + \dots +$ $(-1)^n \left[1 - \exp\left(-\frac{\text{Time} - tdn}{\tau_n}\right) \right]$

TSTEP is the printing increment, and TSTOP is the final time.

The following table of expressions defines the waveform shape:

Table 6 Waveform Shape Definitions

Time	Value
0 to td1	v1
td1 to td2	$v1 + (v2 - v1) \cdot \left[1 - \exp\left(-\frac{\text{Time} - td1}{\tau_1}\right) \right] + (v1 - v2) \cdot \left[1 - \exp\left(-\frac{\text{Time} - td2}{\tau_2}\right) \right] + \dots + (-1)^n \left[1 - \exp\left(-\frac{\text{Time} - tdn}{\tau_n}\right) \right]$
td2 to td3	$v1 + (v2 - v1) \cdot \left[1 - \exp\left(-\frac{\text{Time} - td1}{\tau_1}\right) \right] + (v2 - v1) \cdot \left[1 - \exp\left(-\frac{\text{Time} - td2}{\tau_2}\right) \right]$
....	
td2 to tstop	

Example

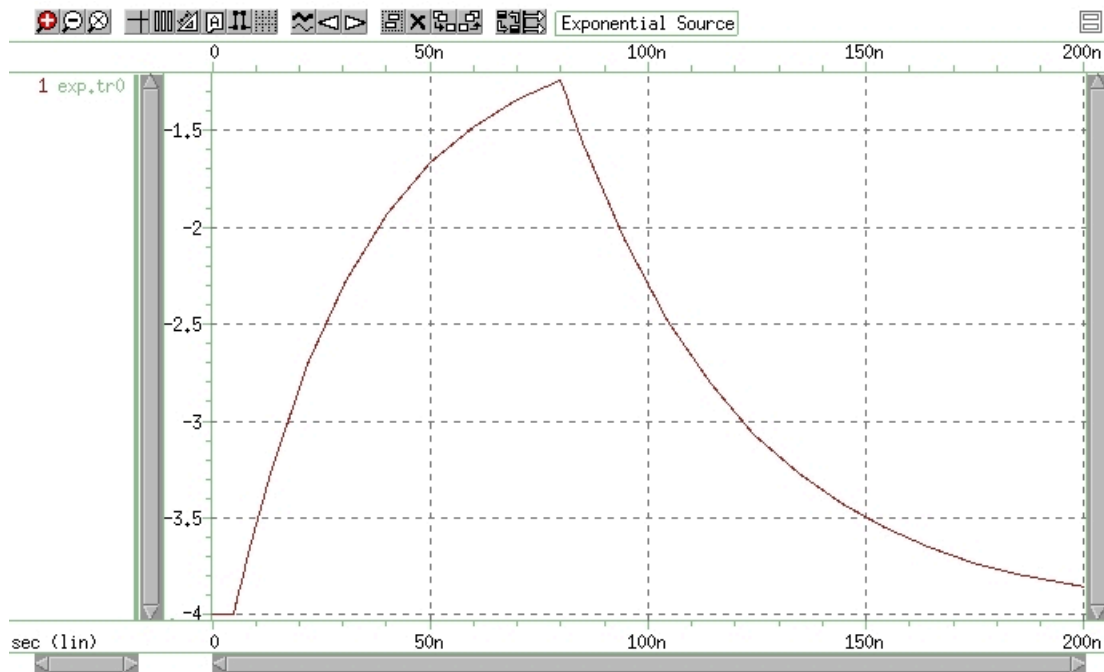
```
VIN 3 0 EXP (-4 -1 2NS 30NS 60NS 40NS)
```

The above example describes an exponential transient source, which connects between nodes 3 and 0. In this source:

- Initial t=0 voltage is -4 V.
- Final voltage is -1 V.
- Waveform rises exponentially from -4 V to -1 V with a time constant of 30 ns.

- At 60 ns, the waveform starts dropping to -4 V again, with a time constant of 40 ns.

Figure 8 Exponential Source Function



This example uses demonstration netlist `exp.sp`, which is available in directory `$installdir/demo/hspice/sources`:

```
*file: exp.sp      exponential independant source
.options post
.param v0=-4 va=-1 td1=5n tau1=30n tau2=40n td2=80n
v 1 0 exp(v0 va td1 tau1 td2 tau2)
r 1 0 1
.tran .05n 200n
.end
```

This example shows an entire netlist, which contains an `EXP` voltage source. In this source:

- Initial $t=0$ voltage is -4 V.
- Final voltage is -1 V.
- Waveform rises exponentially from -4 V to -1 V with a time constant of 30 ns.
- At 80 ns, the waveform starts dropping to -4 V again, with a time constant of 40 ns.

Repeat Option in Exponential Sources

You can use the `R-repeat` option in `EXP` (damping) sources. The syntax is:

```
exp (1 2 10n 30n 60n 20n R=val)
```

Where `val` can be a number specified or a parameter. A syntax of `R=R` is acceptable. For `EXP`, the `R` option means the length of each repeated period. For example:

```
VIN1 in out EXP (1 2 10n 30n 60n 20n R=1.2n)
```

This `EXP` function repeats from the beginning of the `SIN` waveform to `R=1.2n`.

Note:

The `EXP R`-option is not fully consistent with `PWL`. For `PWL`, the `R`-option specifies the start point of the waveform to repeat. In the example below, this `pwl` function repeats from `r=2n` to the last defined points (`5n, 0`).

```
v1 1 0 pwl 0 0 0.5n 0.5 2n 0.8 5n 0 r=2n
```

PWL Source

The PrimeSim Continuum solution provides a `PWL` source function in an independent voltage or current source.

The following sections discuss these topics:

- [PrimeSim Continuum PWL Form](#)
- [MSINC and ASPEC PWL Form](#)
- [Example](#)
- [Data-driven PWL Source](#)
- [File-driven PWL Source](#)

PrimeSim Continuum PWL Form

```
Vxxx n+ n- PWL [(] t1 v1 [t2 v2 t3 v3...]  
[R [=repeat] [rstop=val stopvalue=val [stopslope=val]]]  
+ [TD=delay] [)]  
+ [EDGETYPE=halfsine  
+ [WVFILE='filename']  
+ [WVLABEL='signal_name']  
+ [WVSWEEP=sweep_index]  
  
Ixxx n+ n- PWL [(] t1 v1 [t2 v2 t3 v3...]  
[R [=repeat] [rstop=val stopvalue=val [stopslope=val]]]  
+ [TD=delay] [)]  
+ [EDGETYPE=halfsine]
```



```
+ [WVFILE='filename']  
+ [WVLABEL='signal_name']  
+ [WVSWEEP=sweep_index]
```

Each pair of values (*t1*, *v1*) specifies that the value of the source is *v1* (in volts or amps), at time *t1*.

MSINC and ASPEC PWL Form

```
Vxxx n+ n- PL [(] v1 t1 [v2 t2 v3 t3...] [R= [repeat]]  
+ [TD=delay] [)]  
+ [EDGETYPE=halfsine]  
  
Ixxx n+ n- PL [(] v1 t1 [v2 t2 v3 t3...] [R= [repeat]]  
+ [TD=delay] [)]  
+ [EDGETYPE=halfsine]
```

The PL form of the function accommodates ASPEC style formats, and reverses the order of the time-voltage pairs to voltage-time pairs.

Parameter	Description
Vxxx, Ixxx	Independent voltage source; uses a piecewise linear response.
PL	Keyword for a piecewise linear time-varying source.
v1 v2 ... vn	Current or voltage values at the corresponding timepoint.
t1 t2 ... tn	Timepoint values, where the corresponding current or voltage value is valid.
R=repeat	Keyword and time value to specify a repeating function. With no argument, the source repeats from the beginning of the function. <i>repeat</i> is the time, in units of seconds, which specifies the startpoint of the waveform to repeat. This time needs to be less than the greatest time point, <i>tn</i> .
rstop	Specifies the stop time for the repeat.
stopvalue	Specifies the value of the current/voltage source at the time of <i>rstop</i> . <i>stopvalue</i> can be either a real number or Z for high impedance state.
stopslope	<i>stopslope</i> is the switching time from the last PWL value to the <i>stopvalue</i> . Default value is 30ps, if unspecified.

Note:

- *rstop*, *stopvalue*, and *stopslope* can also be specified in the PWL file '*filename*'.
- Z values are supported in PWL and PWL file sources, exactly the same as original PWLZ sources.
- *.option stopslope* can be used to change the default of *stopslope*.

TD=delay Time, in units of seconds, which specifies the length of time to delay (propagation delay) the piecewise linear function.

Parameter	Description
EDGETYPE	Type of the rising and falling edges of the waveform. Supported types are: LINEAR (default) or HALFSINE. The edges rise and fall by linear or by half-sinusoid, respectively..
WVFILE	Specifies the wave file name to be read.
WVLABEL	Specifies the signal name to be read.
WVSWEET	Specifies the sweep index when there is sweep parameter. The default is 0, means accessing the signal value from the first sweep.

- Linear interpolation between the time points determines the value of the source, at intermediate values of time.
- If you do not specify a time-zero point, thePrimeSim Continuum solution uses the DC value of the source, as the time-zero source value.

The PrimeSim Continuum solution does not force the source to terminate at the `TSTOP` value, specified in the `.TRAN` statement.

If the slope of the piecewise linear function changes below a specified tolerance, the timestep algorithm might not choose the specified time points as simulation time points. To obtain a value for the source voltage or current, the PrimeSim Continuum solution extrapolates neighboring values. As a result, the simulated voltage might deviate slightly from the voltage specified in the `PWL` list. To force the PrimeSim Continuum solution uses the specified values, use `.OPTION SLOPETOL`, which reduces the slope change tolerance.

`R` causes the function to repeat. You can specify a value after this `R`, to indicate the beginning of the function to repeat. The repeat time must equal a breakpoint in the function. For example, if $t1=1$, $t2=2$, $t3=3$, and $t4=4$, then the repeat value can be 1, 2, or 3.

Specify `TD=val` to cause a delay at the beginning of the function. You can use `TD` with or without the repeat function.

Example

This example uses demonstration netlist `pwl.sp`, which is available in directory

```
$installdir/demo/hspice/sources:
```

```
file pwl.sp repeated piecewise linear source
.option post
.tran 5n 500n
v1 1 0 pwl 60n 0v, 120n 0v, 130n 5v, 170n 5v, 180n 0v, r
r1 1 0 1

v2 2 0 p1 0v 60n, 0v 120n, 5v 130n, 5v 170n, 0v 180n, r 60n
```

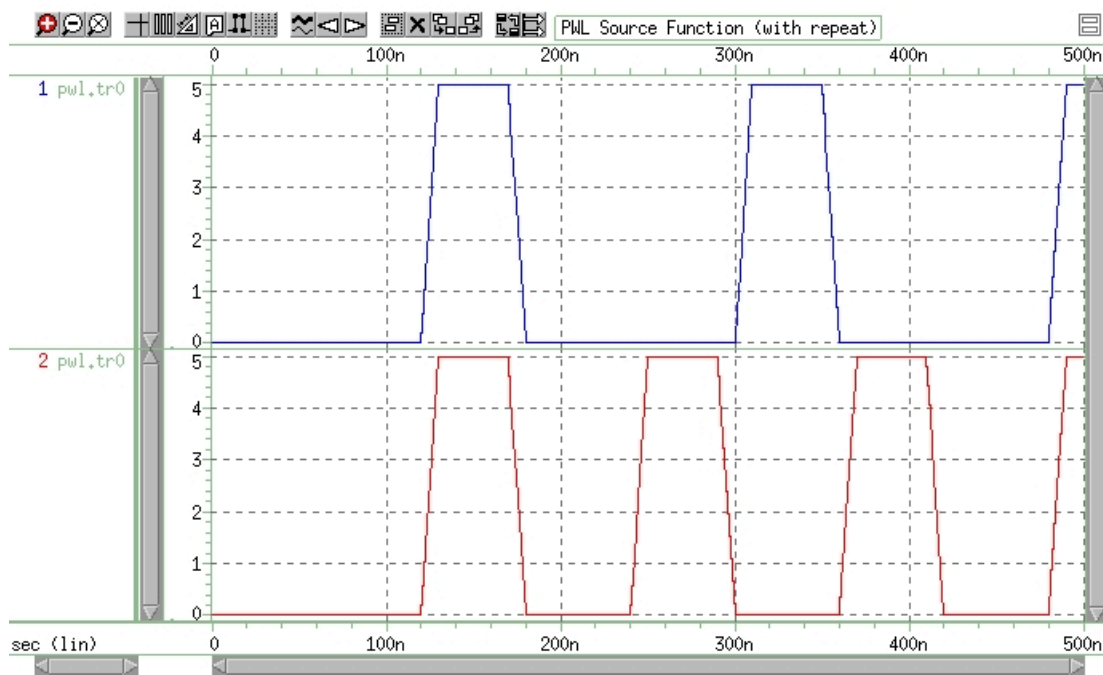
```
r2 2 0 1
.end
```

This example shows an entire netlist, which contains two piecewise linear voltage sources. The two sources have the same function:

- First is in normal format. The repeat starts at the beginning of the function.
- Second is in ASPEC format. The repeat starts at the first timepoint.

See [Figure 9](#) for the difference in responses.

Figure 9 Results of Using the Repeat Function



Data-driven PWL Source

The PrimeSim Continuum solution provides a data-driven PWL source function in an independent voltage or current source.

```
Vxxx n+ n- PWL (TIME,PV)
Ixxx n+ n- PWL (TIME,PV)
.DATA dataname
TIME PVt1 v1t2 v2t3 v3t4 v4. . . .
.ENDDATA
.TRAN DATA=datanam
```

Parameter	Description
TIME	Parameter name for time value, provided in a .DATA statement.
PV	Parameter name for amplitude value, provided in a .DATA statement.

Use with a `.DATA` statement that contains time-value pairs. For each $tn-vn$ (time-value) pair that you specify in the `.DATA` block, the data-driven `PWL` function outputs a current or voltage of the specified tn duration and with the specified vn amplitude. This source enables simulation results reuse as an input source in another simulation. Transient analysis must be data-driven.

Example

This example uses demonstration netlist `datamanual_pwl.sp`

, which is available in directory `$installdir/demo/hspice/sources:`

```
*DATA DRIVEN PIECEWISE LINEAR SOURCE
.options list node post
V1 1 0 PWL(TIME, pv1)
R1 1 0 1
V2 2 0 PWL(TIME, pv2)
R2 2 0 1
.DATA dsrc
TIME pv1 pv2
0n 5v 0v
5n 0v 5v
10n 0v 5v
.ENDDATA
.TRAN DATA=dsrc
.print v(1) v(2)
.END
```

This example is an entire netlist, containing two data-driven, piecewise linear voltage sources. The `.DATA` statement contains the two sets of values referenced in the `pv1` and `pv2` sources. `.TRAN` references the data name; there should be no time in `.TRAN` because `DATA` includes time.

File-driven PWL Source

You can specify a text file containing time, voltage (or current) pairs into a `PWL` source. The data in the text file can be either a table or a list.

```
Vxxx n1 n2 PWL PWLFILE='filename' [coll,[col2]]
[R [=repeat] [rstop=val stopvalue=val [stopslope=val]]]
+ [TD=delay]

Ixxx n1 n2 PWL PWLFILE='filename' [coll,[col2]]
```

```
[R [=repeat] [rstop=val stopvalue=val [stopslope=val]]]  
+ [TD=delay]
```

Parameter	Description
<code>col1</code>	Numerical time value. Should start at 1. The default is <code>col1=1</code> .
<code>col2</code>	Numerical voltage value. Should start at 1. The default is <code>col2=2</code> .
<code>rstop</code>	Specifies the stop time for the repeat.
<code>stopvalue</code>	Specifies the value of the current/voltage source at the time of <code>rstop</code> . <code>stopvalue</code> can be either a real number or Z for high impedance state.
<code>stopslope</code>	<code>stopslope</code> is the switching time from the last PWL value to the <code>stopvalue</code> . Default value is 30ps, if unspecified.

Note:

- `rstop`, `stopvalue`, and `stopslope` can also be specified in the PWL file '*filename*'.
- Z values are supported in PWL and PWL file sources, exactly the same as original PWLZ sources.
- `.option stopslope` can be used to change the default values of `stopslope`.

Examples

Example 1: Table

```
0 0  
1n 0.1  
1n 0.2
```

Example 2: List

Here, `PWLFILE` can use a string parameter.

```
0 0, 1n 0.1, 2n 0.2, ...
```

Note:

Commas in the list are optional.

Example 3

```
Vit n1 n2 PWL PWLFILE='Imod.dat'
```

Example 4

```
.parameter pwl_data_file=str('pwl.dat')
V1 in out PWL PWLFILE=str(pwl_data_file)
```

Example 5

```
I1 VDD 0 pwlfile='myfile'
+ [r=3.5e-6 [rstop=10u stopvalue=0 [stopslope=1n]]] [td=1n]
V1 VDD 0 pwlfile='myfile'
+ [r=3.5e-6 [rstop=10u stopvalue=Z [stopslope=1n]]] [td=1n]
```

High Impedance State PWL (PWLZ)

The high Z state of the PWLZ source adds the capability to disconnect the PWL source for time periods marked with the keyword `z`.

The following sections discuss these topics:

- [Syntax](#)
- [Example](#)

Syntax

```
Vxxx n+ n- PWLZ [(] t1 val1 t2 val2 [t3 z t4 val4 ...]
+ [R [=repeat]] [TD=delay] [)]
```

You can use the keyword `z` in place of the source value. You can disconnect the voltage source for time periods you mark with the keyword `z`. PWLZ sources require at least two time-value pairs.

Parameters	Description
Vxxx	Independent voltage source; capable of having high impedance during periods of times.
PWLZ	Keyword for a piecewise linear time-varying source with high impedance.
val1 val2... valn	Voltage values at the corresponding time points.
t1 t2 ... tn	Time point values.
R=repeat	Keyword and time value to specify a repeating function. With no argument, the source repeats from the beginning of the function. <i>repeat</i> is the time, in units of seconds, which specifies the start point of the waveform to repeat. This time needs to be less than the greatest time point, <i>tn</i> .
TD = delay	Time, in units of seconds, which specifies the length of time to delay (propagation delay) the piecewise linear function.

Example

In this example, node 5 connects to a 0V source at time 0, and rises from 0V to 0.75V in 2 ns. Between 2 ns and 10 ns, the voltage-source value rises from 0.75V to 1.5V. The voltage-source value stays at 1.5V between 10 ns and 50 ns. At 50 ns, node 5 disconnects from the voltage source until 60 ns. It re-connects to a 0.75V voltage source after 60 ns.

```
VXD 5 0 pwlz (0 0 2N 0.75 10N 1.5 50N z 60N 0.75)
```

Single-frequency FM Source

The PrimeSim Continuum solution provides a single-frequency FM source function, in an independent voltage or current source.

The following sections discuss these topics:

- [Syntax](#)
- [Example](#)

Syntax

```
Vxxx n+ n- SFFM [(] vo va [fc [mdi [fs]] [)]  
Ixxx n+ n- SFFM [(] vo va [fc [mdi [fs]] [)]
```

Parameter	Description
Vxxx, Ixxx	Independent voltage source, which exhibits the frequency-modulated response.
SFFM	Keyword for a single-frequency, frequency-modulated, time-varying source.
vo	Output voltage or current offset, in volts or amps.
va	Output voltage or current amplitude, in volts or amps.
fc	Carrier frequency, in Hz. Default=1/TSTOP.
mdi	Modulation index, which determines the magnitude of deviation from the carrier frequency. Values normally lie between 1 and 10. Default=0.0.
fs	Signal frequency, in Hz. Default=1/TSTOP.

The following expression defines the waveform shape:

Equation 6

$$sourcevalue = vo + va \cdot SIN[2 \cdot \pi \cdot fc \cdot Time + mdi \cdot SIN(2 \cdot \pi \cdot fs \cdot Time)]$$

Example

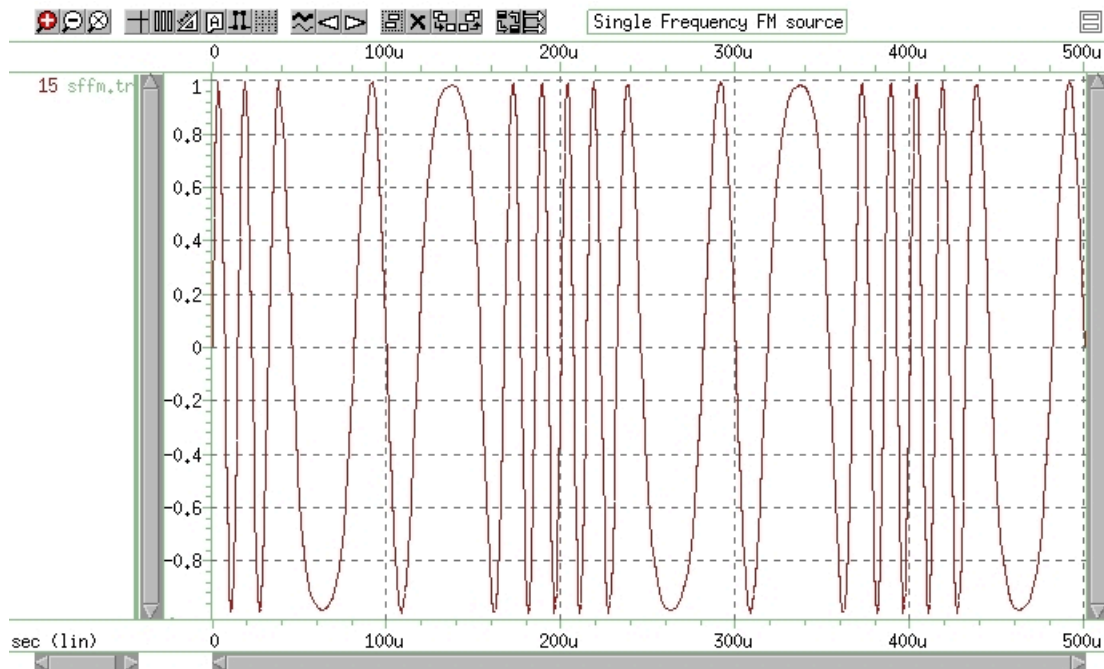
This example uses demonstration netlist `sffm.sp`, which is available in directory `$installdir/demo/hspice/sources`:

```
*file: sffm.sp frequency modulation source
.options post
vsff1 15 0 dc 3v sffm(0v 1v 20k 10 5k)
rssf1 15 0 1
.tran .001ms .5ms
.probe tran v(15)
.end
```

This example shows an entire netlist, which contains a single-frequency, frequency-modulated voltage source. In this source.

- The offset voltage is 0 volts.
- The maximum voltage is 1 millivolt.
- The carrier frequency is 20 kHz.
- The signal is 5 kHz, with a modulation index of 10 (the maximum wavelength is roughly 10 times as long as the minimum).

Figure 10 Single Frequency FM Source



Single-frequency AM Source

The PrimeSim Continuum solution provides a single-frequency AM source function in an independent voltage or current source.

The following sections discuss these topics:

- [Syntax](#)
- [Example](#)

Syntax

```
Vxxx n+ n- AM [ ( ) sa oc fm fc [td] [ ] ]  
Ixxx n+ n- AM [ ( ) sa oc fm fc [td] [ ] ]
```

Parameter	Description
Vxxx, Ixxx	Independent voltage source, which exhibits the amplitude-modulated response.
AM	Keyword for an amplitude-modulated, time-varying source.
sa	Signal amplitude, in volts or amps. Default=0.0.
fc	Carrier frequency, in hertz. Default=0.0.
fm	Modulation frequency, in hertz. Default=1/TSTOP.
oc	Offset constant, a unitless constant that determines the absolute magnitude of the modulation. Default=0.0.
td	Delay time (propagation delay) before the start of the signal, in seconds. Default=0.0.

The following expression defines the waveform shape:

Equation 7

$$sourcevalue = sa \cdot \{ oc + SIN[2 \cdot \pi \cdot fm \cdot (Time - td)] \} \cdot SIN[2 \cdot \pi \cdot fc \cdot (Time - td)]$$

Example

This example uses demonstration netlist `amsrc.sp`, which is available in directory

`$installdir/demo/hspice/sources:`

```
*file amsrc.sp amplitude modulation  
.option post  
.tran .01m 20m  
  
v1 1 0 am(10 1 100 1k 1m)  
r1 1 0 1
```

```
v2 2 0 am(2.5 4 100 1k 1m)
r2 2 0 1

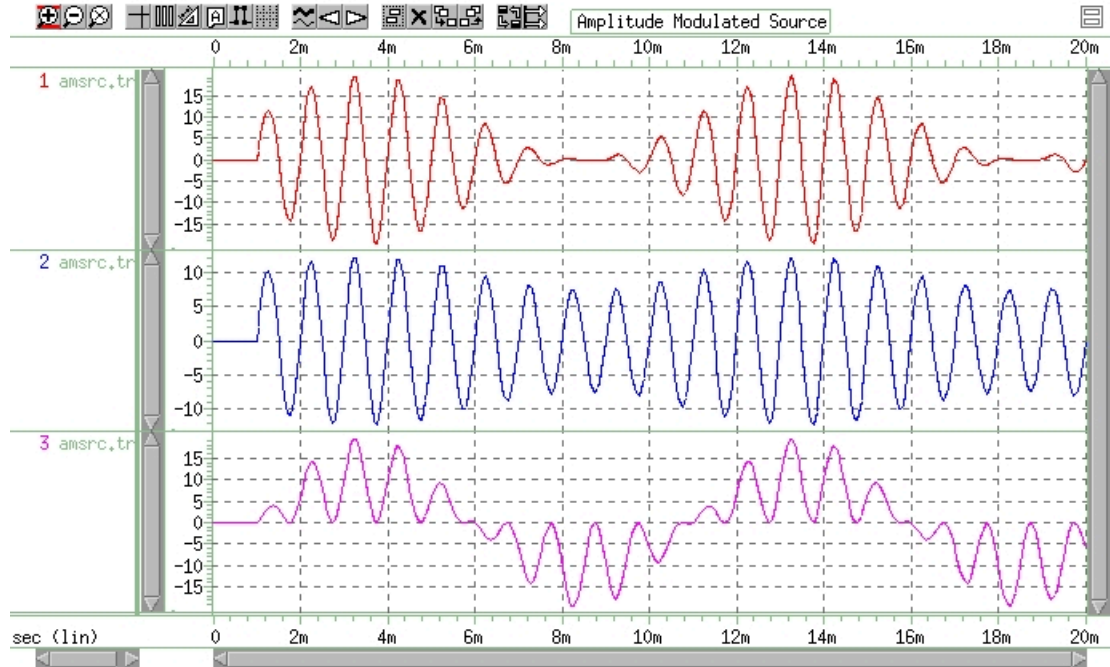
v3 3 0 am(10 1 1k 100 1m)
r3 3 0 1
.end
```

This example shows an entire netlist, which contains three amplitude-modulated voltage sources.

- In the first source:
 - Amplitude is 10.
 - Offset constant is 1.
 - Carrier frequency is 1 kHz.
 - Modulation frequency of 100 Hz.
 - Delay is 1 millisecond.
- In the second source, only the amplitude and offset constant differ from the first source:
 - Amplitude is 2.5.
 - Offset constant is 4.
 - Carrier frequency is 1 kHz.
 - Modulation frequency of 100 Hz.
 - Delay is 1 millisecond.
- The third source exchanges the carrier and modulation frequencies, compared to the first source:
 - Amplitude is 10.
 - Offset constant is 1.
 - Carrier frequency is 100 Hz.
 - Modulation frequency of 1 kHz.

- Delay is 1 millisecond.

Figure 11 Amplitude Modulation Plot



Pattern Source

The PrimeSim Continuum solution provides a pattern source function, in an independent voltage or current source. The pattern source function uses four states, '1', '0', 'm', and 'z', which represent the high, low, middle voltage, or current and high impedance state respectively. The series of these four states is a "b-string."

The PrimeSim Continuum solution supports the following for the PAT keyword:

- 8-bit data byte conversion to a 10-bit transmission character (8B/10B encoding)
- 64-bit data conversion to a 66-bit transmission character (64B/66B encoding)
- 128-bit data conversion to a 130-bit transmission character (128B/132B encoding or 128b/132b encoding)

The following sections discuss these topics:

- [Syntax](#)
- [Examples](#)
- [Nested Structure Pattern Source](#)

- [Pattern-command Driven Pattern Source](#)
- [Defining Pattern Sources with more than 1024 Characters](#)
- [Using String Parameters to Define Pattern Sources](#)
- [High-impedance Z-state in Pattern Sources](#)

Syntax

```
Vxxx n+ n- PAT [(] vhi vlo td tr tf tsample data
+ [RB=val][R=repeat]
+ [ENCODE=DW8b10b [RD_INIT=0|1]]
+ [ENCODE=DW64b66b [TXC=txcdata]]
+ [ENCODE=DW128b130b|DW128b132b [HEAD=DATA|CONTROL]]
+ [MODULATION=PAM4]
+ [PAM4_MAPPING=string]
+ [EDGETYPE=halfsine]
+ [)]

Ixxx n+ n- PAT [(] vhi vlo td tr tf tsample data
+ [RB=val][R=repeat]
+ [ENCODE=DW8b10b [RD_INIT=0|1]]
+ [ENCODE=DW64b66b [TXC=txcdata]]
+ [ENCODE=DW128b130b|DW128b132b [HEAD=DATA|CONTROL]]
+ [MODULATION=PAM4]
+ [PAM4_MAPPING=string]
+ [EDGETYPE=halfsine]
+ [)]
```

Parameter	Description
Vxxx, Ixxx	Independent voltage source that exhibits a pattern response.
PAT	Keyword for a pattern time-varying source.
vhi	High voltage or current value for pattern sources (units of volts or amps).
vlo	Low voltage or current value for pattern sources (units of volts or amps).
td	Delay (propagation) time in seconds from the beginning of the transient interval to the first onset ramp. It can be negative. The state in the delay time is the same as the first state specified in data.
tr	Duration of the onset ramp (in seconds) from the low value to the high value (reverse transit time).
tf	Duration of the recovery ramp (in seconds) from the high value back to the low value (forward transit time).
tsample	Time spent for one symbol which is known as a unit interval (UI). See Table 7 .

Parameter	Description
<i>data</i>	String of '1', '0', 'M', 'Z' representing a pattern source, or 'K' representing 8b/10b encoding control string (or control word). If the first alphabetic character is 'B', which represents it as a binary bit stream, the series is a b-string. '1' represents the value for high voltage or current, '0' is the value for low voltage or current, 'M' represents the value which is equal to $0.5 \cdot (v_{hi} + v_{lo})$. If the first alphabetic character is 'K', which represents the control string in the 8b10b encoder. Then a series binary bit stream (such as k00010001, similar to the original b-string) follows the k-string. In a K-string, the bit stream must be '1' or '0'. You can not use 'M' and 'Z'. The definition of K-string follows the form of "DW_8b10b_enc" in the Synopsys DesignWare Building Block IP. If you do not apply <code>encode=dw8b10b</code> in the netlist, then the K-string takes the same function as B-string.
RB	Keyword to specify the starting bit when repeating. The repeat data starts from the bit indicated by RB. RB must be an integer. The PrimeSim Continuum solution reports an error if the value is larger than the length of the b-string. If the value is less than 1, the PrimeSim Continuum solution resets it to 1 automatically.
R=repeat	Keyword to specify the number of times to execute the repeating operation. With no argument, the source repeats from the beginning of the b-string. If R=-1, it means the repeating operation continues forever. R must be an integer and if it is less than -1, the PrimeSim Continuum solution resets it to 0 automatically.
ENCODE=DW8b10b	Keyword to specify 8b/10b encoding.
RD_INIT=0 1	Initial value of Running Disparity. The one bit memory that recalls the bias of the last unbalanced code word is the Running Disparity. 1: Specifies that a Running Disparity value of zero is synonymous with negative Running Disparity. 0: Specifies that a Running Disparity value of one is synonymous with negative Running Disparity (-) 1: Specifies that a Running Disparity value of one is synonymous with positive Running Disparity (+).
ENCODE=DW64b66b	Keyword to specify 64b/66b encoding.
TXC= <i>txcdata</i>	The input 64-bit pattern signal is divided into 8 blocks with 8 bits each; <i>txcdata</i> is an 8-bit string with only 0 and 1s. It indicates the block type (data block or control block) for each 8-bit block. 0: specifies that the corresponding input block is a data block. 1: specifies that the corresponding input block is a control block. For example: TXC=10000000 indicates that the first block in the input pattern signal (bits 1 to 8) is a control block, and blocks 2, 3, 4, 5, 6, 7, and 8 (that is, 9-16, 17-24, 25-32, 33-40, 41-48, 49-56, and 57-64 bits respectively) are data blocks.
ENCODE=DW128b130b DW128b132b	Keyword to specify 128b/130b or 128b/132b encoding.
HEAD = DATA CONTROL	Indicates whether the pattern input signal is a data signal or control signal. Default value of HEAD is DATA. - DATA: specifies that the 128-bit pattern input signal is a DATA signal. - CONTROL: specifies that the 128-bit pattern input signal is a CONTROL signal.
MODULATION=PAM4	Activates pulse amplitude modulation (PAM) 4 signaling when the source has bit pattern (LFSR/PAT) input. By default, the source element uses the conventional two logic level signaling.

Parameter	Description
PAM4_MAPPING= <i>string</i>	Specifies mapping from a pair of bits to four-level symbols. PAM4_Mapping takes a four character string argument, a combination of '0', '1', '2' and '3' which represent pairs of binary symbols "00", "01", "10", and "11" respectively. The first, second, third, and fourth character positions in the argument string correspond to v0 (lowest), v1, v2, and v3 (highest) voltage levels, respectively. For example, PAM4_MAPPING = "0132" maps "00", "01", "11", and "10" to v0, v1, v2 and v3, respectively. Default value for PAM4_MAPPING (when MODULATION=PAM4) is "0132".
EDGETYPE	Type of the rising and falling edges of the waveform. Supported types are: LINEAR (default) or HALFSINE. The edges rise and fall by linear or by half-sinusoid, respectively.

Table 7 *Binary versus PAM4 Protocol*

	Binary Data	PAM4
Unit interval (tsample)	One bit per unit interval	Two bits per unit interval
Voltage levels	vlo and vhi	vlo, vlo+dv, vlo+2*dv, and vhi, where dv = (vhi-vlo)/3

The time from 0 to the first transition is:

$$tdelay+N*tsample-tr(tf)/2$$

- N is the number of the same bit from the beginning.
- If the first transition is rising, this equation uses tr.
- If the first transition is falling, it uses tf.

Examples

Example 1

The following example shows a pattern source with two b-strings:

```
*FILE: pattern source general form
v1 1 0 pat (5 0 0n 1n 1n 5n b1011 r=1 rb=2 b0m1z)
r1 1 0 1
```

In this pattern:

- High voltage is 5 v
- Low voltage is 0 v

- Time delay is 0 n
- Rise time is 1 n
- Fall time is 1 n
- Sample time is 5 n

The first b-string is 1011, which repeats once and then repeats from the second bit, which is 0. The second b-string is 0m1z. Since the example specifies neither R and RB here, the PrimeSim Continuum solution sets them to the default values, $R=0$, $RB=1$.

Example 2

The following b-string and its repeat time R and repeating start bit RB cannot use a parameter — the PrimeSim Continuum solution considers it as a undivided unit which only the `.PAT` command can define.

```
*FILE:pattern source using parameter
.param td=40ps tr=20ps tf=80ps tsample=400ps
VIN 1 0 PAT (2 0 td tr tf tsample b1010110 r=2)
r1 1 0 1
```

In this pattern:

- High voltage is 2 V.
- Low voltage is 0 V.
- Time delay is 40 ps.
- Rise time is 20 ps.
- Fall time is 80 ps.
- Sample time is 400 ps.
- Data is 1010110.

Nested Structure Pattern Source

The PrimeSim Continuum solution provides a nested structure for the pattern source function to construct complex waveforms. Nested structure is a combination of a b-string and other nested structures defined in a `.PAT` command (see [Pattern-command Driven Pattern Source on page 120](#)).

Syntax

The following general syntax is for a nested structure pattern source.

```
Vxxx n+ n- PAT [(] vhi vlo td tr tf tsample
+ [component 1 ... component n] [RB=val] [R=repeat] [)]
```

```
Ixxx n+ n- PAT [(] vhi vlo td tr tf tsample  
+ [component 1 ... component n] [RB=val] [R=repeat] [)]
```

Parameter	Description
component	Component is the element that makes up NS, which can be a b-string or a patname defined in other PAT commands. You must use Brackets ([]).
RB=val	Keyword to specify the starting component when repeating. The repeat data starts from the component indicated by RB. RB must be an integer. The PrimeSim Continuum solution reports an error if RB is larger than the length of the NS. If RB is less than 1, the PrimeSim Continuum solution resets it automatically to 1.
R=repeat	Keyword to specify number of executions of the repeating operation. With no argument, the source repeats from the beginning of the NS. If R=-1, the repeating operation continues indefinitely. R must be an integer, and if it is less than -1, the PrimeSim Continuum solution resets it automatically to 0.

If the component is a b-string, you can follow it with *R=repeat* and *RB=val* to specify the repeat time and repeating start bit.

Example

```
*FILE: Pattern source using nested structure  
v1 1 0 pat (5 0 0n 1n 1n 5n [b1011 r=1 rb=2 b0m1z] r=2 rb=2)  
r1 1 0 1
```

When expanding the nested structure, you generate the pattern source as:

```
'b1011 r=1 rb=2 b0m1z b0m1z b0m1z'
```

The entire nested structure is repeated twice with the repeated structure starting from the second component defined in the nested structure (*b0m1z*).

Pattern-command Driven Pattern Source

Syntax

The following general syntax is for including a pattern-command driven pattern source in an independent voltage or current source. You can reset the *RB* and *R* of a b-string or *NS* in an independent source. With no argument, the *R* and *RB* are the same when you define it in the pattern command.

```
Vxxx n+ n- PAT [(] vhi vlo td tr tf tsample PatName [RB=val]  
+ [R=repeat] [)]  
  
Ixxx n+ n- PAT [(] vhi vlo td tr tf tsample Patname [RB=val]  
+ [R=repeat] [)]
```


Parameter	Description
<i>PatName</i>	The name of the pattern defined by one or more <code>.PAT</code> commands. For information on the <code>.PAT</code> command, see <code>.PAT</code> in <i>PrimeSim HSPICE® Reference Manual: Commands and Control Options</i> .

Additional syntax applies to the `.PAT` command-driven pattern source:

```
.PAT PatName=data [RB=val] [R=repeat]
.PAT PatName=[component 1...component n] [RB=val] [R=repeat]
```

The `PatName` is the pattern name that has an associated b-string or nested structure.

Examples

Example 1

```
v1 1 0 pat (5 0 0n 1n 1n 5n a1 a2 r=2 rb=2)
.PAT a1=b1010 r=1 rb=1
.PAT a2=b0101 r=1 rb=1
```

The final pattern source is:

```
b1010 r=1 rb=1 b0101 r=2 rb=2
```

When the independent source uses the `.PAT` command to specify its pattern source, you can reset `r` and `rb`.

Example 2

```
*FILE 2: Pattern source driven by pattern command
v1 1 0 pat (5 0 0n 1n 1n 5n [a1 b0011] r=1 rb=1)
.PAT a1=[b1010 b0101] r=0 rb=1
```

The final pattern source is:

```
b1010 b0101 b0011 b1010 b0101 b0011
```

A pattern source can reference the `a1` which is a predefined nested structure.

Defining Pattern Sources with more than 1024 Characters

The PrimeSim Continuum solution limits pattern sources to 1024 characters, including the `.PAT` statement. To define pattern sources with more than 1024 characters, you can define multiple pattern statements and combine them using another pattern statement.

Example

To combine the patterns, first define several separate patterns such as:

```
.pat p1=b100000000001000100001100100001010011000111  
.pat p2=b0100001001010100101101100011010111001111  
.pat p3=b1000010001100101001110100101011011010111
```

Next, define another pattern source that combines the previous ones

```
.pat p4=p1 p2 p3
```

Finally, use the combined pattern source in your source statement:

```
V1 1 0 pat (5 0 0n 20p 30p 200p p4)
```

Using String Parameters to Define Pattern Sources

See the following example:

```
.param pat1=str('b111111111111111')
v1 1 0 pat (5 0 0n 20p 30p 200p str(pat1))
```

High-impedance Z-state in Pattern Sources

In circuit simulation, the equation of voltage source $V(n1, n2)$ is taken as:

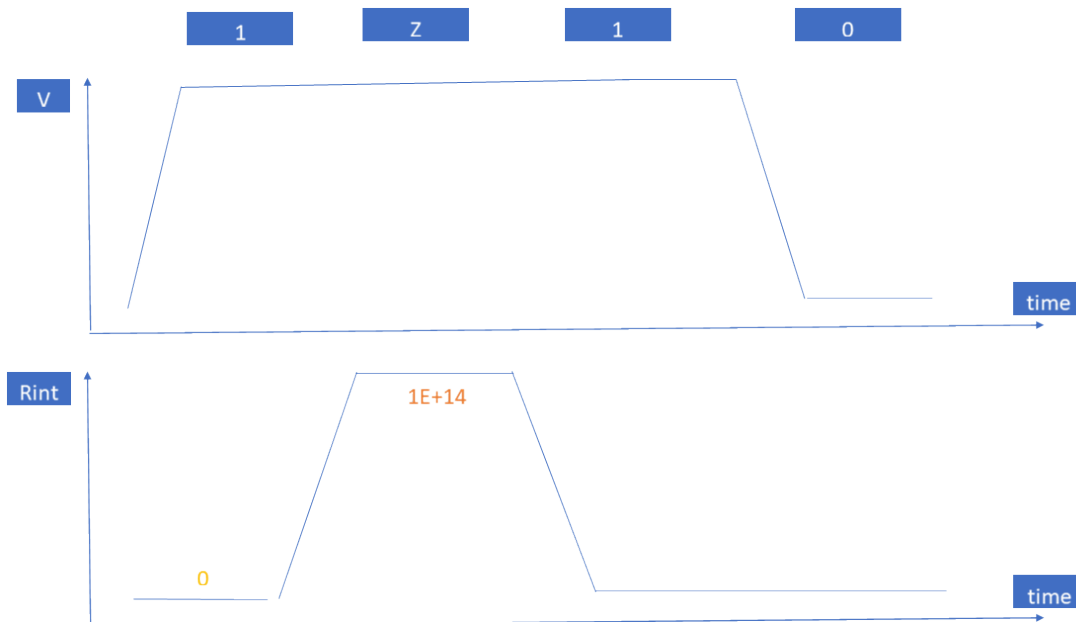
$$V - V_{n1} - V_{n2} - I * R_{int} = 0$$

Here V_{n1} and V_{n2} are the voltage value at node1 and node2 respectively; I is current value through this voltage source. V is input voltage value from the netlist such as PWL, PAT, and LFSR. R_{int} is internal impedance of this voltage source.

A typical pattern source with a high Z state can be controlled by the V and R_{int} value. In the PrimeSim Continuum solution, this is done by creating a two bitstream; one to store 0 or 1 status by changing V 's value and the other to store high Z or non-Z status by changing R_{int} 's value.

The impedance of the pattern source in the Z-state is a fixed value $1E+14$ which represents the high Z status. A typical pattern source with a high Z state is assumed as non-ideal with a resistance in series. This resistance value is ramped up to $1E+14$ when transitioning from 0|1 to Z. Similarly, it is ramped down when transitioning from Z to 1|0.

Figure 12 Impedance of the Pattern source in the Z-state



As shown in [Figure 12 on page 123](#), R is not a sudden change (from 0 to 1E+14). Instead, it is a smooth transition curve and the bitstream of impedance share the same TR/TF/WIDTH with the signal one.

Pseudo Random Bit Generator Source

The PrimeSim Continuum solution provides a Pseudo Random Bit Generator Source (PRBS) function as an independent voltage or current source. You can use this function in several applications from cryptography and bit-error-rate measurement, to wireless communication systems employing spread spectrum or CDMA techniques. The PRBS source uses a Linear Feedback Shift Register (LFSR) to generate a pseudo random bit sequence. The PrimeSim Continuum solution supports 8-bit data byte conversion to a 10-bit transmission character—8B/10B encoding for the LFSR keyword.

The following sections discuss these topics:

- [Syntax](#)
- [Examples](#)
- [Linear Feedback Shift Register](#)
- [Conventions for Feedback Tap Specification](#)
- [Example: Noise Generator Used for a Pulse or DC Level](#)

Syntax

```
Vxxx n+ n- LFSR [(] vlow vhigh tdelay trise tfall rate seed
+ [taps] [rout=val]
+ [DCD=val]
+ [ENCODE=DW8B10B [RD_INIT=0|1]]
+ [ENCODE=DW64b66b [TXC=txcdata]]
+ [ENCODE=DW128b130b|DW128b132b [HEAD=DATA|CONTROL]]
+ [LFSRFILE='filename']
+ [MODULATION=PAM4]
+ [PAM4_MAPPING=string]
+ [rjseed=val] [rjrms=val] [pjamp=val] [pjfreq=val]
+ [pjtype=sine|sawtooth|square] [)]
```

```
Ixxx n+ n- LFSR [(] vlow vhigh tdelay trise tfall rate seed
+ [taps] [rout=val]
+ [DCD=val]
+ [ENCODE=DW8B10B [RD_INIT=0|1]]
+ [ENCODE=DW64b66b [TXC=txcdata]]
+ [ENCODE=DW128b130b|DW128b132b [HEAD=DATA|CONTROL]]
+ [LFSRFILE='filename']
+ [MODULATION=PAM4]
+ [PAM4_MAPPING=string]
```

```
+ [rjseed=val] [rjrms=val] [pjamp=val] [pjfreq=val]
+ [pjtype=sine|sawtooth|square] [ ] ]
```

Parameter	Description
LFSR	Specifies the voltage or current source as PRBS.
vlow	The minimum voltage or current level.
vhigh	The maximum voltage or current level.
tdelay	Specifies the initial time delay to the first transition.
trise	Specifies the duration of the onset ramp (in seconds) from the initial value to the pulse plateau value (reverse transit time).
tfall	Specifies the duration of the recovery ramp (in seconds) from the pulse plateau back to the initial value (forward transit time).
rate	The symbol rate. See Table 8 .
seed	The initial value (in integer form) of the LFSR loaded into the shift register. Because the operation of the register is deterministic, its current (or previous) state completely determines the stream of values produced by the register.
taps	The bits used to generate feedback.
rout	The output resistance.
DCD	DCD value is a real number with unit %. It can be set within the range of [-40,40]. A positive value 'A' means the duty cycle will be reduced by A% of the pulse width. A negative value 'A' means the duty cycle will be increased by A% of the pulse width.
ENCODE=DW8b10b	Keyword to specify 8b/10b encoding.
RD_INIT=0 1	Initial value of Running Disparity. <i>Running Disparity.1</i> is the name of the one-bit memory that recalls the bias of the last unbalanced code word. 0: Specifies that a Running Disparity value of zero is synonymous with negative Running Disparity (-) 1: Specifies that a Running Disparity value of one is synonymous with positive Running Disparity (+).
ENCODE=DW64b66b	Keyword to specify 64b/66b encoding. Note: DW64b66b encoding is valid only for the PAT <code>transient_waveform</code> type.
TXC=txcdata	The input 64-bit pattern signal will be divided into 8 blocks with 8 bits each; <i>txcdata</i> is an 8-bit string with only 0 and 1s. It indicates the block type (data block or control block) for each 8-bit block. 0: specifies that the corresponding input block is a data block. 1: specifies that the corresponding input block is a control block. For example: TXC=10000000 indicates that the first block in the input pattern signal (bits 1 to 8) is a control block, and blocks 2, 3, 4, 5, 6, 7, and 8 (that is, 9-16, 17-24, 25-32, 33-40, 41-48, 49-56, and 57-64 bits respectively) are data blocks.

Parameter	Description
ENCODE=DW128b130b DW128b132b	Keyword to specify 128b/130b or 128b/132b encoding. Note: DW128b130b and DW128b132b encoding are valid only for the PAT transient_waveform type.
HEAD = DATA CONTROL	Indicates whether the pattern input signal is a data signal or control signal. Default value of HEAD is DATA. - DATA: specifies that the 128-bit pattern input signal is a DATA signal. - CONTROL: specifies that the 128-bit pattern input signal is a CONTROL signal.
LFSRFILE	Specifies a user-defined file containing a random bit sequence. Note: LFSRFILE overrides seed and taps, if they are defined.
MODULATION=PAM4	Activates pulse amplitude modulation (PAM) 4 signaling when the source has bit pattern (LFSR/PAT) input. By default, the source element uses the conventional two logic level signaling.
PAM4_MAPPING=string	Specifies mapping from a pair of bits to four-level symbols. PAM4_Mapping takes a four character string argument, a combination of '0', '1', '2', and '3' which represent pairs of binary symbols "00", "01", "10", and "11" respectively. The first, second, third, and fourth character positions in the argument string correspond to v0 (lowest), v1, v2, and v3 (highest) voltage levels, respectively. For example, PAM4_MAPPING = "0132" maps "00", "01", "11", and "10" to v0, v1, v2 and v3, respectively. Default value for PAM4_MAPPING (when MODULATION=PAM4) is "0132".
rjseed	The seed for random number generator, used in generating random jitter for PRBS sources. The default value is 1.
rjrms=val [s]	When set for PRBS source or Bit source, the source has a normally distributed random jitter whose mean is zero and whose standard deviation is the value of rjrms.
pjamp=val [s]	When set for PRBS source or Bit source, the source has multiple periodic jitters whose amplitudes are defined by pjamp and frequencies are defined by pjfreq.
pjfreq=val [Hz]	When set for PRBS source or Bit source, the source has multiple periodic jitters whose amplitudes are defined by pjamp and frequencies are defined by pjfreq.
pjtype	For PRBS source or Bit source, pjtype defines the types of periodic jitters. Possible values are 'sine', 'sawtooth', or 'square'. The default value is 'sine'.

Table 8 Binary versus PAM4 Protocol

	Binary Data	PAM4
Rate	symbol rate = bit rate	2 * symbol rate = bit rate
Voltage levels	vlow and vhigh	vlow, vlow+dv, vlow+2*dv, and vhigh, where dv = (vhigh-vlow)/3

Examples

Example 1

The following example shows the pattern source connected between node in and ground (gnd).

Example of LFSR, output is 1100011111001101:

```
vin out gnd LFSR (0 1 0 1n 1n 10meg 3 [2, 5] rout=10)
.option POST
.tran 10p 10u
.end
```

Where:

- The output low voltage is 0 , and the output high voltage is 1 v.
- The delay time is 0 ms.
- The rise and fall times are each 1 ns.
- The bit rate is 10meg bits/s.
- The seed is 3 (bits: 00011).
- The taps are at the output of the 2nd and 5th shift registers and are xor'd together as the input to the first shift register.
- The output resistance is 10 ohm.

Example 2

The following example shows the pattern source connected between node 1 and node 0:

```
.PARAM tdl=2.5m trl=2n
vin 1 0 LFSR (2 4 tdl trl 1n 6meg 2 [10, 5, 3, 2])
```

Where,

- The output low voltage is 2 v, and the output high voltage is 4 v.
- The delay is 2.5 ms.
- The rise time is 2 ns, and the fall time is 1 ns.
- The bit rate is 6meg bits/s.
- The seed is 2.
- The taps are [10, 5, 3, 2].
- The output resistance is 0 ohm.

Example 3

This example uses demonstration netlist `prbs.sp`, which is available in directory `$installdir/demo/hspice/sources`:

```
* prbs.sp
.OPTION POST
.TRAN 0.5n 50u
V1 1 0 LFSR (0 1 1u 1n 1n 10meg 1 [5, 2] rout=10)
R1 1 0 1
.END
```

Example 4

To generate a PRBS source that includes jitter, use the following steps:

1. Construct your usual linear feedback shift register (LFSR) generator.
2. Construct a matching (T,tr,tf) PULSE source as a clock, but add jitter to it with the PERJITTER keyword.
3. Use the PULSE source to gate (buffer) the LFSR output (through an ideal AND gate, VCCS, or similar function).

Example 5

Consider the following example:

```
vlfsr data 0 DC=0 LFSR(0 1 500p 10p 10p 10g 1 [9 5] )
+ LFSRFILE='bit.txt'
```

Here `LFSRFILE` specifies a random bit sequence defined in the `bit.txt` file. Settings for `SEED` and `TAPS`, if any, are ignored.

Note:

For a SPICE syntax source, the transition from low to high is computed as shown in this example: `v0 vout gnd! dc=0 lfsr ( 0 1 0n 0.1n 0.1n '1/(1n)' 43690 [16,15,13,4] )` The bit rate is 1bit/1ns and the rise time is 0.1ns. The 0% transition happens at 1ns-0.1ns=900ps. Trise is counted into rate from 0 to 1 and Tfall from 1 to 0. This is applicable to both PrimeSim HSPICE and PrimeSim SPICE.

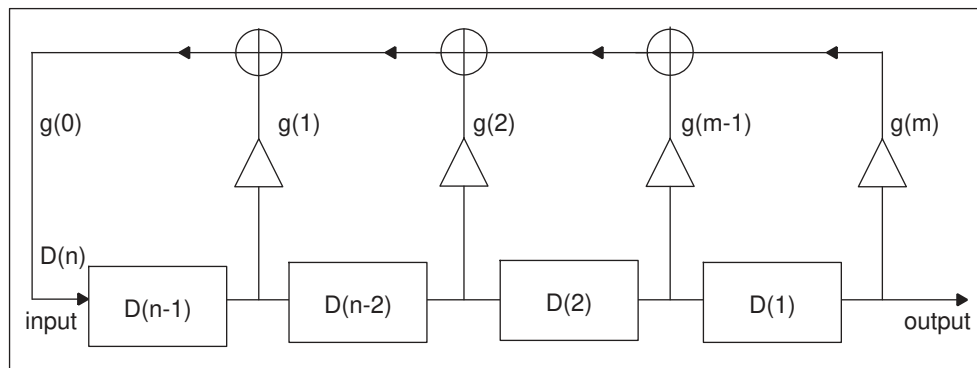
Figure 13 Transition



Linear Feedback Shift Register

A LFSR consists of several simple-shift registers in which a binary-weighted modulo-2 sum of the taps feed back to the input. The modulo-2 sum of two 1-bit binary numbers yields 0 if the two numbers are identical and 1 if they differ (0+0=0, 0+1=1, or 1+1=0). See [Figure 14 on page 129](#).

Figure 14 LFSR Diagram



For any given tap, the weight “ g_i ” is either 0, (meaning “no connection”), or 1, (meaning it feeds back). Two exceptions are g_0 and g_m , which are always 1 and therefore always connected. The g_m is not really a feedback connection, but rather an input-assigned feedback weight of the shift register for mathematical purposes.

The first number in your TAPS definition defines the maximum number of bits. For example [23, 22, 21, 20, 19, 7] denotes a 23-stage LFSR. The TAPS definition is a specific feedback tap sequence that generates an M-Sequence PRB. The LFSR stages can range from 2 to 64 bits. You cannot set the seed to zero; the PrimeSim Continuum solution reports an error and exits the simulation if you set the seed to zero.

Note:

The order of the tap locations in the LFSR statement can be arbitrary. The PrimeSim Continuum solution sorts them in the decreasing order and then construct the LFSR sequence.

Conventions for Feedback Tap Specification

You can express a given set of feedback connections in a convenient and easy-to-use shorthand form with the connection numbers listed within a pair of brackets. The g0 connection is implied and not listed since it is always connected. Although gm is also always connected, it is listed to convey the shift register size (number of registers).

The following line is a set of feedback taps where j is the total number of feedback taps (not including g0), f(1)=m is the highest-order feedback tap (and the size of the LFSR), and f(j) are the remaining feedback taps:

```
[f(1), f(2), f(3), ..., f(j)]
```

Example

The following line shows that the number of registers is 7 and the total number of feedback taps is 4:

```
[7, 3, 2, 1]
```

The following feedback input applies for this specification:

$$D(n) = [D(n-7) + D(n-3) + D(n-2) + D(n-1)] \bmod 2$$

Example: Noise Generator Used for a Pulse or DC Level

The following example creates a DC source voltage with a random noise of 0.1v max amplitude in a spice stimulus.

```
$$$ Noise Generator Used for a Pulse or DC Level
.option probe post
.param VDD = 1.2
.param RNOM = 1k
.param tr = 100p
.param tf = 100p
.param rate = 2G
.param Tb = '1/rate'
.param td = 'Tb/4'
.param tstop = '11100*Tb'
*=====
*Add noise ( amplitude : 100 mVpp ) to supply voltage VDD
*=====
VDD ( vdd supnoise ) 'VDD'
RDD ( vdd 0 ) 1T
x_noise ( supnoise 0 ) tr_nsrc Vpp=0.1
Vclk ( clk 0 ) PULSE( 0 1 td tr tf 'Tb/2-tr' 'Tb' )
```

```

Rclk ( clk 0 ) 1T
.tran 'Tb/1000' 'tstop'
.probe tran v(clk) v(vdd) v(supnoise) v(X_noise.nsum) v(X_noise.*)
•=====
•Noise generator ( output of 10 LFSR added to generate a random bit
sequence )
•=====
.subckt tr_nsrc ( np nn ) Vpp=0.1
*****
•To make the nature of output signal more random ****
•generate 10 separate PRBS sources and sum them up ****
*****
v1 ( n1 0 ) LFSR ( 0 1 td tr tf rate 10001 [20,9,6,3,1] )
v2 ( n2 0 ) LFSR ( 0 1 td tr tf rate 5313 [20,9,6,3,1] )
v3 ( n3 0 ) LFSR ( 0 1 td tr tf rate 9 [10,9,6,3,1] )
v4 ( n4 0 ) LFSR ( 0 1 td tr tf rate 213 [10,9,6,3,1] )
v5 ( n5 0 ) LFSR ( 0 1 td tr tf rate 741 [10,9,6,3,1] )
v6 ( n6 0 ) LFSR ( 0 1 td tr tf rate 397 [10,9,6,3,1] )
v7 ( n7 0 ) LFSR ( 0 1 td tr tf rate 4793 [20,9,6,3,1] )
v8 ( n8 0 ) LFSR ( 0 1 td tr tf rate 7039 [20,9,6,3,1] )
v9 ( n9 0 ) LFSR ( 0 1 td tr tf rate 12031 [20,9,6,3,1] )
va ( na 0 ) LFSR ( 0 1 td tr tf rate 50071 [20,9,6,3,1] )
R1 ( n1 0 ) 1T
R2 ( n2 0 ) 1T
R3 ( n3 0 ) 1T
R4 ( n4 0 ) 1T
R5 ( n5 0 ) 1T
R6 ( n6 0 ) 1T
R7 ( n7 0 ) 1T
R8 ( n8 0 ) 1T
R9 ( n9 0 ) 1T
Ra ( na 0 ) 1T
•*****
•Sum up the 10 random pulse signals
•*****
esum ( nsum 0 )
VOL='v(n1)+v(n2)+v(n3)+v(n4)+v(n5)+v(n6)+v(n7)+v(n8)+v(n9)+v(na) '
•*****
•High pass filter to remove the DC component of the summed voltage
•*****
cs ( nsum nout ) 1p
rs ( nout 0 ) 1k
•*****
•Series of 2 decoupled low pass filters to smooth the edges and eliminate
high frequency components
•*****
e_1 ( nin_f1 0 ) vcvs ( nout 0 ) 1
rf1 ( nin_f1 nout_f1 ) 2k
cf1 ( nout_f1 0 ) 0.35p
e_2 ( nin_f2 0 ) vcvs ( nout_f1 0 ) 1
rf2 ( nin_f2 nout_f2 ) 2k
cf2 ( nout_f2 0 ) 0.35p
•*****

```

```
•Scale the peak to peak value of the noise voltage
•*****
Eout ( np nn ) VCVS ( nout_f2 0 ) Vpp
.ends
•=====
.end
```

Mixed Sources

Mixed sources specify source values for more than one type of analysis. For example, you can specify a DC source, an AC source, and a transient source; all of which connect to the same nodes. In this case, when you run specific analyses, the PrimeSim Continuum solution selects the appropriate DC, AC, or transient source. In a DC analysis, if the mixed source DC value is missing, the simulation replaces it with the zero-time value of its transient source by default. Otherwise, for DC analysis, PrimeSim Continuum solution uses the DC source value for operating point calculation for all analyses except TRAN; TRAN analysis calculates an additional operating point with the zero-time source transient value.

Example

```
VIN 13 2 0.5 AC 1 SIN (0 1 1MEG)
```

where:

- DC source of 0.5 V
- AC source of 1 V
- Transient damped sinusoidal source

Each source connects between nodes 13 and 2.

For DC analysis, the PrimeSim Continuum solution uses its dc value 0.5 volts and selects this operating point for the AC analysis to come. In transient analysis, the PrimeSim Continuum solution calculates another operating point by using zero source value because the sinusoidal source is zero at time zero.

Power Sources

These sections describe independent power sources and controlled power sources.

The following section discusses this topic:

- [Independent Power Sources](#)

Independent Power Sources

A power source is a special kind of voltage or current source that supplies the network with a pre-defined power which varies by time or frequency. The source produces a specific input impedance.

To apply a power source to a network, you can either one of the following:

- A Norton-equivalent circuit (if you specify this circuit and a current source)—the I (current source) element
- A Thevenin-equivalent circuit (if you specify this circuit and a voltage source)—the V (voltage source) element

As with other independent sources, the PrimeSim Continuum solution simulation assumes that positive current flows from the positive node through the source to the negative node. A power source is a time-variant or frequency-dependent utility source; therefore, the value/phase can be a function of either time or frequency.

A power source is a subclass of the independent voltage/current source, with some additional keywords or parameters:

You can use I and V elements in DC, AC, and transient analysis. The I and V elements can be data-driven. Supported power source formats include:

- DC
- SIN, a damped sinusoidal function.

The following sections discuss these topics:

- [Using the Keyword POWER](#)
- [Examples](#)
- [Calculation for Total Dissipated Power and Voltage Source Power](#)
- [Subcircuit Power Calculation](#)

Using the Keyword POWER

If you use the `POWER` keyword in the netlist, then a simulation recognizes a current/voltage source as a power source:

```
Vxxx n+ n- [[DC=] dcval tranfun [AC=acmag acphase]]  
+ power=[0|1|2|w|dbm]
```

```
Ixxx n+ n- [[DC=] dcval tranfun [AC=acmag acphase]]  
+ power=[0|1|2|w|dbm] [M=val]
```

Parameter	Description
<code>DC=dcval</code>	DC source keyword and value in volts. Used for the operating point calculation for all simulations except transient. Transient analysis calculates an additional operating point with the tranfun value at time zero. Default=0.0.
<code>tranfun</code>	Transient source function (When POWER is on, transient source function only supports SIN). The functions specify the characteristics of a time-varying source. See the individual functions for syntax.
<code>AC</code>	AC source keyword for use in AC small-signal analysis.
<code>acmag</code>	Magnitude (RMS) of the AC source in volts or in watts or dbms when the power switch is turned on.
<code>acphase</code>	Phase of the AC source, in degrees. Default=0.0.
<code>power=[0 1 2 W dbm]</code>	Power switch • When 0 (default), element treated as a voltage or current source. • When 1 or W, the element is treated as a power source, and realized as a voltage source with a series impedance or a current source with a parallel impedance. In this case, the source value is interpreted as RMS available power in units of watts. • When 2 or dbm, the element is treated as a voltage source with a series impedance or a current source with a parallel impedance. Values are in dbms. You can use this parameter for transient analysis if the power source is either DC or SIN. When POWER is on, a 50-ohm source impedance is added automatically.

Examples

Example 1

This example applies a 5-W power source to node 10 and node 20, in a Thevenin-equivalent manner. The impedance of this power source is 50 ohms.

```
V11 10 20 power=1 dc=5
```

Example 2

This example applies a 20-dbm power source to node 1 and to ground, in a Norton-equivalent manner. The source impedance is 50 ohms.

```
Iname 1 0 power=2 dc=20
```

Example 3

This example applies a 1 dbm (available) AC power source to node 1 and ground, in a Thevenin-equivalent manner. The impedance of this power source is 50 ohms.

```
V11 1 0 power=dbm ac=1
```

Example 4

This example applies a 1-W (available) sinusoidal power source to node 1 and 0, in a Norton-equivalent manner. The source impedance is 50 ohms.

```
Iname 1 0 power=w sin(0 1 1k)
```

Calculation for Total Dissipated Power and Voltage Source Power

You can use two keywords to calculate power:

- The `POWER` keyword calculates the total dissipated power of the circuit.
- The `SRC_PWR` keyword calculates the total power from the voltage source of the circuit.

For example, if you have a top-level circuit as follows:

```
*****
vin n1 pulse(0 5 0 1n 1n 4n 10n)
r0 n1 n2 1
r1 n2 0 1
c0 n2 0 .2u
*****
```

Note:

The case will fail since only one node is connected to `vin`.

You can use `.MEAS TRAN POWER` to calculate the total dissipated power in resistors. The total dissipated power is $p(r0) + p(r1)$.

You can use `.MEAS TRAN SRC_PWR` to calculate the total power from the voltage source. The total power from the voltage source is $p(r0) + p(r1) + p(c0)$. The PrimeSim Continuum solution can verify this with the following statement:

```
.print tran tot_pwr=par('p(r0) + p(r1) + p(c0)')
```

Note that the PrimeSim Continuum solution does not add the power of each independent source (V and I).

For power measurements, the PrimeSim Continuum solution computes the dissipated or stored power in each passive element (R, L, C), and source (V, I, G, E, F, and H). To compute this power, the PrimeSim Continuum solution multiplies the voltage across an element and the corresponding branch current. However, for semiconductor devices, the PrimeSim Continuum solution and PrimeSim Continuum solution advanced analyses measure only the dissipated power, excluding the power stored in the device junction or parasitic capacitance from the device power computation. The `avg(P(Vdd))` function measures both the dissipated power and the stored power while `avg(P(X))` measures only the dissipated power for semiconductor devices. If the circuit contains semiconductor devices, the results of the `avg(P(X))` function can be erroneous. The recommended

method to measure the average power of the entire circuit in this case is to measure the average power of the source by using the `avg(P(Vdd))` function.

Subcircuit Power Calculation

To print or probe an element or subcircuit power, use the variables, `P(element_name)` or `P(instance_name_of_subckt)`. For example, if you have the following subcircuit:

```
x1 in out inv
.subckt inv in out
    mn out in 0 0 nch W=10u L=1u
    mp out in vdd vdd pch W=10u L=1u
.ends
```

The total circuit POWER is `-( p(vin) + p(vvdd) )`, which is equal to `p(rout)+p(cout)+p(x1)`. You can verify this by using the parameter expression.

Measuring Leakage Power

When you probe `P(Instance_name)`, the PrimeSim Continuum solution includes the gate tunneling current in the power function calculations.

Troubleshooting Differences in Rise/Fall Power Input Signals and Power Measurements

You may note a difference between the AVG power calculations for the rise time and fall time for input signals. There should not be a difference. However, if you measure the AVG subcircuit power using `P(Instance_name)` for an inverter subcircuit, for example, you may note that the PrimeSim Continuum solution excludes the energy stored in the output and includes the energy discharged from the capacitor for fall time.

If there is any difference in the measured results, try running the simulation with the most accurate settings by setting `.OPTION RUNLVL=6`. In addition, you can set `.OPTION DELMAX` to the minimum rise or fall time of your circuit.

You may observe a difference in results of two measurements when using the `avgP(X)` function, where X contains semiconductor devices, and using `avg(P(Vdd))`, the power of the voltage source.

For power measurements, the PrimeSim Continuum solution computes the dissipated or stored power in each passive element (R, L, C), and source (V, I, G, E, F, and H). To compute this power, the PrimeSim Continuum solution multiplies the voltage across an element, and the corresponding branch current. However, for semiconductor devices, the PrimeSim Continuum solution measures only the dissipated power and excludes the power stored in the device junction or parasitic capacitances from the device power computation. The `avg(P(Vdd))` measures both the dissipated power and the stored power while `avg(P(X))` measures only the dissipated power for semiconductor devices. If the circuit contains semiconductor devices, the results of the `avg(P(X))` function can be erroneous. The recommended method to measure the average power of the entire circuit

in this case is to measure the average power of the source by using the (avg P(Vdd)) function.

PrimeSim Continuum Advanced Analog Analyses Sources and Stimuli

The following sections discuss the use of advanced analog analyses-specific sources:

- [Steady-state Voltage and Current Sources](#)
- [Steady-state HB Sources](#)
- [Phase Differences Between HB and SIN Sources](#)
- [Behavioral Noise Sources](#)
- [Function Approximations for Distributed Devices](#)
- [Complex Signal Sources and Stimuli](#)
- [SWEEPBLOCK in Sweep Analyses](#)
- [Clock Source with Random Jitter](#)

Steady-state Voltage and Current Sources

The I (current source) and V (voltage source) elements include extensions that allow you to use them as sources of steady-state sinusoidal signals for HB and HBAC analyses. When you use a power parameter to specify the available power, you also can use these elements as power sources.

For a general description of the I and V elements, see [Power Sources on page 132](#).

I and V Element Syntax

```
Vxxx p n
+ $ **** Voltage or Power Information ****
+ [[dc] mag] [ac [mag [phase]]] [HBAC [mag [phase]]]
+[SNAC [mag [phase]]]
+ [hb [mag [phase] [harm [tone [modharm [modtone]]]]]]
+ [transient waveform] [TRANFORHB=[1|0]]

+ $ **** Power Switch ****
+ [power=[0 | 1 | W | dbm]] [z0=val] [rdc=val] [rac=val]
+ [RHBAC=val] [rhb=val] [rtran=val]

Ixxx p n
+ $ **** Current or Power Information ****
```

Chapter 2: Independent Sources and Stimuli

PrimeSim Continuum Advanced Analog Analyses Sources and Stimuli

```
+ [[dc] mag] [ac [mag [phase]]] [HBAC [mag [phase]]]
+[SNAC [mag [phase]]]
+ [hb [mag [phase] [harm [tone [modharm [modtone]]]]]]
+ [transient waveform] [TRANFORHB=[1|0]]

+ $ **** Power Switch *****
+ [power=[0 | 1 | W | dbm]] [z0=val] [rdc=val] [rac=val]
+ [RHBAC=val] [rhb=val] [rtran=val]
```

Parameter	Description
<code>[[dc] mag]</code>	DC voltage or power source value. You do not need to specify DC explicitly. The default = 0.
<code>[ac [mag [phase]]]</code>	AC voltage or power source value.
<code>[HBAC [mag [phase]]]</code>	(HBAC analysis) HBAC voltage or power source value.
<code>[SNAC [mag [phase]]]</code>	(SNAC analysis) SNAC voltage or power source value.
<code>[hb [mag [phase] [harm [tone [modharm [modtone]]]]]]</code>	(HB analysis) HB voltage, current, or power source value. Multiple HB specifications with different harm, tone, modharm, and modtone values are legal: • <i>phase</i> is in degrees. • <i>harm</i> and <i>tone</i> are indices corresponding to the tones specified in the <code>.HB</code> statement. Indexing starts at 1 (corresponding to the first harmonic of a tone). • <i>modtone</i> and <i>modharm</i> specify sources for multitone simulation. A source specifies a tone and a harmonic, and up to 1 offset tone and harmonic (modtone for tones and modharm for harmonics). The following describes the signal as: $V(\text{or } I) = \text{mag} \cdot \cos(2\pi \cdot (\text{harm} \cdot \text{tone} + \text{modharm} \cdot \text{modtone}) \cdot t + \text{phase})$
<code>[transient waveform]</code>	(Transient analysis) Any one of waveforms: AM, EXP, PULSE, PWL, SFFM, or SIN. Multiple transient descriptions are illegal.
<code>[power=[0 1 W dbm]]</code>	(the PrimeSim Continuum solution advanced analog analyses) Power Switch: • 0: Element treated as a voltage or current source. • 1: Element treated as a power source, realized as a voltage source with a series impedance. In this case, the PrimeSim Continuum interprets the source value as RMS available power in units of watts. • dbm: Element treated as a power source in series with the port impedance. Values are in dbms. You can use this parameter for Transient analysis if the power source is either DC or SIN.
<code>[z0=val]</code>	(LIN analysis) System impedance used when converting to a power source, inserted in series with the voltage source. Currently, this only supports real impedance: • When power=0, z0 defaults to 0. • When power=1, z0 defaults to 50 ohms. You can also enter <code>z0=val</code> .
<code>[rdc=val]</code>	(DC analysis) Series resistance (overrides <code>z0</code>).
<code>[rac=val]</code>	(AC analysis) Series resistance (overrides <code>z0</code>).
<code>[RHBAC=val]</code>	(HBAC analysis) Series resistance (overrides <code>z0</code>).

Parameter	Description
[rhb=val]	(HB analysis) Series resistance (overrides z0).
[rtran=val]	(Transient analysis) Series resistance (overrides z0).
[TRANFORHB=[0 1]]	0 (default): The PrimeSim Continuum solution advanced analog analyses ignores the transient description when an HB or DC value exists. If no DC or HB value exists and TRANFORHB=0, then HB treats the source as a DC source, and the DC source value is the time=0 value. 1: HB analysis uses the transient description if its value is VMRF, SIN, PULSE, PWL, or LFSR. If the type is a nonrepeating PWL source, then a DC source value uses the time=infinity value. For example, the following statement is treated as a DC source with value=1 for HB: v1 1 0 PWL (0 0 1n 1 1u 1) TRANFORHB=1 In contrast, the following statement is a 0V DC source: v1 1 0 PWL (0 0 1n 1 1u 1) TRANFORHB=0 The following statement is treated as a periodic source with a 1 us period that uses PWL values: v1 1 0 PWL (0 0 1n 1 0.999u 1 1u 0) R TRANFORHB=1 To override the global TRANFORHB option, explicitly set TRANFORHB for a V/I source.

Examples

Example 1

This example shows an HB source for a single tone analysis:

```
.hb tones=100MHz harms=7
```

```
I1 1 2 dc=1mA hb 3mA 0. 1 1
```

I1 is a current source with a the following time-domain description:

```
I1=1mA + 3mA*cos(2*pi*1.e8*t)
```

Example 2

This example shows HB sources used for a two-tone analysis:

```
.hb tones=1.e9 1.1e9 intmodmax=5  
Vin lo 0 dc=0. hb 1.5 90 1 1
```

```
Vrf rf 0 dc=0. hb 0.2 0 1 2
```

These sources have the following time-domain descriptions:

```
Vin=1.5*cos(2*pi*1.e9*t - 90*pi/180) V
```

```
Vrf = 0.2*cos(2*pi*1.1e9*t) V
```

Example 3

The following HB source uses a `modtone` and `modharms`:

```
.hb tones=2.e9 1.9e9 harms=5 5
```

```
Vm input gnd dc=0.5 hb 0.2 0. 1 1 -1 2
```

`Vm` has the following time-domain description:

$$V_m = 0.5 + \cos(2\pi \cdot 1.9 \text{e}9 \cdot t)$$

Example 4

This example uses an HB source specified with a `SIN` source and `HBTRANINIT`.

```
.hb tone=1.e8 harms=7  
Vt 1 2 SIN(0.1 1.0 2.e8 0. 0. 90) tranforhb=1
```

Where `Vt` converts to the following HB source:

```
Vt 1 2 dc=0.1 hb 1.0 0.0 2 1
```

Example 5

This example shows a power source (the units are watts).

```
.hb tones=1.1e9 harms=9
```

```
Pt Input Gnd power=1 Z0=50. 1m 0. 1 1
```

`Pt` delivers 1 mW of power through a 50-ohm impedance.

Steady-state HB Sources

Use the `.HB TONES` command to specify the fundamental frequencies used with harmonic balance analysis. Then you can reference these frequencies by their integer indices when specifying steady-state signal sources. For example:

```
.HB TONES=1900MEG,1910MEG INTMODMAX=5
```

This specifies two fundamental frequencies: $f[\text{tone} = 1] = 1.9\text{GHz}$ and $f[\text{tone} = 2] = 1.91\text{GHz}$. Then you can reference their mixing product at 10 MHz using indices as $|f[2] - f[1]|$, while you reference their third-order intermodulation product at 1.89 GHz as $|2f[1] - f[2]|$.

Identify steady-state voltage and current sources with the HB keyword according to:

```
[HB [mag [phase [harm [tone [modharm [modtone]]]]]]]
```

The source is mathematically equivalent to a cosine signal source that follows the equation:

$$A \cos(\omega t + \phi)$$

where:

$$A = mag$$

$$\omega = 2\pi |harm \cdot f[tone] + modharm \cdot f[modtone]|$$

$$\phi = \frac{\pi}{180} \cdot phase$$

Values for tone and modtone (an optional modulating tone) must be non-negative integers that specify index values for the frequencies specified with the `.HB TONES` command. Values for `harm` (harmonic) and `modharm` (modulating tone harmonic) must be integers (negative values are acceptable) that specify harmonic indices.

Examples

Example 1

The following example is a 1.0-V (peak) steady-state cosine voltage source, which is at the fundamental HB frequency with 0 phase and with a 0-V DC value:

```
Vsrc in gnd DC 0 HB 1.0 0 1 1
```

Example 2

The following example is a steady-state cosine power source with 1.0-mW available power, which is implemented with a Norton equivalent circuit and a 50-ohm input impedance:

```
Isrc in gnd HB 1.0e-3 0 1 1 power=1 z0=50
```

Example 3

Five series voltage sources sum to produce a stimulus of five equally spaced frequencies at and above 2.44 GHz using `modharm` and `modtone` parameters. These are commensurate tones (an integer relation exists); therefore, you only need to specify two tones when invoking the HB analysis.

```
.param Vin=1.0
.param f0=2440MEG
.param deltaf=312.5K
.param fcenter='f0 + 2.0*deltaf'
Vrfa in ina HB 'Vin' 0 1 1 $ 2.440625 GHz
Vrfb ina inb HB 'Vin' 0 1 1 -1 2 $ 2.4403125 GHz
Vrfc inb inc HB 'Vin' 0 1 1 -2 2 $ 2.440 GHz
Vrfd inc ind HB 'Vin' 0 1 1 +1 2 $ 2.4409375 GHz
```

```
Vrfe ind gnd HB 'Vin' 0 1 1 +2 2 $ 2.44125 GHz
.HB tones=fcenter,deltaf intmodmax=5
```

Phase Differences Between HB and SIN Sources

The HB steady-state cosine source has a phase variation compared to the `TRAN` time-domain `SIN` source. The `SIN` source (with no offset, delay or damping) follows the equation:

Equation 8

$$A \sin(\omega t + \phi)$$

while the HB sources follow:

Equation 9

$$A \cos(\omega t + \phi)$$

For the two sources to yield identical results, you must align them by setting their phase values accordingly using:

Equation 10

$$A \cos(\omega t + \phi) = A \sin(\omega t + \phi + 90^\circ)$$

Equation 11

$$A \sin(\omega t + \phi) = A \cos(\omega t + \phi - 90^\circ)$$

To specify sources with matching phase for HB and `TRAN` analysis, use a convention similar to:

```
** Example #1 with equivalent HB and SIN sources
** SIN source is given +90 phase shift
.param freq1=2400MEG Vin=1.0
Vsrc in gnd DC 0 HB 'Vin' 0 1 1 SIN(0 'Vin' 'freq1' 0 0 90)
.HB tones=freq1 intmodmax=7
** Example #2 with equivalent HB and SIN sources
** HB source is given -90 phase shift to align with SIN
.param freq1=2400MEG Vin=1.0
Vsrc in gnd DC 0 HB 'Vin' -90 1 1 SIN(0 'Vin' 'freq1' 0)
.HB tones=freq1 intmodmax=7
** Example #3 with equivalent .HB and .TRAN sources
** SIN source is activated for HB using "TRANFORHB"
.param freq1=2400MEG Vin=1.0
Vsrc in gnd DC 0 SIN(0 'Vin' 'freq1' 0) TRANFORHB=1
.HB tones=freq1 intmodmax=7
```

Behavioral Noise Sources

In the PrimeSim Continuum advanced analog analyses, you can use the G-element to specify noise sources. Frequency domain noise analyses (.NOISE, .HBNOISE, and .PHASENOISE) take these noise sources into account.

You can attach noise sources to behavioral models. For example, you can use a G-element with the `VCCAP` parameter to model a varactor which includes a noise model. You also can simulate effects such as substrate noise including its effect on oscillator phase noise. You also can use this G-element syntax to simulate behavioral descriptions of substrate noise during any frequency domain noise analysis, which includes phase noise analysis. For example:

```
gname node1 node2 noise='noise_equation'  
gname node1 node2 node3 node4 noise='noise_equation'
```

The first line creates a simple two-terminal current noise source whose value is described in $A^2/(Hz)$. The output noise generated from this noise source is:

`noise_equation*H`

Where H is the transfer function from the terminal pair (node1,node2) to the circuit output, where the PrimeSim Continuum solution advanced analog analyses measures the output noise.

The second line produces a noise source correlation between the (node1,node2) and (node3,node4) terminal pairs. The resulting output noise is calculated as `noise_equation*sqrt(H1*H2*)`; where:

- H_1 is the transfer function from (node1,node2) to the output
- H_2 is the transfer function from (node3,node4) to the output.

The `noise_equation` expression can involve node voltages and currents through voltage sources.

For the PAC phasenoise simulation to evaluate the frequency-dependent noise, the frequency-dependent noise factor in the phasenoise must be expressed in between the parentheses. For example:

```
gname node1 node2 noise = '(frequency_dependent_noise)*  
bias_dependent_noise'
```

This is only true when the total noise can be expressed in this form and when the frequency-dependent noise can be evaluated in the PAC phasenoise simulation. You also

can input the behavioral noise source as a noise table with the help of predefined Table() function. The Table() function takes two formats:

- Noise table can be input directly through the Table() function. For example:

```
gname node1 node2 noise = 'Table(arg1,f1,v1,f2,v2,.....)'
```

- The *f1,v1,f2,v2,.....* parameters describe the noise table. When *arg1 == f1*, the function returns *v1*. The *arg1* can be an expression of either HERTZ, bias, or both. For example, *arg1 = 'HERTZ * 1.0E+3'*.
- The noise table can be input through a .DATA structure:

```
.DATA d1  
+ x y  
+ f1 v1  
+ f2 v2  
.ENDDATA
```

```
gname node1 node2 noise = 'TABLE(arg1,d1)'
```

The *x, y* parameters in the .DATA structure are two placeholder strings that can be set to whatever you prefer even if they are in conflict with other parameters in the netlist. The *arg1* parameter can be an expression of HERTZ and bias. When *arg1 == f2*, the function will return *v2*.

The following sections discuss these topics:

- [Using Noise Analysis Results as Input Noise Sources](#)
- [Syntax Rules for Behavioral Noise Source Files](#)
- [Power Supply Current and Voltage Noise Sources](#)

Using Noise Analysis Results as Input Noise Sources

By extending the E- and G- voltage-controlled source syntax, the phase noise or noise data in ASCII phase noise and noise files can be used as input for specifying behavioral noise sources.

Usage Model

The syntax for the voltage controlled voltage (E) or current (G) source is:

```
Exxx node1 node2 noisefile='filename' [mname='measname']  
Gxxx node1 node2 noisefile='filename' [mname='measname']
```

where:

noisefile='filename' is the name of the ASCII noise data file. Phase noise analysis and noise analyses can output simulation results as ASCII data using .PRINT statements.

The file name is designated as 'design.printsnpn0' for .SNOSC phase noise and .SNNOISE analyses. For .PHASENOISE, .HBNOISE and .ACPHASENOISE analyses, the file name is designated as 'design.printpn0'. For .NOISE analyses, the file name is designated as 'design.printac0'.

`mname='measname'` is used to select the appropriate noise measurement name to be taken from the *.printpn0, *.printsnpn0, or *.printac0 file.

`measname` can be one of the following:

- `NLP_L(f)` – Selects the `nlp_L(f)` phase noise data in units of dBc/Hz.
- `PAC_L(f)` – selects the `pac_l(f)` phase noise data in units of dBc/Hz.
- `ONoise` – selects the `onoise` data based on .SNNOISE or .NOISE analysis.

The following syntaxes are supported in both the PrimeSim Continuum solution advanced analog analyses and the PrimeSim Continuum solution:

- `Exxx n1 n2 noise data=dataname`
- `Exxx n1 n2 noise data=datablock`
- `Exxx n1 n2 noise file='filename'`
- `Exxx n1 n2 noise='expression'`
- `Exxx n1 n2 noise='Table(arg1,f1,v1,f2,v2...)'`
- `Exxx n1 n2 noise='Table(arg1,dotDataBlockName) ', where dotDataBlockName is the .data statement reference.`

Syntax Rules for Behavioral Noise Source Files

You can use ASCII noise data files not generated by .PRINT statements when you follow these syntax rules:

A noise file always needs a header (do not include frequency or noise points in the header line, as they are ignored). The items in the header must be consistent with the number of data columns. You can use noise keywords in the header (see the next item in this list). The noise value units depend on your use of the `MNAME` keyword.

You can use noise keywords like `ONoise`, and the phase noise keywords, `PAC_L(f)` and `NLP_L(f)`. This is useful for *.printac#, *.printpn#, or *.printsnpn# files and use of the `MNAME` keyword to specify the noise data column to be used.

If `MNAME` is not specified with the behavioral noise source, then the PrimeSim Continuum solution assumes that the second column in the noise file represents the desired noise values, and the `MNAME` string is read from the noise file header. The first column is always assumed to contain frequency values.

The `MNAME` keyword is used to differentiate units. If you use `MNAME=ONoise` and the file contains `ONoise` in the header, then the noise units are interpreted as $V/\sqrt{\text{Hz}}$. The same is true with `MNAME=ONoise (M)` and `MNAME=ONoise (MAG)`. If `MNAME` is set to `MNAME=ONoise (M^2)`, the noise units are interpreted as V^2/Hz or I^2/Hz .

If `MNAME` is set to `PAC_L(f)`, `NLP_L(f)`, or `PHNoise`, then units are interpreted as dBc/Hz . With `MNAME=ONoise (DB)`, units are taken as dB/Hz ($20 \cdot \log(\text{noise})$), as are other `MNAME` keywords that end in `DB` or `(DB)`. Most other `MNAME` keywords are generally interpreted in V^2/Hz units, unless the name ends in `(M)` or `(MAG)`, which are taken to be $V/\sqrt{\text{Hz}}$. Depending on the noise units, different interpolation schemes are used for frequencies needed between noise data points. Noise in dB/Hz or dBc/Hz units use logarithmic interpolation (in x and y), while non-dB units use linear interpolation in V^2/Hz .

Noise File Examples

The following examples provide sample noise results noting the various noise value units:

Example 1: Phase noise file generated by using `.PRINT phasenoise phnoise` statement. The noise units are dBc/Hz :

```
HERTZ  NLP_L(f)
10.0000000000k      -130.2612492663
12.5892541179k      -132.2612484153
15.8489319246k      -134.2612470666
19.9526231497k      -136.2612449295
25.1188643151k      -138.2612415429
31.6227766017k      -140.2612361768
39.8107170553k      -142.2612276743
50.1187233627k      -144.2612142034
63.0957344480k      -146.2611928626
79.4328234724k      -148.2611590575
100.0000000000k      -150.2611055150
125.8925411794k      -152.2610207237
158.4893192461k      -154.2608864692
199.5262314969k      -156.2606739394
251.1886431510k      -158.2603375701
316.2277660168k      -160.2598053187
398.1071705535k      -162.2589632779
501.1872336273k      -164.2576312765
630.9573444802k      -166.2555240055
794.3282347243k      -168.2521885144
1.0000000000x        -170.2469020563
```

Example 2: Noise file generated by using `.PRINT snnoise onoise` statement. The noise units are $V/\sqrt{\text{Hz}}$:

```
HERTZ  ONoise()
100.0000      1.6699u
526.4105k     46.8034n
1.0527x       45.1538n
1.5790x       44.6096n
```

2.1053x	44.3405n
2.6317x	44.1804n
3.1580x	44.0744n
3.6843x	43.9990n
4.2106x	43.9425n
4.7369x	43.8986n
5.2632x	43.8634n
5.7895x	43.8345n
6.3158x	43.8102n
6.8421x	43.7894n
7.3684x	43.7714n
7.8948x	43.7556n
8.4211x	43.7415n
8.9474x	43.7288n
9.4737x	43.7172n
10.0000x	43.7067n

Example 3: User-generated noise file. Units are in dBc/Hz:

HERTZ	NOISE (DB)
10.0	-120
100.0	-125
1.0k	-130
5.0k	-135
10.0k	-140
100.0k	-150

Power Supply Current and Voltage Noise Sources

You can implement the power supply noise source with G- and E-elements. The G-element for the current noise source and the E-element for the voltage noise source. As noise elements, they are two-terminal elements that represent a noise source connected between two specified nodes.

Syntax

Expression form:

```
Gxxx node1 node2 noise='expression'
Exxx node1 node2 noise='expression'
```

The G noise element represents a noise current source and the E noise element represents a noise voltage source. The `xxx` parameter can be set with a value up to 1024 characters. The `node1` and `node2` are the positive and negative nodes that connect to the noise source. The noise expression can contain the bias, frequency, or other parameters.

Data form:

```
Gxxx node1 node2 noise data=dataname
Exxx node1 node2 noise data=dataname
.data dataname
+ pname1 pname2
```

```
+ freq1 noise1
+ freq2 noise2
+ ...
.enddata
```

The data form defines a basic frequency-noise table. The `.DATA` statement contains two parameters: frequency and noise to specify the noise value at each frequency point. The unit for frequency is hertz, and the unit for noise is A^2/Hz (for G current noise source) or V^2/Hz (for E voltage noise source).

Example

The following netlist shows a 1000-ohm resistor (`g1`) using a G-element. The `g1noise` element, placed in parallel with the `g1` resistor, delivers the thermal noise expected from a resistor. The `r1` resistor is included for comparison; the noise due to `r1` should be the same as the noise due to `g1noise`:

```
* Resistor implemented using g-element
v1 1 0 1
r1 1 2 1k
g1 1 2 cur='v(1,2)*0.001'
g1noise 1 2
+ noise='4*1.3806266e-23*(TEMPER+273.15)*0.001'
rout 2 0 1meg
.ac lin 1 100 100
.noise v(2) v1 1
.end
```

Function Approximations for Distributed Devices

High-order rational function approximations constructed for distributed devices used at RF frequencies are obtained in the pole-residue form (also known as Foster canonical form). The popular method of recursive convolution also uses this form.

The PrimeSim Continuum supports the pole-residue form for its frequency-dependent controlled sources (G and E-elements). You can enter the pole-residue form directly without converting to another form.

The following sections discuss these topics:

- [Foster Pole-Residue Form for Transconductance or Gain](#)
- [Advantages of Foster Form Modeling](#)
- [G and E-element Syntax](#)

Foster Pole-Residue Form for Transconductance or Gain

The Foster pole-residue form for transconductance $G(s)$ or gain $E(s)$ has the form:

Equation 12

$$G(s) = k_0 + k_1 s + \sum_{i=1}^N \left(\frac{A_i}{s - p_i} + \frac{A_i^*}{s - p_i^*} \right)$$

where

- k_0, k_1 are real constants.
- Residues A_i and poles p_i are complex numbers (or real as a special case of complex).
- Asterisk (*) denotes the expression's complex conjugate.

Advantages of Foster Form Modeling

The advantages of Foster canonical form modeling are

- Models high-order systems. It can theoretically model systems having infinite poles without numerical problems.
- Equivalent to Laplace and Pole-zero models.
- Popular method of recursive convolution uses this form.

G and E-element Syntax

Transconductance $G(s)$ form

```
Gxxxx n+ n- FOSTER in+ in- k0 k1
+ (Re{A1}, Im{A1}) / (Re{p1}, Im{p1})
+ (Re{A2}, Im{A2}) / (Re{p2}, Im{p2})
+ (Re{A3}, Im{A3}) / (Re{p3}, Im{p3})
+ ...
```

Gain $E(s)$ form

```
Exxxx n+ n- FOSTER in+ in- k0 k1
+ (Re{A1}, Im{A1}) / (Re{p1}, Im{p1})
+ (Re{A2}, Im{A2}) / (Re{p2}, Im{p2})
+ (Re{A3}, Im{A3}) / (Re{p3}, Im{p3})
+ ...
```

In this syntax, parentheses, commas, and slashes are separators—they have the same meaning as a space. A pole-residue pair is represented by four numbers (real and imaginary part of the residue, then real and imaginary part of the pole).

You must ensure that $\text{Re}[p_i] < 0$; otherwise, the simulations will certainly diverge. Also, it is a good idea to ensure passivity of the model (for an N-port admittance matrix Y , $\text{Re}\{Y\}$ should be positive-definite), or the simulation is likely to diverge).

Example

To represent a $G(s)$ in the form:

Equation 13

$$G(s) = 0.001 + 1 \times 10^{-12}s + \frac{0.0008}{s + 1 \times 10^{10}} + \frac{(0.001 - j0.006)}{s - (-1 \times 10^8 + j1.8 \times 10^{10})} + \frac{(0.001 + j0.006)}{s - (-1 \times 10^8 - j1.8 \times 10^{10})}$$

You would input:

```
G1 1 0 FOSTER 2 0 0.001 1e-12
+(0.0004, 0)/(-1e10, 0) (0.001, -0.006)/(-1e8, 1.8e10)
```

Note:

In the case of a real poles, half the residue value is entered because it is essentially applied twice. In the above example, the first pole-residue pair is real, but you still must write it as “ $A1/(s-p1)+A1/(s-p1)$ ”; therefore, 0.0004 is entered rather than 0.0008.

Complex Signal Sources and Stimuli

To predict radio-frequency integrated circuit (RFIC) performance, some analyses require simulations that use representative RF signal sources. Among the representative sources available in the PrimeSim Continuum solution advanced analog analyses is the complex modulated RF source. Also known as the *Vector Modulated* source, it allows digital modulation of an RF carrier using in-phase and quadrature components created from a binary data stream.

The following sections discuss these topics:

- [Vector-modulated RF \(VMRF\) Source](#)
- [Voltage and Current Source Elements](#)

Vector-modulated RF (VMRF) Source

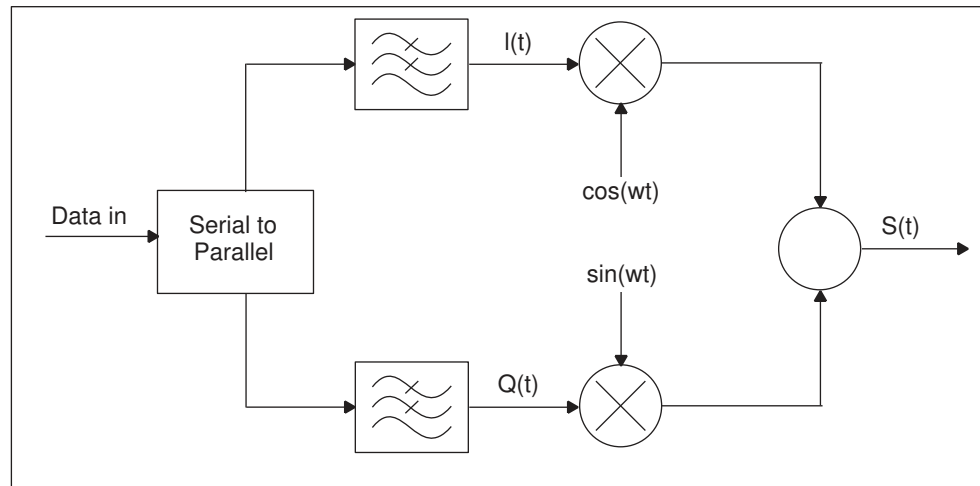
Digital RF waveforms are typically constructed by modulating an RF carrier with in-phase (I) and quadrature (Q) components. In the PrimeSim Continuum solution advanced analog analyses, this is accomplished using the Vector Modulated RF (VMRF) signal source.

The VMRF signal source function is supported both for independent voltage and current sources (V and I elements), and with controlled sources (E, F, G, and H elements):

- When used with independent sources, a baseband data stream can be input in binary or hexadecimal format, and the scheme used to divide the data into I and Q signals can be specified.
- With controlled VMRF sources, the modulating I and Q signals can be separately specified with other signal sources (such as a PWL source) and then used as control inputs into the VMRF source.

VMRF Implementation

The VMRF source is a mathematical implementation of the following block diagram:



The following equation calculates the time and frequency domain stimuli from the quadrature modulated signal sources:

Equation 14

$$s(t) = I(t) \cos(2\pi f_c t + \phi_0) - Q(t) \sin(2\pi f_c t + \phi_0)$$

The discrete ideal I (in-phase) and Q (quadrature) signal components are digital. Discrete values allow uniform scaling of the overall signal. The PrimeSim Continuum solution advanced analog analyses generates data streams for the I and Q signals based on interpreting the data string, breaking the data string into a binary representation, and then using the bit pairs to assign values for the I and Q data streams.

For BPSK (binary phase shift keying) modulation, the discrete signals are scaled so that $\sqrt{I^2 + Q^2} = 1$:

Data In	I Data	Q Data
0	$-\frac{1}{\sqrt{2}}$	$-\frac{1}{\sqrt{2}}$
1	$\frac{1}{\sqrt{2}}$	$\frac{1}{\sqrt{2}}$

For QPSK (quadrature phase shift keying) modulation, the data stream is broken into bit pairs to form the correct I and Q values. This function is represented as the serial to parallel converter:

Data In	I Data	Q Data
00	$-\frac{1}{\sqrt{2}}$	$-\frac{1}{\sqrt{2}}$
01	$-\frac{1}{\sqrt{2}}$	$\frac{1}{\sqrt{2}}$
10	$\frac{1}{\sqrt{2}}$	$-\frac{1}{\sqrt{2}}$
11	$\frac{1}{\sqrt{2}}$	$\frac{1}{\sqrt{2}}$

To generate a continuous-time waveform, the VMRF source takes the resulting digital I and Q data streams and passes them through ideal filters. Rectangular and Nyquist (raised-cosine) filter options are available. Thus, the output waveforms are band-limited according to the specified data rate.

Voltage and Current Source Elements

The V and I elements can include VMRF signal sources that you can use to generate BPSK and QPSK waveforms.

The following sections discuss these topics:

- [V and I Element Syntax and Example](#)
- [E, F, G, and H Element Syntax and Example](#)

V and I Element Syntax and Example

```
Vxxx n+ n- VMRF [ ( ) AMP=sa FREQ=fc PHASE=ph MOD=MOD
+ FILTER=FIL FILCOEF=filpar RATE=Rb BITSTREAM=data
+ [TRANFORHB=0/1] [ ] ]
```

```
Ixxx n+ n- VMRF [ ( ) AMP=sa FREQ=fc PHASE=ph MOD=MOD
+ FILTER=FIL FILCOEF=filpar RATE=Rb BITSTREAM=data
+ [TRANFORHB=0/1] [ ] ]
```

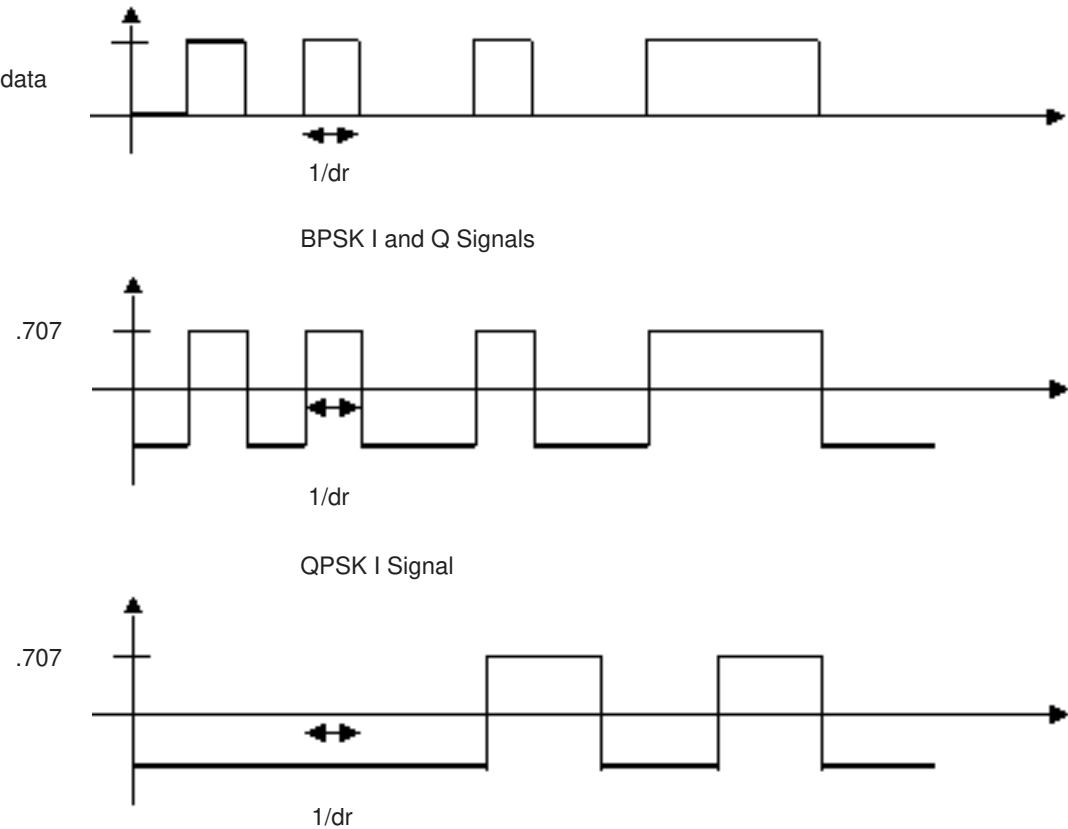
Parameter	Description
Vxxx	Independent voltage source.
Ixxx	Independent current source.
n+ n-	Positive and negative controlled source connecting nodes.
VMRF	Keyword that identifies and activates the Vector Modulated RF signal source.
AMP	Signal amplitude (in volts or amps).
FREQ	Carrier frequency in hertz. Set fc=0.0 to generate baseband I/Q signals. For harmonic balance analysis, the frequency spacing must coincide with the .HB TONES settings.
PHASE	Carrier phase (in degrees). If fc=0.0, - ph=0 and baseband I(t) is generated - ph=-90 and baseband q(t) is generated - Otherwise, $s(t) = i(t) \cos(\phi_0) - q(t) \sin(\phi_0)$
MOD	One of the following keywords identifies the modulation method used to convert a digital stream of information to I(t) and Q(t) variations: - BPSK (binary phase shift keying) - QPSK (quadrature phase shift keying)
FILTER	One of the following keywords identifies the method used to filter the I and Q signals before modulating the RF carrier signal: - COS (raised cosine Nyquist filter) - RECT (rectangular filtering)
FILCOEF	Filter parameter for the COS filter: $0 \leq \text{filpar} \leq 1$
RATE	Bit rate for modulation (bits per second): - For BPSK modulation, the data rate and the symbol rate are the same. - For QPSK modulation, the symbol rate is half the data rate. The Rb value must be greater than zero.

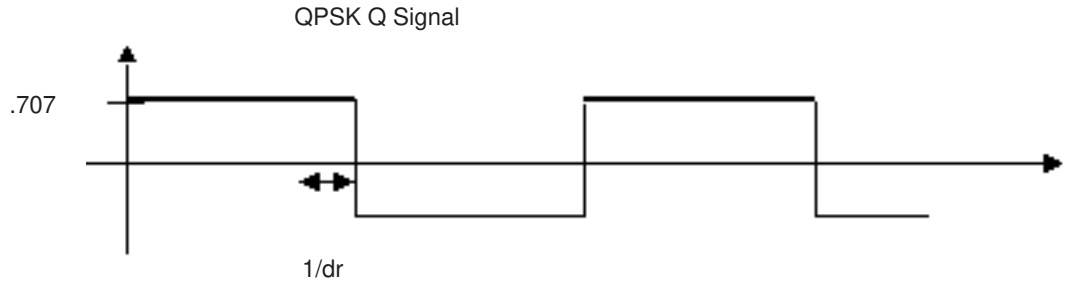
Parameter	Description
BITSTREAM	A binary (b) or hexadecimal (h) string that represents an input data stream. Valid data string characters are: • 0 or 1 for binary (b) mode. • 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, a, b, c, d, e, or f for hexadecimal (h) mode. For example: • 01010011b (binary) • 0F647A30E9h (hexadecimal)

You also can use the standard V source and I source options for nontransient simulations (such as `DC=val` and `AC=mag, ph`) a with the VMRF source.

Example

BITSTREAM=01010010011100b





The R_b parameter represents the data rate. The associated symbol rate represents how fast the I and Q data streams change. The period for each bit of data is:

Equation 15

$$T_b = \frac{1}{R_b}$$

The symbol rate depends on whether you select **BPSK** or **QPSK** modulation

- For **BPSK**, the symbol rate is the same as the data rate:

$$R_s^{BPSK} = R_b$$

- For **QPSK** modulation, two bits are used to create each symbol so the symbol rate is half the data rate.

$$R_s^{QPSK} = \frac{R_b}{2}$$

The period for each symbol is computed as:

Equation 16

$$T_s = \frac{1}{R_s}$$

This value is necessary for establishing the characteristics of Nyquist filters.

The following equation calculates the raised cosine (COS) filter response:

Equation 17

$$H_{rc}(f) = \int_0^{T_s} T_s \cos^2 \left[\frac{\pi T_s}{2\alpha} \left(|f| - \frac{1-\alpha}{2T_s} \right) \right] \frac{1-\alpha}{2T_s} \leq |f| \leq \frac{1+\alpha}{2T_s}$$

$$|f| > \frac{1+\alpha}{2T_s}$$

The VMRF signal source is designed primarily for TRAN and HB analyses and can generate baseband signals. You also can specify DC and AC values as with any of the other PrimeSim Continuum signal source

- In DC analysis, the VMRF source is a constant DC source.
- In AC analysis, the source is a short or an open, unless you specify an AC value.
- In HB analysis, you must specify `.OPTION TRANFORHB` on the source statement line. The `TRANFORHB` option supports the VMRF signal source as well as the SIN, PULSE, and PWL sources.

The VMRF quadrature signal source typically involves an HF carrier signal that is modulated with a baseband signal on a much different time scale. You must set source and simulation control parameters appropriately to avoid time-consuming simulations in both the time and frequency domains.

E, F, G, and H Element Syntax and Example

For E, F, G, and H elements, you can use the VMRF function to modulate I(t) and Q(t) signals with a RF carrier signal. The I and Q signal are driven by PWL sources that might be generated by an external tool, such as MATLAB. The PWL source accepts a text file containing time and voltage (or current) pairs.

When the VMRF function is used with controlled sources, it is anticipated that the in-phase (I) and quadrature (Q) signals are not digital, but continuous-time analog signals. The VMRF function therefore includes no filtering, and merely serves to create the complex modulation on the RF carrier:

```
Exxx n+ n- [VCVS] VMRF [ ( ) Iin+ Iin- Qin+ Qin- FREQ=fc
+ PHASE=ph [SCALE=A] [ ] ]
```

```
Fxxx n+ n- [CCCS] VMRF [ ( ) VI VQ FREQ=fc PHASE=ph
+ [SCALE=A] [ ] ]
```

```
Gxxx n+ n- [VCCS] VMRF [ ( ) Iin+ Iin- Qin+ Qin- FREQ=fc
+ PHASE=ph [SCALE=A] [ ] ]
```

```
Hxxx n+ n- [CCVS] VMRF [ ( ) VI VQ FREQ=fc PHASE=ph
+ [SCALE=A] [ ] ]
```

Parameter	Description
Exxx	Voltage-controlled voltage source.
Fxxx	Current-controlled current source.
Gxxx	Voltage-controlled current source.
Hxxx	Current-controlled current source.

Parameter	Description
VCVS	Keyword for voltage-controlled voltage source.
CCCS	Keyword for current-controlled current source.
VCCS	Keyword for voltage-controlled current source.
CCVS	Keyword for current-controlled current source.
n+ n-	Positive and negative controlled source connecting nodes.
VMRF	Keyword that identifies and activates the vector-modulated RF signal source.
Iin+ Iin-	Node names for input I(t) signal.
Qin+ Qin-	Node names for input Q(t) signal.
VI VQ	
FREQ	Carrier frequency in hertz. Set fc=0.0 to generate baseband I/Q signals.
PHASE	Carrier phase (in degrees). If fc=0.0, - ph=0 and baseband I(t) is generated - ph=-90 and baseband Q(t) is generated
SCALE	Unit-less amplitude scaling parameter.

Example

```
Emod1 inpl inn1 VMRF It_plus It_neg Qt_plus Qt_neg
+ freq=1g phase=0 scale=1.5
```

SWEEPBLOCK in Sweep Analyses

You can use the `.SWEEPBLOCK` statement to specify complicated sweeps. Sweeps affect:

- DC sweep analysis
- Parameter sweeps around TRAN, AC, or HB analyses
- Frequency values used in AC or HBAC analyses

Currently, the PrimeSim Continuum solution supports the following types of sweeps:

- Linear sweeps: sweeps a variable over an interval with a constant increment. The syntax is one of the following:
 - Variable *start stop increment*
 - Variable *lin npoints start stop*

- Logarithmic sweeps: sweeps a variable over an interval. To obtain each point, this sweep multiplies the previous point by a constant factor. You can specify the factor as a number of points per decade or octave as in:
 - Variable *dec npoints start stop*
 - Variable *oct npoints start stop*
- Point sweeps: a variable takes on specific values that you specify as a list. The syntax is:

```
variable poi npoints p1 p2 ...
```

- Data sweeps: a `.DATA` statement identifies the swept variables and their values. The syntax is:

```
data=dataname
```

You can use the `SWEEPBLOCK` feature to combine linear, logarithmic, and point sweeps which creates more complicated sets of values over which a variable is swept.

The `.TRAN`, `.AC`, `.DC`, and `.HB` commands can specify `SWEEPBLOCK=swblockname` as a sweep instead of `LIN`, `DEC`, `OCT`, and so forth. Also, you can use `SWEEPBLOCK` for frequency sweeps with the `.AC`, `.HBAC`, `.PHASENOISE`, and `.HBNOISE` commands.

All commands that can use `SWEEPBLOCK` must refer to the `SWEEPBLOCK` sweep type. In addition, you must specify `SWEEPBLOCK` as one of the syntax types allowed for frequency sweeps with the `.HBAC`, `.PHASENOISE`, and `.HBNOISE` commands.

The following sections discuss these topics:

- [Input Syntax](#)
- [Using SWEEPBLOCK in a DC Parameter Sweep](#)
- [Using in Parameter Sweeps in TRAN, AC, and HB Analyses](#)
- [Using in Parameter Sweeps in HBAC Analyses](#)
- [Limitations](#)

Input Syntax

The `SWEEPBLOCK` feature creates a sweep whose set of values is the union of a set of linear, logarithmic, and point sweeps. To specify the set of values in the `SWEEPBLOCK`, use the `.SWEEPBLOCK` command. This command also assigns a name to the `SWEEPBLOCK`. For example:

```
.SWEEPBLOCK swblockname sweepspec [sweepspec  
+ [sweepspec [...]]]
```

You can use `SWEEPBLOCK` to specify DC sweeps, parameter sweeps, AC and HBAC frequency sweeps, or wherever the PrimeSim Continuum solution accepts sweeps.

You can specify an unlimited number of *sweepspec* parameters. Each *sweepspec* can specify a linear, logarithmic, or point sweep by using one of the following forms:

```
start stop increment  
lin npoints start stop  
dec npoints start stop  
oct npoints start stop  
poi npoints p1 p2 ...
```

Example

The following example specifies a logarithmic sweep from 1 to 1e9 with more resolution from 1e6 to 1e7:

```
.sweepblock freqsweep dec 10 1 1g dec 1000 1meg 10meg
```

Using SWEEPBLOCK in a DC Parameter Sweep

To use the sweepblock in a DC parameter sweep, use the following syntax:

```
.DC swblockname sweepspec [sweepspec [sweepspec [...]]]
```

Each *sweepspec* can be a linear, logarithmic, point, or data sweep, or it can be in the form:

```
variable SWEEPBLOCK=swblockname
```

The `SWEEPBLOCK` syntax sweeps the specified variable over the values contained in the `SWEEPBLOCK`.

Example

```
.dc vin1 0 5 0.1 vin2 sweepblock=vin2vals
```

Using in Parameter Sweeps in TRAN, AC, and HB Analyses

To use the sweepblock in parameter sweeps on `.TRAN`, `.AC`, and `.HB` commands and any other commands that allow parameter sweeps, use the following syntax:

```
variable sweepblock=swblockname
```

Examples

Example 1

```
.tran 1n 100n sweep rout sweepblock=rvals
```

AC and HBAC analysis frequency sweeps can use `sweepblock=swblockname` to specify the frequency values.

Example 2

```
.ac sweepblock=freqsweep
```

Using in Parameter Sweeps in HBAC Analyses

To use the sweepblock in parameter sweeps on `.HBAC` commands with a logarithmic sweep from 1 to 1 GHz that has more resolution from 10 to 100 MHz, and any other commands that allow parameter sweeps, use the following syntax:

```
.sweepblock=freqsweep
```

Example

```
.sweepblock freqsweep  
+ dec 10 1 1g+ dec 1000 1meg 10meg.hb tones=1g nharms=6  
.hbac sweepblock=freqsweep
```

Limitations

- You cannot use recursive `SWEEPBLOCK` specifications. That is, a `.SWEEPBLOCK` command cannot refer to another `SWEEPBLOCK` to build its list of values.
- You cannot include data sweeps in a `.SWEEPBLOCK` statement.

Clock Source with Random Jitter

In many applications involving signal integrity, advanced analog analyses, and mixed-signal design, it is desirable to have an ideal signal source, such as a sine wave or square wave that also includes a nonideal random drift in phase (jitter). Such a source is useful for representing nonideal clock sources during time-domain transient simulation. Modeling jitter in this way can be used to examine eye-diagram behavior or study how jitter may propagate through a circuit or system. A source with jitter is useful for representing nonideal clock sources during time-domain transient simulation.

The `RMSJITTER` and `PERJITTER` parameters allow you to add phase and period jitter to `SIN`, `COS`, and `PULSE` time domain sources.

The following sections discuss these topics:

- [Syntax of SIN, COS, and Pulse Sources](#)

Syntax of SIN, COS, and Pulse Sources

The syntax of `SIN` source is:

```
Vxxx n+ n- SIN [(] vo va [freq [td [q [j ]]]] [)]  
+ [RMSJITTER=val PERJITTER=val SEED=val]]
```



```
Ixxxxn+ n- SIN [(] vo va [freq [td [q [j ]]]] [)]
+ [RMSJITTER=val PERJITTER=val SEED=val]]
```

Parameter	Description
Vxxx, Ixxx	Independent voltage/ current source, which exhibits the pulse response.
PULSE	Keyword for a pulsed time-varying source. The short form is PU.
v1	Initial value of voltage or current before the pulse onset (units: volts/amps).
v2	Pulse plateau value (units of volts or amps).
td	Delay (propagation) time in seconds from the beginning of the transient interval to the first onset ramp. Default=0.0
tr	Duration of the onset ramp (in seconds) from the initial value to the pulse plateau value (reverse transit time). Default = TSTEP
tf	Duration of the recovery ramp (in seconds) from the pulse plateau back to the initial value (forward transit time). Default TSTEP
pw	Pulse width (the width of the plateau portion of the pulse), in seconds. Default =TSTOP.
per	Pulse repetition period, in seconds. Default = TSTEP
rmsjitter	RMS value for random phase jitter, in seconds.
perjitter	RMS value for period jitter, adjusts the magnitude of the random time.
seed	Used to generate random number sequences with different seed value. The default value is 1.

The sine wave behavior following the td time delay now becomes:

Equation 18

$$V(t) + e^{(t-t_d)\theta} = V_0 + V_a \cdot \left[\sin 2\pi f_0(t-t_d) + \frac{\pi}{180} \varphi + \phi t - d_d \right]$$

The syntax of COS source is:

```
Vxxx n+ n- COS [(] vo va [freq [td [q][j]]]] [)]
+ [RMSJITTER=val PERJITTER=val SEED=val]]
```

```
Ixxx n+ n- COS [(] vo va [freq [td [q] [j]]]] [)]
+ [RMSJITTER=val PERJITTER=val SEED=val]]
```

The new cosine wave becomes:

Equation 19

$$V(t) + e^{(t-t_d)\theta} = V_0 + V_a \cdot \left[\cos 2\pi f_0(t-t_d + x(t)) + \frac{\pi}{180} \varphi \right]$$

The syntax for the PULSE source is:

```
Vxxx n+ n- PU[LSE] [(|v1 v2 [td [tr] [tf] [pw] [per]]] []) +
[RMSJITTER=val PERJITTER=val SEED=val]]

Ixxx n+ n- PU[LSE] [(|v1 v2 [td [tr] [tf] [pw] [per]]] [])
+ [RMSJITTER=val PERJITTER=val SEED=val]]
```

The effect of jitter on the PULSE source results in random shifts of the rise and fall transitions that takes place at:

$$\text{RISE edge: } td + n \cdot T_0 \leq t \leq td + tr + n \cdot T_0$$

$$\text{FALL edge: } td + pw + n \cdot T_0 \leq t \leq td + pw + tf + n \cdot T_0$$

The jitter effect is equivalent to introducing random shifts in the period T_0 consistent with the 1st order jitter model based on period jitter. A Gaussian random number generator computes the time deviation $x(t)$ after each leading edge of the clock sources. For flexibility, the SEED parameter (integer) is supported for generating different random number sequences when different SEED integers are used for initialization. SEED does not set a fixed time deviation. It only changes the sequence of random samples. The default value for SEED is 1.

An interpretation of PERJITTER is to view it as causing each period of the PULSE/SIN/COS to be a random variable T_j , where period T_j will have a Gaussian distribution about the (mean) given period value of T_0 . The standard deviation of this Gaussian is the PERJITTER value (it is considered RMS period jitter), which results in a bell curve distribution centered about period T_0 .

Apply the following considerations when using PERJITTER:

- T_j should be forced to be between: $0 < T_j < 2 \cdot T_0$ since period cannot go negative, and the curve should be symmetrical.
- It is reasonable to require that $2 \cdot \text{PERJITTER} < T_0$. Otherwise, the jitter would result in very large period changes and many would be $T_j < 0$.
- To establish a waveform reference, the first period should be T_0 (in other words, no jitter in the first period). This helps to establish good eye diagrams.

As an alternative to using a Verilog-A module, you can generate a pseudo-random binary sequence (PRBS) using the following steps:

1. Construct your usual linear feedback shift register (LFSR) generator.
2. Construct a matching (T,tr,tf) PULSE source as a clock, but add jitter to it with the PERJITTER keyword.
3. Use the PULSE source to gate (buffer) the LFSR output (through an ideal AND gate, VCCS, and so forth).

Specifying a Digital Vector File and Mixed Mode Stimuli

The PrimeSim Continuum solution input netlists support digital vector files. A VEC file consists of three parts:

- Vector Pattern Definition section
- Waveform Characteristics section
- Tabular Data section

To incorporate this information into your simulation, include the `.VEC` command in your netlist.

For paths to full demonstration files (`digstim.vec`—2 bit adder with PWL input, `m2bit.sp`, and `m2bit_v.sp`—same as `m2bit.sp`, except uses vector stimulus file, see Benchmark Examples in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

The following sections discuss these topics:

- [Vector Patterns](#)
- [Defining Tabular Data](#)
- [Waveform Characteristics](#)
- [Modifying Waveform Characteristics](#)
- [Using the Context-Based Control Option \(CBC\)](#)
- [Comment Lines and Line Continuations](#)
- [Parameter Usage](#)
- [Digital Vector File Example](#)

Vector Patterns

The *Vector Pattern Definition* section defines the vectors, their names, sizes, signal direction, sequence or order for each vector stimulus, and so on. A RADIX line must occur first and the other lines can appear in any order in this section. All keywords are case-insensitive.

The following is an example Vector Pattern Definition section:

```
; start of Vector Pattern Definition section
RADIX 1111 1111
VNAME A B C D E F G H
IO IIII IIII
TUNIT ns
```

These four lines are required and appear in the first lines of a VEC file:

- **RADIX** defines eight single-bit vectors.
- **VNAME** gives each vector a name.
- **IO** determines which vectors are inputs, outputs, or bidirectional signals. In this example, all eight are input signals.
- **TUNIT** indicates that the time unit for the tabular data to follow is in units of nanoseconds.

For additional information about these keywords, see [Defining Tabular Data on page 164](#).

Defining Tabular Data

Although the *Tabular Data* section generally appears last in a VEC file (after the *Vector Pattern* and *Waveform Characteristics* definitions), this topic describes it first to introduce the definitions of a vector.

The *Tabular Data* section defines (in tabular format) the values of the signals at specified times. Rows in the Tabular Data section must appear in chronological order because row placement carries sequential timing information. Its general format is:

```
time1 signal1_value1 signal2_value1 signal3_value1...
time2 signal1_value2 signal2_value2 signal3_value2...
time3 signal1_value3 signal2_value3 signal3_value3...
.
.
```

Where *timex* is the specified time, and *signaln_valuen* is the value of specific signals at specific points in time. The set of values for a particular signal (over all times) is a vector

which appears as a vertical column in the tabular data and vector table. The set of all *signal1_valuen* constitutes one vector.

For example:

```
11.0 1000 1000
20.0 1100 1100
33.0 1010 1001
```

This example shows that:

- At 11.0 time units, the value for the first and fifth vectors is 1.
- At 20.0 time units, the first, second, fifth, and sixth vectors are 1.
- At 33.0 time units, the first, third, fifth, and eighth vectors are 1.

The following sections discuss these topics:

- [Input Stimuli](#)
- [Expected Output](#)
- [Verilog Value Format](#)
- [Periodic Tabular Data](#)
- [Accumulated Time](#)

Input Stimuli

The PrimeSim Continuum solution converts each input signal into a PWL voltage source and a series resistance. [Table 9](#) shows the legal states for an input signal. Signal values can have any of these legal states.

Table 9 *Legal States for an Input Signal*

State	Description
0	Drive to ZERO (gnd). Resistance set to 0.
1	Drive to ONE (vdd). Resistance set to 0.
Z, z	Floating to HIGH IMPEDANCE. A <code>TRIZ</code> statement defines resistance value.
X, x	Drive to ZERO (gnd). Resistance set to 0.
L	Resistive drive to ZERO (gnd). An <code>OUT</code> or <code>OUTZ</code> statement defines resistance value. Input Stimuli
H	Resistive drive to ONE (vdd). An <code>OUT</code> or <code>OUTZ</code> statement defines resistance value.
U, u	Drive to ZERO (gnd). Resistance set to 0.

Expected Output

The PrimeSim Continuum solution converts each output signal into a `.DOUT` statement in the netlist. During simulation, the PrimeSim Continuum solution compares the actual results with the expected output vectors. If the states are different, an error message appears. The legal states for expected outputs include the values listed in the following table.

Table 10 *Legal States for an Output Signal*

State	Description
0	Expect ZERO.
1	Expect ONE.
X, x	Don't care.
U, u	Don't care.
Z, z	Expect HIGH IMPEDANCE (don't care). Simulation evaluates Z, z as "don't care" because the PrimeSim Continuum solution cannot detect a high impedance state.

For example:

```
...
IO 0000; start of tabular section data
11.0 1001
20.0 1100
30.0 1000
35.0 xx00
```

where:

- The first line is a comment line because of the semicolon character.
- The second line expects the output to be 1 for the first and fourth vectors, while all others are expected to be low.
- At 20 time units, the PrimeSim Continuum solution expects the first and second vectors to be high, and the third and fourth to be low.
- At 30 time units, the PrimeSim Continuum solution expects only the first vector to be high, and all others low.
- At 35 time units, the PrimeSim Continuum solution expects the output of the first two vectors to be "don't care"; it expects vectors 3 and 4 to be low.

Verilog Value Format

The PrimeSim Continuum solution accepts Verilog-sized format to specify numbers. For example:

```
<size> '<base format> <number>
```

where:

- *<size>* specifies the number of bits, in decimal format.
- *<base format>* indicates:
 - binary ('b or 'B)
 - octal ('o or 'O)
 - hexadecimal ('h or 'H).
- *<number>* values are combinations of the 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, and F characters. Depending on what base format you choose, only a subset of these characters might be legal.

You also can use unknown values (X) and high-impedance (Z) in the *<number>* field. An X or Z sets four bits in the hexadecimal base, three bits in the octal base, or one bit in the binary base.

If the most significant bit of a number is 0, X, or Z, the PrimeSim Continuum solution automatically extends the number (if necessary) to fill the remaining bits with 0, X, or Z, respectively. If the most significant bit is 1, the PrimeSim Continuum solution uses 0 to extend it.

For example:

```
4'b1111  
12'hABx  
32'bZ  
8'h1
```

This example specifies values for:

- 4-bit signal in binary
- 12-bit signal in hexadecimal
- 32-bit signal in binary
- 8-bit signal in hexadecimal

Equivalents of these lines in non-Verilog format, are:

```
1111
AB xxxx
ZZZZ ZZZZ ZZZZ ZZZZ ZZZZ ZZZZ ZZZZ ZZZZ
0000 0001
```

Periodic Tabular Data

Tabular data is often periodic, so you do not need to specify the absolute time at every time point. When you specify the `PERIOD` statement, the Tabular Data section omits the absolute times. For more information, see [Defining Tabular Data on page 164](#).

For example, the `PERIOD` statement in the following sets the time interval to 10 ns between successive lines in the tabular data. This is a shortcut when you use vectors in regular intervals throughout the entire simulation:

```
RADIX 1111 1111
VNAME A B C D E F G H
IO IIII IIII
TUNIT ns
PERIOD 10
; start of vector data section
1000 1000
1100 1100
1010 1001
```

Accumulated Time

Timex specifies the vector increment as absolute time. The character `#` can be added before Timex to specify the accumulated time increment in vector file.

For example:

```
11.0 1000 1000
#20.0 1100 1100
#33.0 1010 1001
```

This example shows that:

- At 11.0 time units, the value for the first and fifth vectors is 1.
- At 31.0 time units, the first, second, fifth, and sixth vectors are 1.
- At 64.0 time units, the first, third, fifth, and eighth vectors are 1.

Waveform Characteristics

The *Waveform Characteristics* section defines various attributes for signals, such as the rise or fall time, the thresholds for logic high or low, and so on. For example:


```
TRISE 0.3 137F 0000
TFALL 0.5 137F 0000
VIH 5.0 137F 0000
VIL 0.0 137F 0000
```

The waveform characteristics are based on a bit-mask. Where:

- The `TRISE` (signal rise time) setting of 0.3ns applies to the first four vectors, but not to the last four.
- The example does not show how many bits are in each of the first four vectors, although the first vector is at least one bit.
- The fourth vector is four bits because F is hexadecimal for binary 1111.
- All bits of the fourth vector have a rise time of 0.3ns for the constant you defined in `TUNIT`. This also applies to `TFALL` (fall time), `VIH` (voltage for logic-high inputs), and `VIL` (voltage for logic-low inputs).

Modifying Waveform Characteristics

The `TDELAY`, `IDELAY`, and `ODELAY` statements define the delay time of the signal, relative to the absolute time of each row in the Tabular Data section:

- `TDELAY` applies to the input and output delay time of input, output, and bidirectional signals.
- `IDELAY` applies to the input delay time of input and bidirectional signals.
- `ODELAY` applies to the output delay time of output and bidirectional signals.

The other statements are defined as follows:

- `SLOPE` statement specifies the rise and fall times for the input signal. To specify the signals to which the *slope* applies, use a mask.
- `TFALL` statement sets an input fall time for specific vectors.
- `TRISE` statement sets an input rise time for specific vectors.
- `TUNIT` statement defines the time unit.
- `TRIZ` statement specifies the output impedance, when the signal (for which the mask applies) is in tristate; `TRIZ` applies only to the input signals.
- `VIH` statement specifies the logic-high voltage for each input signal to which the mask applies.
- `VIL` statement specifies the logic-low voltage for each input signal to which the mask applies.

- the `VREF` statement (similar to the `TDELAY` statement) specifies the name of the reference voltage for each input vector to which the mask applies. `VREF` applies only to input signals.
- `VTH` statement (similar to the `TDELAY` statement) specifies the logic threshold voltage for each output signal to which the mask applies. The threshold voltage determines the logic state of output signals for comparison with the expected output signals.
- `VOH` statement specifies the logic-high voltage for each output signal to which the mask applies.
- `VOL` statement specifies the logic-low voltage for each output signal to which the mask applies.
- `idelay_f` specifies the delay time for falling input vectors, relative to the absolute time of each row in the tabular data section. The delay time can be positive or negative. The default is set to the value specified in the `delay` or `tdelay` statement.

Example

```
idelay_f 10;
```

- `idelay_r` specifies the delay time for rising input vectors, relative to the absolute time of each row in the tabular data section. The delay time can be positive or negative. The default is set to the value specified in the `delay` or `tdelay` statement.

Example

```
idelay_r 10;
```

The `OUT` and `OUTZ` keywords are equivalent and specify output resistance for each signal (for which the mask applies); `OUT` (or `OUTZ`) applies only to input signals.

Using the Context-Based Control Option (CBC)

The `OPTION CBC` (Context-Based Control) specifies the direction of bidirectional signals. A bidirectional signal is an input if its value is 0, 1, or Z; conversely, a bidirectional signal is an output if its value is H, L, U, or X.

For example:

```
RADIX 1 1 1
IO I O B
VNAME A Z B
OPTION CBC
10.0 0 X L
20.0 1 1 H
30.0 1 0 Z
```

This example sets up three vectors, named **A**, **Z**, and **B**. Vector **A** is an input, vector **Z** is an output, and vector **B** is a bidirectional signal (defined in the **IO** statement).

The **OPTION CBC** line turns on context-based control. The next line sets vector **A** to a logic-low at 10.0 ns, and vector **Z** is “do not care.” Because the **L** value is under vector **B**, the PrimeSim Continuum solution expects a logic-low output.

At 20 ns, vector **A** transitions high, and the expected outputs at vectors **Z** and **B** are high. Finally, at 30 ns, the PrimeSim Continuum solution expects vector **Z** to be low, vector **B** changes from an output to a high-impedance input, and vector the **A** signal does not change.

Comment Lines and Line Continuations

Any line in a VEC file that begins with a semicolon (;) is a comment line. Comments also can start at any point along a line. The PrimeSim Continuum solution ignores characters after a semicolon. For example:

```
; This is a comment line  
radix 1 1 4 1234 ; This is a radix line
```

As in netlists, any line in a VEC file that starts with a plus sign (+) is a continuation from the previous line.

Parameter Usage

You can use **.PARAM** statements with some **VEC** statements when you run the PrimeSim Continuum solution. These **VEC** statements fall into the three groups, which are described in the following sections. No other **VEC** statements, except those identified here, support **.PARAM** statements.

First Group

- **PERIOD**
- **TDELAY**
- **IDELAY**
- **ODELAY**
- **SLOPE**
- **TRISE**
- **TFALL**

For these statements, the `TUNIT` statement defines the time unit. If you do not include a `TUNIT` statement, the default time unit value is nanoseconds.

Do not specify absolute unit values in a `.PARAM` statement. For example, if in your netlist:

```
.param myperiod=10ns           $ 'ns' makes this incorrect
```

And in your VEC file:

```
tunit ns
period myperiod
```

What you wanted for the time period is 10 ns; however, because you specified absolute units, 1e-8ns is the value used. In this example, the correct form is:

```
.param myperiod=10
```

Second Group

- `OUT` or `OUTZ`
- `TRIZ`

In these statements, the unit is ohms:

- If you do not include an `OUT` (or `OUTZ`) statement, the default is 0.
- If you do not include a `TRIZ` statement, the default is 1000 M.

The `.PARAM` definition for this group follows the PrimeSim Continuum solution syntax.

For example, if in your netlist:

```
.param myout=10                $ means 10 ohm
.param mytriz= 10Meg           $ means 10,000,000 ohm, don't
$ confuse Meg with M, M means 0.001
```

And in your VEC file:

```
out myout
triz mytriz
```

Then, the PrimeSim Continuum solution returns 10 ohm for `OUT` and 10,000,000 ohm for `TRIZ`.

Third Group

- `VIH`
- `VIL`
- `VOH`

- VOL
- VTH

In these statements, the unit is volts.

- If you do not include a VIH statement, the default is 3.3.
- If you do not include a VIL statement, the default is 0.0.
- If you do not include a VOH statement, the default is 2.64.
- If you do not include a VOL statement, the default is 0.66.
- If you do not include a VTH statement, the default is 1.65.

Digital Vector File Example

```
; specifies # of bits associated with each vector
radix 1 2 444
;*****
; defines name for each vector. For multi-bit vectors,
; innermost [] provide the bit index range, MSB:LSB
vname v1 va[[1:0]] vb[12:1]
;actual signal names: v1, va[0], va[1], vb1, vb2, ... vb12
;*****
; defines vector as input, output, or bi-directional
io i o bbb
; defines time unit
tunit ns
;*****
; vb12-vb5 are output when 'v1' is 'high'
enable v1 0 0 FF0
; vb4-vb1 are output when 'v1' is 'low'
enable ~v1 0 0 00F
;*****
; all signals have a delay of 1 ns
; Note: do not put the unit (such as ns) here again.
; PrimeSim HSPICE multiplies this value by the specified 'tunit'.
tdelay 1.0
; va[1] and va[0] signals have 1.5ns delays
tdelay 1.5 0 3 000
;*****
; specify input rise/fall times (if you want different
; rise/fall times, use the trise/tfall statement.)
; Note: do not put the unit (such as ns) here again.
; PrimeSim HSPICE multiplies this value by the specified 'tunit'.
slope 1.2
;*****
; specify the logic 'high' voltage for input signals
vih 3.3 1 0 000
vih 5.0 0 0 FFF
```

```
; to specify logic low, use 'vil'
;*****
; va & vb switch from 'lo' to 'hi' at 1.75 volts
vth 1.75 0 1 FFF

;*****
; tabular data section
10.0 1 3 FFF
20.0 0 2 AFF
30.0 1 0 888
```

Using the VCD Control File

The VCD control file maps the variable names in the VCD file to the node names in the schematic netlist. All variable names in the VCD control file are case-sensitive, as is the VCD file itself. You can also define various attributes for vectors. If more than one attribute statement of a type is applied to a particular vector, the last attribute takes precedence. A warning is generated if a conflict arises. A comment begins with a semi-colon character (;) at the beginning of a line, or directly in front of a comment—anywhere within the line.

The following procedure enables you to create a VCD control file:

1. Define the bus syntax in the schematic netlist using the `#format` statement.
2. Define the types of vectors in the VCD file to be mapped to the schematic netlist: the `#input` statement defines the input; the `#output` statement defines the output; the `#bidirectional` statement defines the bidirectional vector.
3. Map the variable names defined in the VCD control file to the schematic netlist using the `#alias` statement.
4. Define various waveform characteristics for vectors.

The following sections discuss these topics:

- [Defining Bus Syntax – #format](#)
- [Defining Vector Types – #input, #output, and #bidirectional](#)
- [Defining Mapping Information – #alias](#)
- [Defining Attributes for Vectors](#)
- [Defining the Hierarchical Scope – #scope](#)

Defining Bus Syntax – #format

The `#format` statement specifies the bus naming format. Only one `#format` statement is allowed in a VCD control file.

The syntax is as follows:

```
#format|#fo bus_format
```

The default is `%[#]`. For a character description, see the following table.

Table 11 Character Description

Character	Description
[]	Represents a bus. The bus characters in the VCD control file must match the bus characters used in the schematic netlist. You can use different characters, such as { } or < > to represent a bus.
%	Represents the signal name
#	Represents the bus indexes

The VCD file lists the `tA` variable as follows:

```
$var reg 3 ! tA [2:0] $end
```

This example shows the `#format` statement mapping the `tA` variable to various bus syntax:

Example 1

```
; maps to tA[0] tA[1] tA[02]
#format %[#]

; maps to tA<0> tA<1> tA<02>
#format %<#>

; maps to tA.0 tA.1 tA.02
#format %.#
```

Defining Vector Types – #input, #output, and #bidirectional

The PrimeSim Continuum solution uses all input, output, and bidirectional vectors specified in the VCD control file. Other variables in the VCD file, which are not specified in the VCD control file, are ignored. The full hierarchical name must be specified in the VCD control file to map the variable names in the VCD file. If you prefer not to specify the full hierarchical name, see the [Defining the Hierarchical Scope – #scope](#) section for an alternative method.

The PrimeSim Continuum solution uses input signals as stimulus for the simulation. Each input vector is converted to a PWL voltage source, in series with a resistor. The rise and fall times are used to smoothly transition from one state to another. See the `#trise`,

`#tfall`, and `#slope` statements in the section [Defining Attributes for Vectors](#) for how you define rise time and fall time.

The syntax of `#input` or `.input` is as follows:

```
#input|#in sname1 [... snamen]  
  
.input|.in sname1 [... snamen]
```

Each output vector is converted to an expected output checking statement. During simulation, the actual voltages are converted to a digital value and compared to the expected output checking statement at the specified times. A warning is generated, if the states differ. See the `#voh` and `#vol` statements in the section [Defining Attributes for Vectors](#) for how you define voltage thresholds for logic-high and logic-low.

The syntax of `#output` or `.output` is as follows

```
#output|#out sname1 [... snamen]  
  
.output|.out sname1 [... snamen]
```

Every bidirectional vector requires the `enable` signal(s) to control the direction. The `enable` signal controls the bidirectional vector to behave as input or output for specified conditions.

The syntax of `#bidirectional` is as follows

```
#bidirectional|#bi sname1 [... snamen] \[in|out enable_condition]  
  
.bidirectional|.bi sname1 [... snamen] [in|out enable_condition]
```

Argument	Description
<i>sname1</i> ... <i>sname<i>n</i></i>	Vector names at which the <code>#in</code> , <code>#out</code> , or <code>#bi</code> statement is applied.
<i>enable_condition</i>	The <code>enable</code> condition. The bidirectional vector becomes input (if <code>in</code> is specified) or output (if <code>out</code> is specified), if the <code>enable_condition</code> is true. The following table lists the logic expression operators that can be used to define the <code>enable_condition</code> .

Table 12 *Logic Expression Operators*

Expression Operator	Description
<code>~</code> (or) <code>!</code>	NOT
<code>&</code> (or) <code>&&</code>	AND

Table 12 Logic Expression Operators (Continued)

Expression Operator	Description
$\wedge$	Exclusive OR
$ $ (or) $ $	Inclusive OR

The following example declares

- Input vectors are `adder`, `en`, and `bus m_data`
- Output vectors are `c` and `bus sum`
- bidirectional vector `rom_cs` is controlled by `en`. `rom_cs` is output when `en` is logic-high; otherwise, `rom_cs` is input.
- Bidirectional vector `bus io` is controlled by `en` and `c`. `Busio` is input when both `en` and `c` are logic-high; otherwise, it is output.

Example 2

```
#in adder en m_data*
#out c sum*
#bi rom_cs (out en)
#bi io* (in en & c)
```

Defining Mapping Information – #alias

The `#alias` statement maps the variable names in the VCD file into the node names used in a schematic netlist. The following are situations in which you must use the `#alias` statement

- Variable names (in the VCD file) do not match with node names (in the schematic netlist).
- Different hierarchy levels exist between the variable names in the VCD file and the node names in the schematic netlist.

In the following example, the `#alias` statements in the VCD control file convert the variable names in the VCD file to map with the node names in the schematic netlist. See [Table 13](#) for a conversion and mapping description. See [Table 11](#) for a character description.

Example 3

```
#alias tX64 x_64      ! alias #1
#alias t% %           ! alias #2
```

```
#alias tAP% Ap_%      ! alias #3
#alias tAPE% APE%     ! alias #4
```

Table 13 Converting and Mapping Variable Names to Node Names

Variable Name (VCD file)	Node Name (schematic netlist)	#alias matched
tAPEND	APEND	4
tL2CIB	L2CIB	2
tX64	x_64	1
tAPEB	APEB	4
tGBLB	GBLB	2
tAP0	Ap_0	3
tADD	ADD	2
tAPDD1	Ap_DD1	3

Defining Attributes for Vectors

The VCD control file supports statements that define various attributes for vectors. Multiple specifications of each statement are allowed in the VCD control file. If more than one individual statement is specified to a vector, the last statement overrides the previous ones.

The syntax is as follows

```
KEYWORD value [sname1 ... snamen]
```

Argument	Description
KEYWORD	Any keyword listed in Table 14
value	Value at which the KEYWORD is applied
sname1 ... snamen	Vector names at which the KEYWORD is applied. If <i>snamen</i> is not specified, it is applied to all vectors.

Table 14 Keywords

Keyword	Description
#idelay or .idelay	Specifies the delay time only for the input vectors. The delay can be a positive or negative number. The default is set to 0.
#odelay or .odelay	Specifies the delay time only for the output vectors. The delay can be a positive or negative number. The default value is 0.
#outz or .outz	Specifies the resistance of the voltage source when an input or bidirectional vector is driving in a high impedance state. The value must be greater than or equal to zero. The default is 0 ohms.
#tfall or .tfall	Specifies the fall time for input vectors to transition from high voltage specified in #vih to low voltage specified in #vil. The value must be greater than zero, and a unit must be specified. The default is set to 0.1ns.
#trise or .trise	Specifies the rise time for input vectors to transition from low voltage specified in #vil to high voltage specified in #vih. The value must be greater than zero, and a unit must be specified. The default is set to 0.1ns.
#triz or .triz	Specifies the resistance of the voltage source when an input or bidirectional vector is in a high impedance state or when a bidirectional vector is in an output state. The value must be greater than or equal to zero. The default value is set to 1000M.
#vih or .vih	Specifies the logic-high voltage for the input vectors or bidirectional vectors when in input mode. The default is set to 3.3 volts.
#vil or .vil	Specifies the logic-low voltage for input vectors or bidirectional vectors when in input mode. The default is set to 0 volts.
#voh or .voh	Specifies the threshold for logic-high voltage for output vectors or bidirectional vectors when in output mode. Used for output checking. A voltage value is considered a logic-high state, if it is greater than or equal to the specified voltage threshold. The default is set to 2.64 volts.

Table 14 Keywords (Continued)

Keyword	Description
<code>#vol</code> or <code>.vol</code>	Specifies the threshold for logic-low voltage for output vectors or bidirectional vectors when in output mode. Used for output checking. A voltage value is considered a logic-low state, if it is less than or equal to the specified voltage threshold. The default is set to 0.66 volts.
<code>#vref</code> or <code>.vref</code>	Specifies the reference voltage name for each input or bidirectional vector when in input mode.

The following example declares

- Logic-high voltage for all input signals is 1.5volts
- Logic-low voltage for all input signals is 0.0 volts
- Threshold for logic-high voltage for all output signals is 1.4 volts
- Threshold for logic-low voltage for all output signals is 0.1volts
- Rise time for all input signals is 0.1 ns
- Fall time for all input signals (except for bus data) is 0.1 ns and bus data is 0.2 ns

Example 4

```
#vih 1.5
#vil 0.0
#voh 1.4
#vol 0.1
#trise 0.1ns
#tfall 0.1ns
#tfall 0.2ns data*
```

Defining the Hierarchical Scope – #scope

The `#scope` statement specifies the scope name in the VCD file, from which the variables are used for mapping. By default, if `#scope` is not specified, the PrimeSim Continuum solution processes the variables from the top level. The `#scope` statement is used to avoid specifying the full hierarchical name to the variable names. See the [Defining Vector Types – #input, #output, and #bidirectional](#) section.

Only one `#scope` statement is allowed in a VCD control file. If there are multiple scopes to be used in the VCD file, you must create multiple VCD control files with one `#scope` statement in each file.

The syntax is as follows

```
#scope scopename
```

The *scopename* argument is the scope name of the VCD file.

The following is an example of the VCD file with the `$scope` statement.

Example 5

```
$scope module top $end
$var wire 1 A data_in $end
$var wire 32 B data_out [31:0] $end

$scope module second_A $end
$var wire 1 ` data_in $end
$var wire 32 ^ data_out [31:0] $end
$upscope $end

$scope module second_B $end
$var wire 1 _ data_in $end
$var wire 32 Y data_out [31:0] $end
$upscope $end

$upscope $end
```

In [Example 6](#), the `data_in` and `data_out` variables are presented in every scope in the VCD file. You want to map the variables in the `top.second_B` module to the schematic netlist. If no `#scope` statement is used, you need to specify the full hierarchical names in the VCD control file as follows:

Example 6

```
#input top.second_B.data_in
#output top.second_B.data_out
```

In [Example 7](#), you avoid specifying the full hierarchical names by using the `#scope` statement:

Example 7

```
#scope top.second_B
#input data_in
#output data_out
```

3

Behavioral Modeling and Controlled Sources

Describes element statements for independent sources, power sources, and digital vector sources.

This chapter also explains each type of element and model statement and provides explicit formulas and examples to show how various combinations of parameters affect the simulation.

The PrimeSim Continuum solution ships over a dozen source examples for your use; see Listing of Demonstration Input Files for paths to demo files and Sources Examples, in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

The following section discuss these topics:

- [Voltage and Current Controlled Elements](#)
- [Voltage-Dependent Voltage Sources — E-elements](#)
- [Voltage-dependent Current Sources — G-elements](#)
- [Current-dependent Current Sources - F-elements](#)
- [Current-dependent Voltage Sources - H-elements](#)
- [Polynomial and Piecewise Linear Functions](#)

Voltage and Current Controlled Elements

The PrimeSim Continuum solution provides two voltage-controlled and two current-controlled sources or elements namely:

- Voltage-controlled voltage source (VCVS), or E-element (see [Voltage-Dependent Voltage Sources — E-elements on page 184](#)).
- Voltage-controlled current source (VCCS), or G-element (see [Voltage-dependent Current Sources — G-elements on page 202](#)).

- Current-controlled current source (CCCS), or F-element (see [Current-dependent Current Sources - F-elements on page 227](#)).
- Current-controlled voltage source (CCVS), or H-element (see [Current-dependent Voltage Sources - H-elements on page 231](#)).

You can use these controlled elements to model:

- MOS transistors
- Bipolar transistors
- Tunnel diodes
- SCRs
- Analog functions, such as:
 - Operational amplifiers
 - Summers
 - Comparators
 - Voltage-controlled oscillators
 - Modulators
 - Switched capacitor circuits

Depending on whether you used the polynomial or piecewise linear functions, the controlled elements can be:

- Linear functions of controlling-node voltages
- Non-linear functions of controlling-node voltages
- Linear functions of branch currents
- Non-linear functions of branch currents

The functions of the E, G, F, and H controlled elements are different.

- The E-element can be:
 - A voltage-controlled voltage source
 - A behavioral voltage source
 - An ideal op-amp.
 - An ideal transformer.

- An ideal delay element.
- A piecewise linear, voltage-controlled, multi-input AND, NAND, OR, or NOR gate.
- The G-element can be:
 - A voltage-controlled current source
 - A behavioral current source
 - A voltage-controlled resistor
 - A piecewise linear, voltage-controlled capacitor
 - An ideal delay element
 - A piecewise linear, voltage-controlled multi-input AND, NAND, OR, or NOR gate
- The F-element can be:
 - A current-controlled current source.
 - An ideal delay element.
 - A piecewise linear, current-controlled, multi-input AND, NAND, OR, or NOR gate.
- The H-element can be:
 - A current-controlled voltage source.
 - An ideal delay element.
 - A piecewise linear, current-controlled, multi-input AND, NAND, OR, or NOR gate.

Voltage-Dependent Voltage Sources — E-elements

This section explains E-element syntax statements and defines their parameters.

The following sections discuss these topics:

- [Voltage-Controlled Voltage Source \(VCVS\)](#)
- [E-element Examples](#)
- [Using the E-element for AC Analysis](#)

Voltage-Controlled Voltage Source (VCVS)

The following sections discuss these topics:

- [Linear](#)
- [Polynomial \(POLY\)](#)
- [PWL](#)
- [Multi-Input Gates](#)
- [Delay Element](#)
- [Laplace Transform](#)
- [Pole-Zero Function](#)
- [Frequency Response Table](#)
- [Foster Pole-Residue Form](#)
- [Behavioral Voltage Source](#)
- [Behavioral Time Derivative Voltage Source](#)
- [Behavioral Voltage Noise Source](#)
- [Ideal Op-Amp](#)
- [Ideal Transformer](#)
- [Z Transform Function Call](#)
- [E-element Parameters](#)

Linear

```
Exxx n+ n- [VCVS] in+ in- gain [MAX=val] [MIN=val]  
+ [SCALE=val] [TC1=val] [TC2=val] [ABS=1] [IC=val]
```

For a description of these parameters, see [E-element Parameters on page 194](#).

Polynomial (POLY)

```
Exxx n+ n- [VCVS] POLY(NDIM) in1+ in1- ...  
+ inndim+ inndim-TC1=val [TC2=val] [SCALE=val]  
+ [MAX=val] [MIN=val] [ABS=1] p0 p1... [IC=val]
```

In this syntax, *dim (dimensions)* ≤ 3 . The PrimeSim Continuum solution has no limitation for dimensions while the PrimeSim Continuum solution advanced analog analyses maximum

dimensions are 3. Otherwise, the PrimeSim Continuum solution reports an error message. For a description of these parameters, see [E-element Parameters on page 194](#). For a description of possible POLY syntaxes, see [Polynomial Functions on page 236](#).

PWL

```
Exxx n+ n- [VCVS] PWL(1) in+ in- [DELTA=val]
+ [SCALE=val] [TC1=val] [TC2=val] x1,y1 x2,y2 ...
+ x100,y100 [IC=val]
```

For a description of these parameters, see [E-element Parameters on page 194](#). For a detailed description of PWL functions, see [Piecewise Linear Function on page 241](#).

Multi-Input Gates

```
Exxx n+ n- [VCVS] gatetype(k) in1+ in1- ... ink+ ink-
+ [DELTA=val] [TC1=val] [TC2=val] [SCALE=val]
+ x1,y1 ... x100,y100 [IC=val]
```

In this syntax, *gatetype* can represent an AND, NAND, OR, or NOR gate and *k* represents the number of inputs to the gate. For a description of these parameters, see [E-element Parameters on page 194](#).

Delay Element

```
Exxx n+ n- [VCVS] DELAY in+ in- TD=val [SCALE=val]
+ [TC1=val] [TC2=val] [NPDELAY=val]
```

For a description of these parameters, see [E-element Parameters on page 194](#).

Laplace Transform

Voltage Gain H(s):

```
Exxx n+ n- LAPLACE in+ in- k0, k1, ..., kn / d0, d1, ..., dm
+ [SCALE=val] [TC1=val] [TC2=val]
```

For a description of these parameters, see [E-element Parameters on page 194](#).

H(s) is a rational function, in the following form:

Equation 20

$$H(s) = \frac{k_0 + k_1s + \dots + k_ns^n}{d_0 + d_1s + \dots + d_ms^m}$$

You can use parameters to define the values of all coefficients ($k_0, k_1, \dots, d_0, d_1, \dots$).

Examples

```
Ehipass out 0 LAPLACE in 0 0.0,0.0,0.0,1.0 / 1.0,2.0,2.0,1.0
```

The `Ehipass` element statement describes a third-order high-pass filter with the transfer function:

Equation 21

$$H(s) = \frac{s^3}{1 + 2s + 2s^2 + s^3}$$

For full demonstration netlists using E-elements for Laplace transforms, use the paths shown as follows in Filters Examples in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*:

- `lowloss.sp` (RL line model using Laplace behavioral element)
- `rcline.sp` (RC line model using Laplace elements)
- `ninth.sp` (active low pass filter using Laplace elements)

Pole-Zero Function

Voltage Gain $H(s)$:

```
Exxx n+ n- POLE in+ in- a az1, fz1, ..., azn, fzn / b,  
+ ap1, fp1, ..., apm, fpm [SCALE=val] [TC1=val]  
+ [TC2=val]
```

For a description of these parameters, see [E-element Parameters on page 194](#).

The following equation defines $H(s)$ in terms of poles and zeros:

Equation 22

$$H(s) = \frac{a \cdot (s + \alpha_{z1} - j2\pi f_{z1}) \dots (s + \alpha_{zn} - j2\pi f_{zn}) (s + \alpha_{zn} + j2\pi f_{zn})}{b \cdot (s + \alpha_{p1} - j2\pi f_{p1}) \dots (s + \alpha_{pm} - j2\pi f_{pm}) (s + \alpha_{pm} + j2\pi f_{pm})}$$

The complex poles or zeros are in conjugate pairs. The element description specifies only one of them, and the program includes the conjugate. You can use parameters to specify the a , b , α , and f values.

Examples

```
Elow_pass out 0 POLE in 0 1.0 / 1.0, 1.0,0.0 0.5,0.1379
```

The `Elow_pass` statement describes a low-pass filter with the transfer function:

Equation 23

$$H(s) = \frac{1.0}{1.0 \cdot (s+1)(s+0.5+j2\pi \cdot 0.1379)(s+0.5-(j2\pi \cdot 0.1379))}$$

For a full demonstration netlist of the G- and E-elements used in a behavioral model, use the path shown for `low_pass.sp` in Filters Examples in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

Frequency Response Table

Voltage Gain $H(s)$:

```
Exxx n+ n- FREQ in+ in- f1, a1, f1, ..., fi, ai, f1
+ [DELF=val] [MAXF=val] [SCALE=val] [TC1=val]
+ [TC2=val] [LEVEL=val] [ACCURACY=val]
```

For a description of these parameters, see [E-element Parameters on page 194](#).

where:

- Each `fi` is a frequency point, in hertz.
- `ai` is the magnitude, in dB.
- `f1` is the phase, in degrees.

At each frequency, the PrimeSim Continuum solution uses interpolation to calculate the network response, magnitude, and phase. The PrimeSim Continuum solution interpolates the magnitude (in dB) logarithmically as a function of frequency. It also interpolates the phase (in degrees) linearly as a function of frequency.

Equation 24

$$|H(j2\pi f)| = \left(\frac{a_i - a_k}{\log f_i - \log f_k} \right) (\log f - \log f_i) + a_i$$

Equation 25

$$\angle H(j2\pi f) = \left(\frac{\phi_i - \phi_k}{f_i - f_k} \right) (f - f_i) + \phi_i$$

Examples

```
Eftable output 0 FREQ input 0
+ 1.0k -3.97m 293.7
+ 2.0k -2.00m 211.0
+ 3.0k 17.80m 82.45
+ ... ..
+ 10.0k -53.20 -1125.5
```

- The first column is frequency, in hertz.
- The second column is magnitude, in dB.
- The third column is phase, in degrees.

Set the `LEVEL` to 1 for a high-pass filter.

Set the last frequency point to the highest frequency response value that is a real number with zero phase.

You can use parameters to set the frequency, magnitude, and phase in the table.

For a full demonstration file showing a behavioral model using a G- table element, see the path to the demo file `phaseshift.sp` in Filters Examples in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

Foster Pole-Residue Form

Gain E(s) form

```
Exxx n+ n- FOSTER in+ in- k0 k1
+ (Re{A1}, Im{A1}) / (Re{p1}, Im{p1})
+ (Re{A2}, Im{A2}) / (Re{p2}, Im{p2})
+ (Re{A3}, Im{A3}) / (Re{p3}, Im{p3})
+ ...
```

For a description of these parameters, see [E-element Parameters on page 194](#).

In the above syntax, parenthesis, commas, and slashes are separators—they have the same meaning as a space. Four numbers represent a pole-residue pair (real and imaginary part of the residue, then real and imaginary part of the pole).

You must ensure that $\text{Re}\{p_i\} < 0$; otherwise, the simulations diverge. Also, ensure passivity of the model (for an N-port admittance matrix Y , $\text{Re}\{Y\}$ should be positive-definite), or the simulation is likely to diverge.

Behavioral Voltage Source

The E-element can be used as a behavioral voltage source where the voltage source can be bias and/or frequency dependent. For a behavioral voltage source, use a constant value for '*expression*'.

For voltage sources using expressions, the E-element takes the form:

```
Exxx node1 node2 vol='expression'
```

You can set the `xxx` parameter with a value of up to 1024 characters. The `node1` and `node2` are the positive and negative nodes that connect to the voltage source. The voltage expression can contain the bias, frequency, or other parameters, and involve node voltages and currents through voltage sources.

Behavioral Time Derivative Voltage Source

The E-element can be used as a behavioral time derivative voltage source. The function `ddt( )` computes the time derivative, $\frac{d}{dt}x(t)$, of the argument where the argument is an expression that can include state and input variables, parameters, node voltages or currents.

For voltage sources using the `ddt( )` function, the E-element takes the form:

```
Exxx node1 node2 vol=ddt('expression')
```

Behavioral Voltage Noise Source

You can implement a behavioral voltage noise source with an E-element. As noise elements, these are two-terminal elements that represent a noise source connected between two specified nodes. The E-element specifies a voltage noise source with the noise value $V^{2/Hz}$. The noise source can be bias- and/or frequency-dependent.

For noise sources using expressions, the E-element takes the form:

```
Exxx n+ n- noise='expression'
```

where `Exxx` is the voltage-controlled element name, which must begin with “E”, followed by up to 1023 alphanumeric and special characters. `n+` is the positive node. `n-` is the negative node. `noise='expression'` can contain the bias, frequency, or other parameters.

Data form:

```
Exxx n+ n- noise data=dataname [noise_carrier=devname]
.DATA dataname
+ pname1 pname2
+ freq1 noise1
+ freq2 noise2
+ ...
.enddata
```

The data form defines a basic frequency-noise table. The `.DATA` statement contains two parameters: frequency and noise to specify the noise value at each frequency point. The unit for frequency is hertz, and the unit for noise is dependent on the pattern of the noise name by the following rule:

Unit	Name_pattern
V/sqrt(Hz)	ONoise, or ONoise(), or ***(M), or ***(MAG)
db/Hz	ONoise(DB), or***(DB), or ***DB
dbc/Hz	PHNoise, or PAC_L(f), or BPN_L(f), or NLP_L(f)
V^2/Hz	Otherwise

It should be noted that when noise unit is dbc/Hz, to determine the true noise level, a carrier power is needed. In this scenario, you must specify the `noise_carrier` parameter to communicate to the simulator device on which the carrier power is calculated. This device must be an independent voltage source or a port. And the noise takes effect only in SN or HB noise analyses. For other types of noise unit, the `noise_carrier` parameter does not work.

Noise files applies the same rule for unit setting. See also [Using Noise Analysis Results as Input Noise Sources on page 144](#).

The PrimeSim Continuum solution supports the following syntaxes:

- `Exxx n1 n2 noise data=dataname[noise_carrier=devname]`
- `Exxx n1 n2 noise data=datablock [noise_carrier=devname]`
- `Exxx n1 n2 noisefile='filename' [noise_carrier=devname]`
- `Exxx n1 n2 noise='expression'`
- `Exxx n1 n2 noise='Table(arg1,f1,v1,f2,v2...)'`
- `Exxx n1 n2 noise='Table(arg1,dotDataBlockName) ', where dotDataBlockName is the .data statement reference`

Ideal Op-Amp

```
Exxx n+ n- OPAMP in+ in-
```

You can also substitute `LEVEL=1` in place of `OPAMP`:

```
Exxx n+ n- in+ in- level=1
```

For a description of these parameters, see [E-element Parameters on page 194](#).

Ideal Transformer

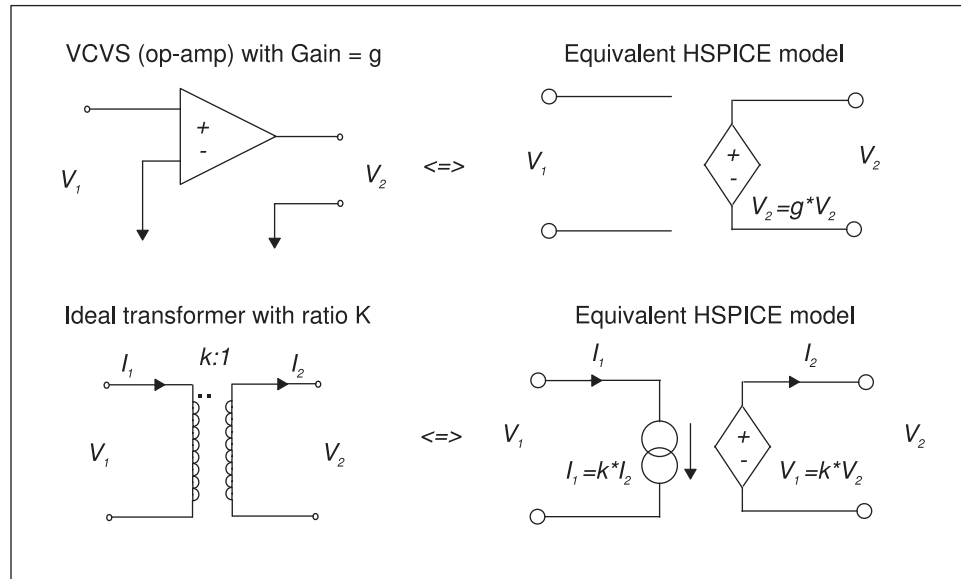
```
Exxx n+ n- TRANSFORMER in+ in- k
```

You can also substitute `LEVEL=2` in place of `TRANSFORMER`:

```
Exxx n+ n- in+ in- level=2 k
```

For a description of these parameters, see [E-element Parameters on page 194](#).

Figure 15 *Equivalent VCVS and Ideal Transformer the PrimeSim Continuum Solution Models*



Voltage-controlled Resistance Switch

Variable-Resistance Switch

```
Gxxx n+ n- VSWITCH in+ in- [voff=val]
+[roff=val] [von=val]
+[ron=val]
```

Short-Transition Switch

```
Gxxx n+ n- VSWITCH in+ in- [roff=val]
+[ron=val] [vt=val]
+[vh=val]
```

Examples

```
Gvrs n1 n2 VSWITCH in1 in2 voff=1 roff=1e+6 von=1.5 ron=50
Gsts n1 n2 VSWITCH in1 in2 roff=1e+4 ron=1 vt=1 vh=0.1
```

The voltage-controlled current source (G) can be used as a voltage-controlled resistance switch. The resistance between controlled nodes ranges from min ($roff, ron$) to max ($roff, ron$). 2 modes are supported for such element: variable-resistance switch and short-transition switch.

Variable-Resistance Switch

For $Von < Voff$

$$V \leq V_{off}: R_s = R_{off};$$

$$V \geq V_{on}, R_s = R_{off};$$

$$V_{off} < V < V_{on}: R_s = \exp \left(L_m + \frac{3L_r(V-V_m)}{2V_d} - \frac{2L_r(V-V_m)^3}{V_d^3} \right)$$

For $V_{on} < V_{off}$

$$V \leq V_{on}: R_s = R_{on};$$

$$V \geq V_{off}, R_s = R_{off};$$

$$V_{on} < V < V_{off}: R_s = \exp \left(L_m - \frac{3L_r(V-V_m)}{2V_d} + \frac{2L_r(V-V_m)^3}{V_d^3} \right)$$

While

$$V = V(\text{in+}, \text{in-}); V_m = \frac{V_{on} + V_{off}}{2}; V_d = |V_{on} - V_{off}|.$$

$$L_m = \ln((R_{on} \cdot R_{off})^{\frac{1}{2}});$$

$$L_r = \ln(R_{on}/R_{off});$$

Short-Transition Switch

If state is OFF, for $V \geq V_T + V_H, R_s = R_{on}$, state is ON.

If state is ON, for $V \leq V_T - V_H, R_s = R_{off}$, state is ON.

The following list describes the voltage-controlled current source parameters.

Parameter	Description
VSWITCH	Keyword for voltage controlled resistance switch. It is a reserved PrimeSim HSPICE keyword; do not use it as a node name.
V _{off}	Control voltage required for off state. Default: 0 - For $V_{off} < V_{on}$: maximum value: $V_{on} - (\text{RELTOL} \cdot \max(V_{on} , V_{off}) + \text{VNTOL})$ - For $V_{off} > V_{on}$: minimum value: $V_{on} + (\text{RELTOL} \cdot \max(V_{on} , V_{off}) + \text{VNTOL})$
V _{on}	Control voltage required for on state. Default: 1
R _{off}	Resistance for off state. Default: 1e+6; Minimum value: RESMIN; Maximum value: 1/GMIN;
R _{on}	Resistance for on state Default: 1; Minimum value: RESMIN; Maximum value: 1/GMIN;
V _T	Threshold control voltage. Default: undefined; Not short-transition switch when undefined;
V _H	Hysteresis control voltage Default: 0

Z Transform Function Call

Z transform functions are used for E-elements (controlled behavioral sources), which is similar to the Laplace form function (see [Laplace Transform on page 186](#) for more details).

$H(z)$ denotes the frequency response, where z is a complex frequency variable. Its value is: $z = e^{(j\omega)}$

where, $\omega = 2 \cdot Pl \cdot f / fs$ and $fs = 2 \cdot MAXF$ $MAXF$ denotes the Nyquist critical frequency.

Z Transform Syntax

Voltage Gain $H(z)$:

```
Exxx n+ n- ZTRANS in+ in- k0, k1, ..., kn
+ / d0, d1, ..., dm
+ [MAXF=val] [SCALE=] [TC1=val] [TC2=val]
```

$H(z)$ is a rational function, in the following form:

$$H(z) = \frac{k_0 + k_1 \cdot z^{-1} + k_2 \cdot z^{-2} + \dots + k_n \cdot z^{-n}}{d_0 + d_1 \cdot z^{-1} + d_2 \cdot z^{-2} + \dots + d_m \cdot z^{-m}}$$

You can use parameters to define the values of all coefficients

$(k_0, k_1, \dots, d_0, d_1, \dots)$

Example:

```
Ehigh_pass out 0 ZTRANS in 0 0.0082 -0.1793
+ 0.6579 -0.1793 / 1
```

The `Ehigh_pass` element statement describes a fourth-order highpass filter, with the transfer function:

$$H(z) = 0.0082 - 0.1793z^{-1} + 0.06579z^{-2} - 0.1793z^{-3}$$

E-element Parameters

The following list describes the E-element parameters.

Parameter	Description
ACCURACY	Used only with the frequency response table. 0: Default. This method generates more accurate results and achieves better performance. 1: Provides more accurate results for frequency table forms, as compared to ACCURACY=0.

Parameter	Description
ABS	Output is an absolute value, if ABS = 1.
DELAY	Keyword for the delay element. Same as for the voltage-controlled voltage source, except it has an associated propagation delay, TD. This element adjusts propagation delay in macro (subcircuit) modeling. DELAY is a reserved word; do not use it as a node name.
DELF	Frequency resolution Δf . The inverse of DELF is the time window over which the PrimeSim Continuum solution calculates $h(t)$ from $H(s)$. A smaller DELF value means a more accurate transient analysis and a longer CPU time. The number of points (N) used to convert $H(s)$ to $h(t)$ is $N=2 \times \text{MAXF}/\text{DELF}$. Because N must be a power of 2, the PrimeSim Continuum solution adjusts the DELF value. The default is $1/\text{TSTOP}$. This parameter only applies when you use the FREQ keyword.
DELTA	Controls the curvature of the piecewise linear corners. Default, delta is computed automatically when not set. The maximum is one-half of the smallest distance between breakpoints.
Exxx	Voltage-controlled element name. Must begin with E followed by up to 1023 alphanumeric and special characters.
gain	Voltage gain.
gatetype (k)	Where gatetype can be AND, NAND, OR, or NOR and the number of inputs of the gate is represented by k. For multi-input gates, only one input determines the state of the output. Note: Do not use the above keywords as node names.
IC	Initial condition: initial estimate of controlling voltage values. If you do not specify IC, the default = 0.0.
in +/-	Positive or negative controlling nodes. Specify one pair for each dimension.
k	Ideal transformer turn ratio: $V(\text{in}+, \text{in}-) = k \cdot V(\text{n}+, \text{n}-)$ or number of gates input.
MAX	Maximum output voltage value. The default is undefined and sets no maximum value.
MAXF	Maximum or the Nyquist critical frequency. The larger the MAXF value, the more accurate the transient results and longer the CPU time. The default is $1024 \times \text{DELF}$. This parameter applies when you use the FREQ or ZTRANS keyword.
MIN	Minimum output voltage value. The default is undefined and sets no minimum value.
n+/-	Positive or negative node of a controlled element.
NDIM	Number of polynomial dimensions. If you do not set POLY(NDIM), the PrimeSim Continuum solution assumes a one-dimensional polynomial. NDIM must be a positive number.
NPDELAY	Sets the number of data points to use in delay simulations. The default value is the larger of either 10, or the smaller of TD/tstep and $\text{tstop}/\text{tstep}$. That is: $\text{NPDELAY}_{\text{default}} = \max \left[\frac{\min \langle \text{TD}, \text{tstop} \rangle}{\text{tstep}}, 10 \right]$ The .TRAN statement specifies tstep and tstop values.

Parameter	Description
LEVEL=0 1 2	Function keyword such as VCVS, VCAP, and so forth: - LEVEL = 0: Voltage-Controlled Current Source (VCCS). - LEVEL = 1: Ideal Op-Amp (OPAMP). - LEVEL = 2: Ideal Transformer (TRANSFORMER). When LEVEL=1 is used in a frequency response table, it indicates that the function is a high pass filter.
OPAMP or Level1=	The keyword for an ideal op-amp element. OPAMP is the PrimeSim Continuum solution reserved word; do not use it as a node name.
P0, P1 ...	The polynomial coefficients. If you specify one coefficient, the PrimeSim Continuum solution assumes that it is P1 (P0 = 0.0), and that the element is linear. If you specify more than one polynomial coefficient, the element is nonlinear, and P0, P1, P2 ... represent them (see Polynomial Functions on page 236).
POLE	Keyword to indicate that the transfer function is described by pole and zero locations.
POLY	Keyword for the polynomial function. If you do not specify <code>POLY(ndim)</code> , the PrimeSim Continuum solution assumes a one-dimensional polynomial. Ndim must be a positive number.
PWL()	Keyword for the piecewise linear function; must include parenthetical ().
SCALE	Multiplier for the element value.
TC1, TC2	First-order and second-order temperature coefficients. Temperature changes update the SCALE: $SCALE_{eff} = SCALE \cdot (1 + TC1 \cdot \Delta t + TC2 \cdot \Delta t^2)$
TD	Keyword for the time (propagation) delay.
TRANSFORMER or LEVEL=2	Keyword for an ideal transformer. TRANSFORMER is a reserved word; do not use it as a node name.
VOL, NOISE	Voltage output that flows from n+ to n-. The expression that you define can be a function of: - node voltages - branch currents - time (time variable) - temperature (temper variable) - frequency (hertz variable)
VCVS	Keyword for a voltage-controlled voltage source. VCVS is a reserved word; do not use it as a node name. The VCVS keyword is optional for E-elements and not required for LAPLACE, FREQ and FOSTER forms.
x1,x2...	Controlling voltage across the in+ and in- nodes. The x values can be in increasing or decreasing order.
y1,y2...	Corresponding element values of x.
LAPLACE	Laplace transform resolves a function into its moments (see Laplace Transform on page 186)
FREQ	Response table (see Frequency Response Table on page 188)

Parameter	Description
FOSTER	Pole residue form (see Foster Pole-Residue Form on page 189)
ZTRANS	Keyword to indicate that the transfer function is described by Z-transform coefficients.
NOISEFILE	File name that contains noise data.
NOISE_CARRIER	Device name to calculate carrier power, used when noise unit is dbc/Hz in RF analyses.

For a listing of output templates for voltage-controlled voltage source E-elements models (PrimeSim HSPICE only), see the *Voltage-Controlled Voltage Source (E-element)* table in Element Template Listings in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

E-element Examples

For full demonstration examples of circuits using E-elements, see the paths to the following netlist files in Behavioral Application Examples in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*:

- `behave.sp` (AND/NAND gates)
- `compar.sp` (behavioral comparator with hysteresis)
- `pwl7.sp` (modeling inverter by using a PWL VCVS)
- `sampling.sp` (Sampling a sine wave using an expression)
- `swcap5.sp` (Fifth-order elliptic switched capacitor filter—OPAMP(LEVEL=1))
- `idealop.sp` (Ninth-order low-pass filter, OPAMP)
- `integ.sp` (Integrator circuit, GAIN)
- `pll_bvp.sp` (PLL build with behavioral source, PWL)

The following sections provide these examples:

- [Ideal OpAmp](#)
- [Voltage Summer](#)
- [Polynomial Function](#)
- [Zero-Delay Inverter Gate](#)
- [Delayed and Inverted Signal](#)
- [Differential Amplifiers and Opamp Signals](#)

- [Ideal Transformer](#)
- [Voltage-Controlled Oscillator](#)
- [Switching between Two Voltage Sources Connected to the Same Node](#)
- [VCVS as the Controlling Element for a CCCS](#)

Ideal OpAmp

You can use the voltage-controlled voltage source to build a voltage amplifier with supply limits.

- The output voltage across nodes 2,3 is $v(14,1) * 2$.
- The value of the voltage gain parameter is 2.
- The `MAX` parameter sets a maximum E1 voltage of 5 V.
- The `MIN` parameter sets a minimum E1 voltage output of -5 V.

Example

If $V(14,1) = -4V$, then the PrimeSim Continuum solution sets E1 to -5 V, and not -8 V as the equation suggests:

```
Eopamp 2 3 14 1 MAX=+5 MIN=-5 2.0
```

To specify a value for polynomial coefficient parameters, use the following format:

```
.PARAM CU=2.0  
E1 2 3 14 1 MAX=+5 MIN=-5 CU
```

For a full demonstration netlist of the E-element used for an active low pass filter using behavioral opamp models, use the path shown for `low_pass9a.sp` in Filters Examples in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

Voltage Summer

An ideal voltage summer specifies the source voltage as a function of three controlling voltages

- $V(13,0)$
- $V(15,0)$
- $V(17,0)$

To describe a voltage source, the voltage summer uses this value:

$$V(13,0) + V(15,0) + V(17,0)$$

This example represents an ideal voltage summer. It initializes the three controlling voltages for a DC operating point analysis, to 1.5, 2.0, and 17.25 V.

```
EX 17 0 POLY(3) 13 0 15 0 17 0 0 1 1 1 IC=1.5,2.0,17.25
```

Polynomial Function

A voltage-controlled source also can output a nonlinear function using a polynomial. In the following example, the voltage at V(3,4) is a function of the voltage at V(21,17):

```
V(3,4)= '10.5 + 2.1 *V(21,17) + 1.75 *V(21,17)**2'
```

The two-dimensional polynomial can be written as:

```
E2 3 4 POLY(2) 21 17 21 17 10.5 2.1 0 0 0 1.75
```

For more information on polynomial functions, see [Polynomial and Piecewise Linear Functions on page 236](#).

Using behavioral equations, the polynomial can be replaced by the following:

```
E2 3 4 VOL='10.5 + 2.1*V(21,17) + 1.75*V(21,17)**2'
```

Note:

For the power function, the PrimeSim Continuum solution accepts either '**' or '^'.

Zero-Delay Inverter Gate

Use a piecewise linear transfer function to build a simple inverter with no delay:

```
Einv out 0 PWL(1) in 0 .7v,5v 1v,0v
```

Delayed and Inverted Signal

You can use an E-element to invert a signal to generate a delayed signal which is the inverse of another signal. Use the following syntax to generate a delayed signal (TD); it is the inverse of the input signal (in):

```
.option list node post
Vin in 0 pwl(0 0 10n 0 13n 2v 23n 2v 24n 0v)    $$ input signal
to be inverted and delayed
Edelay in_delay 0 DELAY in 0 TD=2n              $$ signal
    "in_delay" is the "in" signal delayed by TD
Edelay_inv out_inv 0 PWL(1) in_delay 0 0v 2v 2v 0v    $$ signal
    "out_inv" is the inverted and delayed "in" signal
.tran .1n 30n
.print v(in_delay) v(out_inv) v(in)
.end
```

Differential Amplifiers and Opamp Signals

E-elements define a differential voltage source to use with differential amplifiers or op-amps. E-elements also can provide differential signals to drive circuits requiring differential signals

```
*****spice definition*****
VID 7 0 DC 0 ac=1 SIN (0 0.70 1MEG)
E+ in+ 10 7 0 0.5 $ differential signal 1, level can be varied by
    changing gain
E- in- 10 7 0 -0.5 $ differential signal 2
VIC 10 0 DC 0.3V $VIC is the common mode signal source
```

For a full netlist differential block analysis example, see also: `$installdir/demo/hspice/behave/diff.sp`.

Ideal Transformer

If the turn ratio is 10 to 1, the voltage relationship is $V(\text{out})=V(\text{in})/10$.

```
Etrans out 0 TRANSFORMER in 0 10
```

You can also substitute `LEVEL=2` in place of `TRANSFORMER`:

```
Etrans out 0 in 0 level=2 10
```

Voltage-Controlled Oscillator

In this example, controlled-sources control VCO output:

```
.param pi=3.1415926
.param twopi='2*pi'
.subckt vco in inb out outb f0=100k kf=50k out_off=0.0 out_amp=1.0
gs 0 s poly(2) c 0 in inb 0 'twopi*1e-9*f0' 0 0 'twopi*1e-9*kf'
gc c 0 poly(2) s 0 in inb 0 'twopi*1e-9*f0' 0 0 'twopi*1e-9*kf'
cs s 0 1e-9 ic=0
cc c 0 1e-9 ic=1
eout out 0 vol='out_off+(out_amp*v(s))'
eoutb outb 0 vol='out_off+(out_amp*v(c))'
.ic v(c)=1 v(s)=0
.ends
```

Switching between Two Voltage Sources Connected to the Same Node

You can use the PrimeSim Continuum solution voltage controlled voltage source (E-element) to design a behavioral switch by creating a netlist as follows:

```
.option post
Vin1 in1 0 pwl ... $ source 1
Vin2 in2 0 pwl ... $ source 2
```



```
Ein in 0 vol='v(in1)*v(ctrl) + v(in2)*(1-v(ctrl))'
                                           $ behavioral switch
vctrl ctrl 0 pw1 0 1 49n 1 50n 0          $ control voltage
.tran 1n 100n
.end
```

In this example, the `v(ctrl)` value is 1 initially so that `v(in)=v(in1)`. At 50 ns, the control voltage changes to 0 and `v(in)=v(in2)`.

VCVS as the Controlling Element for a CCCS

A VCVS can be used as the controlling element for a CCCS. See the following example:

```
f0 v_out 0 CCCS evref_i 1.0
v1 net03 v_2v dc=0
v0 net04 0 dc=1
r1 v_out 0 r=1e3
r0 net03 0 r=1e3
evref_i v_2v 0 VCVS net04 0 2
```

See Also:

- Using G- and E-elements in the *PrimeSim HSPICE User Guide: Advanced Analog Simulation and Analysis*.
- Filters Examples in *PrimeSim HSPICE User Guide: Basic Simulation and Analysis* (`bandstop1.sp`) for a demonstration of the E-element as a filter.

Using the E-element for AC Analysis

The following equation describes an E-element:

```
E1 ee 0 vol=f(v(1), v(2))
```

An AC analysis computes voltage as follows:

```
v(ee)=A*delta_v1+B*delta_v2
```

Where:

- `A` is the derivative of $f(v(1), v(2))$ to $v(1)$ at the operating point.
- `B` is the derivative of $f(v(1), v(2))$ to $v(2)$ at the operating point.
- `delta_v1` is the AC voltage variation of $v(1)$.
- `delta_v2` is the AC voltage variation of $v(2)$.

Example

This example uses demonstration netlist `ee1m.sp`, which is available in directory `$installdir/demo/hspice/sources`:

```
*****
***** E element for AC analysis
.option post
.op
*CASE1-Mixed and zero time unit has zero value(tran)
v_n1 n1 gnd dc=6.0 pw1 0.0 6.0 1.0n 6.0 ac 5.0
v_n2 n2 gnd dc=4.0 pw1 0.0 4.0 1.0n 6.0 ac 2.0
e1 n3 gnd vol='v(n1)+v(n2)'
e2 n4 gnd vol='v(n1)*v(n2)'
r1 n1 gnd 1
r2 n2 gnd 1
r3 n3 gnd 1
r4 n4 gnd 1
.tran 10p 3n
.ac dec 1 1 100meg
.print ac v(n?)
.end
*****
```

The AC voltage of node n3 is:

$$\begin{aligned} v(n3) &= 1.0 * v(n1) (ac) + 1.0 * v(n2) (ac) \\ &= 1.0 * 5.0 + 1.0 * 2.0 \\ &= 7.0 (v) \end{aligned}$$

The AC voltage of node n4 is:

$$\begin{aligned} v(n4) &= v(n2) (op) * v(n1) (ac) + v(n1) (op) * v(n2) (ac) \\ &= 4.0 * 5.0 + 6.0 * 2.0 \\ &= 32.0 (v) \end{aligned}$$

Voltage-dependent Current Sources — G-elements

This section explains G-element syntax statements, and their parameters.

The following sections discuss these topics:

- [Voltage-controlled Current Source \(VCCS\)](#)
- [G-element Examples](#)

Voltage-controlled Current Source (VCCS)

The following sections discuss these topics:

- [Linear](#)
- [Polynomial \(POLY\)](#)
- [PWL](#)
- [Multi-Input Gate](#)
- [Delay Element](#)
- [Z Transform Function Call](#)
- [Laplace Transform](#)
- [Pole-Zero Function](#)
- [Frequency Response Table](#)
- [Foster Pole-Residue Form](#)
- [Behavioral Current Source](#)
- [Behavioral Time Derivative Current Source](#)
- [Behavioral Current Noise Source](#)
- [Voltage-controlled Resistor \(VCR\)](#)
- [Voltage-Controlled Capacitor \(VCCAP\)](#)
- [Magnetic Tunnel Junction MRES0 Mode](#)
- [G-element Parameters](#)

Linear

```
Gxxx n+ n- [VCCS | Level=0] in+ in- transconductance [MAX=val]  
+ [MIN=val] [SCALE=val] [M=val] [TC1=val] [TC2=val]  
+ [ABS=1] [IC=val]
```

For a description of the G-element parameters, see [G-element Parameters on page 217](#).

Polynomial (POLY)

```
Gxxx n+ n- [VCCS | Level=0] POLY(NDIM) in1+ in1- ... [inndim+ inndim-]  
+ [MAX=val] [MIN=val] [SCALE=val] [M=val] [TC1=val]  
+ [TC2=val] [ABS=1] P0[P1...] [IC=val]
```

For a description of the G-element parameters, see [G-element Parameters on page 217](#).
For a description of possible POLY syntaxes, see [Polynomial Functions on page 236](#).

PWL

```
Gxxx n+ n- [VCCS | Level=0] PWL(1) in+ in- [DELTA=val]
+ [SCALE=val] [M=val] [TC1=val] [TC2=val]
+ x1,y1 x2,y2 ... x100,y100 [IC=val] [SMOOTH=val]
Gxxx n+ n- [VCCS | Level=0] NPWL(1) in+ in- [DELTA=val]
+ [SCALE=val] [M=val] [TC1=val] [TC2=val]
+ x1,y1 x2,y2 ... x100,y100 [IC=val] [SMOOTH=val]
Gxxx n+ n- [VCCS | Level=0] PPWL(1) in+ in- [DELTA=val]
+ [SCALE=val] [M=val] [TC1=val] [TC2=val]
+ x1,y1 x2,y2 ... x100,y100 [IC=val] [SMOOTH=val]
```

For a detailed description of PWL functions, see [Piecewise Linear Function on page 241](#).

Multi-Input Gate

```
Gxxx n+ n- [VCCS | Level=0] gatetype(k) in1+ in1- ...
+ ink+ ink- [DELTA=val] [TC1=val] [TC2=val] [SCALE=val]
+ [M=val] x1,y1 ... x100,y100 [IC=val]
```

In this syntax, *gatetype* can represent an AND, NAND, OR, or NOR gate and *k* represents the number of inputs to the gate. For a description of the G-element parameters, see [G-element Parameters on page 217](#).

Delay Element

```
Gxxx n+ n- [VCCS | Level=0] DELAY in+ in- TD=val [SCALE=val]
+ [TC1=val] [TC2=val] NPDELAY=val
```

For a description of the G-element parameters, see [G-element Parameters on page 217](#).

Z Transform Function Call

Z transform functions are used for G- elements (controlled behavioral sources), which is similar to the Laplace form function (see [Laplace Transform on page 205](#) for more details).

$H(z)$ denotes the frequency response, where z is a complex frequency variable. Its value is: $z = e^{j\omega}$

where, $\omega = 2 \cdot \text{Pl} \cdot f / fs$ and $fs = 2 \cdot \text{MAXF}$ MAXF denotes the Nyquist critical frequency.

Z Transform Syntax

Transconductance H(z):

```
Gxxx n+ n- ZTRANS in+ in- k0, k1, ..., kn
+ / d0, d1, ..., dm
+ [MAXF=val] [SCALE=val] [TC1=val] [TC2=val] [M=val]
```

H(z) is a rational function, in the following form:

$$H(z) = \frac{k_0 + k_1 \cdot z^{-1} + k_2 \cdot z^{-2} + \dots + k_n \cdot z^{-n}}{d_0 + d_1 \cdot z^{-1} + d_2 \cdot z^{-2} + \dots + d_m \cdot z^{-m}}$$

You can use parameters to define the values of all coefficients.

(k0, k1, ..., d0, d1, ...)

Example:

```
Glow_pass 0 out ZTRANS in 0 0.0317 0.0951
+ 0.0951 0.0317 / 1.0 -1.459 0.9104 -0.1978
```

The Glow_pass element statement describes a third-order low-pass filter, with the transfer function:

$$H(z) = \frac{0.0317 + 0.0951z^{-1} + 0.0951z^{-2} + 0.0317z^{-3}}{1.0 - 1.459z^{-1} + 0.9104z^{-2} + 0.1978z^{-3}}$$

Laplace Transform

Transconductance H(s):

```
Gxxx n+ n- LAPLACE in+ in- k0, k1, ..., kn / d0, d1, ..., dm
+ [SCALE=val] [TC1=val] [TC2=val] [M=val]
```

H(s) is a rational function, in the following form:

Equation 26

$$H(s) = \frac{k_0 + k_1 s + \dots + k_n s^n}{d_0 + d_1 s + \dots + d_m s^m}$$

You can use parameters to define the values of all coefficients (k0, k1, ..., d0, d1, ...).

For a description of the G-element parameters, see [G-element Parameters on page 217](#).

Note:

The LAPLACE form of G-elements do not require the VCR keyword.

Example

```
Glowpass 0 out LAPLACE in 0 1.0 / 1.0 2.0 2.0 1.0
```

The `Glowpass` element statement describes a third-order low-pass filter with the transfer function:

Equation 27

$$H(s) = \frac{1}{1 + 2s + 2s^2 + s^3}$$

Pole-Zero Function

Transconductance $H(s)$:

```
Gxxx n+ n- POLE in+ in- a az1, fz1, ..., azn, fzn / b,
+ ap1, fp1, ..., apm, fpm [SCALE=val] [TC1=val]
+ [TC2=val] [M=val]
```

The following equation defines $H(s)$ in terms of poles and zeros:

Equation 28

$$H(s) = \frac{a \cdot (s + \alpha_{z1} - j2\pi f_{z1}) \dots (s + \alpha_{zn} - j2\pi f_{zn}) (s + \alpha_{zn} + j2\pi f_{zn})}{b \cdot (s + \alpha_{p1} - j2\pi f_{p1}) \dots (s + \alpha_{pm} - j2\pi f_{pm}) (s + \alpha_{pm} + j2\pi f_{pm})}$$

The complex poles or zeros are in conjugate pairs. The element description specifies only one of them, and the program includes the conjugate. You can use parameters to specify the a , b , α , and f values.

For a description of the G-element parameters, see [G-element Parameters on page 217](#).

Example

```
Ghigh_pass 0 out POLE in 0 1.0 0.0,0.0 / 1.0 0.001,0.0
```

The `Ghigh_pass` statement describes a high-pass filter with the transfer function:

Equation 29

$$H(s) = \frac{1.0 \cdot (s + 0.0 + j \cdot 0.0)}{1.0 \cdot (s + 0.001 + j \cdot 0.0)}$$

Frequency Response Table

Transconductance $H(s)$:

```
Gxxx n+ n- FREQ in+ in- f1, a1, ph1, ..., fi, ai, ph1
+ [DELF=val] [MAXF=val] [SCALE=val] [TC1=val]
+ [TC2=val] [M=val] [LEVEL=val] [ACCURACY=val]
```

Where:

- Each f_i is a frequency point, in hertz.
- a_i is the magnitude, in dB.
- ph_i is the phase, in degrees.

At each frequency, the PrimeSim Continuum solution uses interpolation to calculate the network response, magnitude, and phase. The PrimeSim Continuum solution interpolates the magnitude (in dB) logarithmically as a function of frequency. It also interpolates the phase (in degrees) linearly as a function of frequency.

Equation 30

$$|H(j2\pi f)| = \left(\frac{a_i - a_k}{\log f_i - \log f_k} \right) (\log f - \log f_i) + a_i$$

Equation 31

$$\angle H(j2\pi f) = \left(\frac{\phi_i - \phi_k}{f_i - f_k} \right) (f - f_i) + \phi_i$$

For a description of the G-element parameters, see [G-element Parameters on page 217](#).

Note:

The FREQ form of G-elements do not require the VCR keyword.

Foster Pole-Residue Form

Transconductance G(s) form:

```
Gxxx n+ n- FOSTER in+ in- k0 k1
+ (Re{A1}, Im{A1}) / (Re{p1}, Im{p1})
+ (Re{A2}, Im{A2}) / (Re{p2}, Im{p2})
+ (Re{A3}, Im{A3}) / (Re{p3}, Im{p3})
+ ...
```

In the above syntax, parenthesis, commas, and slashes are separators—they have the same meaning as a space. Four numbers represent a pole-residue pair (real and imaginary part of the residue, then real and imaginary part of the pole).

You must ensure that $\text{Re}\{p_i\} < 0$; otherwise, the simulations diverge. Also, ensure passivity of the model (for an N-port admittance matrix Y , $\text{Re}\{Y\}$ should be positive-definite), or the simulation is likely to diverge.

For a description of the G-element parameters, see [G-element Parameters on page 217](#).

Note:

The FOSTER form of G-elements do not require the VCR keyword.

Example

To represent a $G(s)$ in the form:

Equation 32

$$G(s) = 0.001 + 1 \times 10^{-12}s + \frac{0.0008}{s + 1 \times 10^{10}} + \frac{(0.001 - j0.006)}{s - (-1 \times 10^8 + j1.8 \times 10^{10})} + \frac{(0.001 + j0.006)}{s - (-1 \times 10^8 - j1.8 \times 10^{10})}$$

You would input:

```
G1 1 0 FOSTER 2 0 0.001 1e-12
+(0.0004, 0)/(-1e10, 0) (0.001, -0.006)/(-1e8, 1.8e10)
```

Note:

For real poles, enter half the residue value because the netlist applies it twice. In the above example, the first pole-residue pair is real. You write it as “A1/(s-p1)+A1/(s-p1)”; therefore, enter 0.0004 rather than 0.0008.

Behavioral Current Source

The G-element can be used as a behavioral current source where the current source can be bias and/or frequency dependent. For a behavioral current source, use a constant value for '*expression*'.

For current sources using expressions, the G-element takes the form:

```
Gxxx node1 node2 cur[rent]='expression'
```

Note:

In the preceding syntax, both `cur` and `current` are acceptable.

You can set the `xxx` parameter with a value up to 1024 characters. The `node1` and `node2` are the positive and negative nodes that connect to the current source. The current expression can contain the bias, frequency, or other parameters, and involve node voltages and currents through voltage sources.

Behavioral Time Derivative Current Source

The G-element can be used as a behavioral time derivative current source. The function `ddt ( )` computes the time derivative, $\frac{d}{dt}x(t)$, of the argument where the argument is an expression that can include state and input variables, parameters, node voltages or currents.

For current sources using the `ddt ( )` function, the G-element takes the form:

```
Gxxx node1 node2 cur='ddt('expression')
```


Behavioral Current Noise Source

The following sections discuss these topics:

- [Expression Form](#)
- [Data Form](#)
- [Example](#)

Expression Form

The G-element specifies a current noise source with the noise value in $A^{2/Hz}$. The noise source can be bias and/or frequency dependent. For a white noise source, use a constant value for 'expression'.

For noise sources using expressions, the G-element takes the form:

```
Gxxx node1 node2 noise='noise_expression'
```

You can set the *xxx* parameter with a value up to 1024 characters. The *node1* and *node2* are the positive and negative nodes that connect to the noise source. The noise expression can contain the bias, frequency, or other parameters, and involve node voltages and currents through voltage sources.

For a description of the G-element parameters, see [G-element Parameters on page 217](#).

This syntax creates a simple two-terminal current noise source; A^2/Hz describes its value. The output noise generated from this noise source is:

```
noise_expression*H
```

H is the transfer function from the terminal pair (node1,node2) to the circuit output which measures the output noise.

You also can implement a behavioral noise source with an E-element. As noise elements, they are two terminal elements that represent a noise source connected between two specified nodes:

```
gname node1 node2 node3 node4 noise='expression'
```

This syntax produces a noise source correlation between the terminal pairs (node1 node2) and (node3 node4). Compute the resulting output noise as:

```
noise_expression*sqrt(H1*H2*)
```

- *H1* is the transfer function from (node1,node2) to the output.
- *H2* is the transfer function from (node3,node4) to the output.

The noise expression can involve node voltages and currents through voltage sources.

Data Form

```
Gxxx node1 node2 noise data=dataname [noise_carrier=devname]
.DATA dataname
+ pname1 pname2
+ freq1 noise1
+ freq2 noise2
+ ...
.enddata
```

The data form defines a basic frequency-noise table. The `.DATA` statement contains two parameters: frequency and noise to specify the noise value at each frequency point. The unit for frequency is hertz, and the unit for noise is

dependent on the pattern of the noise name by the following rule:

Unit	Name_pattern
V/sqrt(Hz)	ONoise, or ONoise(), or *** (M), or *** (MAG)
db/Hz	ONoise (DB), or *** (DB), or ***DB
dbc/Hz	PHNoise, or PAC_L(f), or BPN_L(f), or NLP_L(f)
V ² /Hz	Otherwise

It should be noted that when noise unit is dbc/Hz, to determine the true noise level, a carrier power is needed. In this scenario, you must specify the `noise_carrier` parameter to communicate to the simulator device on which the carrier power is calculated. This device must be an independent voltage source or a port. And the noise takes effect only in SN or HB noise analyses. For other types of noise unit, the `noise_carrier` parameter does not work.

The G-element with noise sources supports the following syntaxes:

- `Gxxx n1 n2 noise data=dataname [noise_carrier=devname]`
- `Gxxx n1 n2 noise data=datablock [noise_carrier=devname]`
- `Gxxx n1 n2 noisefile=filename [noise_carrier=devname]`
- `Gxxx n1 n2 noise=expression`
- `Gxxx n1 n2 noise=Table(arg1,f1,v1,f2,v2...)`
- `Gxxx n1 n2 noise=Table(arg1,dotDataBlockName)`, where `dotDataBlockName` is the `.data` statement reference

For a description of the G-element parameters, see [G-element Parameters on page 217](#).

Example

The following netlist shows a 1000-ohm resistor (`g1`) using a G-element. The `g1noise` element, placed in parallel with the `g1` resistor, delivers the thermal noise expected from a

resistor. The following example includes the `r1` resistor for comparison: The noise due to `r1` should be the same as the noise due to `g1noise`:

```
* Resistor implemented using g-element
v1 1 0 1
r1 1 2 1k
g1 1 2 cur='v(1,2)*0.001'
g1noise 1 2
+ noise='4*1.3806266e-23*(TEMPER+273.15)*0.001'
rout 2 0 1meg
.ac lin 1 100 100
.noise v(2) v1 1
.end
```

Voltage-controlled Resistor (VCR)

Note:

The `LEVEL=1` keyword can be used instead of the `VCR` keyword for each form of VCR.

The following sections discuss these topics:

- [Linear](#)
- [Polynomial \(POLY\)](#)
- [PWL](#)
- [EPWL](#)
- [Multi-Input Gates](#)

Linear

```
Gxxx n+ n- VCR in+ in- transfactor [MAX=val] [MIN=val]
+ [SCALE=val] [M=val] [TC1=val] [TC2=val] [IC=val]
```

For a description of the G-element parameters, see [G-element Parameters on page 217](#).

Polynomial (POLY)

```
Gxxx n+ n- VCR POLY (NDIM) in1+ in1- ...
+ [inndim+ inndim-] [MAX=val] [MIN=val] [SCALE=val]
+ [M=val] [TC1=val] [TC2=val] P0 [P1...] [IC=vals]
```

For a description of the G-element parameters, see [G-element Parameters on page 217](#).

PWL

```
Gxxx n+ n- VCR PWL(1) in+ in- [DELTA=val] [SCALE=val]
+ [M=val] [TC1=val] [TC2=val] x1,y1 x2,y2 ... x100,y100
+ [IC=val] [SMOOTH=val]
```

```
Gxxx n+ n- VCR NPWL(1) in+ in- [DELTA=val] [SCALE=val]
+ [M=val] [TC1=val] [TC2=val] x1,y1 x2,y2 ... x100,y100
+ [IC=val] [SMOOTH=val]
Gxxx n+ n- VCR PPWL(1) in+ in- [DELTA=val] [SCALE=val]
+ [M=val] [TC1=val] [TC2=val] x1,y1 x2,y2 ... x100,y100
+ [IC=val] [SMOOTH=val]
```

For a description of the G-element parameters, see [G-element Parameters on page 217](#).

EPWL

```
Gxxx n+ n- VCR EPWL(1) in+ in- [DELTA=val] [SCALE=val]
+ [M=val] [TC1=val] [TC2=val] x1,y1 x2,y2 ... x100,y100
+ [IC=val] [SMOOTH=val]
```

See also [Exponential Piecewise Linear Function on page 241](#).

For a description of the G-element parameters, see [G-element Parameters on page 217](#).

Multi-Input Gates

```
Gxxx n+ n- VCR gatetype(k) in1+ in1- ... ink+ ink-
+ [DELTA=val] [TC1=val] [TC2=val] [SCALE=val] [M=val]
+ x1,y1 ... x100,y100 [IC=val]
```

For a description of the G-element parameters, see [G-element Parameters on page 217](#).

Voltage-Controlled Capacitor (VCCAP)

```
Gxxx n+ n- VCCAP PWL(1) in+ in- [DELTA=val]
+ [SCALE=val] [M=val] [TC1=val] [TC2=val]
+ x1,y1 x2,y2 ... x100,y100 [IC=val] [SMOOTH=val]
CTYPE=0|1
```

The PrimeSim Continuum solution uses either `LEVEL=2` (NPWL) or `LEVEL=3` (PPWL) based on the relationship of the (n+, n-) and (in+, in-) nodes. For a description of the G-element parameters, see [G-element Parameters on page 217](#).

Use the `NPWL` and `PPWL` functions to interchange the *n+* and *n-* nodes, but use the same transfer function.

The following sections summarize these actions:

- [NPWL Function](#)
- [PPWL Function](#)

NPWL Function

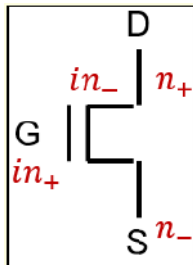
For the in_- node connected to n_+ :

- If $v(n_+, n_-) < 0$, then the controlling voltage is $v(in_+, in_-)$.
- Otherwise, the controlling voltage is $v(in_+, n_-)$.

For the in_- node connected to n_- :

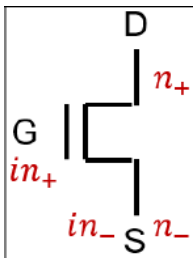
- If $v(n_+, n_-) > 0$, then the controlling voltage is $v(in_+, in_-)$.
- Otherwise, the controlling voltage is $v(in_+, n_+)$.
- NPWL is to model bidirectional switch or transfer gates of NMOS.
- For example, there is a normal NMOS with D, G, and S terminals.
- Let $n_+ = D$, $n_- = S$, $in_+ = G$, $in_- = D$, so the in_- node is connected to n_+

Figure 16



- If $v(n_+, n_-) < 0$, it means $v_{DS} < 0$, the current is from S to D. Therefore, the controlling voltage is v_{GD} , that is, $v(in_+, in_-)$.
- If $v(n_+, n_-) > 0$, it means $v_{DS} > 0$, the current is from D to S. Therefore, the controlling voltage is v_{GS} , that is, $v(in_+, n_-)$.
- Let $n_+ = D$, $n_- = S$, $in_+ = G$, $in_- = S$, so the in_- node is connected to n_-

Figure 17



- If $v(n_+, n_-) > 0$, it means $v_{DS} > 0$, the current is from D to S. Therefore, the controlling voltage is v_{GS} , that is $v(in_+, in_-)$.
- Otherwise, if $v(n_+, n_-) < 0$, it means $v_{DS} < 0$, the current is from S to D. Therefore, the controlling voltage is v_{GD} , that is, $v(in_+, n_+)$.

PPWL Function

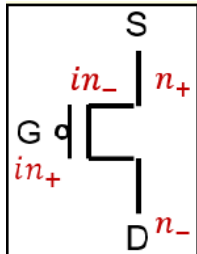
For the in_- node, connected to n_+ :

- If $v(n_+, n_-) > 0$, then the controlling voltage is $v(in_+, in_-)$.
- Otherwise, the controlling voltage is $v(in_+, n_-)$.

For the in_- node, connected to n_- :

- If $v(n_+, n_-) < 0$, then the controlling voltage is $v(in_+, in_-)$.
- Otherwise, the controlling voltage is $v(in_+, n_+)$.
- PPWL is to model bidirectional switch or transfer gates of PMOS.
 - Note that the controlling voltage of PMOS is a negative value.
- For example, there is a normal PMOS with S, G, and D terminals.
- Let $n_+ = S, n_- = D, in_+ = G, in_- = S$, so the in_- node is connected to n_+

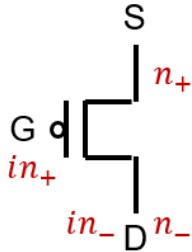
Figure 18



- If $v(n_+, n_-) > 0$, it refers $v_{SD} > 0$, the current is from S to D. Therefore, the controlling voltage is v_{GS} , that is, $v(in_+, in_-)$.
- Otherwise, if $v(n_+, n_-) < 0$, it refers to $v_{SD} < 0$, the current is from D to S. Therefore, the controlling voltage is v_{GD} that is, $v(in_+, n_-)$.

- Let $n_+ = S$, $n_- = D$, $in_+ = G$, $in_- = D$, so the in_- node is connected to n_-

Figure 19



- If $v(n_+, n_-) < 0$, it means $v_{DS} > 0$, the current is from D to S. Therefore, the controlling voltage is v_{GD} , that is $v(in_+, in_-)$.
- If $v(n_+, n_-) > 0$, it means $V_{DS} < 0$, the current is from S to D. Therefore, the controlling voltage is v_{GS} , that is $v(in_+, n_+)$.

If the in_- node does not connect to either n_+ or n_- , then the PrimeSim Continuum solution changes NPWL and PPWL to PWL.

Voltage-controlled Resistance Switch

Variable-Resistance Switch

```
Gxxx n+ n- VSWITCH in+ in- [voff=val]
+[roff=val] [von=val]
+[ron=val]
```

Short-Transition Switch

```
Gxxx n+ n- VSWITCH in+ in- [roff=val]
+[ron=val] [vt=val]
+[vh=val]
```

Examples

```
Gvrs n1 n2 VSWITCH in1 in2 voff=1 roff=1e+6 von=1.5 ron=50
Gsts n1 n2 VSWITCH in1 in2 roff=1e+4 ron=1 vt=1 vh=0.1
```

The voltage-controlled current source (G) can be used as a voltage-controlled resistance switch. The resistance between controlled nodes ranges from min ($roff, ron$) to max ($roff, ron$). 2 modes are supported for such element: variable-resistance switch and short-transition switch.

Variable-Resistance Switch

For $Von < Voff$

$$V \leq V_{off}: R_s = R_{off};$$

$$V \geq V_{on}, R_s = R_{off};$$

$$V_{off} < V < V_{on}: R_s = \exp \left(L_m + \frac{3L_r(V-V_m)}{2V_d} - \frac{2L_r(V-V_m)^3}{V_d^3} \right)$$

For $V_{on} < V_{off}$

$$V \leq V_{on}: R_s = R_{on};$$

$$V \geq V_{off}, R_s = R_{off};$$

$$V_{on} < V < V_{off}: R_s = \exp \left(L_m - \frac{3L_r(V-V_m)}{2V_d} + \frac{2L_r(V-V_m)^3}{V_d^3} \right)$$

While

$$V = \underline{V}(\text{in+}, \text{in-}); V_m = \frac{V_{on} + V_{off}}{2}; V_d = |V_{on} - V_{off}|.$$

$$L_m = \ln((R_{on} \cdot R_{off})^{\frac{1}{2}});$$

$$L_r = \ln(R_{on}/R_{off});$$

Short-Transition Switch

If state is OFF, for $V \geq V_T + V_H, R_s = R_{on}$, state is ON.

If state is ON, for $V \leq V_T - V_H, R_s = R_{off}$, state is ON.

The following list describes the voltage-controlled current source parameters.

Parameter	Description
VSWITCH	Keyword for voltage controlled resistance switch. It is a reserved PrimeSim HSPICE keyword; do not use it as a node name.
V _{off}	Control voltage required for off state. Default: 0 - For $V_{off} < V_{on}$: maximum value: $V_{on} - (\text{RELTOL} \cdot \max(V_{on} , V_{off}) + \text{VNTOL})$ - For $V_{off} > V_{on}$: minimum value: $V_{on} + (\text{RELTOL} \cdot \max(V_{on} , V_{off}) + \text{VNTOL})$
V _{on}	Control voltage required for on state. Default: 1
R _{off}	Resistance for off state. Default: 1e+6; Minimum value: RESMIN; Maximum value: 1/GMIN;
R _{on}	Resistance for on state Default: 1; Minimum value: RESMIN; Maximum value: 1/GMIN;
V _T	Threshold control voltage. Default: undefined; Not short-transition switch when undefined;
V _H	Hysteresis control voltage Default: 0

Magnetic Tunnel Junction MRES0 Mode

```
Gxx p n MRES0
+ rmin=val
+ rmax=val
+ ihi=val
+ ilo=val
+ [ic=val]
+ [iwin=val]
```

Argument	Description
rmin	Minimum resistance of 'parallel' state of the MRAM cell dipoles.
rmax	Maximum resistance of 'anti-parallel' state of the MRAM cell dipoles.
ihi	Upper threshold for i
ilo	Lower threshold for i
ic	Initial state of the MRAM cell; default=0. Boolean value: 0 ~ rmin; 1 ~ rmax.
iwin	Time duration window for which current must be above threshold (ihi ilo); default=0.
Note: This current threshold condition must always be followed: ihi > ilo. iwin is used for the following check: when current crosses the threshold, the current remains above (or below) that threshold for a certain period of time (specified by iwin) to cause a switch. If the current falls before meeting this iwin criteria, the exam time data is reset.	

For more information, see also the *PrimeSim XA™ MRAM Core Cell Models* chapter in the *PrimeSim XA™ User Guide*.

G-element Parameters

The following list describes the G-element parameters.

Parameter	Description
ACCURACY	Used only with the frequency response table. 0: Default. This method generates more accurate results and achieves better performance. 1: Provides more accurate results for frequency table forms, as compared to ACCURACY=0.

Parameter	Description
ABS	Output is an absolute value, if ABS = 1.
CUR, VALUE, NOISE	Current output that flows from n+ to n-. The expression that you define can be a function of: • node voltages • branch currents • time (<code>time</code> variable) • temperature (<code>temper</code> variable) • frequency (<code>hertz</code> variable) Note: CUR is preferred over CURRENT.
DELAY	Keyword for the delay element. Same as in the voltage-controlled current source, but has an associated propagation delay, TD. Adjusts propagation delay in macro (subcircuit) modeling. DELAY is a keyword; do not use it as a node name.
DELF	Frequency resolution Δf. The inverse of DELF is the time window, over which the PrimeSim Continuum solution calculates $h(t)$ from $H(s)$. A smaller DELF value means more accurate transient analysis, and longer CPU time. The number of points (N) used to convert $H(s)$ to $h(t)$ is $N=2 \times \text{MAXF}/\text{DELF}$. Because N must be a power of 2, the PrimeSim Continuum solution adjusts the DELF value. The default is $1/\text{TSTOP}$. In the G-element, with FREQ and ACCURACY = 0 or 1, to perform circular convolution for periodic input, the PrimeSim Continuum solution limits the period. To do this, the PrimeSim Continuum solution sets $\text{DELF} = 1/T : T < \text{TSTOP}$.
DELTA	Controls curvature of piecewise linear corners. Default, delta is computed automatically when not set. Maximum is 1/2 of the smallest distance between breakpoints.
GXXX	Name of the voltage-controlled element. Must begin with G followed by up to 1023 alphanumeric and special characters.
gatetype(k)	Where gatetype can be AND, NAND, OR, or NOR and the number of inputs of the gate is represented by k. For multi-input gates, only one input determines the state of the output. Note: Do not use the above keywords as node names.
LEVEL=0 1 2 3	Function keyword such as VCCS, VCAP, and so forth. • LEVEL=0: Voltage-controlled current source (VCCS). • LEVEL=1: Voltage-controlled resistor (VCR). • LEVEL=2: Voltage-controlled capacitor (VCCAP), negative piecewise linear (NPWL). • LEVEL=3: VCCAP, positive piecewise linear (PPWL). When LEVEL=1 is used in a frequency response table, it indicates that the function is a high pass filter.
IC	Initial condition. Initial estimate of the values of controlling voltages. If you do not specify IC, the default 0.0.
in +/-	Positive or negative controlling nodes. Specify one pair for each dimension.

Parameter	Description
M	Number of G-elements in parallel. Default is 1.
MAX	Maximum value of the current or resistance. The default is undefined and sets no maximum value.
MAXF	Maximum or Nyquist critical frequency. The larger the MAXF value, the more accurate the transient results and longer the CPU time. The default is 1024*DELF. This parameter applies only when you also use the FREQ parameter.
MIN	Minimum value of the current or resistance. The default is undefined and sets no minimum value.
n+/-	Positive or negative node of the controlled element.
NDIM	Number of polynomial dimensions. If you do not specify <i>POLY(NDIM)</i> , the PrimeSim Continuum solution assumes an one-dimensional polynomial. <i>NDIM</i> must be a positive number.
NPDELAY	Sets the number of data points to use in delay simulations. The default is the larger of either 10, or the smaller of TD/tstep and tstop/tstep. That is: $NPDELAY_{default} = \max\left[\frac{\min\langle TD, tstop \rangle}{tstep}, 10\right]$.The .TRAN statement specifies the tstep and tstop values.
NPWL	Models symmetrical bidirectional switch/transfer gate, NMOS.
P0, P1 ...	The polynomial coefficients: • If you specify one coefficient, the PrimeSim Continuum solution assumes that it is P1 (P0 = 0.0), and the element is linear. • If you specify more than one polynomial coefficient, the element is nonlinear, and the coefficients are P0, P1, P2 ... (see Polynomial Functions on page 236).
POLE	Keyword to indicate that the transfer function is described by pole and zero locations.
POLY	Keyword for the polynomial dimension function. If you do not specify <i>POLY(ndim)</i> , the PrimeSim Continuum solution assumes that it is an one-dimensional polynomial. <i>Ndim</i> must be a positive number.
PWL()	Keyword for the piecewise linear function; must include parenthetical ().
PPWL	Models symmetrical bidirectional switch/transfer gate, PMOS.
SCALE	Multiplier for the element value.

Parameter	Description
SMOOTH	For piecewise-linear, dependent-source elements, SMOOTH selects the curve-smoothing method. See Turning off Smoothing. A curve-smoothing method simulates exact data points that you provide. You can use this method to make the PrimeSim Continuum solution simulates specific data points, which correspond to either measured data or data sheets. Choices for SMOOTH are 1 or 2: • Selects the smoothing method used in the PrimeSim Continuum solution versions before release H93A. Use this method to maintain compatibility with simulations that you ran using releases older than H93A. • Selects the smoothing method which uses data points that you provide. This is the default for the PrimeSim Continuum solution versions since release H93A.
TC1, TC2	First-order and second-order temperature coefficients. Temperature changes update the SCALE: $SCALE_{eff} = SCALE \cdot (1 + TC1 \cdot \Delta t + TC2 \cdot \Delta t^2)$.
EPWL ()	Keyword for the exponential piecewise linear function; must include the parenthesis (). Used after VCR keyword, representing one kind of exponential-ramping Voltage Controlled Resistor.
TD	Keyword for the time (propagation) delay.
transconductance	Voltage-to-current conversion factor.
transfactor	Voltage-to-resistance conversion factor.
VCCAP	Keyword for voltage-controlled capacitance element. VCCAP is the reserved PrimeSim Continuum solution keyword; do not use it as a node name.
VCCS	Keyword for the voltage-controlled current source. VCCS is the reserved PrimeSim Continuum solution keyword; do not use it as a node name.
VCR	Keyword for the voltage controlled resistor element. VCR is the reserved PrimeSim Continuum solution keyword; do not use it as a node name. The LAPLACE, FREQ and FOSTER forms of G elements do not require the VCR keyword.
x1, ...	Controlling voltage, across the <i>in+</i> and <i>in-</i> nodes. Specify the x values in increasing or decreasing order.
y1, ...	Corresponding element values of x.
LAPLACE	Laplace transform resolves a function into its moments (see Laplace Transform on page 186)
FREQ	Response table (see Frequency Response Table on page 188)
FOSTER	Pole residue form (see Foster Pole-Residue Form on page 189)
CTYPE	For G VCCAP: Determines capacitance charge calculation for elements with capacitance equations. If the C capacitance is a function of V(n1[,n2]), set CTYPE=0. Use this setting correctly, to ensure proper capacitance calculations, and correct simulation results. Default=0. Set CTYPE=1 for simulation results consistent with those from the PrimeSim XA simulator.

Parameter	Description
NOISEFILE	File name that contains noise data.
NOISE_CARRIER	Device name to calculate carrier power, used when noise unit is dbc/Hz in RF analyses.

For a list of output templates for voltage-controlled current source G-element models (HSPICE only), see the *Voltage-Controlled Current Source (G-element)* table in Element Template Listings in *HSPICE® User Guide: Basic Simulation and Analysis*.

G-element Examples

For a set of demonstration files for voltage-dependent current sources using G-elements, see the paths to the following example netlists in Behavioral Application Examples in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*:

- For PWL examples:
 - `switch.sp` (test for PWL switch element)
 - `swrc.sp` (switched capacitor RC circuit)
 - `pwl2.sp` (PPW-VCCS with a gain of 1 amp/volt)
 - `pwl4.sp` (eight-input NAND gate)
 - `ivx.sp` (characteristics of the PMOS and NMOS as a switch)
 - `vcob.sp` (voltage-controlled oscillator by using PWL functions)
- For a polynomial example:
 - `pll_bvp.sp` (PLL build with behavioral source)
- For NPWL/PPWL/NAND circuits:
 - `det_dff.sp` (double edge-triggered flip-flop)
 - `diode.sp` (behavioral diode by using a PWL VCCS)
 - `dlatch.sp` (CMOS D-latch by using behavioral models)
- For AND/NAND gates that uses the G-element:
 - `behave.sp`

For a description of the G-element parameters, see [G-element Parameters on page 217](#).

The following sections provide additional illustrative examples for the G-element:

- [Modeling Switches](#)
- [Switch-level MOSFET](#)
- [Runtime Current Source with Equation Containing Output Variable](#)
- [Voltage-controlled Capacitor](#)
- [Zero-delay Gate](#)
- [Delay Element](#)
- [Diode Equation](#)
- [Diode Breakdown](#)
- [Diode Lookup Table \(versus Model\)](#)
- [Triodes](#)
- [Behavioral Noise Model](#)
- [Turning off Smoothing](#)
- [Using Dependent Sources to Convert I to V and V to I](#)
- [Additional Full Demonstration Netlists Using G-Element](#)

Modeling Switches

You can model a switch to be open or closed based on simulation time or a pair of controlling nodes.

Switch Example 1: Time-varying switch—use the built-in function TIME to change the value of R from 0 (closed) to 100g ohm (open) when the simulator reaches time value T1:

```
R1 n1 n2 '100g*(TIME <= T1)'
```

As long as $TIME \leq T1$, the expression evaluates to zero and so does the resistor (switch) value. You could easily rewrite the resistor to switch from closed to open:

```
R1 n1 n2 '100g*(TIME >= T1)'
```

Switch Example 2: Voltage-controlled switch—use a voltage-controlled resistor and the PWL (piece-wise linear) function. The point-value pair represents the controlling input voltage and output resistance, respectively:

```
G_Switch n1 n2 VCR PWL(1) c1 c2 0v,100g 1v,1p
```

Where:

- $n1$ and $n2$ are the poles of the switch.
- $c1$ and $c2$ are the control nodes

In the following sample netlist, the switch is controlled by the PWL voltage source to switch at 1us:

```
* g-element switch
.option post
V_ctrl c1 0 PWL (0 0v .99u 0v 1u 1v)
G_Switch n1 n2 VCR PWL(1) c1 0 0v,100g 1v,1p
V_ref n1 0 10v
R_load n2 0 100
.tran .1u 2u
.end
```

Switch Example 3:

```
vc1 c1 c2 pw1 0n 1 5n 2
gs a gnd vcr epw1(1) c1 c2 v1,r1 v2,r2
```

Switch-level MOSFET

To model a switch-level n-channel MOSFET, use the N-piecewise linear resistance switch. The resistance value does not change when you switch the d and s node positions.

```
Gnmos d s VCR NPWL(1) g s LEVEL=1 0.4v,150g
+ 1v,10meg 2v,50k 3v,4k 5v,2k
```

For a full demonstration example, see the path to the `ac1.sp` netlist in Behavioral Application Examples in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

Runtime Current Source with Equation Containing Output Variable

The PrimeSim Continuum solution does not support a runtime output variable such as `v(gate)` in the example equation that follows. If the `.DATA` block has a runtime current source (I-element) where an equation contains runtime output variable such as `v(gate)`, as in this example equation:

```
I0 1 0 '(1-a0*v(gate))/b0'
vg gate 0 '(gt_v1)' $ (gt_v1)
```

Then the recommended method is to use the G-element:

```
g0 1 0 cur='((1-(a0*v(gate)))/b0)'
```

For a full demonstration example of runtime current (amplitude modulator with pulse waveform carrier), see the path to the `amp_mod.sp` netlist in Behavioral Application Examples in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*. See

also the path to `galgl.sp` for sampling a sine wave with a current source in Behavioral Application Examples in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

Voltage-controlled Capacitor

The capacitance value across the (*out*,0) nodes varies linearly (from 1 p to 5 p), when voltage across the *ctrl*,0 nodes varies between 2 and 2.5 V. The capacitance value remains constant at 1 picofarad when below the lower voltage limit, and at 5 picofarads when above the upper voltage limit:

```
Gcap out 0 VCCAP PWL(1) ctrl 0 2v,1p 2.5v,5p
```

Zero-delay Gate

To implement a two-input AND gate, use an expression and a piecewise linear table:

- The inputs are voltages at the *a* and *b* nodes.
- The output is the current flow from the *out* to 0 node.
- The PrimeSim Continuum solution multiplies the current by the `SCALE` value—which in this example, is the inverse of the load resistance, connected across the *out*,0 nodes:

```
Gand out 0 AND(2) a 0 b 0 SCALE='1/rload' 0v,0a 1v,.5a  
+ 4v,4.5a 5v,5a
```

Delay Element

A delay is a low-pass filter type delay, similar to that of an op-amp. In contrast, a transmission line has an infinite frequency response. A glitch input to a G-delay attenuates in a way that is similar to a buffer circuit. In this example, the output of the delay element is the current flow from the *out* node to the 1 node with a value equal to the voltage across the (*in*, 0) nodes, multiplied by the `SCALE` value, and delayed by the `TD` value:

```
Gdel out 0 DELAY in 0 TD=5ns SCALE=2 NPDELAY=25
```

For a full demonstration example of delay parameter (five-stage ring oscillator – macromodel CMOS inverter), see the path to the `ring5bm.sp` netlist.

Diode Equation

To model forward-bias diode characteristics from node 5 to ground, use a runtime expression. The saturation current is 1e-14 amp and the thermal voltage is 0.025 V:

```
Gdio 5 0 CUR='1e-14*(EXP(V(5)/0.025)-1.0)'
```


Diode Breakdown

You can model the diode breakdown region to a forward region. When voltage across a diode is above or below the piecewise linear limit values (–2.2 V, 2 V), the diode current remains at the corresponding limit values (–1a, 1.2a):

```
Gdiode 1 0 PWL(1) 1 0 -2.2v,-1a -2v,-1pa .3v,.15pa
+.6v,10ua 1v,1a 2v,1.2a
```

Diode Lookup Table (versus Model)

In the PrimeSim Continuum solution you can use the diode lookup-table values in a G-element (VCCS) as a PWL table. The following is a simple netlist using the lookup table values in a G-element:

```
.option post
Gdiode 1 0 PWL(1) 1 0 -2.2v,-1a -2v,-1pa .3v,.15pa
+ .6v,10ua 1v,1a 2v,1.2a
V1 1 0 2v
.op
.print i(*)
.tran 1n 10n
.end
```

The only limitation is that the maximum number of I-V value pairs for a PWL G-element is 100.

Triodes

Both of the following voltage-controlled current sources implement a basic triode:

- The first example uses the poly(2) operator to multiply the anode and grid voltages together and to scale by 0.02.
- The second example uses the explicit behavioral algebraic description:

```
gt i_anode cathode poly(2) anode,cathode
+ grid,cathode 0 0 0 0 .02
gt i_anode cathode
+ cur='20m*v(anode,cathode)*v(grid,cathode)'
```

Behavioral Noise Model

The following netlist shows a 1000-ohm resistor (g1) implemented using a G-element. The g1noise element, placed in parallel with the g1 resistor, delivers the thermal noise expected from a resistor. The following example includes the r1 resistor for comparison. The noise due to r1 should be the same as the noise due to g1noise:

```
* Resistor implemented using g-element
v1 1 0 1
r1 1 2 1k
g1 1 2 cur='v(1,2)*0.001'
```

```
glnoise 1 2 noise='sqrt(4*1.3806266e-23*(TEMPER+273.15)*0.001) '
rout 2 0 1meg
.ac lin 1 100 100
.noise v(2) v1 1
.end
```

Turning off Smoothing

By default, a PWL controlled source performs smoothing on corners which may introduce some error. To turn off the smoothing, use `DELTA=0` in the G-element definition:

```
Gtest BB 0 VCCS PWL(1) B E 0,0 0.1,0 1,1 2,2 DELTA=0
```

You also can set `DELTA` to a small value to achieve some smoothing that can help convergence, but reduce the difference from the ideal PWL.

Using Dependent Sources to Convert I to V and V to I

You can convert voltage to current using the G-element and current to voltage by using the H-element (see [Current-controlled Voltage Source \(CCVS\)—H-Element](#)). To use the H-element for a I to V conversion, you also must use a dummy voltage source. For example:

```
* I-V and V-I conversion
.option post list node
*** I-V Conversion ****
I_in      p1 0 1
vdummy    p2 p1 0 $ dummy voltage source
H_in      p2 0 CCVS vdummy 1
R_fc      p2 p3 10
C_fc      p3 0 500fF
*** V-I Conversion ****
G_in      p4 p5 VCCS p3 0 1
Rs        p5 p6 1
C1        p4 p5 500fF
C2        p4 p6 500fF
.tran 10p 1n
.probe tran v(p?) i(g*)
.end
```

Additional Full Demonstration Netlists Using G-Element

For an additional set of full demonstration files for voltage-dependent current sources using G-element parameters examples, see the paths to the example netlists as follows in Behavioral Application Examples in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*:

- `pdp.sp` (phase detector by using behavioral NAND gates Multi-Input NAND).
- `ringb.sp` (ring oscillator by using behavioral model—NAND).
- `vcob.sp` (voltage-controlled oscillator by using PWL functions—PWL/NAND).

- `rtest.sp` (voltage-controlled resistor, inverter chain—Level=1).
- `vcob.sp` (voltage-controlled oscillator by using PWL functions).
- See Filters Examples in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis* (`lcline.sp`) for a demonstration of a G-element as a filter.

See also: Using G- and E-elements in *PrimeSim HSPICE® User Guide: Advanced Analog Simulation and Analysis*.

Current-dependent Current Sources - F-elements

This section explains the F-element syntax and parameters.

Note:

G-elements with algebraic make F-elements obsolete. You can still use F-elements for backward-compatibility with existing designs.

The following sections discuss these topics:

- [Current-controlled Current Source \(CCCS\)](#)
- [F-element Examples](#)

Current-controlled Current Source (CCCS)

The following sections discuss these topics:

- [Linear](#)
- [Polynomial \(POLY\)](#)
- [PWL](#)
- [Multi-Input Gates](#)
- [Delay Element](#)
- [F-element Parameters](#)

Linear

```
Fxxx n+ n- [CCCS] vn1 gain [MAX=val] [MIN=val] [SCALE=val]  
+ [TC1=val] [TC2=val] [M=val] [ABS=1] [IC=val]
```

For a description of these parameters, see [F-element Parameters on page 228](#).

Polynomial (POLY)

```
Fxxx n+ n- [CCCS] POLY(ndim) vn1 [... vnndim] [MAX=val]
+ [MIN=val] [TC1=val] [TC2=val] [SCALE=val] [M=val]
+ [ABS=1] p0 [p1...] [IC=val]
```

In this syntax, *dim (dimensions)* ≤ 3.

For a description of these parameters, see [F-element Parameters on page 228](#). For a description of possible POLY syntaxes, see [Polynomial Functions on page 236](#).

PWL

```
Fxxx n+ n- [CCCS] PWL(1) vn1 [DELTA=val] [SCALE=val]
+ [TC1=val] [TC2=val] [M=val] x1,y1 ... x100,y100 [IC=val]
```

For a description of these parameters, see [F-element Parameters on page 228](#). For a detailed description of PWL functions, see [Piecewise Linear Function on page 241](#).

Examples

For a full demo file example, go to Behavioral Application Examples in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis* and take the path to the `pw18.sp` file (smoothing the triangle waveform by using the PWL CCCS).

Multi-Input Gates

```
Fxxx n+ n- [CCCS] gatetype(k) vn1, ... vnk [DELTA=val]
+ [SCALE=val] [TC1=val] [TC2=val] [M=val] [ABS=1]
+ x1,y1 ... x100,y100 [IC=val]
```

In this syntax, *gatetype* can represent an AND, NAND, OR, or NOR gate and *k* represents the number of inputs to the gate. For a description of these parameters, see [F-element Parameters on page 228](#).

Delay Element

```
Fxxx n+ n- [CCCS] DELAY vn1 TD=val [SCALE=val]
+ [TC1=val] [TC2=val] NPDELAY=val
```

For a description of these parameters, see [F-element Parameters on page 228](#).

F-element Parameters

The following list describes the F-element parameters:

Parameter	Description
ABS	Output is an absolute value, if ABS = 1.

Parameter	Description
CCCS	Keyword for current-controlled current source. CCCS is the PrimeSim Continuum solution reserved keyword; do not use it as a node name.
DELAY	Keyword for the delay element. Same as for a current-controlled current source, but has an associated propagation delay, TD. Adjusts the propagation delay in the macro model (subcircuit) process. DELAY is a reserved word; do not use it as a node name.
DELTA	Controls the curvature of piecewise linear corners. Default, delta is computed automatically when not set. The maximum is 1/2 of the smallest distance between breakpoints.
Fxxx	Element name of the current-controlled current source. Must begin with F, followed by up to 1023 alphanumeric and special characters.
gain	Current gain.
gatetype (k)	Where gatetype can be AND, NAND, OR, or NOR and the number of inputs of the gate is represented by k. For multi-input gates, only one input determines the state of the output. Note: Do not use the above keywords as node names.
IC	Initial condition (estimate) of the controlling current(s), in amps. If you do not specify IC, the default = 0.0.
M	Number of replications of the element in parallel.
MAX	Maximum output current. Default=undefined; sets no maximum.
MIN	Minimum output current. Default = undefined; sets no minimum.
n+/-	Connecting nodes for a positive or negative controlled source.
NDIM	Number of polynomial dimensions. If you do not specify POLY(NDIM), the PrimeSim Continuum solution assumes a one-dimensional polynomial. NDIM must be a positive number.
NPDELAY	Number of data points to use in delay simulations. The default value is the larger of either 10 or the smaller of TD/tstep and tstop/tstep. That is, $NPDELAY_{default} = \max\left[\frac{\min\langle TD, tstop \rangle}{tstep}, 10\right]$ The .TRAN statement specifies the tstep and tstop values.
P0, P1 ...	The polynomial coefficients. If you specify one coefficient, the PrimeSim Continuum solution assumes it is P1 (P0=0.0), and the source element is linear. If you specify more than one polynomial coefficient, then the source is nonlinear, and the PrimeSim Continuum solution assumes that the polynomials are P0, P1, P2 ... See Polynomial Functions on page 236 .
POLY	Keyword for the polynomial function. If you do not specify POLY(ndim), the PrimeSim Continuum solution assumes that this is a one-dimensional polynomial. Ndim must be a positive number.
PWL	Keyword for the piecewise linear function.

Parameter	Description
SCALE	Multiplier for the element value.
TC1, TC2	First-order and second-order temperature coefficients. Temperature changes update the SCALE: $SCALE_{eff} = SCALE \cdot (1 + TC1 \cdot \Delta t + TC2 \cdot \Delta t^2)$
TD	Keyword for the time (propagation) delay.
vn1 ...	Names of voltage sources, through which the controlling current flows. Specify one name for each dimension.
x1, ...	Controlling current, through the vn1 source. Specify the x values in increasing or decreasing order.
y1, ...	Corresponding output current values of x.

For a listing of output templates for current-controlled current source F-element (PrimeSim HSPICE only) models, see the *Current-Controlled Current Source (F-element)* table in Element Template Listings in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

F-element Examples

Example 1

This example describes a current-controlled current source connected between nodes 13 and 5. The current which controls the value of the controlled source flows through the voltage source named VSENS.

```
F1 13 5 VSENS MAX=+3 MIN=-3 5
```

Note:

To use a current-controlled current source, you can place a dummy independent voltage source into the path of the controlling current.

The defining equation is:

Equation 33

$$I(F1) = 5 \cdot I(VSENS)$$

- Current gain is 5.
- Maximum current flow through F1 is 3 A.
- Minimum current flow is -3 A.

If $I(VSENS)=2$ A, then this examples sets $I(F1)$ to 3 amps, not 10 amps (as the equation suggests). You can define a parameter for the polynomial coefficient(s):

```
.PARAM VU=5  
F1 13 5 VSENS MAX=+3 MIN=-3 VU
```

Example 2

This example is a current-controlled current source, with the value:

$$I(F2)=1e-3 + 1.3e-3 \cdot I(VCC)$$

Current flows from the positive node, through the source, to the negative node. The positive controlling-current flows from the positive node, through the source, to the negative node of *vnam* (linear), or to the negative node of each voltage source (nonlinear).

```
F2 12 10 POLY VCC 1MA 1.3M
```

Example 3

This example is a delayed, current-controlled current source.

```
Fd 1 0 DELAY vin TD=7ns SCALE=5
```

Example 4

This example is a piecewise-linear, current-controlled current source.

```
Filim 0 out PWL(1) vsrc -1a,-1a 1a,1a
```

Current-dependent Voltage Sources - H-elements

This section explains H-element syntax statements and defines their parameters.

Note:

E-elements with algebraics make H-elements obsolete. You can still use H-elements for backward-compatibility with existing designs.

For a listing of output templates for current-controlled voltage source (H-element) models (the PrimeSim Continuum solution only), see the *Current-Controlled Voltage Source (H-element)* table in Element Template Listings in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

The following sections introduce the topics for the CCVS source:

- [Current-controlled Voltage Source \(CCVS\)—H-Element](#)
- [H-element Examples](#)

Current-controlled Voltage Source (CCVS)—H-Element

The following sections discuss these topics:

- [Linear](#)
- [Polynomial \(POLY\)](#)
- [PWL](#)
- [Multi-Input Gate](#)
- [Delay Element](#)
- [H-element Parameters](#)

Linear

```
Hxxx n+ n- [CCVS] vn1 transresistance [MAX=val] [MIN=val]  
+ [SCALE=val] [TC1=val] [TC2=val] [ABS=1] [IC=val]
```

Polynomial (POLY)

```
Hxxx n+ n- [CCVS] POLY(NDIM) vn1 [... vnndim]  
+ [MAX=val] [MIN=val] [TC1=val] [TC2=val] [SCALE=val]  
+ [ABS=1] P0 [P1...] [IC=val]
```

For a description of possible POLY syntaxes, see [Polynomial Functions on page 236](#).

Note:

If `POLY(NDIM)` is not specified, it is treated as `POLY(1)`.

PWL

```
Hxxx n+ n- [CCVS] PWL(1) vn1 [DELTA=val] [SCALE=val]  
+ [TC1=val] [TC2=val] x1,y1 ... x100,y100 [IC=val]
```

For a detailed description of PWL functions, see [Piecewise Linear Function on page 241](#).

Multi-Input Gate

```
Hxxx n+ n- [CCVS] gatetype(k) vn1, ...vnk [DELTA=val] [SCALE=val]  
+ [TC1=val] [TC2=val] x1,y1 ... x100,y100 [IC=val]
```

In this syntax, *gatetype* can represent an AND, NAND, OR, or NOR gate and *k* represents the number of inputs to the gate.

Delay Element

Hxxx n+ n- [CCVS] DELAY vn1 TD=val [SCALE=val] [TC1=val]
+ [TC2=val] [NPDELAY=val]

H-element Parameters

The following list describes the H-element parameters:

Parameter	Description
ABS	Output is an absolute value, if ABS = 1.
CCVS	Keyword for the current-controlled voltage source. CCVS is the PrimeSim Continuum solution reserved keyword; do not use it as a node name.
DELAY	Keyword for the delay element. Same as for a current-controlled voltage source, but has an associated propagation delay, TD. Use this element to adjust the propagation delay in the macro (subcircuit) model process. DELAY is the PrimeSim Continuum solution reserved keyword; do not use it as a node name.
DELTA	Controls curvature of piecewise linear corners. Default, delta is computed automatically when not set. Maximum is 1/2 of the smallest distance between breakpoints.
gatetype (k)	Where gatetype can be AND, NAND, OR, or NOR and the number of inputs of the gate is represented by k. For multi-input gates, only one input determines the state of the output. Note: Do not use the above keywords as node names.
Hxxx	Element name of current-controlled voltage source. Must start with H followed by up to 1023 alphanumeric and special characters.
IC	Initial condition (estimate) of the controlling current(s), in amps. If you do not specify IC, the default = 0.0.
MAX	Maximum voltage. Default is undefined; sets no maximum.
MIN	Minimum voltage. Default is undefined; sets no minimum.
n+/-	Connecting nodes for positive or negative controlled source.
NDIM	Number of polynomial dimensions. If you do not specify POLY(NDIM), the PrimeSim Continuum solution assumes a one-dimensional polynomial. NDIM must be a positive number.
NPDELAY	Number of data points to use in delay simulations. The default value is the larger of either 10, or the smaller of TD/tstep and tstop/tstep. That is: $NPDELAY_{default} = \max \left[\frac{\min \langle TD, tstop \rangle}{tstep}, 10 \right]$.The .TRAN statement specifies the tstep and tstop values.

Parameter	Description
<i>P0, P1 ...</i>	Polynomial coefficients: • If you specify one polynomial coefficient, the source is linear, and the PrimeSim Continuum solution assumes that the polynomial is P1 (P0 = 0.0). • If you specify more than one polynomial coefficient, the source is nonlinear. PrimeSim HSPICE assumes the polynomials are P0, P1, P2 ... See Polynomial Functions on page 236.
POLY	Keyword for polynomial dimension function. If you do not specify <i>POLY(ndim)</i> , the PrimeSim Continuum solution assumes a one-dimensional polynomial. <i>Ndim</i> must be a positive number.
PWL	Keyword for a piecewise linear function.
SCALE	Multiplier for the element value.
<i>TC1, TC2</i>	First-order and second-order temperature coefficients. Temperature changes update the SCALE: $SCALE_{eff} = SCALE \cdot (1 + TC1 \cdot \Delta t + TC2 \cdot \Delta t^2)$
TD	Keyword for the time (propagation) delay.
transresistance	Current-to-voltage conversion factor.
<i>vn1 ...</i>	Names of voltage sources, through which controlling current flows. You must specify one name for each dimension.
<i>x1, ...</i>	Controlling current, through the <i>vn1</i> source. Specify the x values in increasing or decreasing order.
<i>y1, ...</i>	Corresponding output voltage values of x.

H-element Examples

This section provides illustrative examples for the H-element.

Example 1

The following example selects a linear current-controlled voltage source. The controlling current flows through the dependent voltage source, called VCUR:

```
HX 20 10 VCUR MAX=+10 MIN=-10 1000
```

Example 2

The defining equation of the C CVS is:

Equation 34

$$HX = 1000 \cdot I(VCUR)$$

The defining equation specifies that the voltage output of HX is 1000 times the value of the current flowing through VCUR:

- If the equation produces a value of HX greater than +10 V, the **MAX** parameter sets HX to 10 V.
- If the equation produces a value of HX less than -10 V, the **MIN** parameter sets HX to -10 V.

VCUR is the name of the independent voltage source through which the controlling current flows. If the controlling current does not flow through an independent voltage source, you must insert a dummy independent voltage source.

Example 3

The following example describes a dependent voltage source:

```
.PARAM CT=1000  
HX 20 10 VCUR MAX=+10 MIN=-10 CT  
HXY 13 20 POLY(2) VIN1 VIN2 0 0 0 0 1 IC=0.5, 1.3
```

The value is as follows:

Equation 35

$$V = I(VIN1) \cdot I(VIN2)$$

This two-dimensional polynomial equation specifies:

- FA1=VIN1
- FA2=VIN2
- P0=0
- P1=0
- P2=0
- P3=0
- P4=1

The initial controlling current is .5 mA through VIN1, and 1.3 mA for VIN2.

Positive controlling current flows from the positive node through the source to the negative node of vnam (linear). The (nonlinear) polynomial specifies the source voltage as a function of the controlling currents.

Example 4: CCVS as the Controlling Element for a CCCS

A CCVS can be used as the controlling element for a CCCS. See the following example:

```
v4 e3_m gnd! dc=0
v2 v2_p e3_m dc=2 ac=3 sin ( 0 2 5e+6 0 0 0 )
h15 h15_p gnd! ccvs v2 2 abs=0
f18 f18_p gnd! cccs h15 1 abs=0
```

For full demonstration examples of H-element PWL netlist files, go to Behavioral Application Examples in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis* and follow the paths to these two netlists:

- `op_amp.sp` (OPAMP from Chua and Lin)
- `pwl10.sp` (OPAMP used as a voltage follower)

Polynomial and Piecewise Linear Functions

This section describes the polynomial and piecewise linear functions. For full PWL examples, see PWL/DATA/VEC Converter in *PrimeSim HSPICE® User Guide: Basic Simulation and Analysis*.

The following sections discuss these topics:

- [Polynomial Functions](#)
- [Piecewise Linear Function](#)
- [Exponential Piecewise Linear Function](#)

Polynomial Functions

You can use the controlled element statement to define the controlled output variable (current, resistance, or voltage), as a polynomial function of one or more voltages or branch currents. You can select several polynomial equations, by using the `POLY (NDIM)` parameter in the E, F, G, or H-element statement. Syntax can be either `POLY=INTEGER_NUMBER` or `POLY ( INTEGER_NUMBER )`

For example, either of the following are legitimate statements for an E-element instance with the `POLY` function:

```
E1 e1 0 POLY=2 e11 0 e12 0 1 2 3
```

or:

```
E1 e1 0 POLY(2) e11 0 e12 0 1 2 3
```

Polynomial values can be:

Value	Description
POLY(1)	One-dimensional equation (function of one controlling variable).
POLY(2)	Two-dimensional equation (function of two controlling variables).
POLY(3)	Three-dimensional equation (function of three controlling variables).
POLY(n)	Multi-dimensional equation (function of n controlling variables). The PrimeSim Continuum solution advanced analog analyses has a maximum allowable dimension limit of 3.

Each polynomial equation includes polynomial coefficient parameters (P0, P1 ... Pn), which you can set to explicitly define the equation.

The following sections discuss these topics:

- [One-Dimensional Function](#)
- [Two-Dimensional Function](#)
- [Three-Dimensional Function](#)
- [N-Dimensional Function](#)

One-Dimensional Function

If the function is one-dimensional (a function of one branch current or node voltage), the following expression determines the FV function value:

Equation 36

$$FV = P0 + (P1 \cdot FA) + (P2 \cdot FA^2) + (P3 \cdot FA^3) + (P4 \cdot FA^4) + (P5 \cdot FA^5) + \dots$$

Parameter	Description
FV	Controlled voltage or current from the controlled source.
P0 . . . PN	Coefficients of a polynomial equation.
FA	Controlling branch current, or nodal voltage.

Note:

If you specify one coefficient in a one-dimensional polynomial, the PrimeSim Continuum solution assumes that the coefficient is P1 (P0=0.0). Use this as input for linear controlled sources.

The following controlled source statement is a one-dimensional function. This voltage-controlled voltage source connects to nodes 5 and 0.

```
E1 5 0 POLY(1) 3 2 1 2.5
```

In the above source statement, the single-dimension polynomial function parameter, `POLY(1)`, informs the PrimeSim Continuum solution that E1 is a function of the difference of one nodal voltage pair. In this example, the voltage difference is between nodes 3 and 2, so $F_A = V(3, 2)$.

The dependent source statement then specifies that $P_0=1$ and $P_1=2.5$. From the one-dimensional polynomial equation above, the defining equation for $V(5,0)$ is:

Equation 37

$$V(5, 0) = 1 + 2.5 \cdot V(3, 2)$$

You can also express $V(5,0)$ as *E1*:

Equation 38

$$E1 = 1 + 2.5 \cdot V(3, 2)$$

Two-Dimensional Function

If the function is two-dimensional (that is, a function of two node voltages or two branch currents), the following expression determines FV :

Equation 39

$$FV = P_0 + (P_1 \cdot F_A) + (P_2 \cdot F_B) + (P_3 \cdot F_A^2) + (P_4 \cdot F_A \cdot F_B) + (P_5 \cdot F_B^2) + (P_6 \cdot F_A^3) + (P_7 \cdot F_A^2 \cdot F_B) + (P_8 \cdot F_A \cdot F_B^2) + (P_9 \cdot F_B^3) + \dots$$

For a two-dimensional polynomial, the controlled source is a function of two nodal voltages or currents. To specify a two-dimensional polynomial, set `POLY(2)` in the controlled source statement.

For example, generate a voltage-controlled source that specifies the controlled voltage, $V(1,0)$, as:

Equation 40

$$V(1, 0) = 3 \cdot V(3, 2) + 4 \cdot V(7, 6)^2$$

or:

Equation 41

$$E1 = 3 \cdot V(3, 2) + 4 \cdot V(7, 6)^2$$

To implement this function, use this controlled-source element statement:

```
E1 1 0 POLY(2) 3 2 7 6 0 3 0 0 0 4
```

This example specifies a controlled voltage source, which connects between nodes 1 and 0. Two differential voltages control this voltage source:

- Voltage difference between nodes 3 and 2.
- Voltage difference between nodes 7 and 6.

That is, $F_A = V(3, 2)$, and $F_B = V(7, 6)$. The polynomial coefficients are:

- $P_0 = 0$
- $P_1 = 3$
- $P_2 = 0$
- $P_3 = 0$
- $P_4 = 0$
- $P_5 = 4$

Three-Dimensional Function

For a three-dimensional polynomial function, with F_A , F_B , and F_C as its arguments, the following expression determines the F_V function value:

Equation 42

$$F_V = P_0 + (P_1 \cdot F_A) + (P_2 \cdot F_B) + (P_3 \cdot F_C) + (P_4 \cdot F_A^2)$$

For example, generate a voltage-controlled source that specifies the voltage as:

Equation 43

$$V(1, 0) = 3 \cdot V(3, 2) + 4 \cdot V(7, 6)^2 + 5 \cdot V(9, 8)^3$$

or:

Equation 44

$$E1 = 3 \cdot V(3, 2) + 4 \cdot V(7, 6)^2 + 5 \cdot V(9, 8)^3$$

The resulting three-dimensional polynomial equation is:

$$F_A = V(3, 2)$$

$$F_B = V(7, 6)$$

$$FC = V(9, 8)$$

$$P1 = 3$$

$$P7 = 4$$

$$P19 = 5$$

Substitute these values into the voltage controlled voltage source statement:

```
E1 1 0 POLY(3) 3 2 7 6 9 8 0 3 0 0 0 0 0 4 0 0 0 0 0 0
+ 0 0 0 0 0 0 5
```

The preceding example specifies a controlled voltage source, which connects between nodes 1 and 0. Three differential voltages control this voltage source:

- Voltage difference between nodes 3 and 2.
- Voltage difference between nodes 7 and 6.
- Voltage difference between nodes 9 and 8.

That is:

- $FA = V(3, 2)$
- $FB = V(7, 6)$
- $FC = V(9, 8)$

The statement defines the polynomial coefficients as:

- $P1 = 3$
- $P7 = 4$
- $P19 = 5$
- Other coefficients are zero.

N-Dimensional Function

Express an N-dimensional polynomial function as:

Equation 45

$$FV = p_0 + \sum_{j=1}^k (p_{ij} Fx_1 + p_{2j} Fx_2 + \dots + p_{ij} Fx_k)^j$$

where, $Fx_1, Fx_2, \dots, F_k$, represent the k independent controlling branch current, or nodal voltage, and p_{ij} , $i = 1, 2, \dots, k = 1, 2, \dots, n$ are the coefficients.

Piecewise Linear Function

You can use the one-dimensional piecewise linear (PWL) function to model special element characteristics, such as those of:

- Tunnel diodes
- Silicon-controlled rectifiers
- Diode breakdown regions

To describe the piecewise linear function, specify measured data points. Although data points describe the device characteristic, the PrimeSim Continuum solution automatically smooths the corners, to ensure derivative continuity. This, in turn, results in better convergence.

The `DELTA` parameter controls the curvature of the characteristic at the corners. The smaller the `DELTA`, the sharper the corners are. The maximum `DELTA` limits to half of the smallest breakpoint distance. If the breakpoints have sufficient separation, specify the `DELTA` to a proper value.

- You can specify up to 100 point pairs.
- You must specify at least two point pairs (each point consists of an x and a y coefficient).

To model bidirectional switch or transfer gates, G-elements use the `NPWL()` and `PPWL()` functions, which behave the same way as NMOS and PMOS transistors.

You can also use the piecewise linear function to model multi-input AND, NAND, OR, and NOR gates. In this usage, only one input determines the state of the output.

- In AND and NAND gates, the input with the smallest value determines the corresponding output of the gates.
- In OR and NOR gates, the input with the largest value determines the corresponding output of the gates.

Exponential Piecewise Linear Function

In the PrimeSim Continuum solution, the `EPWL` keyword is used after the `VCR` keyword, in a G-element syntax, to simulate/represent a behavioral resistor. For the complete syntax, see [EPWL on page 212](#).

For example, the following syntax:

```
gs a gnd vcr epwl(1) c1 c2 v1,r1 v2,r2
```

represents the following resistor:

```
R='max(exp(min(p1,p2)), min(exp(max(p1,p2)), exp(p1 + (V(c1,c2) - v1)*(p2 - p1)/(v2 - v1))))'
```

where:

$p1 = \log(r1)$ and

$p2 = \log(r2)$.

4

Modeling Input/Output Buffers Using IBIS Files

Describes the IBIS syntaxes for input/output buffers and buffer types.

The following sections discuss these topics:

- [IBIS Syntax Conventions for I/O Buffers](#)
- [Buffer Types](#)
- [Examples](#)
- [Specifying Required and Optional Common Keywords](#)

IBIS Syntax Conventions for I/O Buffers

The general syntax of an element card for I/O buffers is:

```
Bxxx node_1 node_2 ... node_N
+ file='filename' model='model_name'
+ keyword_1=value_1 ... [keyword_M=value_M]
+ M=num
```

Parameter	Description
Bxxx	Buffer element name. The name must begin with B, which can be followed by up to 1023 alphanumeric characters.
node_1 node_2 ... node_N	List of I/O buffer external nodes. The number of nodes and corresponding rules are specific to different buffer types.
file=filename	Name of the IBIS file.
model=model_name	Name of the model.
keyword_i=value_i	Assigns the <i>value_i</i> value to the <i>keyword_i</i> keyword. Specify optional keywords in brackets (<i>[]</i>) (see Specifying Required and Optional Common Keywords on page 271 for more information).
M=num	Multiplier: Instance parameter where <i>num</i> is a positive integer.

The gnd node on the circuit diagram for buffers denotes the ideal SPICE ground node (the node 0 [zero] notation is also used). This node is always available in the simulation device models. Do not include this node in the node list on the buffer card. If the gnd node appears on a circuit diagram, simulation connects the node to the ideal ground. The gnd node on circuit diagrams explains the connection of individual parts inside buffers.

In some cases, buffer nodes have different rules than nodes for other elements. Some nodes might already be connected to voltage sources (simulation makes such connections) so do not connect a voltage source to such nodes. Conversely, some nodes should be connected to voltage sources and you need to connect voltage sources to these nodes.

Note:

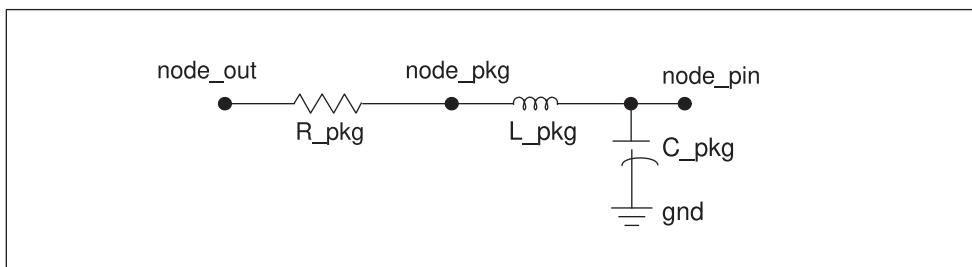
See [Specifying Required and Optional Common Keywords on page 271](#) and the sections about individual buffer types for detailed explanations on how to use these nodes.

Buffers correspond to models in IBIS files and do not include packages. In this case, you need to manually add the corresponding packages. For example, if node_out and node_pin are nodes for output of the output buffer and corresponding pin, then add the following lines to the netlist:

```
R_pkg node_out node_pkg R_pkg_value  
L_pkg node_pkg node_pin L_pkg_value  
C_pkg node_pin gnd C_pkgvalue
```

The preceding lines use the IBIS file to find the R_pkg, L_pkg, and C_pkg values (see [Figure 20 on page 244](#) for the circuit diagram).

Figure 20 *Circuit Diagram for Package*



The keyword `pkg_fork` for the `.ibis` command selects the forked section connection of pkg model. The syntax is:

```
pkg_fork = fork (default) | branch
```

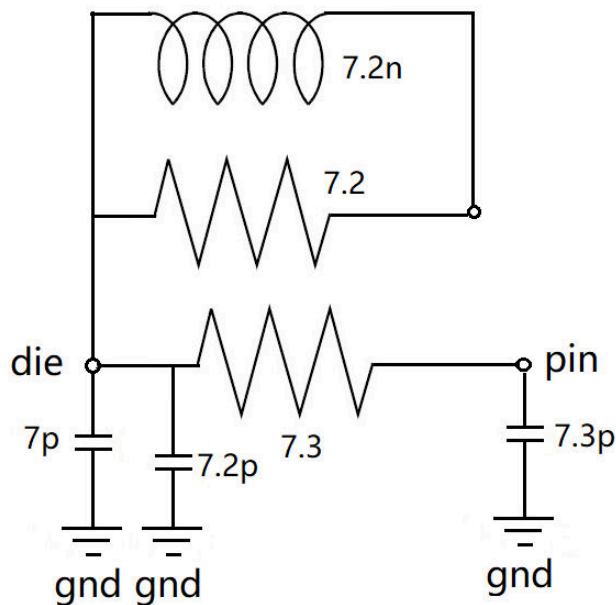
When `pkg_fork= fork`, the forked section is connected in parallel mode. By default it is `fork`. If `pkg_fork=branch`, then the forked section is created as new branch with dangling node.

Consider the following example for the PKG model,

```
B7 Len=0 C=7pF/  
Fork  
Len=0 L=7.2nH / Len=0 R=7.2 / Len=0 C=7.2pF /  
Endfork  
Len=0 R=7.3/ Len=0 C=7.3pF/
```

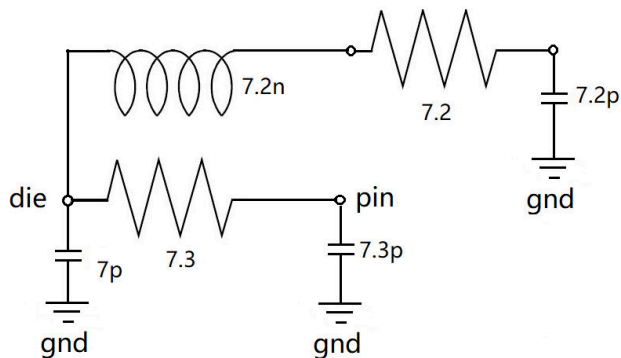
For pin B7, the package between die pad and pin is connected as below [Figure 21](#) which is default.

Figure 21 PKG Model With Default Settings



If `pkg_fork=branch`, then the package is connected as shown in [Figure 22](#).

Figure 22 *PKG Model With Branch*



Buffer Types

This section describes buffers as used in the Synopsys IBIS device models. Refer to [Specifying Required and Optional Common Keywords on page 271](#) for details about how to use keywords that are in the syntax examples in the following sections.

The following sections discuss these topics:

- [Input Buffer](#)
- [Output Buffer](#)
- [Tristate Buffer](#)
- [Input/Output Buffer](#)
- [Open Drain, Open Sink, Open Source Buffers](#)
- [I/O Open Drain, I/O Open Sink, I/O Open Source Buffers](#)
- [Input ECL Buffer](#)
- [Output ECL Buffer](#)
- [Tristate ECL Buffer](#)
- [Input-Output ECL Buffer](#)
- [Terminator Buffer](#)
- [Series Buffer](#)
- [Series Switch Buffer](#)
- [Multilingual Model Support](#)

Input Buffer

The syntax of an input buffer element card is:

```
B_INPUT nd_pc nd_gc nd_in nd_out_of_in
+ file='filename' model='model_name'
+ [typ=typ|min|max|fast|slow] [power=on|off]
+ [buffer=1|input]
+ [interpol=1|2]
+ [nowarn]
+ [c_com_pc=c_com_pc_value]
+ [c_com_gc=c_com_gc_value]
+ [pc_scal=pc_scal_value]
+ [gc_scal=gc_scal_value]
```

In the preceding syntax, the total number of external nodes is 4.

If you specify the `power=on` keyword (default), the `nd_pc` and `nd_gc` nodes are connected to voltage sources with values taken from the IBIS file. Such voltage sources are created internally by the PrimeSim Continuum solution. So do *not* connect the nodes of `nd_pc` and `nd_gc` to other voltage sources. Names for these nodes are provided for you to print out the voltage values, if required.

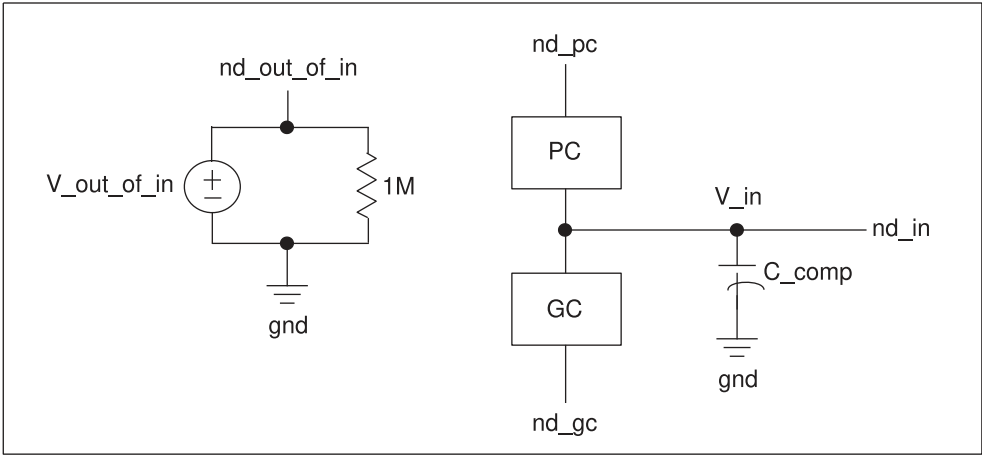
For example:

```
.PRINT V(nd_pc) V(nd_gc)
```

If you specify the `power=off` keyword, the simulation does not connect these nodes to voltage sources. You need to connect the nodes to voltage sources directly through an RLC network, or through a transmission line.

You can connect node `nd_in` to I, E, F, G, and H -elements. The buffer measures and processes the voltage on this node and sends a response to the `nd_out_of_in` node. The `nd_out_of_in` node is connected to the voltage source as shown in [Figure 23](#). It is an error to connect this node to a voltage source. If `power=off`, you can connect the `nd_pc` and `nd_gc` nodes to the ground, which sets the voltage to zero on these nodes.

Figure 23 *Input Buffer*



V_out_of_in is a digital signal that assumes values of either 0, 0.5, or 1 depending on the V_in, Vinh, Vinl, and Polarity voltages. The simulation processes V_out_of_in according to the following rules:

Table 15 *IBIS Input Buffer*

If:	Then:
Polarity=Non-Inverting	Initially V_out_of_in is set to 0 if V_in < (Vinh+Vinl)/2 and to 1 in the opposite case.
V_in>Vinh	V_out_of_in is set to 1
V_in<Vinl	V_out_of_in is set to 0
else Vinl<=V_in<=Vinh	V_out_of_in is set to 0.5
Polarity=Inverting	Initially V_out_of_in is set to 0 if V_in > (Vinh+Vinl)/2 and to 1 in the opposite case
V_in>Vinh	V_out_of_in is set to 0
V_in<Vinl	V_out_of_in is set to 1
else Vinl<=V_in<=Vinh	V_out_of_in is set to 0.5

Figure 23 shows a single circuit specified on a single element card. V_out_of_in is a voltage source whose value is a function of V_in (and of Vinl, Vinh thresholds, and Polarity parameter). You can use it to drive other circuits.

If you specify pc_scal or gc_scal arguments, and pc_scal_value or gc_scal_value do not equal 1.0, then the PrimeSim Continuum solution uses the pc_scal_value or gc_scal_value to adjust the PC or GC I/V curve.

Output Buffer

The syntax for an output buffer element card is:

```
B_OUTPUT nd_pu nd_pd nd_out nd_in [nd_pc nd_gc]
+ file='file_name' model='model_name'
+ [typ=typ|min|max|fast|slow] [power=on|off]
+ [buffer=2|output]
+ [xv_pu=state_pu] [xv_pd=state_pd]
+ [interpol=1|2]
+ [ramp_fwf=0|1|2] [ramp_rwf=0|1|2]
+ [fwf_tune=fwf_tune_value] [rwf_tune=rwf_tune_value]
+ [nowarn]
+ [c_com_pu=c_com_pu_value]
+ [c_com_pd=c_com_pd_value]
+ [c_com_pc=c_com_pc_value]
+ [c_com_gc=c_com_gc_value]
+ [pu_scal=pu_scal_value]
+ [pd_scal=pd_scal_value]
+ [pc_scal=pc_scal_value]
+ [gc_scal=gc_scal_value]
+ [rwf_scal=rwf_scal_value]
+ [fwf_scal=fwf_scal_value]
+ [spu_scal=spu_scal_value]
+ [spd_scal=spd_scal_value]
```

The *nd_pc* and *nd_gc* nodes are optional. However, you can specify both nodes or none of them. The total number of external nodes is either 4 or 6; any other number is an error. If you do not specify the *nd_pc* and *nd_gc* nodes on the element card, but Power_Clamp or Ground_Clamp I/V curves are present in the model in question, then the simulator simply connects Power_Clamp or Ground_Clamp to the corresponding *nd_pu* (pullup) or *nd_pd* (pulldown).

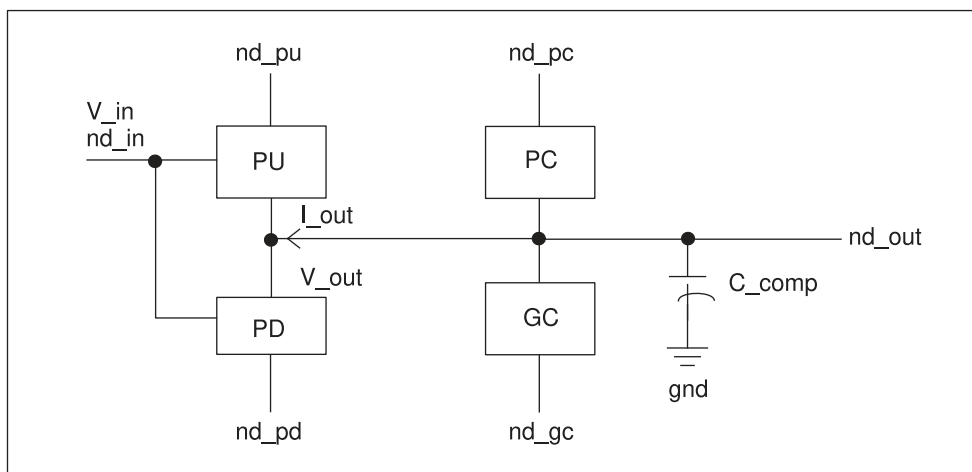
However, you need the optional *nd_pc* and *nd_gc* nodes if:

- The POWER Clamp Reference and GND Clamp Reference IBIS keywords are present in the IBIS model and have different values than the IBIS keywords Pullup Reference and Pulldown Reference, or
- The Pullup Reference and Pulldown Reference IBIS keywords do not exist and POWER Clamp Reference and GND Clamp Reference have different values than those determined by the Voltage Range IBIS keyword.

If your circuit needs the *nd_pc* and *nd_gc* optional nodes, but they are not in the element card, the simulation connects *nd_pc* to *nd_pu* and *nd_gc* to *nd_pd* and issues this warning:

```
**warning** nodes of pc and pu(or gc and pd) are merged, although they
have different reference voltage!
```

Figure 24 Output Buffer



If you specify the `power=on` (default) keyword, then the `nd_pu` and `nd_pd` nodes, and if specified, the `nd_pc` and `nd_gc` nodes, are connected to voltage sources with values taken from the IBIS file. Such voltage sources are created internally by the PrimeSim Continuum. So do *not* connect the nodes of `nd_pu` and `nd_pd`, (and, if specified, `nd_pc` and `nd_gc`) to other voltage sources. Specify names for these nodes so you can print out the voltage values if required. For example:

```
.PRINT V(nd_pu) V(nd_pd)
```

If you specify the `power=off` keyword, the simulation does not connect these nodes to voltage sources. Connect the nodes to voltage sources directly, through an RLC network, or through a transmission line.

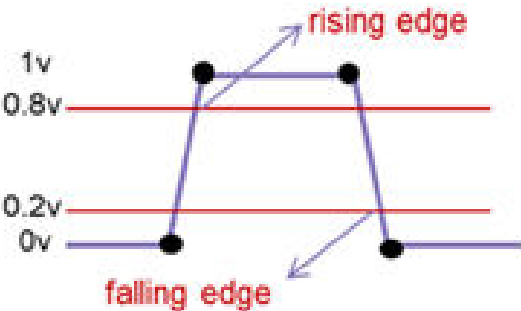
No special rules apply for the `nd_out` node. The voltage on this node is controlled by the digital signal on the `nd_in` node. You can connect any voltage source, current source, voltage controlled voltage source, voltage controlled current source, current controlled voltage source, or current controlled current source to the `nd_in` node as shown in the following example:

```
V_in nd_in gnd 0V pulse(0V 1V 1p 1p 0.1n 7.5n 15n)
```

If `power=off`, you can connect the `nd_pu`, `nd_pd`, `nd_pc`, and `nd_gc` nodes to the ground if you want zero voltage on these nodes.

V_in is a controlling signal that represents a digital signal with values 0 and 1. However, the simulation can use any signal and process, according to the following rules:

A small Tr/f of input signal to B element is helpful for accuracy. Since the PrimeSim Continuum solution detects the rising/falling edge by voltage thresholds of 0.2&0.8 as follows:



For voltage sources, the PrimeSim Continuum solution always add breakpoints (transient calculation points) at above black dots, no matter if there are other breakpoints between them. So smaller Tr/f means the PrimeSim Continuum solution can detect the time points of rising/falling edge with less error. For example, if you use 1p for Tr/f, then the error is definitely less than 1p.

Table 16 IBIS Output Buffer

If:	Then:
Polarity=Non-inverting	At t=0 for transient analysis (or for DC analysis), the buffer goes to HIGH state if V_in > 0.5 and to LOW in the opposite case. Next, if the buffer is in HIGH state, it goes to LOW state if V_in < 0.2. If the buffer is in LOW state, it goes to HIGH state if V_in > 0.8.
Polarity=Inverting	At t=0 for transient analysis (or for DC analysis), the buffer goes to HIGH state if V_in < 0.5 and to LOW in the opposite case. Next, if the buffer is in HIGH state, it goes to LOW state if V_in > 0.8. If the buffer is in LOW state, it goes to HIGH state if V_in < 0.2.

If the pc_scal (or gc_scal, pu_scal, pd_scal) argument exists and the pc_scal_value (or gc_scal_value, pu_scal_value, pd_scal_value) does not equal 1.0, then the simulation adjusts the PC (or GC, PU, PD) I/V curve using the pc_scal_value (or gc_scal_value, pu_scal_value, pd_scal_value).

If the `rwf_scal` (or `fwf_scal`) argument exists and `rwf_scal_value` (or `fwf_scal_value`) does not equal 1.0, then the simulation adjusts the rising and falling vt curves using the `rwf_scal_value` (or `fwf_scal_value`).

If the `spu_scal` (or `spd_scal`) argument exists and `spu_scal_value` (or `spd_scal_value`) is greater than 0 and `power=off` and `V(nd_pu)-V(nd_pd)` does not equal the corresponding value in the `*.ibs` file, then the simulation adjusts the I/V curves of PU (or PD) using the `spu_scal_value` (or `spd_scal_value`).

Probing Current Flowing through C_comp when Using a B-element

While you cannot probe the current through a C_comp directly, you can use this workaround:

1. Move C_comp to C_fixture in VT waveforms (i.e., set `C_fixture = C_fixture + C_comp` and `C_comp = 0`).
2. Add a capacitor (with value = C_comp) from the output node of the B-element to Ground.
3. Probe the current flowing through the added capacitor.

Tristate Buffer

The syntax for a tristate buffer element card is:

```
B_3STATE nd_pu nd_pd nd_out nd_in nd_en [nd_pc nd_gc]
+ file='file_name' model='model_name'
+ [typ=typ|min|max|fast|slow] [power=on|off]
+ [buffer=4|three_state]
+ [xv_pu=state_pu] [xv_pd=state_pd]
+ [interp=1|2]
+ [ramp_fwf=2|1|0] [ramp_rwf=2|1|0]
+ [fwf_tune=fwf_tune_value] [rwf_tune=rwf_tune_value]
+ [nowarn]
+ [c_com_pu=c_com_pu_value]
+ [c_com_pd=c_com_pd_value]
+ [c_com_pc=c_com_pc_value]
+ [c_com_gc=c_com_gc_value]
+ [pu_scal=pu_scal_value]
+ [pd_scal=pd_scal_value]
+ [pc_scal=pc_scal_value]
+ [gc_scal=gc_scal_value]
+ [rwf_scal=rwf_scal_value]
+ [fwf_scal=fwf_scal_value]
+ [spu_scal=spu_scal_value]
+ [spd_scal=spd_scal_value]
```

The `nd_pc` and `nd_gc` nodes are optional, and you can specify either both nodes or none of them. The total number of external nodes is either 5 or 7; any other number is an error.

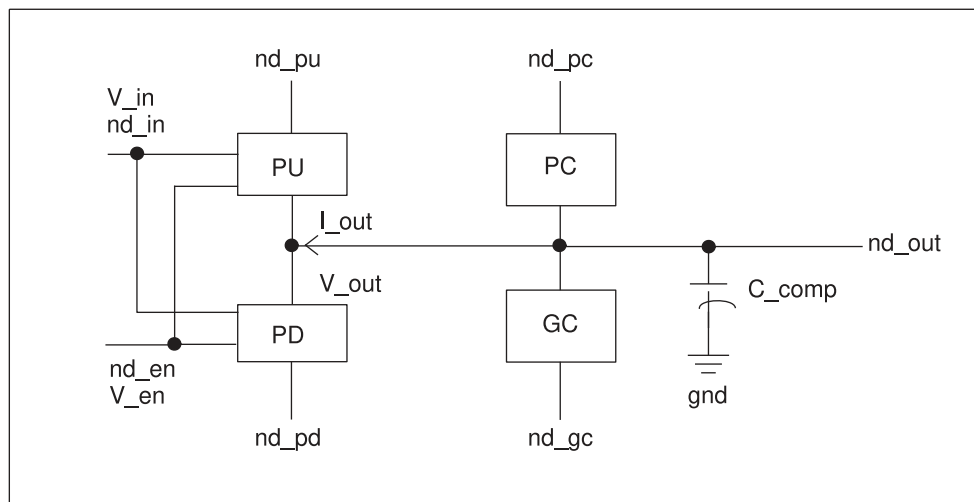
If the element card does not specify the *nd_pc* and *nd_gc* nodes, but the model contains Power_Clamp or Ground_Clamp I-V curves, then the simulator adds Power_Clamp or Ground_Clamp I-V curves data to the corresponding Pull_Up or Pull_Down I-V curves data.

However, you need the *nd_pc* and *nd_gc* optional nodes if:

- The POWER Clamp Reference and GND Clamp Reference IBIS keywords are present in the IBIS model and have different values than the IBIS keywords Pullup Reference and Pulldown Reference, or
- The Pullup Reference and Pulldown Reference IBIS keywords do not exist and POWER Clamp Reference and GND Clamp Reference have different values than those determined by the Voltage Range IBIS keyword.

If your circuit needs *nd_pc* and *nd_gc* optional nodes, but they are not in the element card, the simulation issues a warning and connects *nd_pc* to *nd_pu* and *nd_gc* to *nd_pd*.

Figure 25 Tristate Buffer



If you specify the *power=on* (default) keyword, then the *nd_pu* and *nd_pd* nodes, (and, if specified, *nd_pc* and *nd_gc*), are connected to voltage sources with values taken from the IBIS file. Do not connect these nodes to voltage sources.

However, specify names for these nodes in the netlist so you can print out the voltage values if required. For example:

```
.PRINT V(nd_pu) V(nd_pd)
```

If you specify the `power=off` keyword, the simulation does not connect these nodes to voltage sources. Connect the nodes to voltage sources directly through an RLC network, or through a transmission line.

No special rules apply for the `nd_out` node. The voltage on this node is controlled by the digital signal on the `nd_in` and `nd_en` nodes. Voltage sources must be connected to the nodes `nd_in`, `nd_en` as shown in the following example:

```
V_in nd_in gnd 0V pulse( 0V 1V 1n 0.1n 0.1n 7.5n 15n )
V_en nd_en gnd 0V pulse( 0V 1V 3n 0.1n 0.1n 7.5n 15n )
```

You can connect the `nd_pu`, `nd_pd`, `nd_pc`, and `nd_gc` nodes to the ground if you want to have zero voltage on these nodes. The `nd_in` and `nd_en` nodes cannot be connected to the ground.

`V_in` and `V_en` are controlling signals representing digital signals with 0 and 1 values. The simulation can use any signal and process according to the rules in [Table 17](#). The enable signal, `V_en`, supersedes the input signal `V_in`.

Table 17 IBIS Tristate Buffer

If:	Then:
ENABLE = Active-High	At t=0 for transient analysis (or for DC analysis), the buffer goes to the ENABLE state if <code>V_en</code> > 0.5 and to DISABLE in the opposite case.
ENABLE = Active-Low	At t=0 for transient analysis (or for DC analysis), the buffer goes to ENABLE state if <code>V_en</code> < 0.5 and to DISABLE in the opposite case.
The buffer is in ENABLE state	Begins transition to DISABLE state if <code>V_en</code> < 0.2 (where Enable = Active-High) and if <code>V_en</code> > 0.8 (where Enable = Active-Low).
The buffer is in DISABLE state or in transition from ENABLE to DISABLE state	Begins transition to ENABLE state if <code>V_en</code> > 0.8 (where Enable = Active-High) and if <code>V_en</code> < 0.2 (where Enable = Active-Low).
The buffer is in ENABLE state	Response to the input signal, <code>V_in</code> , is the same as the output buffer.
Polarity=Non-Inverting	At t=0 for transient analysis (or for DC analysis), the buffer goes to HIGH state if <code>V_in</code> > 0.5 and to LOW in the opposite case. Next, if the buffer is in HIGH state, it goes to LOW state if <code>V_in</code> < 0.2. If the buffer is in LOW state, it goes to HIGH state if <code>V_in</code> > 0.8.
Polarity=Inverting	At t=0 for transient analysis (or for DC analysis), the buffer goes to HIGH state if <code>V_in</code> < 0.5 and to LOW in the opposite case. Next, if the buffer is in HIGH state, it goes to LOW state if <code>V_in</code> > 0.8. If the buffer is in LOW state, it goes to HIGH state if <code>V_in</code> < 0.2.

Input/Output Buffer

The syntax of an input/output buffer element card is:

```
B_IO nd_pu nd_pd nd_out nd_in nd_en nd_out_of_in [nd_pc nd_gc]
+ file='file_name' model='model_name'
+ [typ=typ|min|max|fast|slow] [power=on|off]
+ [buffer=3|input_output]
+ [xv_pu=state_pu] [xv_pd=state_pd]
+ [interpol=1|2]
+ [ramp_fwf=2|1|0] [ramp_rwf=2|1|0]
+ [fwf_tune=fwf_tune_value] [rwf_tune=rwf_tune_value]
+ [nowarn]
+ [c_com_pu=c_com_pu_value]
+ [c_com_pd=c_com_pd_value]
+ [c_com_pc=c_com_pc_value]
+ [c_com_gc=c_com_gc_value]
+ [pu_scal=pu_scal_value]
+ [pd_scal=pd_scal_value]
+ [pc_scal=pc_scal_value]
+ [gc_scal=gc_scal_value]
+ [rwf_scal=rwf_scal_value]
+ [fwf_scal=fwf_scal_value]
+ [spu_scal=spu_scal_value]
+ [spd_scal=spd_scal_value]
```

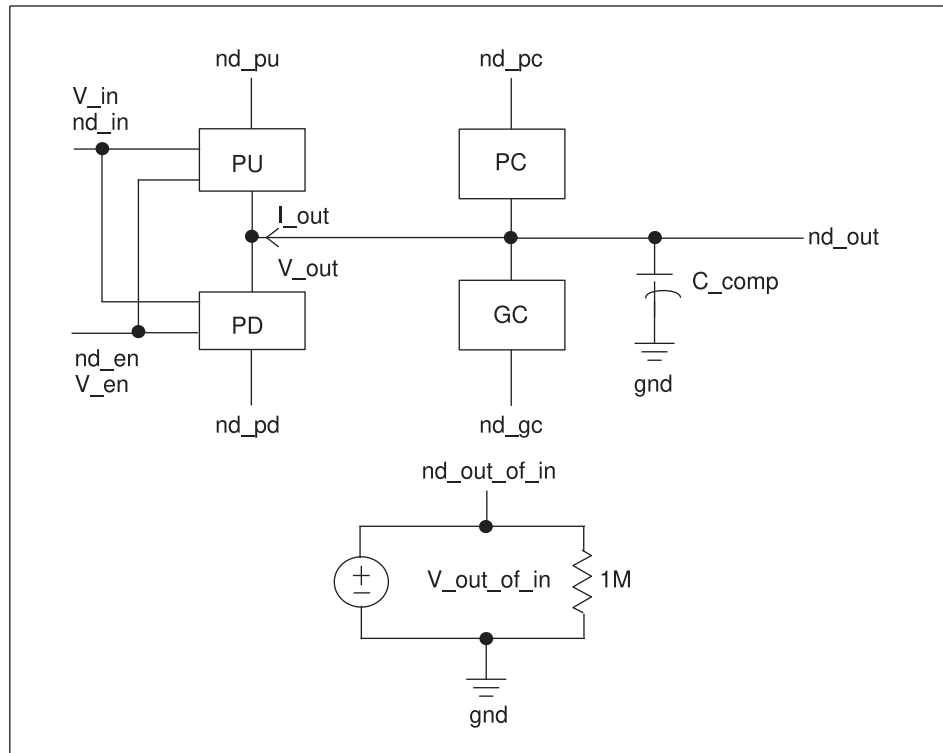
The `nd_pc` and `nd_gc` nodes are optional. However, you can specify either both nodes or none of them. The total number of external nodes is either 6 or 8; any other number is an error. If the element card does not specify the `nd_pc` and `nd_gc` nodes, but the model contains Power_Clamp or Ground_Clamp I-V curves, then the simulator adds Power_Clamp or Ground_Clamp I-V curves data to the corresponding Pull_Up or Pull_Down I-V curves data.

However, you need the `nd_pc` and `nd_gc` optional nodes if:

- The IBIS POWER Clamp Reference and GND Clamp Reference keywords are present in the IBIS model and have different values than the Pullup Reference and Pulldown Reference IBIS keywords, or
- The IBIS Pullup Reference and Pulldown Reference keywords do not exist and POWER Clamp Reference and GND Clamp Reference have different values than those determined by the Voltage Range IBIS keyword.

If you need the `nd_pc` and `nd_gc` optional nodes, and you omitted them from the element card, the simulation issues a warning and connects `nd_pc` to `nd_pu` and `nd_gc` to `nd_pd`.

Figure 26 Input-Output Buffer



If you specify the `power=on` (default) keyword, then the `nd_pu` and `nd_pd` nodes, (and if specified, `nd_pc` and `nd_gc`), are connected to voltage sources with values taken from the IBIS file. Such voltage sources are created internally by the PrimeSim Continuum solution. So do *not* connect the nodes of `nd_pu` and `nd_pd`, (and, if specified, `nd_pc` and `nd_gc`) to other voltage sources. However, you should specify names for these nodes in the netlist so you can print out the voltage values if required. For example:

```
.PRINT V(nd_pu) V(nd_pd)
```

If you specify the `power=off` keyword, the simulation does not connect these nodes to voltage sources. You should connect the nodes to voltage sources directly through an RLC network, or through a transmission line.

No special rules apply for the `nd_out` node. The voltage on this node is controlled by the digital signal on the `nd_in` and `nd_en` nodes. Voltage sources must be connected to the `nd_in` and `nd_en` nodes as shown in the following example:

```
V_in nd_in gnd 0V pulse (0V 1V 1n 0.1n 0.1n 7.5n 15n)
V_en nd_en gnd 0V pulse (0V 1V 3n 0.1n 0.1n 7.5n 15n)
```


You can connect the `nd_pu`, `nd_pd`, `nd_pc`, and `nd_gc` nodes to the ground if you want zero voltage on these nodes.

The `nd_out_of_in` node is connected to a voltage source (see [Figure 26](#)). Connecting this node to a voltage source or the ground causes an error to occur.

The input-output buffer is a combination of the tristate buffer and the input buffer. See [Input Buffer on page 247](#) and [Tristate Buffer on page 252](#) for more information.

The input-output buffer can function as an input buffer. In this case, the resultant `V_out_of_in` digital signal on the `nd_out_of_in` node is controlled by the `V_out` voltage on the `nd_out` node.

For the input buffer, this controlling voltage is called `V_in` and any corresponding node is called `nd_in`.

The input-output buffer uses `V_in` and `nd_in` notations to denote the controlling voltage and controlling input node for the output part of the buffer.

If the input-output buffer is not in the DISABLE state (this includes ENABLE state and transitions to ENABLE->DISABLE and DISABLE->ENABLE), then it functions as a tristate buffer. If the input-output buffer is in the DISABLE state, it functions as an input buffer.

However, there is a difference in the digital output of the input part of the buffer (voltage `V_out_of_in`). `V_out_of_in` is not always defined (for example, if the buffer is in ENABLE state, or $V_{inl} < V_{out} < V_{inh}$ at the time moment when the transition to DISABLE state is completed). Also, you need to preserve logical LEVELs 0 and 1 for LOW and HIGH states. Therefore, `V_out_of_in` uses the value 0.5 when it is undefined.

[Figure 26](#) shows a single circuit specified on a single element card. The `V_out_of_in` is a voltage source whose value is a function of `V_out` (of the `V_{inl}` and `V_{inh}` thresholds and the Polarity parameter). It can be used to drive other circuits.

Open Drain, Open Sink, Open Source Buffers

Open drain and open sink buffers do not include pullup circuitry. Open source buffers do not include pulldown circuitry. However, the element cards for these three buffers coincide with the element card for the output buffer. Accordingly, you should always specify names for pullup and pulldown nodes, `nd_pu` and `nd_pd`, even if the buffer does not include pullup or pulldown circuitry.

All rules in [Output Buffer on page 249](#) apply to open drain, open sink, and open source buffers with the following exceptions:

- Because open drain and open sink buffers do not have pullup circuitry, do not specify the `xv_pu=nd_state_pu` option.
- Similarly, because open source buffers have no pulldown circuitry, do not specify the `xv_pd=nd_state_pd` option.

I/O Open Drain, I/O Open Sink, I/O Open Source Buffers

I/O open drain and I/O open sink buffers do not include pullup circuitry. I/O open source buffers do not include pulldown circuitry. However, the element cards for these three buffers coincide with the element card for the input-output buffer. Accordingly, you should always specify names for pullup and pulldown nodes, `nd_pu` and `nd_pd`, even if the buffer does not include pullup or pulldown circuitry.

All rules in [Input/Output Buffer on page 255](#) apply to I/O open drain, I/O open sink, and I/O open source buffers with the following exceptions:

- Because I/O open drain and I/O open sink buffers do not have pullup circuitry, do not specify the `xv_pu=nd_state_pu` option.
- Similarly, because I/O open source buffers do not include pulldown circuitry, do not specify the `xv_pd=nd_state_pd` option.

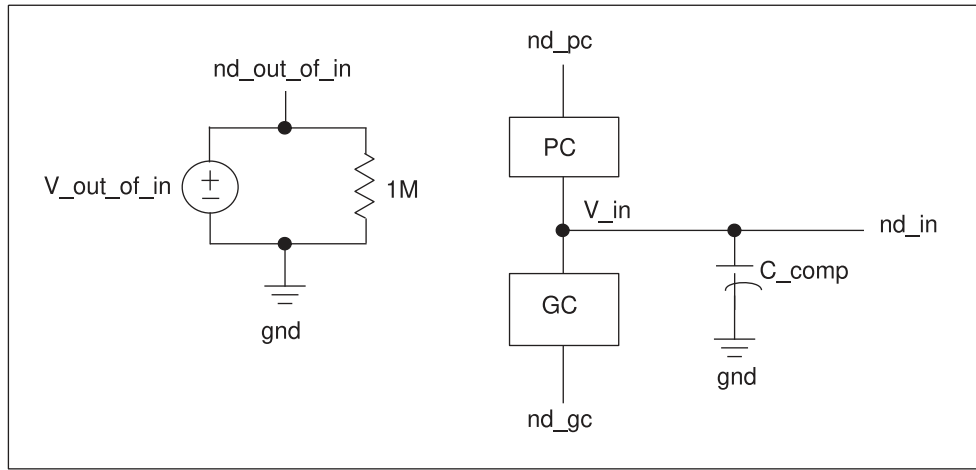
Input ECL Buffer

The syntax of the input ECL buffer element card is:

```
B_INPUT_ECL nd_pc nd_gc nd_in nd_out_of_in
+ file='file_name' model='model_name'
+ [typ=typ|min|max|fast|slow] [power=on|off]
+ [buffer=1|input_ecl]
+ [interpol=1|2]
+ [nowarn]
+ [c_com_pc=c_com_pc_value]
+ [c_com_gc=c_com_gc_value]
+ [pc_scal=pc_scal_value]
+ [gc_scal=gc_scal_value]
```

The input ECL buffer is similar to the input buffer. The only difference is in default values for `Vinl` and `Vinh`.

Figure 27 Input ECL Buffer



Output ECL Buffer

The syntax of the output ECL buffer element card is:

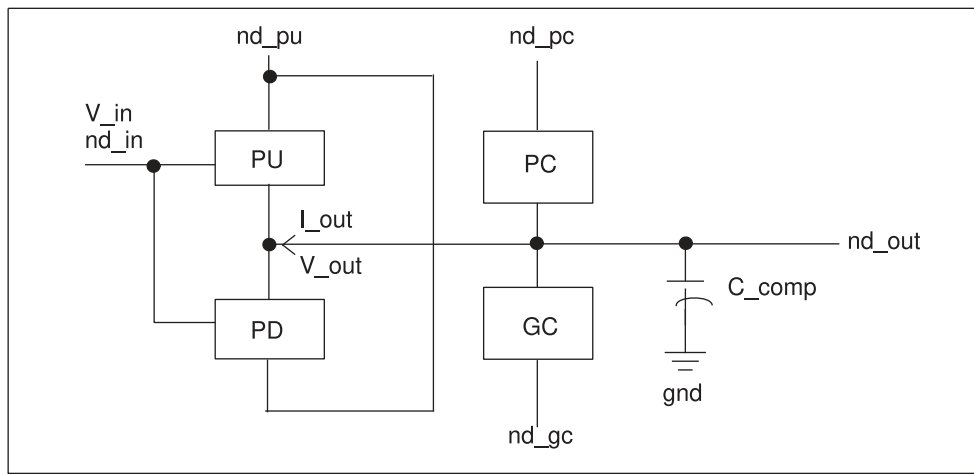
```
B_OUTPUT_ECL nd_pu nd_out nd_in [nd_pc nd_gc]
+ file='file_name' model='model_name'
+ [typ=typ|min|max|fast|slow] [power=on|off]
+ [buffer=l2|output_ecl]
+ [xv_pu=state_pu] [xv_pd=state_pd]
+ [interpol=1|2]
+ [ramp_fwf=0|1|2] [ramp_rwf=0|1|2]
+ [fwf_tune=fwf_tune_value] [rwf_tune=rwf_tune_value]
+ [nowarn]
+ [c_com_pu=c_com_pu_value]
+ [c_com_pc=c_com_pc_value]
+ [c_com_gc=c_com_gc_value]
+ [pu_scal=pu_scal_value]
+ [pd_scal=pd_scal_value]
+ [pc_scal=pc_scal_value]
+ [gc_scal=gc_scal_value]
+ [rwf_scal=rwf_scal_value]
+ [fwf_scal=fwf_scal_value]
+ [spu_scal=spu_scal_value]
+ [spd_scal=spd_scal_value]
```

The *nd_pc* and *nd_gc* nodes are optional. However, you can specify both nodes or none of them. The total number of external nodes is either 3 or 5; any other number is an error. The output ECL buffer does not have a pulldown node. The pulldown table in the IBIS file is referenced in respect to pullup voltage.

If you do not specify the *nd_pc* and *nd_gc* nodes on the element card, but the Power_Clamp or Ground_Clamp I-V curves are present in the model in question, then the simulator issues an error message (this simulator behavior is different from that for the output buffer).

In other respects, the output ECL buffer is similar to the output buffer. For more information, see [Output Buffer on page 249](#).

Figure 28 Output ECL Buffer



Tristate ECL Buffer

The syntax for the tristate ECL buffer element card is:

```
B_3STATE_ECL nd_pu nd_out nd_in nd_en [nd_pc nd_gc]
+ file='file_name' model='model_name'
+ [typ=typ|min|max|fast|slow] [power=on|off]
+ [buffer=14|three_state_ecl]
+ [xv_pu=state_pu] [xv_pd=state_pd]
+ [interpol=1|2]
+ [ramp_fw=2|1|0] [ramp_rwf=2|1|0]
+ [fwf_tune=fwf_tune_value] [rwf_tune=rwf_tune_value]
+ [nowarn]
+ [c_com_pu=c_com_pu_value]
+ [c_com_pc=c_com_pc_value]
+ [c_com_gc=c_com_gc_value]
+ [pu_scal=pu_scal_value]
+ [pd_scal=pd_scal_value]
+ [pc_scal=pc_scal_value]
+ [gc_scal=gc_scal_value]
+ [rwf_scal=rwf_scal_value]
+ [fwf_scal=fwf_scal_value]
```

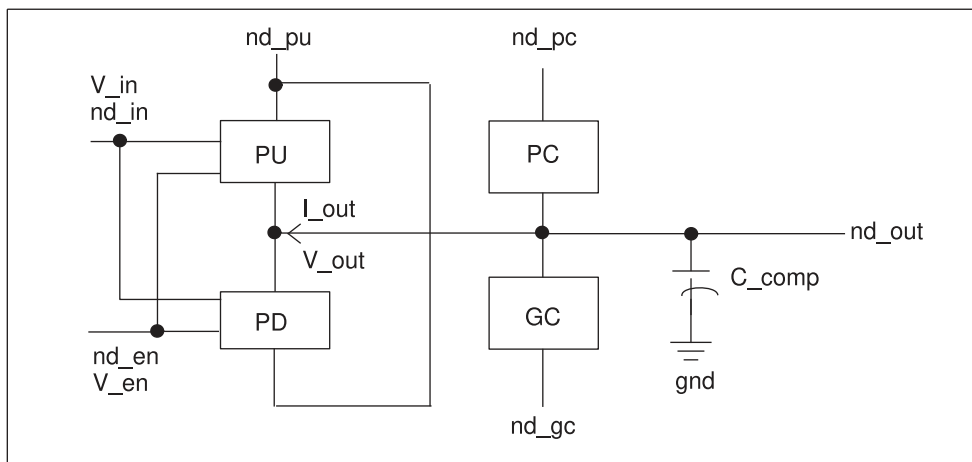
```
+ [spu_scal=spu_scal_value]
+ [spd_scal=spd_scal_value]
```

The *nd_pc* and *nd_gc* nodes are optional. However, either both or none can be specified. The total number of external nodes is either 4 or 6, any other number is an error. The tristate ECL buffer does not have a pulldown node. The pulldown table in the IBIS file is referenced in respect to pullup voltage.

If you do not specify the *nd_pc* and *nd_gc* nodes on the element card, but the Power_Clamp or Ground_Clamp I-V curves are present in the model in question, then the simulator issues an error message (this simulator behavior is different from that for the tristate buffer).

In other respects, the tristate ECL buffer is similar to the tristate buffer. See [Tristate Buffer on page 252](#) for more information.

Figure 29 Tristate ECL Buffer



Input-Output ECL Buffer

The syntax for the input-output ECL buffer element card is:

```
B_IO_ECL nd_pu nd_out nd_in nd_en nd_out_of_in [nd_pc nd_gc]
+ file='file_name' model='model_name'
+ [typ=typ|min|max|fast|slow] [power=on|off]
+ [buffer=13|io_ecl]
+ [xv_pu=state_pu] [xv_pd=state_pd]
+ [interpol=1|2]
+ [ramp_fw=2|1|0] [ramp_rwf=2|1|0]
+ [fwf_tune=fwf_tune_value] [rwf_tune=rwf_tune_value]
+ [nowarn]
+ [c_com_pu=c_com_pu_value]
```

Chapter 4: Modeling Input/Output Buffers Using IBIS Files

Buffer Types

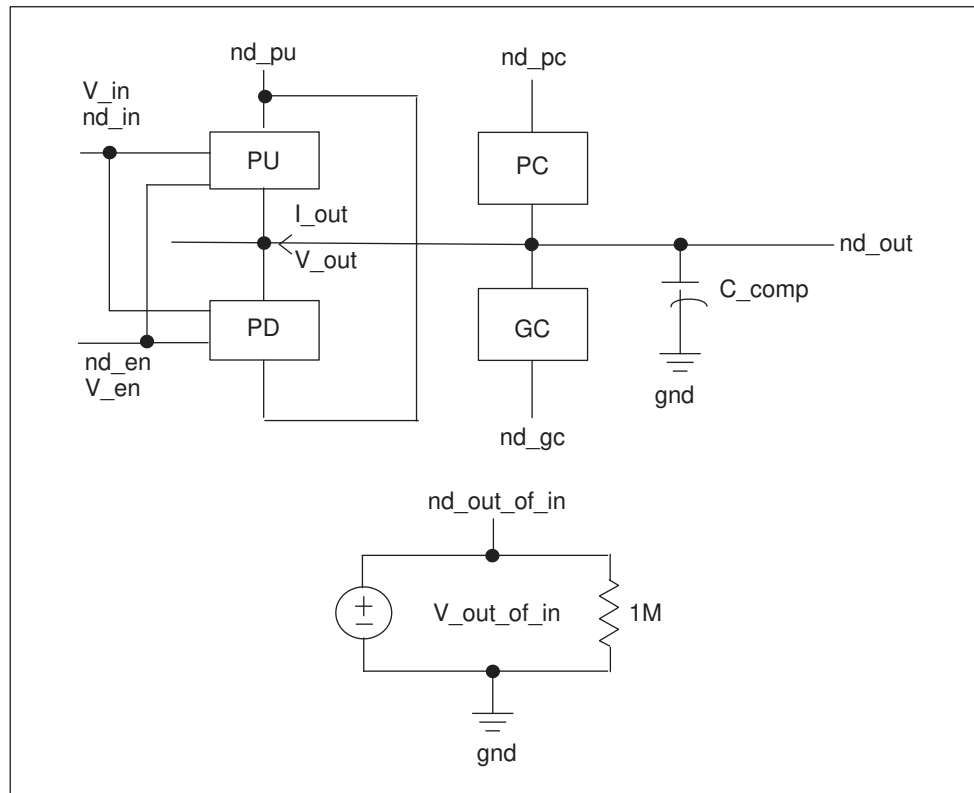
```
+ [c_com_pc=c_com_pc_value]
+ [c_com_gc=c_com_gc_value]
+ [pu_scal=pu_scal_value]
+ [pd_scal=pd_scal_value]
+ [pc_scal=pc_scal_value]
+ [gc_scal=gc_scal_value]
+ [rwf_scal=rwf_scal_value]
+ [fwf_scal=fwf_scal_value]
+ [spu_scal=spu_scal_value]
+ [spd_scal=spd_scal_value]
```

The *nd_pc* and *nd_gc* nodes are optional. However, you can specify either both nodes or none of them. The total number of external nodes is either 5 or 7; any other number is an error. The tristate ECL buffer does not include a pulldown node. The pulldown table in the IBIS file is referenced in respect to pullup voltage.

If you specify the *nd_pc* and *nd_gc* nodes on the element card but Power_Clamp or Ground_Clamp I-V curves are present in the model in question, then the simulator issues an error message (this simulator behavior is different from the Input-Output buffer).

In other respects, the input-output ECL buffer is similar to the input-output buffer. See [Input/Output Buffer on page 255](#) for more information.

Figure 30 Input-Output ECL Buffer



Terminator Buffer

The syntax for the terminator buffer element card is:

```
B_TERMINATOR nd_pc nd_gc nd_out
+ file='filename' model='model_name'
+ [typ=typ|min|max|fast|slow] [power=on|off]
+ [buffer=17|terminator]
+ [interpol=1|2]
+ [nowarn]
+ [c_com_pc=c_com_pc_value]
+ [c_com_gc=c_com_gc_value]
+ [pc_scal=pc_scal_value]
+ [gc_scal=gc_scal_value]
```

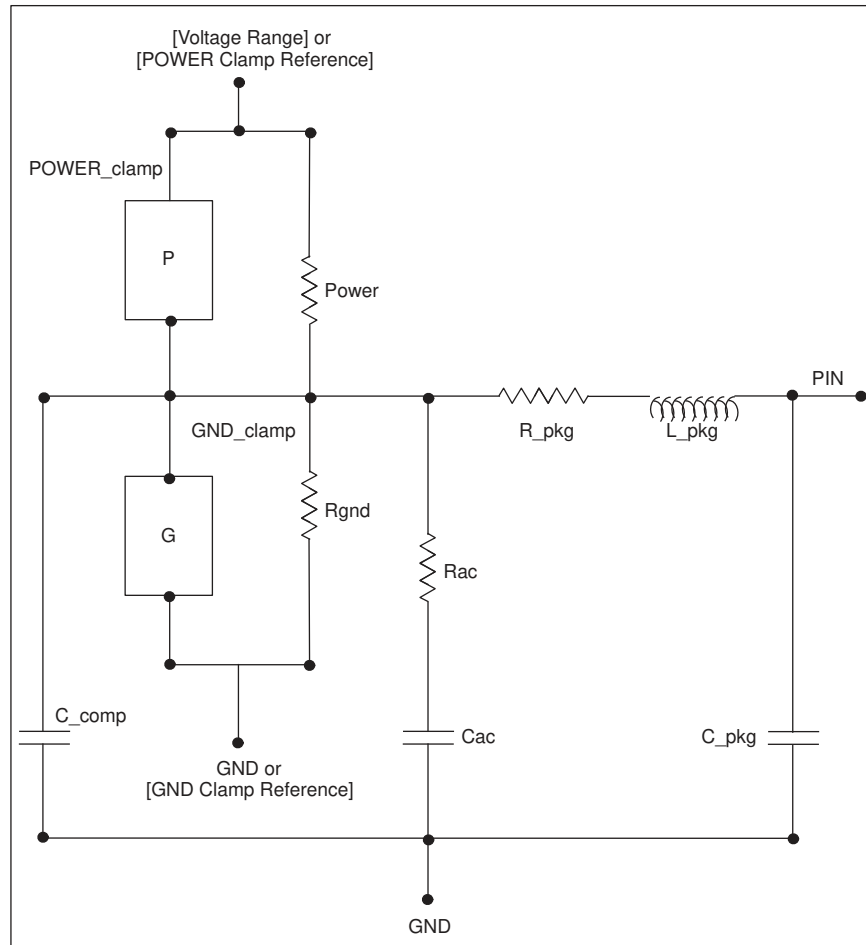
In the preceding syntax, the total number of external nodes is 3. If you specify the `power=on` keyword (default), the `nd_pc` and `nd_gc` nodes are connected to voltage sources with values taken from the IBIS file. Such voltage sources are created internally by the PrimeSim Continuum solution. So do *not* connect the nodes of `nd_pc` and `nd_gc`

to other voltage sources. Names for these nodes are provided so you can print out the voltage values if required.

The schematic in [Figure 31](#) shows a terminator buffer:

```
.PRINT V(nd_pc) V(nd_gc)
```

Figure 31 Terminator Buffer Schematic



If you specify the `power=off` keyword, the simulation does not connect these nodes to voltage sources. You need to connect the nodes to voltage sources directly through an RLC network or transmission line. You can connect `node_in` to I-, E-, F-, G-, and H-elements.

Note:

`Rac` and `Cac` are the resistance and capacitance values for an AC terminator.

Series Buffer

The syntax for the series buffer element card is:

```
B_SERIES nd_in nd_out  
+ file='filename' model='model_name'  
+ [typ=typ|min|max|fast|slow]  
+ [buffer=15|series]  
+ [interpol=1|2]  
+ [nowarn]  
+ [all_sm=0|1]
```

In the preceding syntax, the total number of external nodes is 2. This buffer type is for series models that can be described by the following model keywords:

- [R Series]
- [L Series]
- [RI Series]
- [C Series]
- [Lc Series]
- [Rc Series]
- [Series Current]
- [Series MOSFET]

[Figure 32](#) shows a schematic for the series buffer.

Series Switch Buffer

The syntax for the series switch buffer element card is:

```
B_SER_SW nd_in nd_out  
+ file='filename' model='model_name'  
+ [typ=typ|min|max|fast|slow]  
+ [buffer=16|series_switch]  
+ [ss_state=on|off]  
+ [interpol=1|2]  
+ [nowarn]  
+ [all_sm=0|1]
```

In this syntax, the total number of external nodes is 2. This buffer type is for series switch models which can be described by the following model keywords:

- [On]
- [Off]
- [R Series]
- [L Series]
- [RI Series]
- [C Series]
- [Lc Series]
- [Rc Series]
- [Series Current]
- [Series MOSFET]

Voltage Thresholds

Voltages applied to the input and enable nodes are digital signals. They should be either 0 or 1. You can specify input voltage as:

```
V_in nd_in 0 pulse (0 3.3 0 0.5n 0.5n 4n 8n)
```

However, IC circuit simulation currently detects only two thresholds, 20% and 80% of [0,1] swing, that is, 0.2V and 0.8V. If a buffer is non-inverting and in a LOW state, it starts the transition to a HIGH state, if $V_{in} > 0.8V$. If the buffer is in HIGH state, it starts the transition to LOW state, if $V_{in} < 0.2V$. Specifying input voltage in the range [0, 3.3V] as in the above example does not make LOW → HIGH transitions better in any way, but can add uncertainty over the 0.5ns time interval when the transition actually occurs.

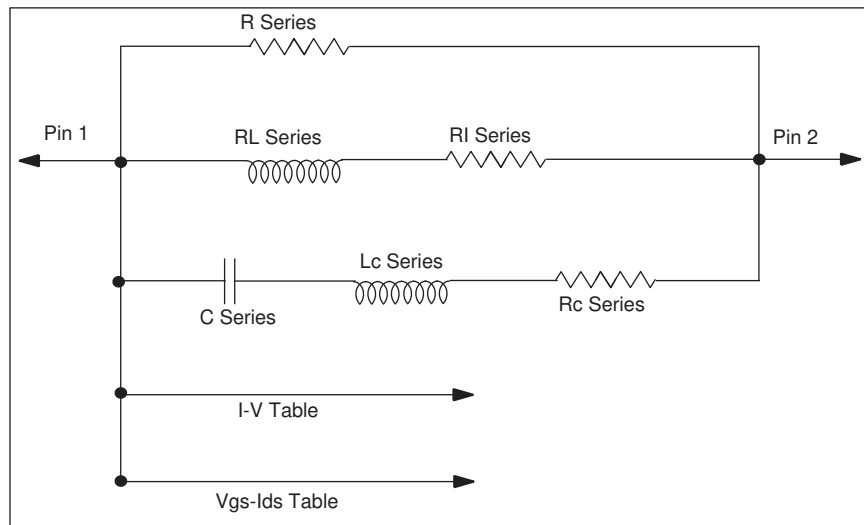
The schematic in [Figure 32](#) is similar to a series switch buffer, except that the Vgs-Ids table is not available in the OFF state.

The `all_sm` subparameter is optional. It is used to control the method of the Series MOSFET. When `all_sm=0`, only the first Vgs-Ids table (`vds!=0`) is used. The PrimeSim Continuum solution uses the following formula to implement this method:

$$ids = Ids(vgs, Vds) * vds/Vds$$

Otherwise, when `all_sm=1`, all Vgs-Ids tables are used for the Series MOSFET.

Figure 32 Series Buffer Schematic



Multilingual Model Support

The PrimeSim Continuum solution supports buffers of an external model described either in SPICE or in Verilog-A.

The syntax to call SPICE or Verilog-A formatted buffers is:

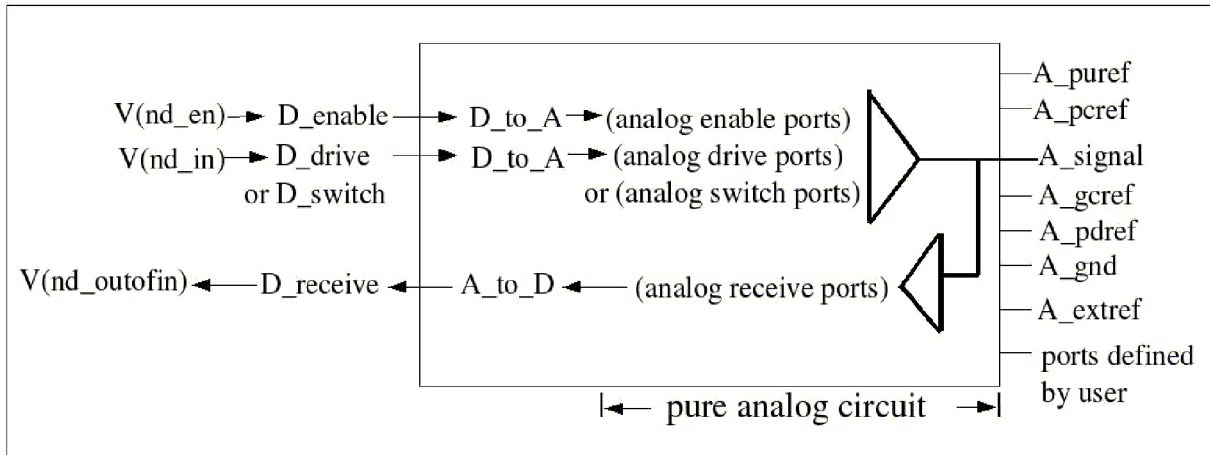
```
B_SPICE node1 node2 node3 ...
+ file='ibis_filename' model='model_name'
+ [nd_in=node_input]
+ [nd_en=node_enable]
+ [nd_outofin=node_out_of_in]
+ [typ={typ|min|max}] [power={on|off}]
+ [nowarn]
+ [para_begin para1=value1 para2=value2
+ ...
+ para_end]
```

In the preceding syntax

- The list of nodes must map port names declared in [Ports] of [External Model].
- Specify a [Corner] in [External Model] from which the current simulation extracts data. If min or max [Corner] is not available, typ [Corner] is used. The default is typ=typ.

- The file of subcircuit (or module) listed in [Corner] of [External Model] must be manually included in netlist if the subcircuit (or module) is used by the external model.
- Keywords para_begin/para_end are used to assign values to parameters which are declared in [External Model] with Verilog-A language. Using para_begin/para_end for a B-element which is not a [External Model] causes an error.

Figure 33 SPICE or Verilog-A Formatted Buffer in the PrimeSim Continuum



The keyword *nd_in* is only used for Output, IO, series switch, and Tri-state buffer, while *nd_en* is only used for IO or Tri-state buffer.

The two keywords specify two nodes: node_input and node_enable. They are similar to nd_in and nd_en in preceding buffers. So voltages at such nodes are controlling the signal that represents the digital signal with values 0 and 1 (see Table 16, IBIS Output Buffer, where 1 is the signal triggering buffer to high, and 0 is the signal triggering buffer to low). This digital signal decides the voltage value between analog ports of D_drive (or D_enable) according to [D_to_A] rules in the [External Model] for DC or TRAN analysis.

The format of [D_to_A] specification is:

```
D_to_A D_drive port1 port2 vlow vhigh trise tfall corner_name D_to_A D_enable port1
port2 vlow vhigh trise tfall corner_name
```

To illustrate the procedure, denote the digital signal from node_input (or node_enable) by D_sti.

Then, for DC or initial TRAN analysis:

V(port1,port2)=Vhigh, if D_sti =1;

V(port1,port2)=vlow, if D_sti =0.

For subsequent TRAN analysis:

$V(\text{port1}, \text{port2})$ changes from v_{low} to v_{high} linearly within time t_{rise} , if D_{sti} switches from 0 to 1 and $V(\text{port1}, \text{port2})$ changes from v_{high} to v_{low} linearly within time t_{fall} , if D_{sti} switches from 1 to 0.

If polarity is inverting, preceding port1 and port2 should be swapped.

If nd_in or in_en is needed for related type of buffer but it is omitted from the buffer card, then the PrimeSim Continuum solution still adds the subcircuit or module from [External Model], but without $V(\text{port1}, \text{port2})$ added, and it reports a warning.

The keyword $nd_outofin$ is only used for IO or Input buffer. The buffer measures and processes voltages between analog ports in [A_to_D] of [External Model] and sends a response to the $nd_out_of_in$ node. The $nd_out_of_in$ node is connected to the voltage source as shown in [Figure 23](#).

The digital signal $V_out_of_in$ assumes values of 0, 1, or 0.5— depending on the voltage difference of two ports, v_{low} , v_{high} thresholds in [A_to_D] and Polarity.

The simulation processes $V_out_of_in$ according to the following rules:

IF	THEN
Polarity=Non-Inverting	Initially $V_out_of_in$ is set to 0, if $V(\text{port1}, \text{port2}) < (v_{\text{low}} + v_{\text{high}})/2$ and to 1 in the opposite case
$V(\text{port1}, \text{port2}) > v_{\text{high}}$	$V_out_of_in$ is set to 1
$V(\text{port1}, \text{port2}) < v_{\text{low}}$	$V_out_of_in$ is set to 0
else $v_{\text{low}} \leq V(\text{port1}, \text{port2}) \leq v_{\text{high}}$	$V_out_of_in$ is set to 0.5
Polarity=Inverting	Initially $V_out_of_in$ is set to 0, if $V(\text{port1}, \text{port2}) > (v_{\text{low}} + v_{\text{high}})/2$ and to 1 in the opposite case
$V(\text{port1}, \text{port2}) > v_{\text{high}}$	$V_out_of_in$ is set to 0
$V(\text{port1}, \text{port2}) < v_{\text{low}}$	$V_out_of_in$ is set to 1
else $v_{\text{low}} \leq V(\text{port1}, \text{port2}) \leq v_{\text{high}}$	$V_out_of_in$ is set to 0.5

If an IO buffer is driving, then $V_out_of_in$ is set to 0.5. The keywords `typ`, `power`, and `nowarn` are similar as in ordinary buffer.

Examples

This section provides a few examples of modeling I/O buffers using IBIS files. For additional examples, see Using IBIS Files in the *PrimeSim HSPICE® User Guide: Signal Integrity Modeling and Analysis*.

Example 1

The Input buffer name is B1. The four terminals names are nd_pc, nd_gc, nd_in and nd_out_of_in. The IBIS model named IBIS_IN is in the file named test.ibs. The values specified in IBIS_IN guide the PrimeSim Continuum solution to connect nd_pc and nd_gc to the voltage sources. Do *not* manually connect voltage sources to these nodes.

```
B1 nd_pc nd_gc nd_in nd_out_of_in
+ file='test.ibs'
+ model='IBIS_IN'
```

Example 2

The output buffer name is B2. The six terminals names are nd_pu, nd_pd, nd_out, nd_in, and nd_pc, nd_gc. Here, nd_pc, nd_gc are optional terminals. If you do not set these names, the netlist uses the optional terminal names instead. The IBIS model named IBIS_OUT in the file named test.ibs. The values specified in IBIS_OUT guide the PrimeSim Continuum solution to connect nd_pc and nd_gc to the voltage sources. Do *not* manually connect voltage sources to these nodes.

```
B2 nd_pu nd_pd nd_out nd_in [nd_pc nd_gc]
+ file = 'test.ibs'
+ model = 'IBIS_OUT'
```

Example 3

The I/O buffer name is B3. The eight terminals names are nd_pu, nd_pd, nd_out, nd_in, nd_en, nd_out_of_in, and nd_pc, nd_gc. Here, nd_pc, nd_gc are optional terminals. If you do not set these names, the netlist uses the optional terminal names instead. The IBIS model named IBIS_IO is in the IBIS file named test.ibs. The values specified in IBIS_IO guide the PrimeSim Continuum solution to connect nd_pc and nd_gc to the voltage sources. Do *not* manually connect voltage sources to these nodes.

```
B3 nd_pu nd_pd nd_out nd_in nd_en nd_out_of_in [nd_pc nd_gc]
+ file = 'test.ibs'
+ model = 'IBIS_IO'
```

Example 4

The output buffer `bspice` calls the external model `spitest` defined in the IBIS model file named `test_new.ibs`. The SPICE buffer model `IO_buf1.inc` is included in the netlist. The input, enable, and output nodes of the buffer are mapped to the IBIS model using the keywords `nd_in`, `nd_en`, and `nd_outofin`.

```
.include 'IO_buf1.inc'

bspice in_b2 out_b2_o pu_bus2_o pd_bus2_o en_b2
+ file= 'test_new.ibs'
+ model= 'spitest'
+ nd_in= in_b1
+ nd_en= en_b1
+ nd_outofin= out_b2
+ power=off
```

Specifying Required and Optional Common Keywords

This section describes how to specify the most commonly used keywords in the PrimeSim Continuum solution. Required keywords are [file](#) and [model](#).

If optional keywords are not used, the default values are selected. Optional keywords are enclosed in square brackets [] in the buffer syntax. Optional keywords can be set with a parameter and use the keyword=parameter_name format. In cases where a keyword has multiple values, [keyword=*val_1* | *val_2* | ... | *val_n*], the default value is listed first.

You cannot sweep the `typ` and `hsp_ver` keywords during analysis.

The following keywords are optional:

- [file](#)
- [model](#)
- [buffer](#)
- [typ](#)
- [hsp_ver](#)
- [power](#)
- [interpol](#)
- [xv_pu](#) | [xv_pd](#)
- [ramp_fwf](#) | [ramp_rwf](#)
- [fwf_tune](#) | [rwf_tune](#)

- [rwf_pd_dly](#) | [fwf_pu_dly](#)
- [pd_scal](#) | [pu_scal](#) | [pc_scal](#) | [gc_scal](#) | [rwf_scal](#) | [fwf_scal](#)
- [dt_scale](#)
- [ss_state](#)
- [rm_dly_rwf](#)|[rm_dly_fwf](#)|[rm_tail_rwf](#)|[rm_tail_fwf](#)
- [nowarn](#)
- [c_com_pu](#) | [c_com_pd](#) | [c_com_pc](#) | [c_com_gc](#)
- [detect_oti_mid](#)
- [time_control](#)
- [.OPTION D_IBIS](#)

file

Syntax

```
file = 'file_name'
```

Required. Identifies the IBIS file. The `file_name` must be lower case and must specify either the absolute path for the file or the relative path with respect to the directory from which you run the simulator. File path names are restricted to 128 characters.

Example

```
file = '.ibis/at16245.ibs'  
file = '/home/oneuser/ibis/models/abc.ibs'
```

model

Syntax

```
model = 'model_name'
```

Required. Identifies the model for a buffer from the IBIS file, specified with the `file='...'` keyword. The `model_name` keyword is case-sensitive and must match one of the models from the IBIS file. Model names are restricted to 60 characters.

Example

```
model = 'ABC_1234_out'
model = 'abc_1234_IN'
```

buffer

Syntax

```
buffer = Buffer_Number | Buffer_Type
```

In this syntax, *Buffer_Number* is an integer within the range $1 \leq N \leq 17$. Each buffer has an assigned number.

Table 18 IBIS Buffer Types and Numbers

Buffer Type	Buffer No.	Number of nodes (nominal or min/max)
INPUT	1	4
OUTPUT	2	4/6
INPUT_OUTPUT	3	6/8
THREE_STATE	4	5/7
OPEN_DRAIN	5	4/6
IO_OPEN_DRAIN	6	6/8
OPEN_SINK	7	4/6
IO_OPEN_SINK	8	6/8
OPEN_SOURCE	9	4/6
IO_OPEN_SOURCE	10	6/8
INPUT_ECL	11	4
OUTPUT_ECL	12	3/5
IO_ECL	13	5/7
THREE_STATE_ECL	14	4/6
SERIES	15	2
SERIES_SWITCH	16	2
TERMINATOR	17	3

The value of *Buffer_Number* and *Buffer_Type* must match the *Model_type* entry in the IBIS [*Model*] specified by the `model='...'` keyword. The `buffer= Buffer_Number | Buffer_Type` keyword provides an extra check for the input netlist. If you omit the keyword, this extra check is not performed.

typ

Syntax

`typ = typ|min|max|fast|slow`

If the value of the `typ` buffer parameter is either `typ`, `min`, or `max`, then this value signifies a column in the IBIS file from which the current simulation extracts data. The default is `typ=typ`. If `min` or `max` data are not available, `typ` data are used instead.

If the value of the `typ` buffer parameter is `fast` or `slow`, then the simulation uses certain combinations of `min` and `max` data. [Table 20](#) specifies the exact type of data used for `fast` and `slow` values.

The `typ` keyword can also be set to a number for a different `typ` data of an IBIS model. To use a parameter for the `typ` keyword, you can only set the parameter to 0, -1, 1, 2, or -2. [Table 19](#) shows the keywords that can be used:

Table 19 Possible Keywords for the *typ* Buffer Parameter

Parameter	Number
typ	0
min	-1
max	1
fast	2
slow	-2

[Table 20](#) lists all parameters and data types for all buffers. Specific buffers use relevant data only. No buffer uses all data in the table (for example, only the terminator specifies and uses the `Rgnd`, `Rpower`, `Rac`, `Cac` parameters).

Table 20 Fast and Slow Data for IBIS Buffers

Parameter/Data	Fast	Slow
C_comp	min	max

Table 20 Fast and Slow Data for IBIS Buffers (Continued)

Parameter/Data	Fast	Slow
Temp_Range	max	min
Voltage_Range	max	min
Pullup_Ref	max	min
Pulldown_Ref	min	max
POWER_Clamp_Ref	max	min
GND_Clamp_Ref	min	max
Rgnd	max	min
Rpower	max	min
Rac	max	min
Cac	min	max
Pulldown	max	min
Pullup	max	min
GND_Clamp	max	min
POWER_Clamp	max	min
Ramp	max	min
Rising_waveform	max	min
Falling_waveform	max	min
V_fixture	max	min
Model Spec	max	min

hsp_ver

Syntax

```
hsp_ver = hspice_version
```

The default is the current version of the PrimeSim Continuum simulators. If you prefer the previous version of the IBIS buffer, use the following statement:

```
hsp_ver = version number
```

power

Syntax

```
power = on|off
```

The default is `power=on`. To connect buffers to power sources that are specified in the IBIS file, use the Voltage Range, Pullup Reference, Pulldown Reference, POWER Clamp Reference, and GND Clamp Reference keywords.

By default, the simulation connects the required voltage sources for such external nodes as Pullup, Pulldown, Power_Clamp, and Ground_Clamp if applicable. Do not connect these nodes to voltage sources. However, you should specify names for these nodes so you can print out the voltage values if required.

If `power=off`, use internal voltage sources are not included in the buffer, and you must add external voltage sources. Use this option if the voltage source is not connected directly to buffer nodes but through a circuit to account for parasitic RLC to simulate power/ground bounce, and so on.

interpol

Syntax

```
interpol = 1|2
```

Default is `interpol=1` (recommended). The I/V curves and V(t) curves need to be interpolated. The `interpol=1` keyword uses linear interpolation and `interpol=2` uses quadratic bi-spline interpolation.

xv_pu | xv_pd

Syntax

```
xv_pu = nd_state_pu  
xv_pd = nd_state_pd
```

The buffers with output function (output, input-output, tristate, and so on) are controlled by one (input) or two (input and enable) controlling signals. To describe the state of a buffer

at any moment, use two state variables, St_{pu} and St_{pd} , which vary from 0 to 1. For example:

- If the output buffer is in LOW state, then $St_{pu}=0$, $St_{pd}=1$.
- If the output buffer transitions from a LOW state to HIGH state, then St_{pu} continuously changes from 0 to 1, while St_{pd} goes from 1 to 0.

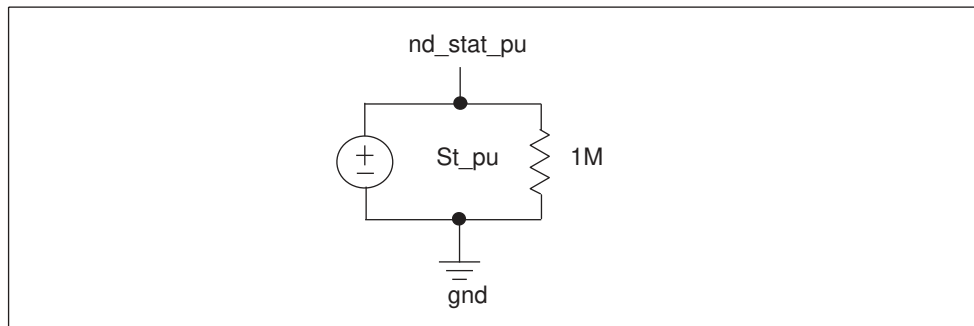
The actual time dependence for such a transition is derived from either ramp data or waveforms.

You might want to know exactly how the transition takes place. The $xv_{pu}=nd_state_pu$, $xv_{pd}=nd_state_pd$ keywords provide such information. Here nd_state_pu and nd_state_pd are names of additional nodes (which must be unique, and are treated as any other node from the netlist, except for a 16-character limitation). If you include the keywords, then the simulation adds voltage sources (with 1M Ω parallel resistor).

The values of the voltages are St_{pu} and St_{pd} . You can print or display them as follows:

```
.PRINT V(nd_state_pu) V(nd_state_pd)]
```

Figure 34 Equivalent Circuit for $xv_{pu}=nd_state_pu$ Keyword



ramp_fwf | ramp_rwf

Syntax

```
ramp_fwf = 2|1|0  
ramp_rwf = 2|1|0
```

Default is $ramp_fwf=2$ [Falling_Waveform] and $ramp_rwf=2$ [Rising_Waveform]. If ramp or waveform data are available, you can use these options to choose which data to use.

The $ramp_fwf$ parameter controls falling waveforms/ramp. The $ramp_rwf$ parameter controls rising waveforms/ramp.

- Value 0 denotes use ramp data.
- Value 1 denotes use of one waveform:
 - For `ramp_fwf=1`, if more than one falling waveform is available, the simulation uses the first falling waveform for the model.
 - For `ramp_rwf=1`, if more than one rising waveform is available, the simulation uses the first rising waveform for the model.
- Value 2 (default) denotes use of two waveforms:
 - For `ramp_fwf=2`, if more than two falling waveforms are available, the PrimeSim Continuum solution uses the first two falling waveforms found for the model.
 - For `ramp_rwf=2`, if more than two rising waveforms are available, the PrimeSim Continuum solution uses the first two rising waveforms found for the model.

If IC circuit simulation cannot perform a specified type of processing (for example, if you specify `ramp_fwf=2`, but only one falling waveform is found), it decrements values of `ramp_fwf` or `ramp_rwf` by one and attempts to process the new values of `ramp_fwf` and/or `ramp_rwf`. In this case, a warning is printed (unless the `nowarn` option is set).

Note:

The `ramp_fwf` and `ramp_rwf` parameters are independent, and can have different values.

The PrimeSim Continuum solution selects two proper VT waveforms in a sampling method according to the initial voltages of VT waveforms and V(out) before a buffer's translation. For example: assume there are 3 rising waveforms whose initial voltage are: 0v, 0.2v, and 0.4v, respectively. Assuming at a current time point, the buffer is in low state (PU off, PD on) with V(out)=0.1v. If input is a rising edge, then the buffer starts translation from low to high. The PrimeSim Continuum solution selects the two rising waveforms with initial voltages of 0v and 0.2v. In other words, it selects two waveforms so that $\text{Initial_VT1} \leq V(\text{out}) \leq \text{Initial_VT2}$.

Invalid Ramp Warning

By default, the PrimeSim Continuum solution sets the parameters `ramp_rwf` and `ramp_fwf` to 2. The PrimeSim Continuum solution then looks for two [Rising Waveform] and two [Falling Wave Form] tables in the IBIS file. The invalid ramp warning is issued when you do not have the same number of tables in the IBIS file.

You can avoid this warning by setting the proper values for `ramp_rwf` and `ramp_fwf`. If you do not have any waveform tables then, set both parameters to 0 and the [Ramp] data from the IBIS file is used.

fwf_tune | rwf_tune

Syntax

```
fwf_tune = fwf_tune_value
rwf_tune = rwf_tune_value
```

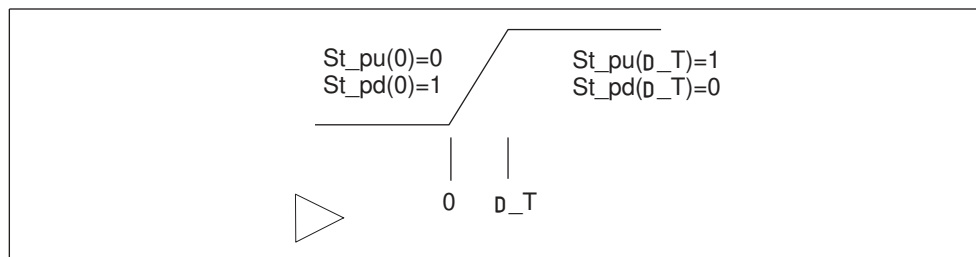
The `fwf_tune_value` and `rwf_tune_value` keywords are numbers between 0 and 1. For ramp data, the default is `fwf_tune=0.1` and `rwf_tune=0.1`; For single VT waveform, the default is `fwf_tune=0.25` and `rwf_tune=0.25`. The `fwf_tune`, `rwf_tune` parameters specify transition time for circuitry (either pullup or pulldown) that goes from the ON to OFF state. This is specified as a fraction of time (`delta_T`) for a transition for the opposite circuitry (either pulldown or pullup) from OFF to ON state. Here, `delta_T` means the real transition time. For ramp data, `delta_T` is `dT/0.6`; for single VT waveform, `delta_T` is obtained by subtracting the non switching time of the waveform from the total time of it.

The following two parameters control the algorithm for processing ramp and a single VT waveform.

- Use `fwf_tune` only when `ramp_fwf` is 0 or 1.
- Use `rwf_tune` only when `ramp_rwf` is 0 or 1.

Figure 35 shows the effect of these parameters when switching the output buffer from LOW to HIGH.

Figure 35 Change in Values of $St_{pu}(t)$ and $St_{pd}(t)$ When a Buffer is Switched from LOW to HIGH



Initially, $St_{pd}=1$, $St_{pu}=0$. Both ramp data and a single rising waveform provide information about the switching process. A time interval, δ_T occurs during the transition from LOW $\rightarrow$ HIGH. The difference between the two data types (ramp and a single rising waveform) is that the shape of the waveform for ramp is fixed as a linearly growing function from LOW to HIGH. By contrast, an actual waveform accounts for an arbitrary time dependence.

However, this is not enough information to determine $St_{pu}(t)$ and $St_{pd}(t)$ [recall that $St_{pu}(0)=0$, $St_{pd}(0)=1$, $St_{pu}(\delta_T)=1$, $St_{pd}(\delta_T)=0$]. Mathematically,

this represents one linear equation with two unknowns that have an infinite number of solutions. To resolve this problem, you must impose additional conditions on `St_pu` and `St_pd`.

Synopsys IBIS device models use the following approach.

The circuitry that goes from ON to OFF (for rising waveforms, pulldown circuitry) usually undergoes this transition much faster than the circuitry that goes from OFF to ON (for rising waveforms, pullup circuitry), we specify a fraction of time in units of `delta_T`, during which the circuitry that goes from ON to OFF undergoes the transition.

Therefore, if `rwf_tune=0.1`, then during $0.1 * \text{delta_T}$, the pulldown circuitry switches from ON to OFF. The transition is a linear function of time. After imposing this additional condition, you can uniquely find the rate of transition for the circuitry that goes from the OFF state to ON state.

This approach is also valid for the `fwf_tune` parameter. If resultant `St_pu`, `St_pd` are not reasonable, (e.g., they are far from the span between 0 and 1, then the preceding approach is discarded and the additional condition $\text{St_pu} + \text{St_pd} = 1$ is used instead. In this situation, `rwf_tune` and `fwf_tune` are useless and ignored.

The `fwf_tune` and `rwf_tune` parameters are optimization parameters. The significance of these parameters strongly depends on I/V curves for pullup and pulldown circuitries. A change in `fwf_tune` and `rwf_tune` can be insignificant or very significant, depending on the I/V curves. Adjust these parameters slightly to evaluate the accuracy of the model.

If you use two waveforms, the corresponding system of equations is completely defined mathematically and the `fwf_tune` and `rwf_tune` parameters are not used (ignored if specified). However, if the data in two waveforms are inaccurate or inconsistent with other data, circuit simulation can use a single waveform or ramp data instead of two waveforms (issues a warning). If this occurs, `fwf_tune` and/or `rwf_tune` are used even if `ramp_fwf=2` and `ramp_rwf=2`.

If the two-waveform data is inconsistent or inaccurate, the results can be less accurate than ramp or one-waveform results. You can compare that two-waveform results against ramp and one-waveform results.

You can use the `xv_pu=nd_state_pu` and `xv_pd=nd_state_pd` keywords to print or view the `St_pu(t)` and `St_pd(t)` state evolution functions.

`rwf_pd_dly` | `fwf_pu_dly`

Syntax

```
rwf_pd_dly = rwf_pd_dly_value  
fwf_pu_dly = fwf_pu_dly_value
```


The `rwf_pd_dly` and `fwf_pu_dly` parameters can improve the accuracy of a single VT waveform-based IBIS buffer. They provide internal delay between external stimulus triggering and pulldown circuit turn off for rising edge, or pullup circuit turn off for falling edge.

The `rwf_pd_dly_value` and `fwf_pu_dly_value` keywords are numbers between 0 and 1. The default is the ratio of beginning non switching time in the related VT waveform to the total transition time of the VT waveform (i.e., last time point of it).

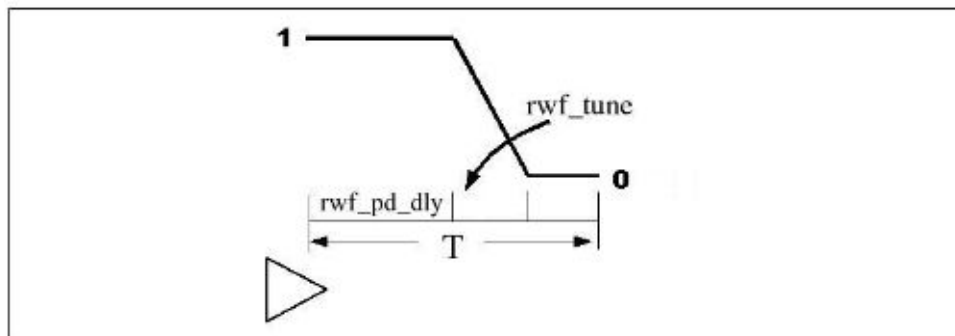
- Use `rwf_pd_dly` only when `ramp_rwf=1`
- Use `fwf_pu_dly` only when `ramp_fwf=1`

A `rwf_pd_dly` parameter is used with a `rwf_tune` parameter to provide a more real pulldown circuit turn off transition for a rising edge, like that shown in Figure 35 for `St_pd(t)`. Similarly, a `fwf_pu_dly` parameter is used with a `fwf_tune` parameter to provide a more real pullup circuit turn off transition for a falling edge. In Figure 36, the internal delay is $rwf_pd_dly \cdot T$, where T is the transition time of the single rising or falling waveform.

Note:

$rwf_pd_dly_value + rwf_tune$ should be less than 1. Otherwise, the PrimeSim Continuum solution resets `rwf_tune` to $1 - rwf_pd_dly_value$. Similarly, $fwf_pd_dly_value + fwf_tune$ should be less than 1. Otherwise, the PrimeSim Continuum solution resets `fwf_tune` to $1 - fwf_pd_dly_value$.

Figure 36 Pulldown Circuit Turn Off Transition and Internal Delay Calculation



pd_scal | pu_scal | pc_scal | gc_scal | rwf_scal | fwf_scal

Syntax

```
pd_scal=pd_scal_value
pu_scal=pu_scal_value
pc_scal=pc_scal_value
```

```
gc_scal=gc_scal_value  
rwf_scal=rwf_scal_value  
fwf_scal=fwf_scal_value
```

All IBIS and I-V curves can be scaled in the PrimeSim Continuum solution. The `rwf_scal` and `fwf_scal` parameters scale the rising and falling V-T curves. The `pd_scal`, `pu_scal`, `pc_scal`, and `gc_scal` parameters scale the pullup, pulldown, ground and power clamp I-V curves of the buffer. The default for all parameters is 1. If you set the `rwf_scal` and `fwf_scal` parameters, you affect the rise (for `rwf_scal`) and fall (for `fwf_scal`) times and the delay of buffer. Scaling the I-V curves of the buffer effectively scales the drive strength of the buffer.

dt_scale

Syntax

```
dt_scale=on|off
```

For the `ramp` keyword in IBIS model, there will always be a sub-param of `dv/dt_r` (or `dv/dt_f`). `dv` means the 20% to 80% voltage swing of output, and `dt` means the time it takes to swing such a voltage. In fact, the `dv` can be obtained by the DC computation for the buffer in low and high states respectively. If there is inconsistency between the computed `dv` and the value from ramp data, then the PrimeSim Continuum solution resets the `dv` according to DC computation. If `dt_scale=on`, the PrimeSim Continuum solution scales the `dt` according to the `dv` scale. If `dt_scale=off`, the PrimeSim Continuum solution keeps `dt` unchanged.

The default value for `dt_dscale` is `off`.

ss_state

Syntax

```
ss_state = on|off
```

The `ss_state` keyword turns `on` or `off` the state electrical models in the `.ibis` file. The default is `ss_state=on`, but only for the series switch buffer model data.

rm_dly_rwf|rm_dly_fwf|rm_tail_rwf|rm_tail_fwf

Syntax

```
rm_dly_rwf=default | rdly_time_value  
rm_dly_fwf=default | fdly_time_value
```

```
rm_tail_rwf=default | rtail_time_value  
rm_tail_fwf=default | ftail_time_value
```

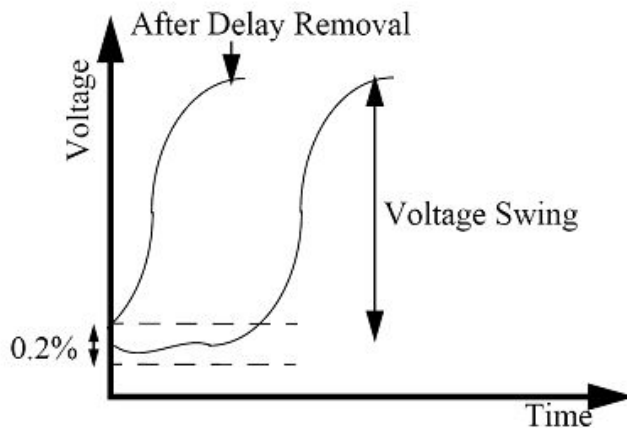
These four keywords are used to remove non-switching time from VT waveforms in IBIS model.

- `rm_dly_rwf` and `rm_dly_fwf` remove initial delays from [Rising Waveform] and [Falling Waveform], respectively. (It is strongly recommended that `rdly_time_value` be the same as `fdly_time_value` so that the duty cycle of output waveform is not changed after using these keywords.)
- `rm_tail_rwf` and `rm_tail_fwf` remove flat tails from [Rising Waveform] and [Falling Waveform], respectively.

For every keyword, if the value is set as a positive number, then that amount of time is removed from all corresponding VT waveforms. For example, if `rm_dly_rwf=1ns`, then 1ns initial delay is removed from each [Rising Waveform] in the related IBIS model.

For every keyword, if the value is set as 'default', then for each VT waveform, the PrimeSim Continuum solution computes the amount of no-switching time based on the criteria that the voltage change should be less than 0.2% of full voltage swing, then selects the minimum one as the final value used. For example, if `rm_dly_rwf=default`, then the PrimeSim Continuum solution computes the initial delay of each [Rising Waveform] (Figure 37), it then selects the minimum value of such delays as the value of the keyword. With the G-2012.06 release, if the keyword `rm_dly_rwf` and `rm_dly_fwf` is not set in a B-element card, then it is the equivalent of "`rm_dly_fwf=default`".

Figure 37 Removal of initial delay of [Rising Waveform] by `rm_dly_rwf=default`



The PrimeSim Continuum solution issues a warning if the value of any keyword is more than the default, (more than the value obtained by the 0.2% of full voltage swing criteria).

Note:

For a IBIS models with a sub model like the driver schedule, the value of every keyword set in the B-element card is applied on all sub models.

nowarn

Syntax

```
nowarn
```

The `nowarn` keyword suppresses warning messages from the IBIS parser. There is no equal sign “=” and value after the `nowarn` keyword. Do not use `nowarn` as the first keyword after the nodes list. Use at least one keyword followed by “=” and a value between the list of nodes and the `nowarn` keyword.

c_com_pu | c_com_pd | c_com_pc | c_com_gc

Syntax

```
c_com_pu = c_com_pu_value  
c_com_pd = c_com_pd_value  
c_com_pc = c_com_pc_value  
c_com_gc = c_com_gc_value
```

By default, (default 1) the C_comp die capacitance connects between node_out (nd_in for input buffer) and node_gc (or node_pd, if node_gc does not exist in B element terminals). To simulate power bounce and ground bounce, split C_comp into several parts. Then connect between node_out (nd_in for input buffer) and some (or all) of the node_pu, node_pd, node_pc, and node_gc nodes.

If you specify at least one of the optional parameters (`c_com_pu`, `c_com_pd`, `c_com_pc`, and `c_com_gc`), then the default (1) does not apply, and unspecified parameters have a value of zero (default 2). The `c_com_pu`, `c_com_pd`, `c_com_pc`, and `c_com_gc` values are dimensionless, and denote fractions of C_comp connected between node_out (nd_in for input buffer) and respective nodes (either node_pu, node_pd, node_pc, or node_gc). For example, C_comp*c_com_pu is capacitance connected between node_out and node_pu.

Do not specify negative values for `c_com_pu`, `c_com_pd`, `c_com_pc`, and `c_com_gc`.

It is expected that $c_com_pu + c_com_pd + c_com_pc + c_com_gc = 1$. However, the PrimeSim Continuum solution -based simulators do not enforce this requirement, and warn you only if the requirement is not satisfied.

In this case, deriving the states assumes that the IBIS files specifies `C_comp` for the die. The simulation uses different value of `C_comp`, namely:

```
C_comp*(c_com_pu+c_com_pd+c_com_pc+c_com_gc)
```

Effectively, it means that some additional capacitance connects in parallel to `C_comp` (possibly negative).

For the output, input-output, and 3-state buffer types, if you do not specify the `node_pc` and `node_gc` nodes in the netlist, `c_com_pc` is added to `c_com_pu` and `c_com_gc` is added to `c_com_pd`. After that, `c_com_pc` and `c_com_gc` are not used anymore.

For the open drain, open sink, input-output open drain, and input-output open sink buffer types, if you do not specify the `node_pc` and `node_gc` nodes in the netlist, `c_com_pc` if given is ignored, and `c_com_gc` is added to `c_com_pd`. After that, `c_com_gc` is not used anymore.

For the open source, and input-output open source buffer types, if you do not specify the `node_pc` and `node_gc` nodes in the netlist, `c_com_gc` if given is ignored, and `c_com_pc` is added to `c_com_pu`. After that, `c_com_pc` is not used anymore.

For the output ECL, input-output ECL, and 3-state ECLbuffer types, if you do not specify the `node_pc` and `node_gc` nodes in the netlist, `c_com_pc` and `c_com_gc` are ignored (assign zero values).

For the output ECL, input-output ECL, and 3-state ECLbuffer types, if `c_com_pd` is not zero, it is added to `c_com_pu` (`c_com_pd` is not used after that).

Note:

The PrimeSim Continuum solution supports `C_comp_pullup`, `C_comp_pulldown`, `C_comp_power_clamp`, and `C_comp_gnd_clamp` in the IBIS model only when `C_comp` is *not* specified in the same model. In this situation, keywords `c_com_*` (following B element) is ignored and a warning is issued.

In an IBIS model file, if there is a [C Comp Corner] keyword, then the `c_comp` data in [C Comp Corner] block will be used by the PrimeSim Continuum solution with the same rules as described previously in this section. And any `c_comp` data outside of [C Comp Corner] is ignored.

detect_oti_mid

Syntax

```
detect_oti_mid=1|0
```

The default is `detect_oti_mid=1`. If `detect_oti_mid=1`, `V_out_of_in` is calculated according to the rules in the description of `Input_Buffer`. Otherwise, if `detect_oti_mid=0`, then the only difference with respect to `detect_oti_mid=1` is that when a buffer is behaving as a receiver, `V_out_of_in` does not take the value 0.5 if `V_in` is between `V_inl` and `V_inh`. Instead, it keeps the same value as at last time point. This keyword is only valid for `Input_Buffer` or `IO_Buffer`.

time_control

Syntax

```
time_control=1|0
```

IBIS simulation was dependent on the PrimeSim Continuum solution transient time step control methods. When using `RUNLVL`, the PrimeSim Continuum solution can relax the time stepping to improve performance but this may not be ideal for IBIS simulations in certain cases. In order to improve IBIS simulation accuracy, a customized time step control for IBIS buffers is used. The default setting for `time_control` is 1, meaning that the IBIS time step control is enabled.

.OPTION D_IBIS

The `D_IBIS` option specifies the directory containing the IBIS files. For the syntax and examples, refer to `.OPTION D_IBIS` in the *PrimeSim HSPICE® Reference Manual: Commands and Control Options*.

A

Lumped Transmission Line — U-Model

Describes lossy transmission lines using the U-element. The U-model computes transmission line electrical properties from common wire and interconnect cross-sectional geometries. Currently, the U-element is rarely used.

This appendix contains the following sections:

- [Using the U-element for Lossy Transmission Lines](#)
- [U Model for Transmission Lines](#)
- [U-element Examples, Models, and Applications](#)

Using the U-element for Lossy Transmission Lines

The U-element is useful for modeling lossy non-ideal transmission lines in terms of lumped-element equivalent circuits. The U-elements can model effects such as

- Time delay
- Phase shift
- Coupling and Crosstalk
- Distortion

The U-elements do not support the [M=va/] multiplier function. If a U-element is used in a subcircuit and an instance of the subcircuit has a multiplier applied, that multiplier is not applied to the U-element.

A warning message similar to the following is issued in both the status file (.st0) and the output file (.lis) if the smallest transmission line delay is less than TSTOP/10e6:

```
**warning**: the smallest T-line delay (TD) = 0.245E-14 is too small
Please check TD, L and SCALE specification
```

This feature is an aid to finding errors that cause excessively long simulations.

Note:

All transmission lines have a ground reference for the signal conductors. In this appendix, the ground reference is called the reference plane so that it is not confused with SPICE ground. The reference plane is the shield or the ground plane of the transmission line element. The reference plane nodes may or may not be connected to SPICE ground.

Syntax

```
Uxxx in1 [in2 ...in5] refin out1 [out2 ...out5]  
+ refout mname L=val [LUMPS=val]
```

Parameter	Description
Uxxx	Lossy (U-element) transmission line element name. Must begin with U, followed by up to 1023 alphanumeric characters.
inx	Signal input node for the xth transmission line (in1 is required).
refin	Ground reference for the input signal.
outx	Signal output node for the x th transmission line (each input port must have a corresponding output port).
refout	Ground reference for the output signal.
mname	Model reference name for the U-model lossy transmission-line.
L	Physical length of the transmission line, in units of meters.
LUMPS	Number of lumped-parameter sections used to simulate the element.

In this syntax, the number of ports on a single transmission line is limited to five in and five out. One input and output port, the ground references, a model reference, and a length are all required.

The U-element models single and coupled lossy transmission lines for various planar, coaxial, and twinlead structures. When U-element is included in your netlist, simulation creates an internal network of R, L, C, and G-elements to represent up to five lines and their coupling capacitances and inductances.

You can specify interconnect properties in three ways:

- Specify the R, L, C, and G (conductance) parameters in a matrix form (`ELEV=2`).
- Provide common electrical parameters, such as characteristic impedance and attenuation factors (`ELEV=3`).
- Specify the geometry and the material properties of the interconnect (`ELEV=1`).

This section initially describes how to use the third method.

The U-element is optimized for typical geometries used in ICs, MCMs, and PCBs. The model's closed form expressions are optimized via measurements and comparisons with several different electromagnetic field solvers.

The U-element geometric model handles one to five uniformly spaced transmission lines, all at the same height. Also, the transmission lines can be on top of a dielectric (microstrip), buried in a sea of dielectric (buried), have reference planes above and below them (stripline), or have a single reference plane and dielectric above and below the line (overlay). Thickness, conductor resistivity, and dielectric conductivity allow for calculating loss as well.

The U-element statement contains the element name, the connecting nodes, the U model reference name, the length of the transmission line, and, optionally, the number of lumps in the element. You can create two kinds of lossy lines: lines with a reference plane inductance (LRR, controlled by the model parameter `LLEV`) and lines without a reference plane inductance. Wires on integrated circuits and printed circuit boards require reference plane inductance.

The reference ground inductance and the reference plane capacitance to SPICE ground are set by the `HGP`, `CMULT`, and optionally, the `CEXT` parameters.

U Model for Transmission Lines

The following shows the netlist syntax for the U model.

```
.MODEL mname U LEVEL=3 ELEV=val <PLEV=val <DLEV=val>  
+ <LLEV=val> + <Pname=val> ...
```

Parameter	Description
LEVEL=3	Selects the lossy transmission line model
ELEV= <i>val</i>	Selects the electrical specification format including the geometric model <i>val</i> =1
PLEV= <i>val</i>	Selects the transmission line type
DLEV= <i>val</i>	Selects the dielectric and ground reference configuration

Parameter	Description
<code>LLEV=val</code>	Selects the use of reference plane inductance and capacitance to the PrimeSim Continuum solution ground.
<code>Pname=val</code>	Specifies a physical parameter, such as NL or WD (see Table 21) or a loss parameter, such as RHO or NLAY (see Table 22).

[Table 21](#) and [Table 22](#) list the model parameters.

In the above table, a) sea, `DLEV=0`; b) microstrip, `DLEV=1`; c) stripline, `DLEV=2`; and d) overlay, `DLEV=3`.

The following sections discuss these topics:

- [Selecting U Models](#)
- [Lossy U Model Parameters for Planar Geometries](#)
- [Lossy U Model Parameters for Geometric Coax \(PLEV=2, ELEV=1\)](#)
- [Lossy U Model Parameters Geometric Twinlead \(PLEV=3, ELEV=1\)](#)

Selecting U Models

The U model allows three different description formats: geometric/physical, precomputed, and electrical. It provides equally natural description of vendor parts, physically described shapes, and parametric input from field solvers. The description format is specified by the required model parameter `ELEV` as follows:

- `ELEV=1` – geometric/physical description such as width, height, and resistivity of conductors. It is used by board designers dealing with physical design rules.
- `ELEV=2` – precomputed parameters. These are available with some commercial packaging, or as a result of running a field solver on a physical description of commercial packaging.
- `ELEV=3` – electrical parameters such as delay and impedance, available with purchased cables. It allows one conductor and ground plane for `PLEV=1`.

The U model explicitly supports transmission lines with several types of geometric structures. The geometric structure type is indicated by the `PLEV` model parameter as follows:

- `PLEV=1` – Selects planar structures, such as microstrip and stripline, (the usual conductor shapes on integrated circuits and printed-circuit boards).
- `PLEV=2` – Selects coax, which frequently is used to connect separated instruments.

- **PLEV=3** – Selects twinlead, which is used to connect instruments and to suppress common mode noise coupling.

Figure 38 U Model Geometric Structure (PLEV) Options

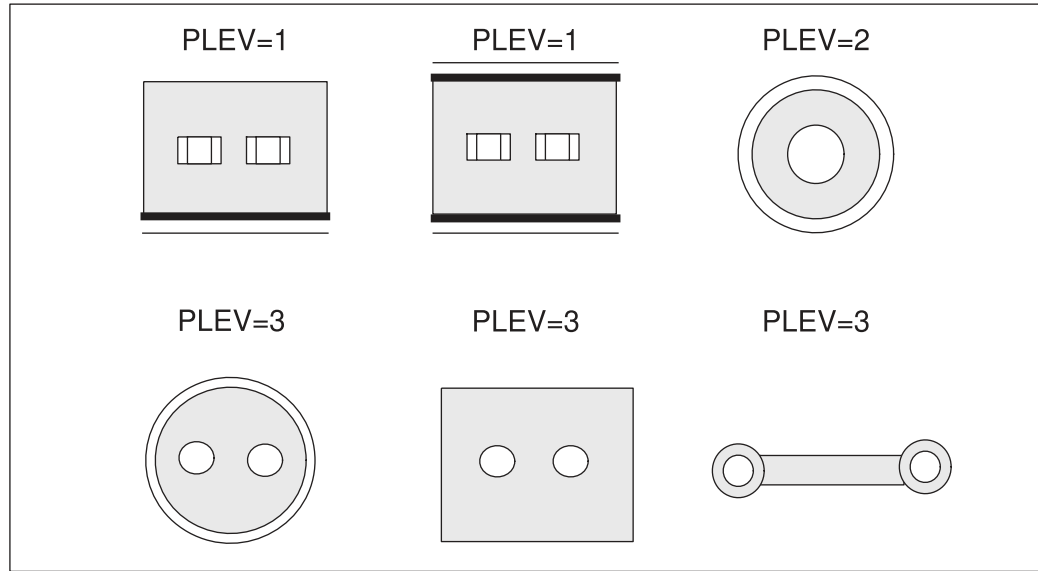
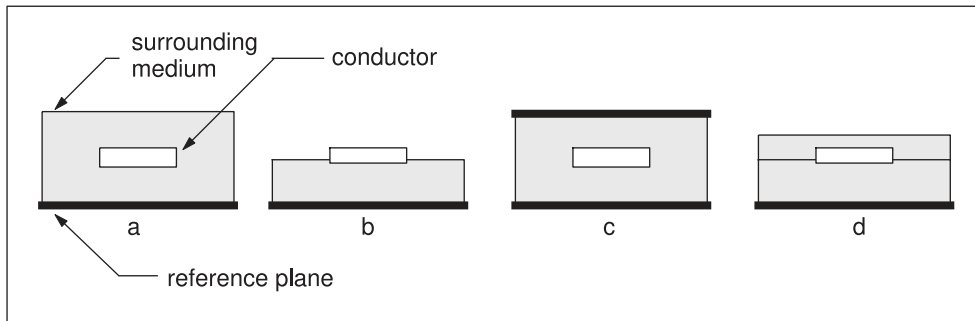


Figure 39 shows the four dielectric (DLEV) configurations available for the geometric U model. You use the **DLEV** switch to specify one of these configurations. The geometric U model uses **ELEV=1**, and the configurations shown are for **PLEV=1** planar geometries.

Figure 39 Dielectric and Reference Plane (DLEV) Configuration Options



In the above figure, a) sea, **DLEV=0**; b) microstrip, **DLEV=1**; c) stripline, **DLEV=2**; and d) overlay, **DLEV=3**.

Lossy U Model Parameters for Planar Geometries

The following sections discuss these topics:

- [Common Planar Model Parameters](#)
- [Physical Parameters](#)
- [Loss Parameters](#)
- [Geometric Parameter Recommended Ranges](#)
- [Reference Planes and the PrimeSim Continuum Ground with LLEV](#)
- [Estimating the Skin Effect Frequency](#)
- [Number of Lumped-Parameter Sections](#)
- [Ringing](#)
- [Geometric Parameters \(ELEV=1\)](#)

Common Planar Model Parameters

The parameters for U models are listed in [Table 21](#).

Table 21 *U-element Physical Parameters*

Parameter	Units	Default	Description
LEVEL		required	(=3) required for lossy transmission lines model
ELEV		required	Electrical model (=1 for geometry)
DLEV			Dielectric model (=0 for sea, =1 for microstrip, =2 stripline, =3 overlay; default is 1)
PLEV		required	Transmission line physical model (=1 for planar)
LLEV			Omit or include the reference plane inductance (=0 to omit, =1 to include; default is 0)
NL			Number of conductors (from 1 to 5)
WD	m		Width of each conductor
HT	m		Height of all conductors
TH	m		Thickness of all conductors
THB	m		Reference plane thickness
TS	m		Distance between reference planes for stripline (default for DLEV=2 is 2 HT + TH. TS is not used when DLEV=0 or 1)

Table 21 *U-element Physical Parameters (Continued)*

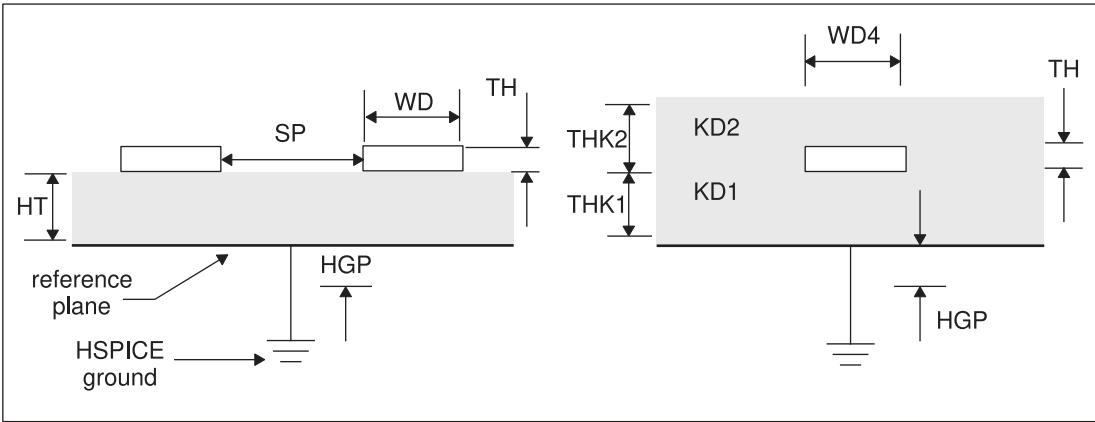
Parameter	Units	Default	Description
SP	m		Spacing between conductors (required if NL > 1)
KD			Dielectric constant
XW	m		Perturbation of conductor width added (default is 0)
CEXT	F/m		External capacitance between reference plane and ground. Only used when LLEV=1, this overrides the computed characteristic.
CMULT		1	Dielectric constant of material between reference plane and ground (default is 1 – only used when LLEV=1)
HGP	m		Height of the reference plane above the PrimeSim Continuumground. Used for computing reference plane inductance and capacitance to ground (default is 1.5*HT – HGP is only used when LLEV=1).
CORKD			Perturbation multiplier for dielectric (default is 1)
WLUMP		20	Number of lumps per wavelength for error control
MAXL		20	Maximum number of lumps per element

The U model has two parametric adjustments: **XW** and **CORKD**. **XW** adds to the width of each conductor, but does not change the conductor pitch (spacing plus width). It is useful for examining the effects of conductor etching. **CORKD** is a multiplier for the dielectric value. Some board materials vary more than others. It provides an easy way to test tolerance to dielectric variations.

Physical Parameters

The dimensions for one and two-conductor planar transmission lines are shown in [Figure 40](#).

Figure 40 U-element Conductor Dimensions



Loss Parameters

Table 22 lists the loss parameters for the U model.

Table 22 U-element Loss Parameters

Parameter	Units	Description
RHO	ohm·m	Conductor resistivity (default is rho of copper, 17E-9 ohm·m)
RHOB	ohm·m	Reference plane resistivity (default value is for copper)
NLAY		Number of layers for conductor resistance computation (=1 for DC resistance or core resistance, =2 for core and skin resistance at skin effect frequency)
SIG	mho/m	Dielectric conductivity

Losses have a large impact on circuit performance, especially as clock frequencies increase. RHO, RHOB, SIG, and NLAY are parameters associated with losses. Time domain simulators, such as SPICE, cannot directly handle losses that vary with frequency. Both the resistive skin effect loss and the effects of dielectric loss create loss variations with frequency. NLAY is a switch that turns on skin effect calculations in circuit simulation and analysis. The skin effect resistance is proportional to the conductor and backplane resistivities, RHO and RHOB.

Dielectric conductivity is included using SIG. The U model computes skin effect resistance at a single frequency and uses that resistance as a constant. The dielectric SIG computes a fixed conductance matrix, which is constant for all frequencies. To closely approximate losses, compute resistances and conductances at the frequency of maximum power dissipation. In AC analysis, resistance increases as the square root of frequency above the skin-effect frequency, and resistance is constant below the skin effect frequency.

Geometric Parameter Recommended Ranges

The U-element analytic equations compute quickly, but have a limited range of validity. The U-element equations were optimized for typical IC, MCM, and PCB applications. [Table 23](#) lists the recommended minimum and maximum values for U-element parameter variables.

Table 23 *Recommended Ranges*

Parameter	Minimum	Maximum
NL	1	5
KD	1	24
WD/HT	0.08	5
TH/HT	0	1
TH/WD	0	1
SP/HT	0.15	7.5
SP/WD	1	5

The U-element equations lose their accuracy when you use values outside the recommended ranges. Because the single-line formula is optimized for single lines, you can see a difference between the parameters of single lines and two coupled lines at a very wide separation. The absolute error for a single line parameter is less than 5% when used within the recommended range. The main line error for coupled lines is less than 15%. Coupling errors can be as high as 30% in cases of very small coupling. Since the largest errors occur at small coupling values, actual waveform errors are kept small.

Reference Planes and the PrimeSim Continuum Ground with LLEV

The schematic for a single lump of the U model with $L_{LEV}=0$, is shown in [Figure 41](#). If $L_{LEV}=1$, the schematic includes inductance in the reference path as well as capacitance to the PrimeSim Continuum ground.

Figure 41 Lossy Line with Reference Plane when LLEV=0

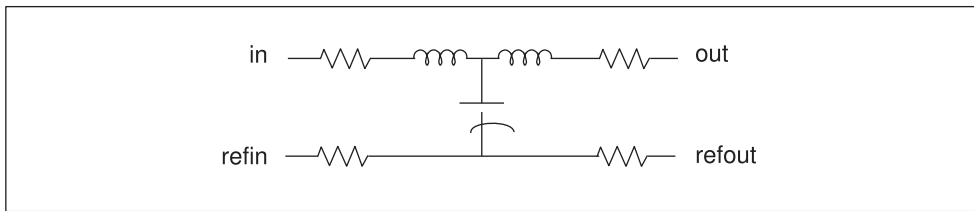
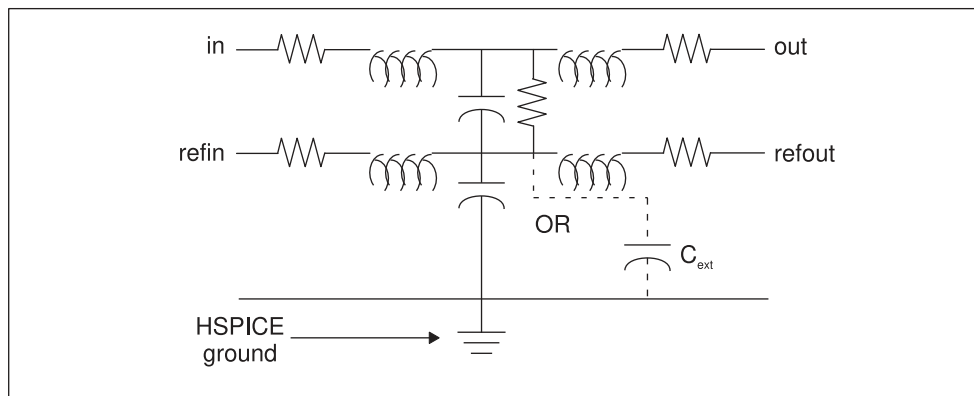


Figure 42 shows a single lump of a U model for a single line with reference plane inductance. If $LLEV=1$, the reference plane inductance is computed, and capacitance from the reference plane to the PrimeSim Continuum ground is included in the model. The reference plane is the ground plane of the conductors in the U model.

Figure 42 Schematic of a U-element Lump when LLEV=1



The model reference plane is not necessarily the same as the PrimeSim Continuum solution ground. For example, a printed circuit board with transmission lines can have a separate reference plane above a chassis. Simulation with the U model uses either HGP , the distance between the reference plane and the PrimeSim Continuum solution ground, or C_{ext} to compute the parameters for the ground-to-reference transmission line.

If you use HGP , the capacitance per meter of the ground-to-reference line is computed based on a planar line of width $(NL+2)(WD+SP)$ and the height of HGP above the SPICE ground. $CMULT$ is the dielectric constant of the ground-to-reference transmission line. If you specify C_{ext} , then C_{ext} is the capacitance per meter for the ground-to-reference line. The inductance of the ground-to-reference line is computed from the capacitance per meter and an assumed propagation at the speed of light.

Estimating the Skin Effect Frequency

Most of the power in a transmission line is dissipated at the clock frequency. As a first choice, simulation estimates the maximum dissipation frequency, or skin effect frequency,

from the risetime parameter. The risetime parameter is set with the `.OPTION` statement (for example, `.OPTION RISETIME=0.1ns`).

Some designers use $0.35/trise$ to estimate the skin effect frequency. This estimate is good for the bandwidth occupied by a transient, but not for the clock frequency, at which most of the energy is transferred. In fact, a frequency of $0.35/trise$ is far too high and results in excessive loss for almost all applications. Simulation computes the skin effect frequency from $1/(15*trise)$. If you use precomputed model parameters (`ELEV=2`), compute the resistance matrix at the skin effect frequency.

When the risetime parameter is not given, simulation uses other parameters to compute the skin effect frequency. The circuit simulator examines the `.TRAN` statement for `tstep` and `delmax` and examines the source statement for `trise`. If you set any one of the parameters `tstep`, `delmax`, and `trise`, simulation uses the maximum of these parameters as the effective risetime. In AC analysis, the skin effect is evaluated at the frequency of each small-signal analysis. Below the computed skin effect frequency (`ELEV=1`) or `FR1` (`ELEV=3`), the AC resistance is constant. Above the skin effect frequency, the resistance increases as the square root of frequency.

Number of Lumped-Parameter Sections

The number of sections (lumps) in a transmission line model also affects the transmission line response. Simulation computes the default number of lumps from the line delay and the signal risetime. There should be enough lumps in the transmission line model to ensure that each lump represents a length of line that is a small fraction of a wavelength at the highest frequency used. It is easy to compute the number of lumps from the line delay and the signal risetime by using an estimate of $0.35/trise$ as the highest frequency.

For the default number of lumps, simulation uses the smaller of 20 or $1+(20*TD_{eff}/trise)$, where TD_{eff} is the line delay. In most transient analysis cases by using more than 20 lumps gives a negligible bandwidth improvement at the cost of increased simulation time. In AC simulations over many decades of frequency with lines over one meter long, more than 20 lumps may be needed for accurate simulation.

Ringings

Sometimes a transmission line simulation shows ringing in the waveforms as in [Figure 59 on page 318](#). If the ringing is not verifiable by measurement, it might be due to an incorrect number of lumps in the transmission line models or due to the simulator integration method. Increasing the number of lumps in the model or changing the integration method to Gear reduces the amount of ringing due to simulation errors. The default integration method is `TRAP` (trapezoidal), but you can change it to Gear with the statement `.OPTION METHOD=GEAR`.

See [Oscillations Due to Simulation Errors on page 334](#) for more information about the number of lumps and ringing.

The next section covers parameters for geometric lines. It discusses coaxial and twinlead transmission lines, and the previously-described planar type.

Geometric Parameters (ELEV=1)

Geometric parameters provide a description of a transmission line in terms of the geometry of its construction and the physical constants of each layer or other geometric shape involved.

PLEV=1, ELEV=1 Geometric Planar Conductors

Planar conductors are used to model printed circuit boards, packages, and integrated circuits. The geometric planar transmission line is restricted to:

- One conductor height (H_T or H_{T1})
- One conductor width (W_D or W_{D1})
- One conductor thickness (T_H or T_{H1})
- One conductor spacing (S_P or S_{P12})
- One dielectric conductivity (S_{IG} or S_{IG1})
- One or two relative dielectric constants (K_D or K_{D1} , and K_{D2} only if $D_{LEV}=3$)

Common planar conductors include:

- $D_{LEV}=0$ – Microstrip sea of dielectric. This planar conductor has a single reference plane and a common dielectric surrounding conductor (see [Figure 43](#)).
- $D_{LEV}=1$ – Microstrip dual dielectric. This planar conductor has a single reference plane and two dielectric layers (see [Figure 45](#)).
- $D_{LEV}=2$ – Stripline. This planar conductor has an upper and lower reference plane (see [Figure 44](#)). Both symmetric and asymmetric spacing are available.
- $D_{LEV}=3$ – Overlay dielectric. This planar conductor has a single reference plane and an overlay of dielectric material covering the conductor (see [Figure 46](#)).

Figure 43 Planar Transmission Line, $DLEV=0$, Sea of Dielectric

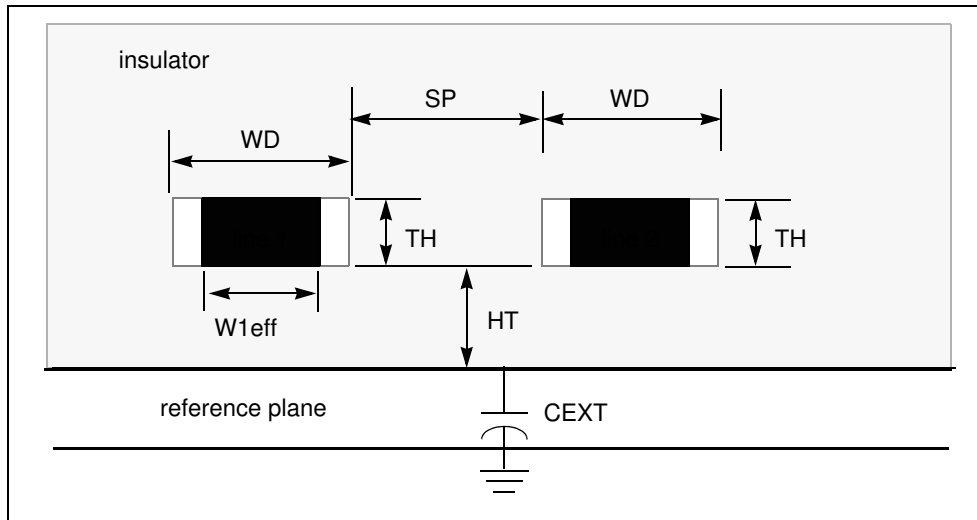


Figure 44 Planar Transmission Line, $DLEV=2$, Stripline

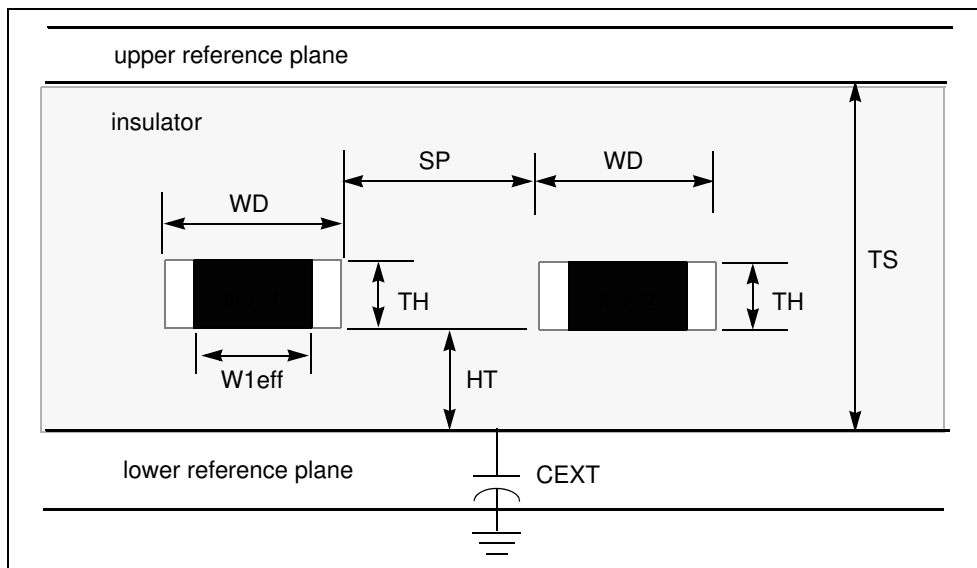


Figure 45 Planar Transmission Line, DLEV=1, Microstrip

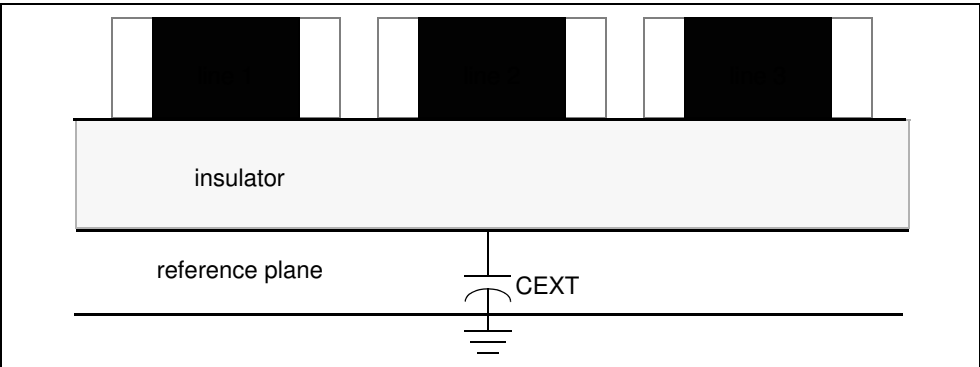


Figure 46 Planar Transmission Line, DLEV=3, Overlay Dielectric

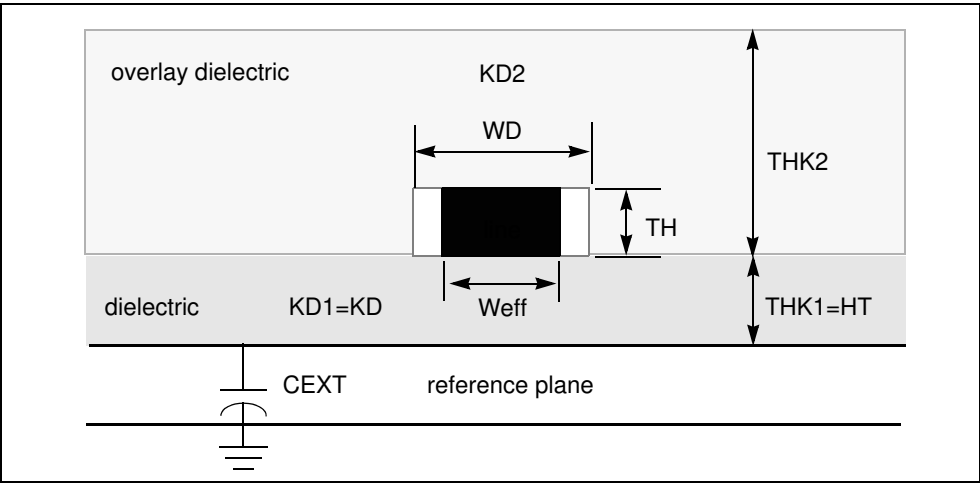


Table 24 Geometric Parameters

Name (Alias)	Units	Default	Description
DLEV	—	1.0	0: microstrip sea of dielectric 1: microstrip layered dielectric 2: stripline
NL	—	1	Number of conductors
NLAY	—	1.0	Layer algorithm: 1: DC cross section only 2: skinddepth cross section on surface plus DC core
HT (HT1)	m	req	Conductor height
WD (WD1)	m	req	Conductor width
TH (TH1)	m	req	Conductor thickness

Table 24 *Geometric Parameters (Continued)*

Name (Alias)	Units	Default	Description
THK1	m	HT	Dielectric thickness for DLEV=3
THK2	m	0.0	Overlay dielectric thickness for $DLEV=30 \leq THK2 < 3 \cdot HT$ (see note after this table)
THB	m	calc	Reference conductor thickness
SP (SP12)	m	req	Spacing: line 1 to line 2 required for $nl > 1$
XW	m	0.0	Difference between drawn and realized width
TS	m	calc	Height from bottom reference plane to top reference plane $TS=TH+2 \cdot HT$ (DLEV=2, stripline only)
HGP	m	HT	Height of reference plane above SPICE ground – LLEV=1
CMULT	—	1.0	CPR multiplier for dielectric constant of material between shield and SPICE ground if LLEV=1 and CEXT is not present
CEXT	F/m	und	External capacitance from reference plane to circuit ground point – used only to override HGP and CMULT computation
RHO	ohm·m	17E-9	Resistivity of conductor material – defaults to value for copper
RHOB	ohm·m	rho	Resistivity of reference plane material
SIG1 (SIG)	mho/m	0.0	Conductivity of dielectric
KD1 (KD)	—	4.0	Relative dielectric constant of dielectric
KD2		KD	Relative dielectric constant of overlay dielectric for $DLEV=3$ $1 < KD1 < 4 \cdot KD$
CORKD	—	1.0	Correction multiplier for KD

Note:

If $THK2$ is greater than three times HT , simulation accuracy decreases. A warning message is issued to indicate this. A reference plane is a ground plane, but it is not necessarily at SPICE ground potential.

Lossy U Model Parameters for Geometric Coax (PLEV=2, ELEV=1)

Figure 47 Geometric Coaxial Cable

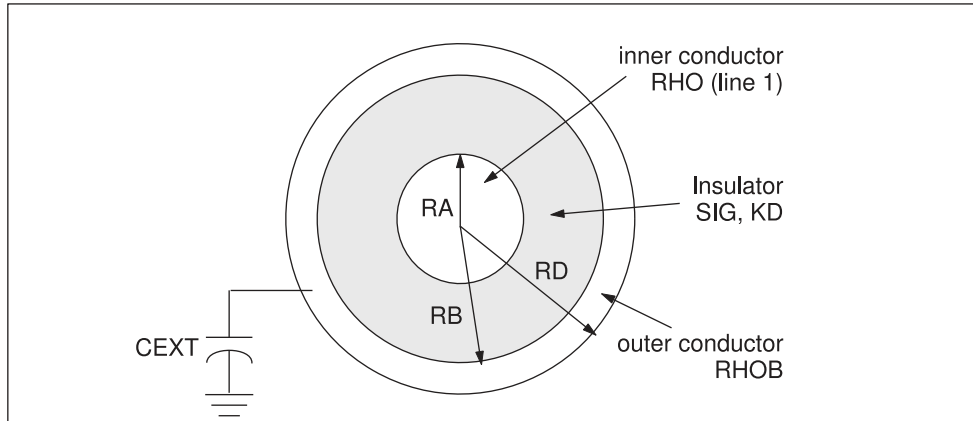


Table 25 Lossy U Model Parameters

Name (Alias)	Units	Default	Description
RA	m	req	Outer radius of inner conductor
RB	m	req	Inner radius of outer conductor (shield)
RD	m	ra+rb	Outer radius of outer conductor (shield)
HGP	m	RD	Distance from shield to SPICE ground
RHO	ohm·m	17E-9	Resistivity of conductor material – defaults to value for copper
RHOB	ohm·m	rho	Resistivity of shield material
SIG	mho/m	0.0	Conductivity of dielectric
KD	—	4.0	Relative dielectric constant of dielectric
CMULT	—	1.0	Multiplier (used in defining CPR) for dielectric constant of material between shield and SPICE ground when LLEV=1 and CEXT is not present
CEXT	F/m	und.	External capacitance from shield to SPICE ground – used only to override HGP and CMULT computation
SHTHK	m	2.54E-4	Coaxial shield conductor thickness

Lossy U Model Parameters Geometric Twinlead (PLEV=3, ELEV=1)

Figure 48 Geometric Embedded Twinlead, DLEV=0, Sea of Dielectric

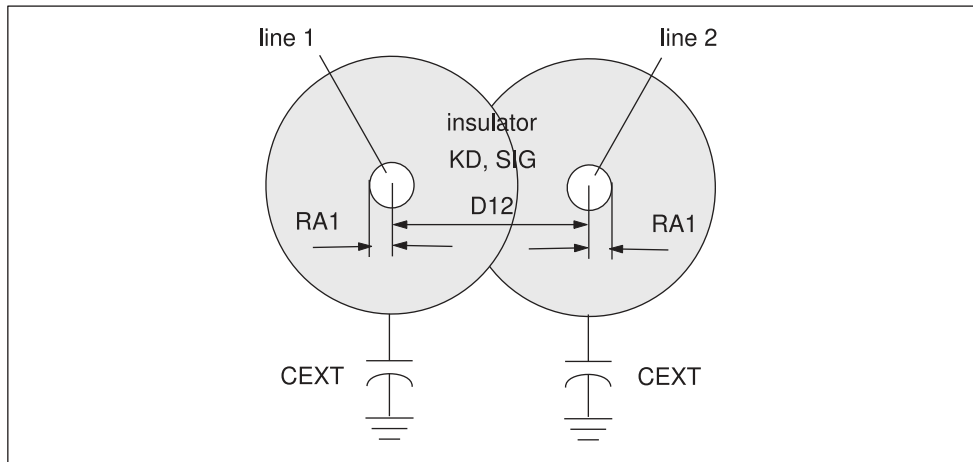


Figure 49 Geometric Twinlead, DLEV=1 with Insulating Spacer

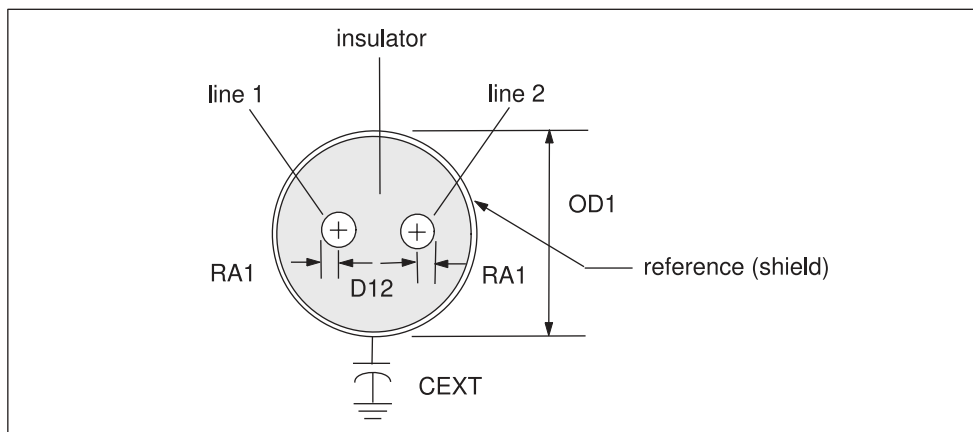


Figure 50 Geometric Twinlead, DLEV=2, Shielded

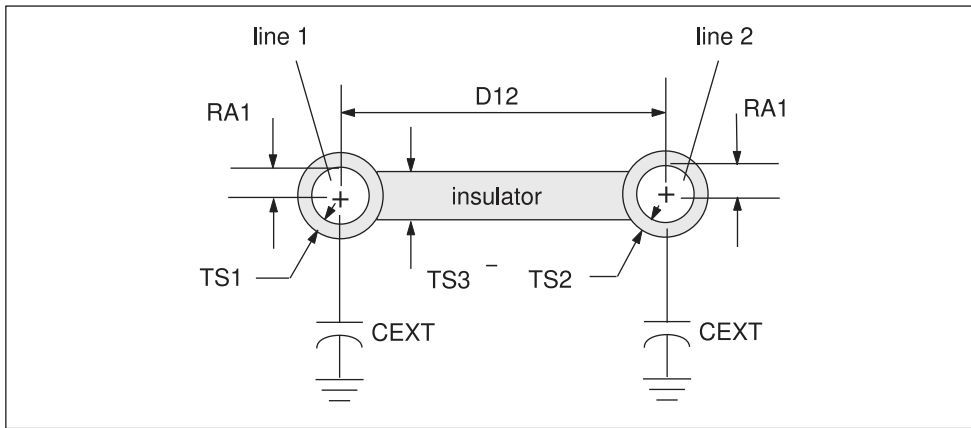


Table 26 Geometric Twinlead Parameters (ELEV=1)

Name (Alias)	Units	Default	Description
DLEV	—	0.0	0: embedded twinlead 1: spacer twinlead 2: shielded twinlead
RA1	m	req.	Outer radius of each conductor
D12	m	req.	Distance between the conductor centers
RHO	ohm·m	17E-9	Resistivity of first conductor material – defaults to value for copper
KD		4.0	Relative dielectric constant of dielectric
SIG	mho/m	0.0	Conductivity of dielectric
HGP	m	d12	Distance to reference plane
CMULT	—	1.0	Multiplier {used in defining CPR} for dielectric constant of material between reference plane and SPICE ground when LLEV=1 and CEXT is not present.
CEXT	F/m	undef.	External capacitance from reference plane to SPICE ground point (overrides LRR when present)
TS1	m	req.	Insulation thickness on first conductor
TS2	m	TS1	Insulation thickness on second conductor
TS3	m	TS1	Insulation thickness of spacer between conductor

The following parameters apply to shielded twinlead:

RHOB	ohm·m	rho	Resistivity of shield material (if present)
OD1	m	req.	Maximum outer dimension of shield

Table 26 Geometric Twinlead Parameters (ELEV=1) (Continued)

Name (Alias)	Units	Default	Description
SHTHK	m	2.54E-4	Twinlead shield conductor thickness

The following sections discuss these topics:

- [Precomputed Model Parameters \(ELEV=2\)](#)
- [Conductor Width Relative to Reference Plane Width](#)
- [Alternative Multiconductor Capacitance/Conductance Definitions](#)
- [Measured Parameters \(ELEV=3\)](#)

Precomputed Model Parameters (ELEV=2)

Precomputed parameters allow the specification of up to five signal conductors and a reference conductor. These parameters may be extracted from a field solver, laboratory experiments, or packaging specifications supplied by vendors. The parameters supplied include:

- Capacitance/length. Each conductor has a capacitance to all other conductors.
- Conductance/length. Each conductor has a conductance to all other conductors due to dielectric leakage.
- Inductance/length. Each conductor has a self inductance and mutual inductances to all other conductors in the transmission line.
- Resistance/length. Each conductor has two resistances, high frequency resistance due to skin effect and bent wires and DC core resistance.

[Figure 51](#) identifies the precomputed components for a three-conductor line with a reference plane. The names for the resistance, capacitance, and conductance components for up to five lines are shown in [Figure 52](#).

Figure 51 Precomputed Components for 3 Conductors and a Reference Plane

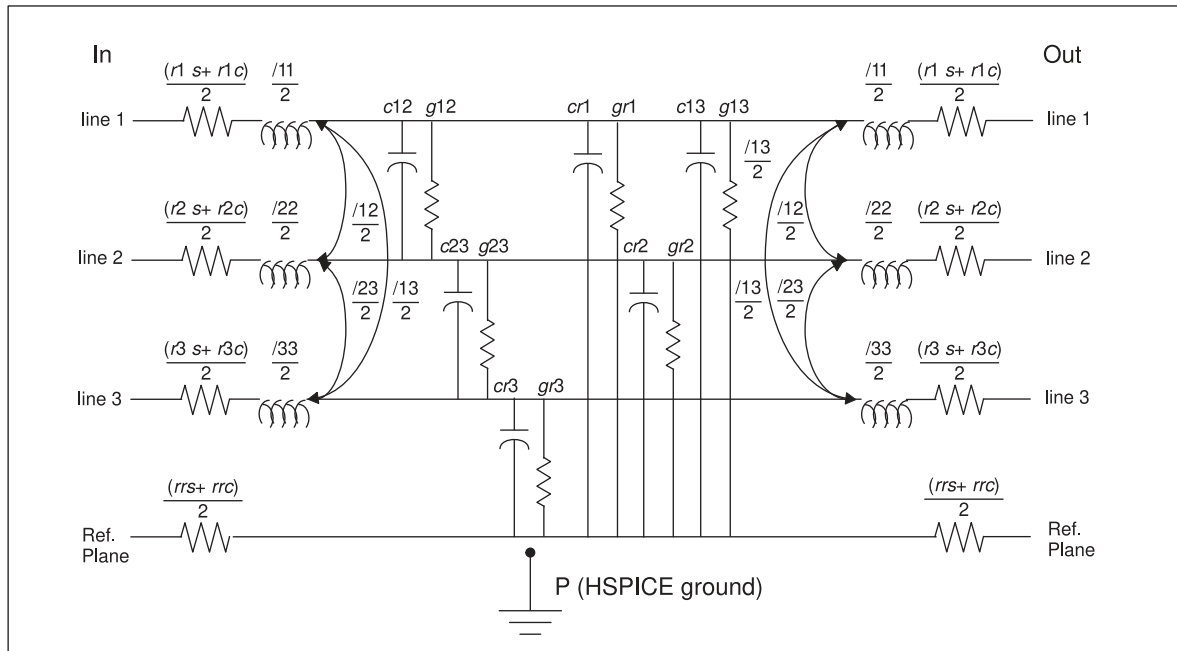


Figure 52 *ELEV=2 Model Keywords for Conductor PLEV=1*

	Ref. plane	line 1	line 2	line 3	line 4	line 5	
HSPICE ground	CPR GPR	CP1 GP1	CP2 GP2	CP3 GP3	CP4 GP4	CP5 GP5	
Ref. plane		CR1 GR1	CR2 GR2	CR3 GR3	CR4 GR4	CR5 GR5	
	LRR	LR1	LR2	LR3	LR4	LR5	
line 1		R11 L11	C12 G12 L12	C13 G13 L13	C14 G14 L14	C15 G15 L15	LLEV=1 parameter only
line 2			R22 L22	C23 G23 L23	C24 G24 L24	C25 G25 L25	
line 3				R33 L33	C34 G34 L34	C35 G35 L35	
line 4					R44 L44	C45 G45 L45	
line 5						R55 L55	

All precomputed parameters default to zero except `CEXT`, which is not used unless it is defined. The units are standard MKS in every case, namely:

Table 27 *Precomputed Model
Parameters*

Parameter	Units
capacitance	F/m
inductance	H/m
conductance	mho/m
resistance	ohm/m

The following are four additional parameters, `LLEV` (which defaults to 0), `CEXT`, and `GPR`:

- `LLEV=0` – reference plane conductor is resistive only (default).
- `LLEV=1` – reference plane inductance is included as well as common mode inductance and capacitance to SPICE ground for all conductors.
- `CEXT` – external capacitance from the reference plane to SPICE ground. When `CEXT` is specified, it overrides `CPR`.
- `GPR` – conductance to circuit ground; is zero except for immersion in a conductive medium.

Conductor Width Relative to Reference Plane Width

For the precomputed lossy U model (`ELEV=2`), the conductor width must be smaller than the reference plane width, which makes the conductor inductance smaller than the reference plane inductance. If the reference plane inductance is greater than the conductor inductance, simulation reports an error.

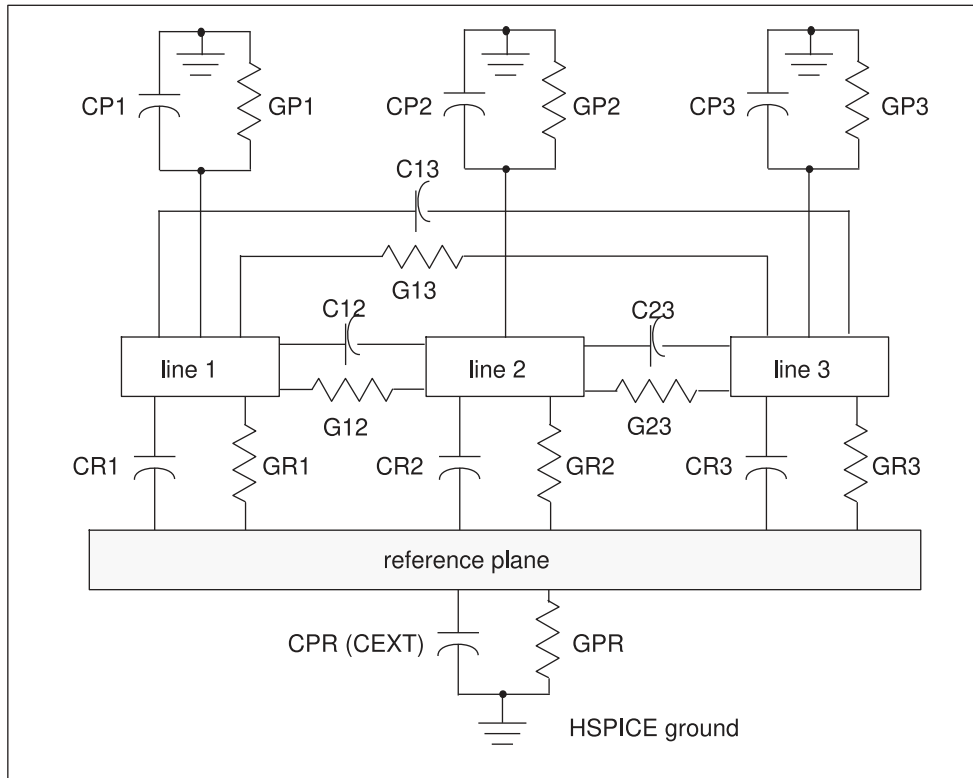
Alternative Multiconductor Capacitance/Conductance Definitions

Three different definitions of capacitances and conductances between multiple conductors are currently used. In this manual, relationships are written explicitly only for various capacitance formulations, but they apply equally well to corresponding conductance quantities, which are electrically in parallel with the capacitances. The symbols used in this section, and where you are likely to encounter these usages, are:

- `CXY`: branch capacitances, input, and circuit models.
- `Cjk`: Maxwell matrices for capacitance, multiple capacitor stamp for MNA (modified nodal admittance) matrix, which is a SPICE and the PrimeSim Continuum solution internal. Also the output of some field solvers.
- `CX`: capacitance with all conductors except X grounded. The output of some test equipment.
- `GXY`, `Gjk`, `GX`: conductances corresponding to above capacitances

The following example uses a multiple conductor capacitance model, a typical U model transmission line. The U-element supports up to five signal conductors plus a reference plane, but the three conductor case, [Figure 53](#), demonstrates the three definitions of capacitance. The branch capacitances are specified in the PrimeSim Continuum solution notation.

Figure 53 Single-Lump Circuit Capacitance



The branch and Maxwell matrices are completely derivable from each other. The “O.C.G.” (“other conductors grounded”) matrix is derivable from either the Maxwell matrix or the branch matrix. Thus:

Equation 46

$$CX = \sum CXY$$

$$X \neq Y$$

$C_{jk} = CX$ on diagonal

$= -CXY$ off diagonal

The matrices for the example given above provide the following “O.C.G.” capacitances:

$$C1 = CR1 + C12 + C13$$

$$C2 = CR2 + C12 + C23$$

$$C3 = CR3 + C13 + C23$$

$$CR = CR1 + CR2 + CR3 + CPR$$

Also, the Maxwell matrix is specified as:

Equation 47

$$C_{jk} = \begin{bmatrix} CR & -CR1 & -CR2 & -CR3 \\ -CR1 & C1 & -C12 & -C13 \\ -CR2 & -C12 & C2 & -C23 \\ -CR3 & -C13 & -C23 & C3 \end{bmatrix}$$

The branch capacitances also may be obtained from the Maxwell matrices. The off-diagonal terms are the negative of the corresponding Maxwell matrix component. The branch matrix terms for capacitance to circuit ground are the sum of all the terms in the full column of the maxwell matrix with signs intact:

$$\begin{aligned} CPR &= \text{sum } (C_{jk}), \quad j=R, \quad k=R:3 \\ CP1 &= \text{sum } (C_{1k}), \quad j=1, \quad k=R:3 \\ CP2 &= \text{sum } (C_{2k}), \quad j=2, \quad k=R:3 \\ CP3 &= \text{sum } (C_{3k}), \quad j=3, \quad k=R:3 \end{aligned}$$

CP1, ... CP5 are not computed internally with the geometric (`ELEV=1`) option, although CPR is. This, and the internally computed inductances, are consistent with an implicit assumption that the signal conductors are completely shielded by the reference plane conductor. This is true, to a high degree of accuracy for stripline, coaxial cable, and shielded twinlead, and to a fair degree for MICROSTRIP. If accurate values of CP1 and so forth are available from a field solver, they can be used with `ELEV=2` type input.

If the currents from each of the other conductors can be measured separately, then all of the terms in the Maxwell matrix may be obtained by laboratory experiment. By setting all voltages except that on the first signal conductor equal to 0, for instance, you can obtain all of the Maxwell matrix terms in column 1.

Equation 48

$$\begin{bmatrix} IR \\ I1 \\ I2 \\ I3 \end{bmatrix} = \begin{bmatrix} CR & -CR1 & -CR2 & -CR3 \\ -CR1 & C1 & -C12 & -C13 \\ -CR2 & -C12 & C2 & -C23 \\ -CR3 & -C13 & -C23 & C3 \end{bmatrix} \cdot j\omega \cdot \begin{bmatrix} 0.0 \\ 1.0 \\ 0.0 \\ 0.0 \end{bmatrix} = j\omega \cdot \begin{bmatrix} -CR1 \\ C1 \\ -C12 \\ -C13 \end{bmatrix}$$

The advantage of using branch capacitances for input derives is that only one side of the off-diagonal matrix terms are input. This makes the input less tedious and provides fewer opportunities for error.

Measured Parameters (ELEV=3)

When measured parameters are specified in the input, the program calculates the resistance, capacitance, and inductance parameters using TEM transmission line theory with the `LLEV=0` option. If redundant measured parameters are given, the program recognizes the situation, and discards those which are usually presumed to be less accurate. For twinlead models, `PLEV=3`, the common mode capacitance is one thousandth of that for differential-mode, which allows a reference plane to be used.

The `ELEV=3` model is limited to one conductor and reference plane for `PLEV=1`.

Table 28 *Measured Parameters*

Name (Alias)	Units	Default	Description
PLEV			1: planar 2: coax 3: twinlead
ZK	ohm	calc	Characteristic impedance
VREL	—	calc	Relative velocity of propagation ($d_{\text{elen}} / (\text{delay} \cdot c_{\text{light}})$)
DELAY	sec	calc	Delay for length d_{elen}
CAPL		1.0	Linear capacitance in length d_{len}
AT1		1.0	Attenuation factor in length d_{len} . Use dB scale factor when specifying attenuation in dB.
DELEN	m	1.0	Unit of length for delay (for example, ft.)
CLEN	m	1.0	Unit of length for capacitance
ATLEN	m	1.0	Unit of length for attenuation
FR1	Hz	req.	Frequency at which AT1 is valid. Resistance is constant below FR1, and increases as (frequency) above FR1.

Parameter Combinations

You can use several combinations of measured parameters to compute the L and C values used internally. The full parameter set is redundant. If you input a redundant parameter set, the program discards those that are presumed to be less accurate. [Table 30](#) shows how each of seven possible parameter combinations are reduced, if need be, to a unique set and then used to compute C and L.

Three different delays are used in discussing transmission lines:

Table 29 *Delays Used with Transmission Lines*

Delay	Description
DELAY	U Model input parameter that is the delay required to propagate a distance “ d_{len} ”
TD	T-element input parameter signifying the delay required to propagate one meter
TDeff	Internal variable, which is the delay required to propagate the length of the transmission line T-element or U-element.

Table 30 Lossless Parameter Combinations

Input Parameters	Basis of Computation
ZK, DELAY, DELEN, CAPL, CLEN	Redundant. Discard CAPL and CLEN.
ZK, VREL, CAPL, CLEN	Redundant. Discard CAPL and CLEN.
ZK, DELAY, DELEN	$VREL = DELEN / (DELAY \cdot CLIGHT)$
ZK and VREL	$C = 1 / (ZK \cdot VREL \cdot CLIGHT)$ $L = ZK / (VREL \cdot CLIGHT)$
ZK, CAPL, CLEN	$C = CAPL / CLEN$ $L = C \cdot ZK^2$
CAPL, CLEN, DELAY, DELEN	$VREL = DELEN / (DELAY \cdot CLIGHT)$
CAPL, CLEN, VREL	$LC = CAPL / CLEN$ $LL = 1 / (C \cdot VREL^2 \cdot CLIGHT^2)$

Loss Factor Input

You can specify the attenuation per unit length as either an attenuation factor or a decibel attenuation. Because the data might be available either as input/output or output/input, decibels greater than 0, or factors greater than 1 are assumed to be input/output. The following example shows the four ways that one may specify that an input of 1.0 is attenuated to an output of 0.758.

Table 31 Input Attenuation Variations

AT1 Input	Computation of attenuation factor and linear resistance
AT1 = -2.4dB	$v(out)/v(in) = 0.758 = 10^{(+AT1/20)}$ (for dB < 0)
AT1 = +2.4dB	$v(out)/v(in) = 0.758 = 10^{(-AT1/20)}$ (for dB > 0)
AT1 = 1.318	$v(out)/v(in) = 0.758 = 1/AT1$ (for AT1 < 1)
AT1 = 0.758	$v(out)/v(in) = 0.758 = AT1$ (for AT1 > 1)

The attenuation factor is used to compute the exponential loss parameter and linear resistance.

Equation 49

$$\alpha = \frac{\ln((v(in)) / (v(out)))}{ATlin}$$

Equation 50

$$LR = 2 \cdot \alpha \cdot \sqrt{(LL) / (LC)}$$

U-element Examples, Models, and Applications

The following examples show the results of simulating a stripline geometry using the U Model in a PCB scale application and in an IC scale application.

The following sections discuss these topics:

- [Three Coupled Lines, Stripline Configuration](#)
- [Three Coupled Lines, Sea of Dielectric Configuration](#)
- [Simulation Output](#)
- [IcWire Output Section](#)
- [Capacitance and Inductance Matrices](#)
- [Five Coupled Lines, Stripline Configuration](#)
- [U Model Applications](#)
- [Solving Ringing Problems with U-elements](#)

Three Coupled Lines, Stripline Configuration

[Figure 54](#) shows three coupled lines in a stripline configuration on an FR4 printed circuit board. A simple circuit using three coupled striplines is shown in [Figure 55](#).

This example is based on demonstration netlist *stripline.sp*, which is available in directory `$<installdir>/demo/hspice/tline`:

Figure 54 *Three Coupled Striplines (PCB Scale)*

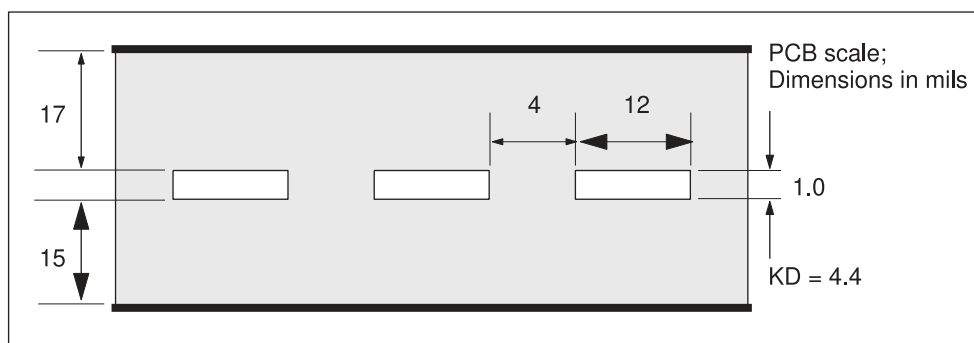
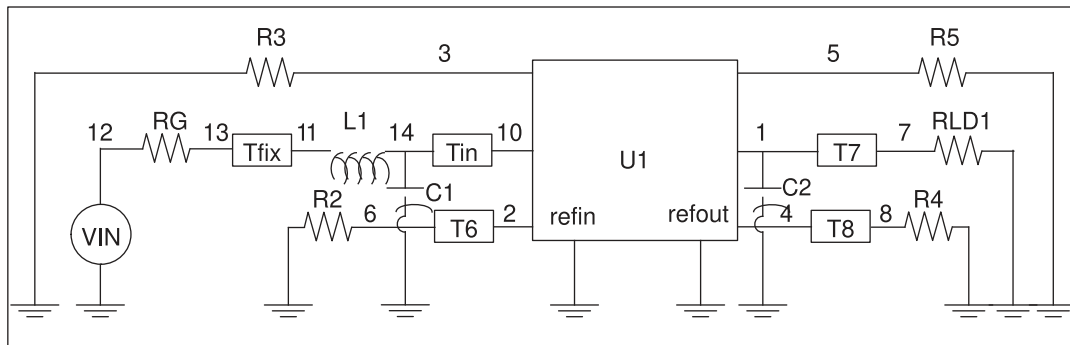


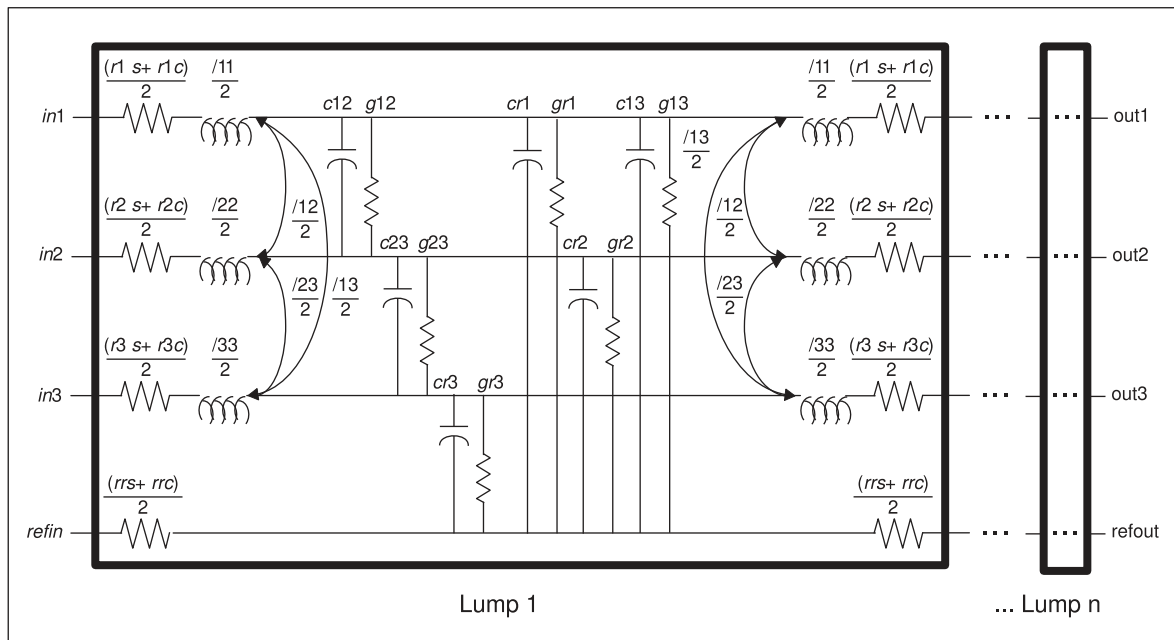
Figure 55 Schematic Using the Three Coupled Striplines U Model



```
* Stripline circuit
.Tran 50ps 7.5ns
.OPTION Post NoMod Accurate Probe Method=Gear
VIN 12 0 PWL 0 0v 250ps 0v 350ps 2v
L1 14 11 2.5n
C1 14 0 2p
Tin 14 0 10 0 ZO=50 TD=0.17ns
Tfix 13 0 11 0 ZO=45 TD=500ps
RG 12 13 50
RLD1 7 0 50
C2 1 0 2p
U1 3 10 2 0 5 1 4 0 USTRIP L=0.178
T6 2 0 6 0 ZO=50 TD=0.17ns
T7 1 0 7 0 ZO=50 TD=0.17ns
T8 4 0 8 0 ZO=50 TD=0.17ns
R2 6 0 50
R3 3 0 50
R4 8 0 50
R5 5 0 50
.Model USTRIP U LEVEL=3 PLev=1 Elev=1 Dlev=2 Nl=3 Ht=381u
+ Wd=305u Th=25u Sp=102u Ts=838u Kd=4.7
.Probe v(13) v(7) v(8) v(6)
.End
```

Figure 56, Figure 57, and Figure 58 show the main line and crosstalk responses. The rise time and delay of the waveform are sensitive to the skin effect frequency, since losses reduce the slope of the signal rise. The main line response shows some differences between simulation and measurement. The rise time differences are due to layout parasitics and the fixed resistance model of skin effect. The differences between measured and simulated delays are due to errors in the estimation of dielectric constant and the probe position.

Figure 56 Measured versus Computed Through-Line Response



The gradual rise in response between 3 ns and 4 ns is due to skin effect. During this period, the electric field driving the current penetrates farther into the conductor so that the current flow increases slightly and gradually. This affects the measured response as shown for the period between 3 ns and 4 ns.

Figure 57 shows the backward crosstalk response. The amplitude and delay of this backward crosstalk are very close to the measured values. The risetime differences are due to approximating the skin effect with a fixed resistor, while the peak level difference is due to errors in the LC matrix solution for the coupled lines.

Figure 57 Measured Versus Computed Backward Crosstalk Response

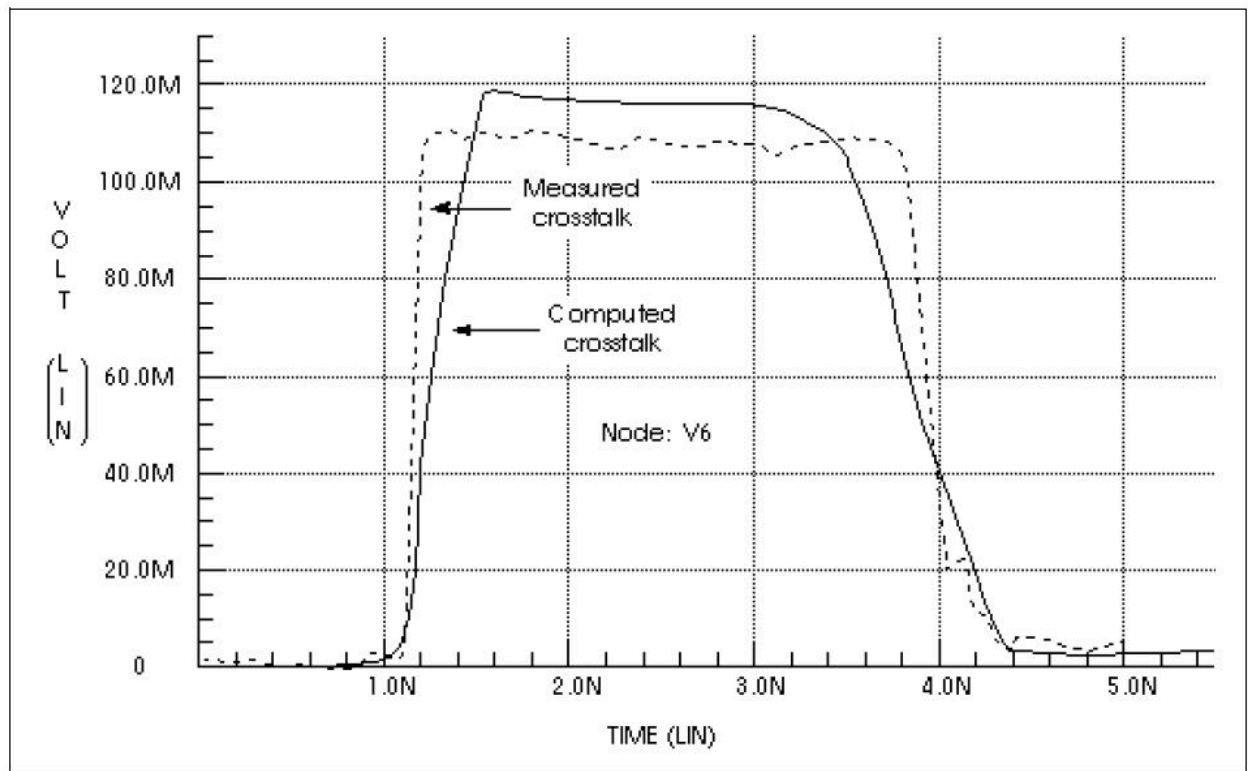
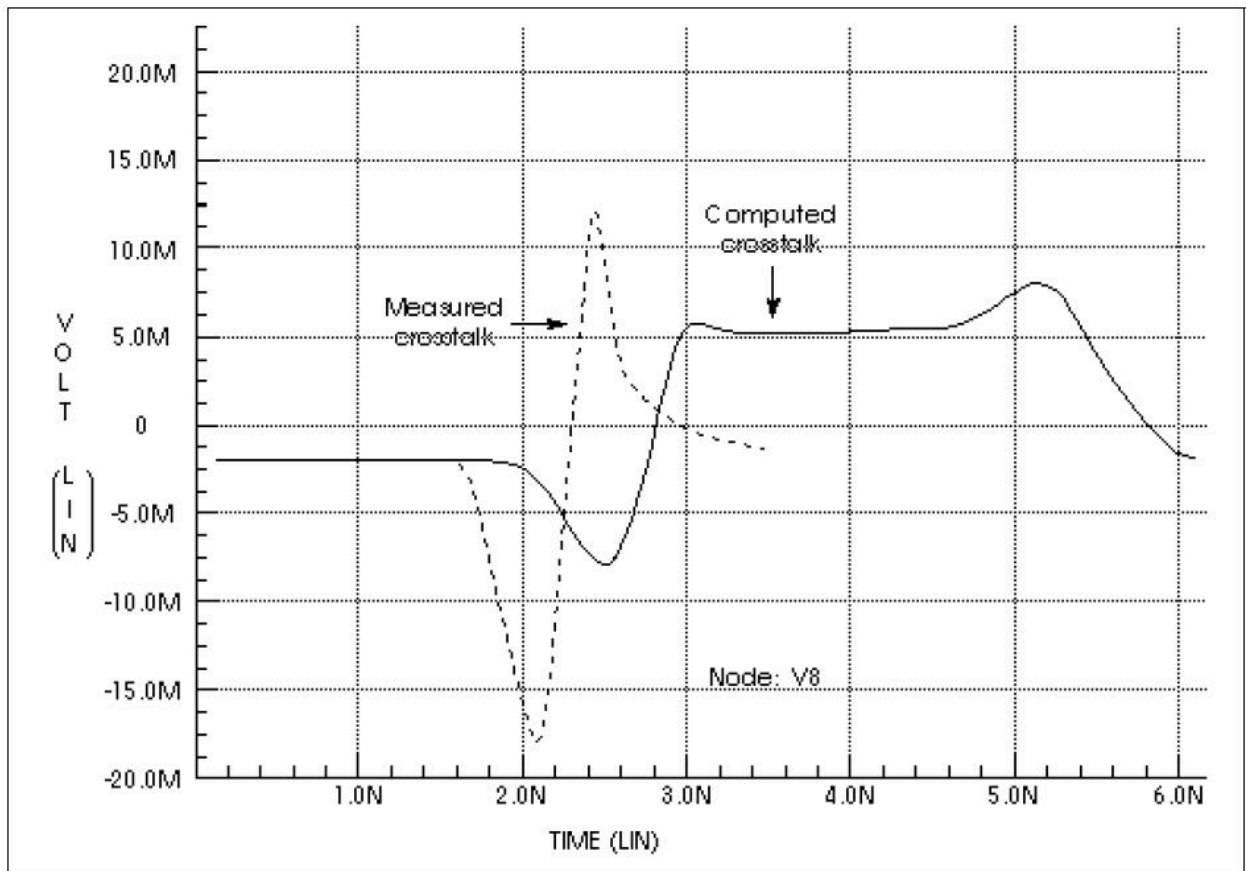


Figure 58 shows the forward crosstalk response. This forward crosstalk shows almost complete signal cancellation in both measurement and simulation. The forward crosstalk levels are about one tenth the backward crosstalk levels. The onset of ringing of the forward crosstalk has reasonable agreement between simulation and measurement. However, the trailing edge of the measured and simulated responses differ. The measured response trails off to zero after about 3 ns, while the simulated response does not trail down to zero until 6 ns. Errors in simulation at this voltage level can easily be due to board layout parasitics that have not been included in the simulation.

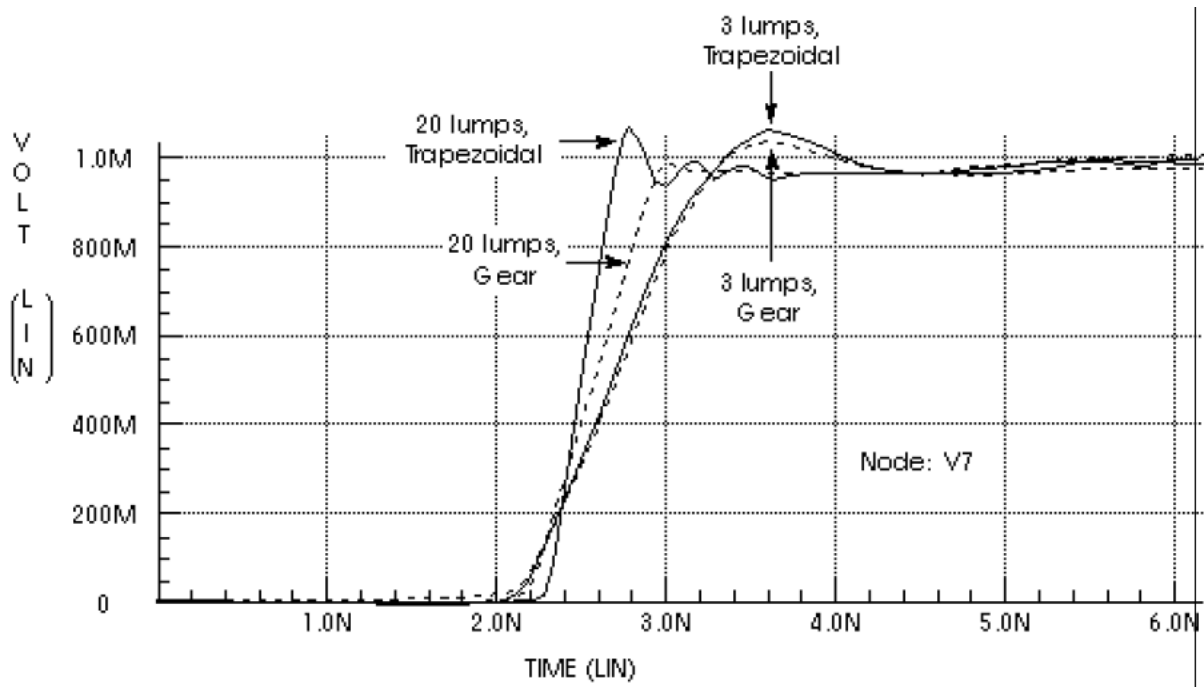
Figure 58 Measured Versus Computed Forward Crosstalk Response



Simulation methods can have a significant effect on the predicted waveforms. [Figure 59](#) shows the main line response at Node 7 of [Figure 55](#) as the integration method and the number of lumped elements change. With the recommended number of lumps, 20, the Trapezoidal integration method shows a fast risetime with ringing.

The Gear integration method shows a fast risetime and a well damped response. When the number of lumped elements changes to 3, both Trapezoidal and Gear methods show a slow risetime with ringing. In this situation, the Gear method with 20 lumps gives the more accurate simulation.

Figure 59 Computed Responses for 20 Lumps and 3 Lumps, Gear and Trapezoidal Integration Methods



Three Coupled Lines, Sea of Dielectric Configuration

This example shows the U-element analytic equations for a typical integrated circuit transmission line application. Three 200 μ m-long aluminum wires in a silicon dioxide dielectric are simulated to examine the through-line and coupled line response.

The U model uses the transmission line geometric parameters to generate a multisection lumped-parameter transmission line model. Use a single U-element statement to create an internal network of three 20-lump circuits.

[Figure 60](#) shows the IC-scale coupled line geometry.

Figure 60 Three Coupled Lines with One Reference Plane in a Sea of Dielectric (IC Scale)

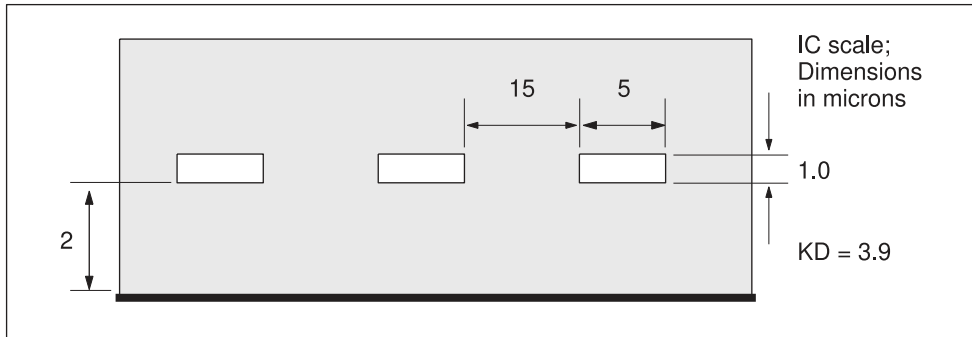


Figure 61 shows one lump of the lumped-parameter schematic for the three-conductor stripline configuration of Figure 60. This internal circuitry represents one U-element instantiation. Internal elements are described in [Simulation Output on page 322](#).

Figure 61 Schematic for Three Coupled Lines with One Reference Plane

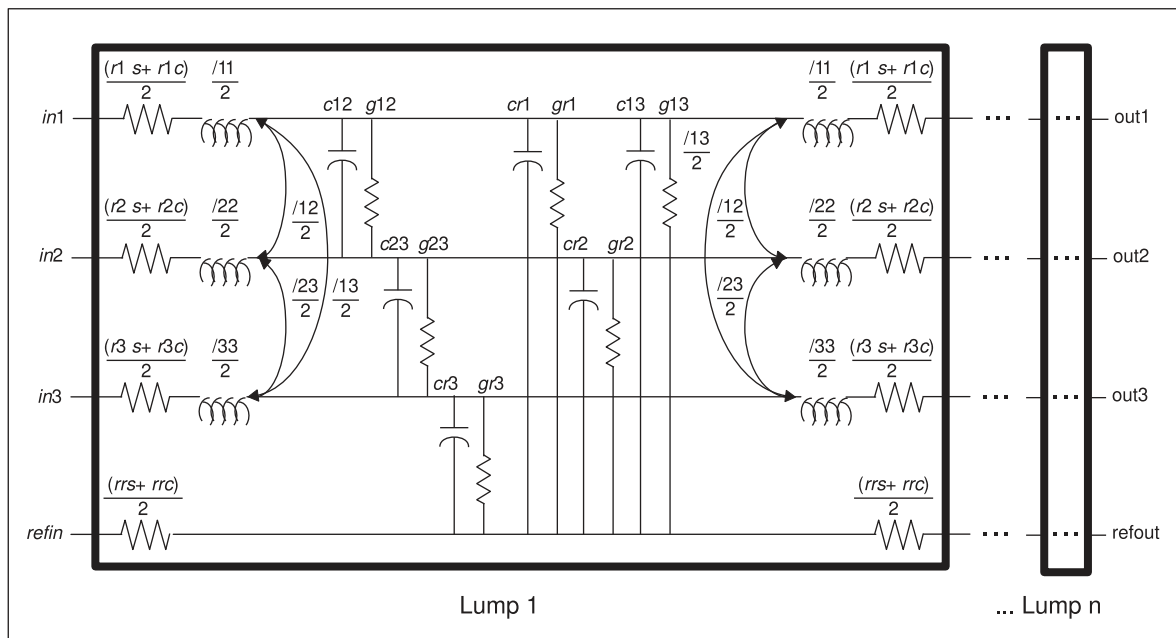
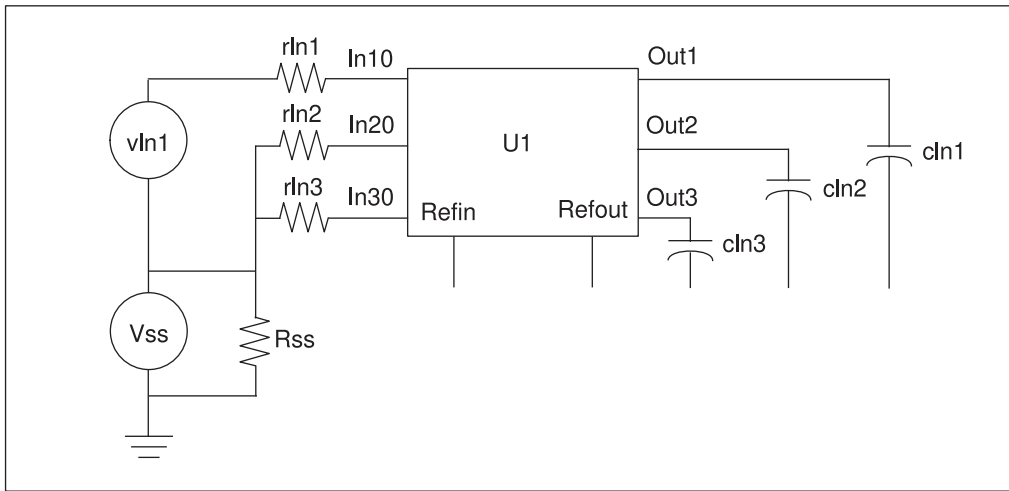


Figure 62 shows a schematic using the U-element of Figure 61. In this simple circuit, a pulse drives a three-conductor transmission line source terminated by 50 resistors and loaded by 1pF capacitors.

Figure 62 Schematic Using the Three Coupled Lines U Model



The input file for the U-element solution is shown below.

This example is based on demonstration netlist uele.sp, which is available in directory `$install_dir/demo/hspice/tline`:

```
*The input file for the U Element solution
.Tran 0.1ns 20ns
.OPTION Post Accurate NoMod Brief Probe
Vss Vss 0 0v
Rss Vss 0 1x
vIn1 In1 Vss Pwl 0ns 0v 11ns 0v 12ns 5v 15ns 5v 16ns 0v
rIn1 In1 In10 50
rIn2 Vss In20 50
rIn3 Vss In30 50
u1 In10 In20 In30 Vss Out1 Out2 Out3 Vss IcWire L=200um
cIn1 Out1 Vss 1pF
cIn2 Out2 Vss 1pF
cIn3 Out3 Vss 1pF
.Probe v(Out1) v(Out2) v(Out3)
.Model IcWire U LEVEL=3 Dlev=0 Nl=3 Nlay=2 Plev=1 Elev=1
+ Llev=0 Ht=2u Wd=5u Sp=15u Th=1u Rho=2.8e-8 Kd=3.9
.End
```

The U-element uses the conductor geometry to create length-independent RLC matrices for a set of transmission lines. You can then input any length, and simulation computes the number of circuit lumps that are required.

Figure 63, Figure 64, and Figure 65 show the through and coupled responses, computed using U-element equations.

Figure 63 Computed Through-Line Response

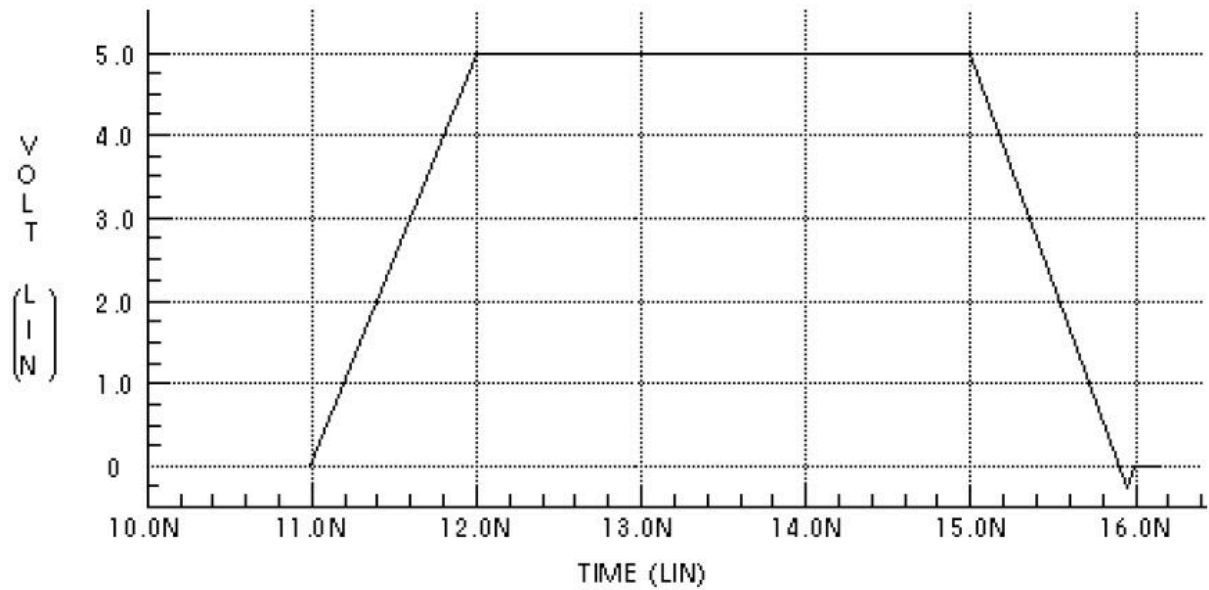


Figure 64 shows the nearest coupled line response. This response occurs only during signal transitions.

Figure 64 Computed Nearest Coupled-Line Response

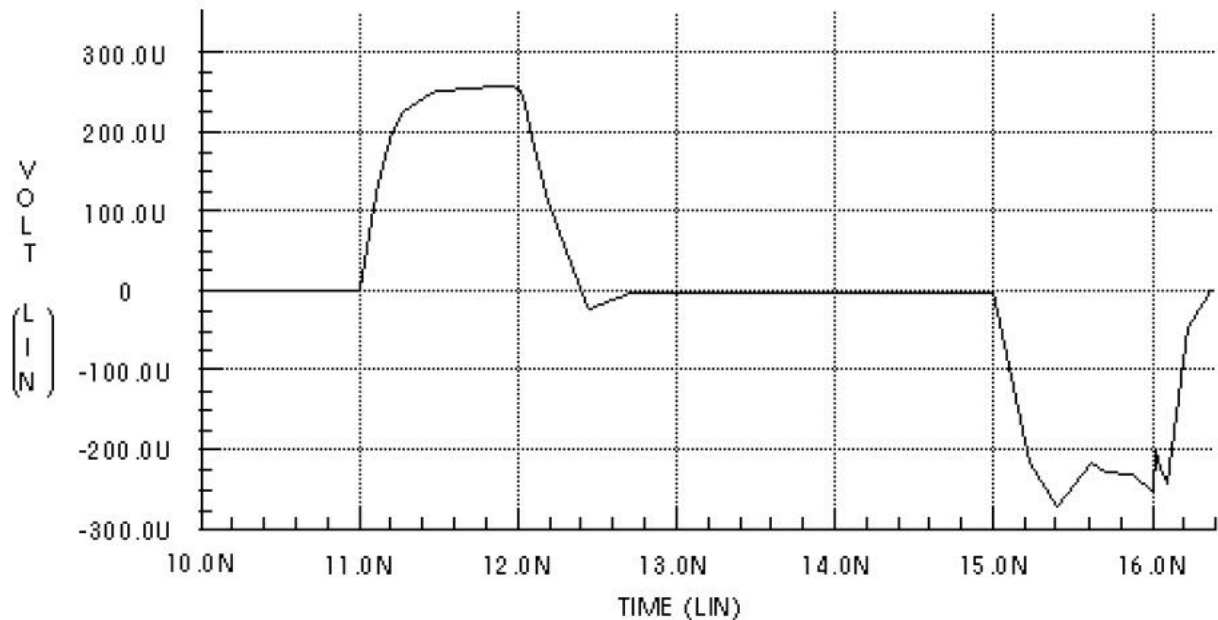
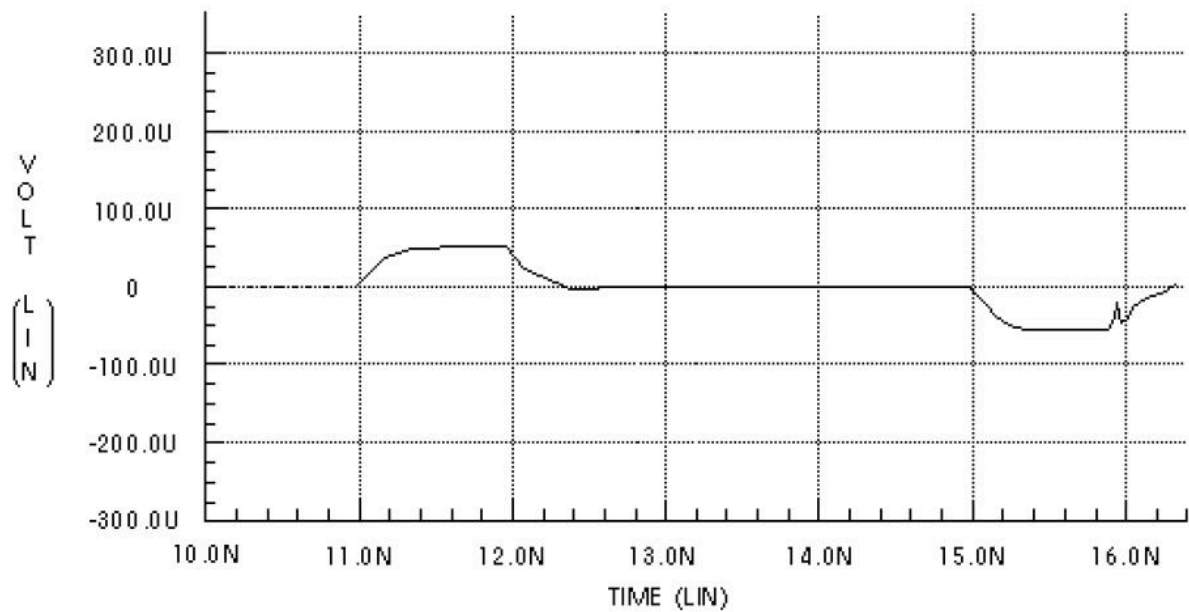


Figure 65 shows the third coupled line response. The predicted response is about 1/100000 of the main line response.

Figure 65 Computed Furthest Coupled-Line Response



By default, simulation prints the model values, including the LCRG matrices for the U-element. All of the printed LCRG parameters are identified in the following section.

Simulation Output

The listing below is part of the output from a simulation using the PrimeSim Continuum solution input deck for the IcWire U model. Descriptions of the parameters specific to U-elements follows the listing. (Parameters not listed in this section are described in [Table 21 on page 292](#) and [Table 22 on page 294](#).)

IcWire Output Section

```
*** model name: 0:icwire ***

names      values      units      names      values      units      names      values      units
--- u-model control parameters ---
maxl=      20.00      #lumps    wlump=     20.00            elev=      1.00
plev=      1.00            llev=      0.            nlay=     2.00      #layrs
nl=        3.00      #lines    nb=        1.00      #refpl
--- begin type specific parameters ---
```

Appendix A: Lumped Transmission Line — U-Model U-element Examples, Models, and Applications

dlev=	0.	#	kd=	3.90		corkd=	1.00	
sig=	0.	mho/m	ho=	28.00	n ohm*m	rhob=	28.00	n ohm*m
th=	1.00u	meter	thb=	1.42u	meter	skin=	10.31u	meter
skinb=	10.31u	meter	wd2	5.00u	meter	ht2	2.00u	meter
th2	1.00u	meter	sp1	15.00u	meter	wd3	5.00u	meter
ht3	2.00u	meter	th3	1.00u	meter	sp2	15.00u	meter
cr1=	170.28p	f/m	gr1=	0.	mho/m	l11=	246.48n	h/m
cr2=	168.25p	f/m	gr2=	0.	ee	c12=	5.02p	f/m
g12=	0.	mho/m	l12=	6.97n	h/m	l22=	243.44n	h/m
cr3=	170.28p	f/m	gr3=	0.	mho/m	c13=	649.53f	f/m
g13=	0.	mho/m	l13=	1.11n	h/m	c23=	5.02p	f/m
g23=	0.	mho/m	l23=	6.97n	h/m	l33=	246.48n	h/m
--- two layer (skin and core) parameters ---								
rrs=	1.12k	ohm/m	rrc=	0.	ohm/m	r1s=	5.60k	ohm/m
rlc=	0.	ohm/m	r2s=	5.60k	ohm/m	r2c=	0.	ohm/m
r3s=	5.60k	ohm/m	r3c=	0.	ohm/m			

Table 32 *U-element Parameters*

Parameter	Description
cij	Coupling capacitance from conductor i to conductor j (positive)
crj	Self-capacitance/m of conductor j to the reference plane
cpr	Capacitance of the reference plane to the PrimeSim Continuum solution ground
gij	Conductance/m from conductor i to conductor j (zero if sig=0)
grj	Conductance/m from conductor j to the reference plane (0 if sig=0)
gpr	Conductance from the reference plane to the PrimeSim Continuum solution ground, always=0
hti	Conductor i height above reference plane (only ht is input; all heights same)
lri	Inductance/m from conductor i to the reference plane
lrr	Inductance/m of the reference plane
lij	Inductance/m from conductor i to conductor j

Table 32 U-element Parameters (Continued)

Parameter	Description
ljj	Self-inductance/m of conductor j
rrc	Reference plane core resistance/m (if NLAY = 2, zero if skin depth > 90% of thb)
rrr	Resistance/m of the reference plane (if NLAY = 1)
rrs	Skin resistance/m of the reference plane (if NLAY = 2)
ris	Skin resistance/m of conductor i (if NLAY = 2)
ric	Conductor i core resistance/m (if NLAY = 2, zero if skin depth > 50% of th)
rjj	Resistance/m of conductor j (if NLAY = 1)
skin	Skin depth
skinb	Skin depth of the reference plane
spi	Spacing between conductor i and conductor i + 1 (only sp is input; same spacings)
thi	Thickness of conductor i (only th is input; all thicknesses are the same)
wdi	Width of conductor i (only wd is input—all widths are the same)

The total conductor resistance is indicated by rjj when NLAY=1, or by ris + ric when NLAY=2.

As shown in the next section, some difference between the PrimeSim Continuum solution and field solver results is to be expected. Within the range of validity shown in [Table 23 on page 295](#) for the U model, simulation comes very close to field solver accuracy. In fact, discrepancies between results from different field solvers can be as large as their discrepancies with simulation. The next section compares some Synopsys physical circuit models to models derived using field solvers.

Capacitance and Inductance Matrices

Simulation places capacitance and inductance values for U-elements in matrix form, for example:

Equation 51

$$\begin{bmatrix} C_{ii} & \dots & C_{ji} \\ \vdots & \ddots & \vdots \\ C_{ij} & \dots & C_{jj} \end{bmatrix}$$

[Table 33](#) shows the capacitance and inductance matrices for the three-line, buried microstrip IC-scale example shown in [Figure 60](#).

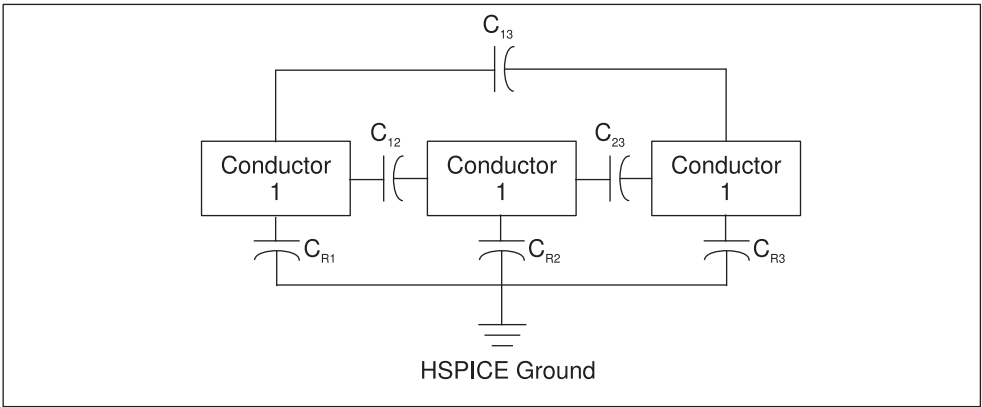
Table 33 *Capacitance and Inductance Matrices for the Three-Line, IC-Scale Interconnect System*

Capacitance (pF/m)	176 -5.02 -0.65 -5.02 178 -5.02 -0.65 -5.02 176
Inductance (nH/m)	246 6.97 1.11 6.97 243 6.97 1.11 6.97 246

The capacitance matrices in Table 33 are based on the admittance matrix of the capacitances between the conductors. The negative values in the capacitance matrix are due to the sign convention for admittance matrices. The inductance matrices are based on the impedance matrices of the self and mutual inductance of the conductors. Each matrix value is per meter of conductor length. The actual lumped values use a conductor length equal to the total line length divided by the number of lumps.

The capacitance matrix can be related directly to the output of the IC-scale example. Simulation uses the branch capacitance matrix for internal calculations. For the three-conductors in this example, Figure 66 shows the equivalent capacitances in terms of simulation device model parameters.

Figure 66 *Conductor Capacitances for IC-scale Example*



The capacitances of Figure 66 are those shown in Figure 61. The PrimeSim Continuum nodal capacitance matrix of Table 33 is shown below by using the capacitance terms that are listed in the PrimeSim Continuum output.

$$\begin{bmatrix} C_{R1} + C_{12} + C_{13} & -C_{12} & -C_{13} \\ -C_{12} & C_{R2} + C_{12} + C_{23} & -C_{23} \\ -C_{13} & -C_{23} & C_{R3} + C_{13} + C_{23} \end{bmatrix}$$

Off-diagonal terms are the negative of coupling capacitances (to conform to the sign convention).

The diagonal terms require some computation, for example:

$$\begin{aligned} C_{11} &= C_{R1} + C_{12} + C_{13} \\ &= 170.28 + 5.02 + 0.65 \\ &= 175.95, \text{ or } 176 \text{ pF/m} \end{aligned}$$

Note that the matrix values on the diagonal in [Table 33](#) are large: they indicate self-capacitance and inductance. The diagonal values show close agreement among the various solution methods. As the coupling values become small compared to the diagonal values, the various solution methods give very different results. Third-line coupling capacitances of 0.65 pF/m, 1.2 pF/m, and 0.88 pF/m are shown in [Table 33](#). Although the differences between these coupling capacitances seem large, they represent a negligible difference in waveforms because they account for only a very small amount of voltage coupling. [Table 33](#) represents very small coupling because the line spacing is large (about seven substrate heights).

[Table 34](#) shows the parameters for the stripline in [Figure 67](#).

Figure 67 Stripline Geometry Used in MCM Technology

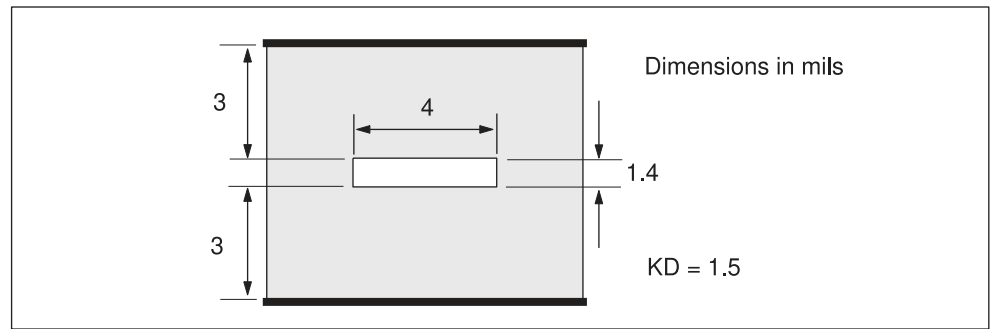


Table 34 Capacitance and Inductance
for the Single Line MCM-Scale Stripline

Capacitance (pF/m)	164.4
Inductance (nH/m)	236.5

Five Coupled Lines, Stripline Configuration

This example shows a five-line interconnect system in a PCB technology. [Table 35](#) shows the matrix parameters for the line configuration of [Figure 68](#).

Figure 68 Five Coupled Lines on a PCB

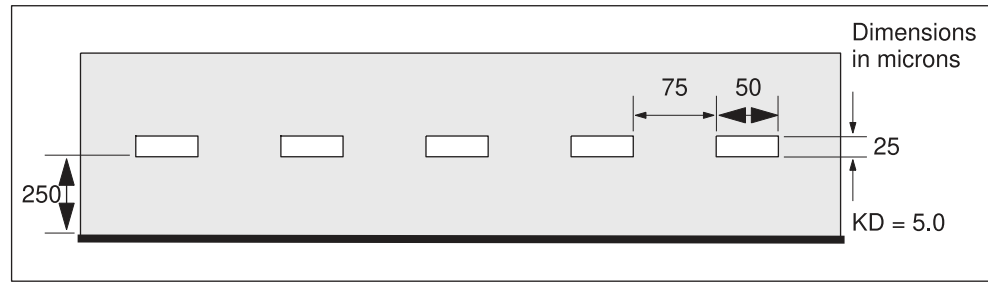


Table 35 Capacitance/Inductance for 5-Line PCB-Scale Interconnect System

Capacitance (pF/m)	59	-19	-2.5	-0.8	-0.4	-19	69	-18
	-2.2	-0.8	-2.5	-18	69	-18	-2.5	-0.8
	-2.2	-18	69	-19	-0.4	-0.8	-2.5	-19
	57							
Inductance (nH/m)	676	309	179	116	81	309	641	297
	116	179	297	637	297	179	116	174
	641	309	81	116	179	309	676	

U Model Applications

This section gives examples of use, and explains some of the aspects of ringing (impulse-initiated oscillation) in real and simulated transmission line circuits.

The following sections discuss these topics:

- [Data Entry Examples](#)
- [Printed Circuit Board Models](#)
- [Via Modeling for PCBs in the PrimeSim Continuum Solution](#)
- [Coax Models](#)
- [Twinlead Models](#)
- [Two Coupled Microstrips](#)

Data Entry Examples

Coax Geometry Entry (ELEV=1, PLEV=2) with ground reference (LLEV=1) and skin effect (NLAY=2):

```
uc in1 3 out1 4 wire2 l=1
.model wire2 u LEVEL=3 nlay=2 plev=2 elev=1 llev=1
+ ra=1m rb=7.22m hgp=20m rho=1.7e-8 kd=2.5
```

Matrix Entry (ELEV=2):

```
u1 In1 In2 In3 Vss Out1 Out2 Out3 Vss Wire3 L=0.01
.model Wire3 U LEVEL=3 NL=3 Elev=2 Llev=0
+ rrr=1.12k r11=5.6k r22=5.6k r33=5.6k c13=0.879pF
+ cr1=176.4pF cr2=172.6pF cr3=176.4pF c12=4.7pF c23=4.7pF
+ L11=237nH L22=237nH L33=237nH L12=5.52nH L23=5.52nH
+ L13=1.34nH
```

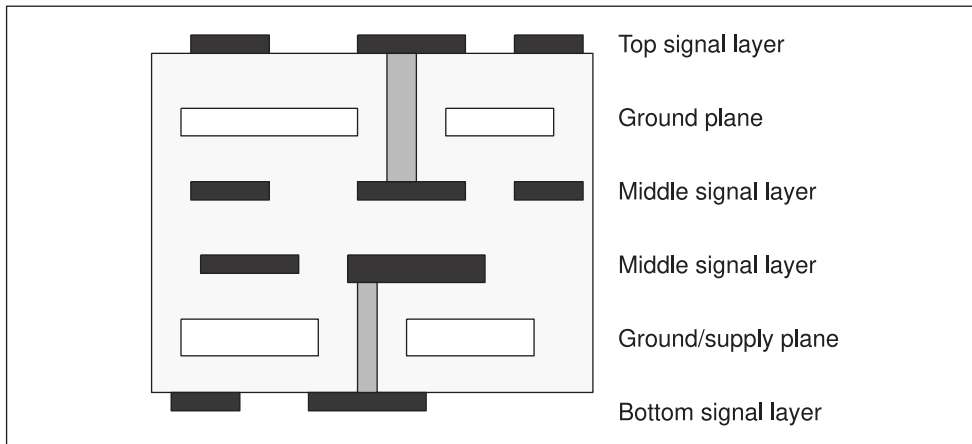
Coax Measured Data Entry (ELEV=3, PLEV=2):

```
u10 1 0 2 0 rg58 l=12
.model rg58 u LEVEL=3 plev=2 elev=3
+ zk=50 capl=30.8p clen=1ft vrel=0.66
+ fr1=100meg at1=5.3db atlen=100ft
```

Printed Circuit Board Models

Figure 69 illustrates a small cross section of a six-layer printed circuit board. The top and bottom signal layers require a microstrip U model (DLEV=1), while the middle signal layers use a stripline U model (DLEV=2).

Figure 69 Six-Layer Printed Circuit Board



Important aspects of such a circuit board are:

- Trace impedance is difficult to control because of etch variation
- 6 mil effective trace widths
- 8 mil drawn widths
- 10 mil insulator thickness
- 1 ounce copper 1.3 mil thick
- Microstrip model TOP used for top and bottom
- Stripline model MID used for middle signal layers

Example:

Top and bottom layer model:

```
.MODEL TOP U LEVEL=3 ELEV=1 PLEV=1 TH=1.3mil HT=10mil KD=4.5  
+ DLEV=1 WD=8mil XW=-2mil
```

Middle layer model:

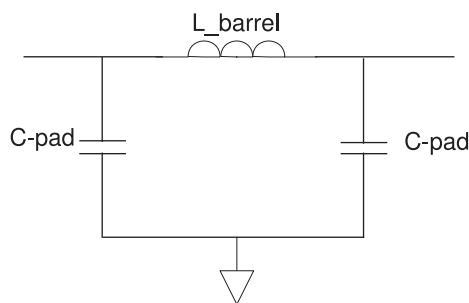
```
.MODEL MID U LEVEL=3 ELEV=1 PLEV=1 TH=1.3mil HT=10mil KD=4.5  
+ DLEV=2 WD=8mil XW=-2mil TS=32mil
```

Via Modeling for PCBs in the PrimeSim Continuum Solution

You can do modeling of vias for PCBs in pre-layout signal integrity phase of the PrimeSim Continuum. A 3D field solver is required to accurately model vias, you can use the built-in the 2D field solver to approximate a via model using the following method:

1. Create a SPICE netlist for a via. The via can be modeled as a lumped pi-model if the via delay is less than 0.1 of the signal edge.

Figure 70 Lumped pi-model



Where: L_{barrel} is the inductance of the barrel of the via and C_{pad} is the capacitance of the pad

2. Use the .LIN command to extract an S-parameter model file in Touchstone format.
3. Use the S-element to use the model file.

Other Considerations

If you are working with a PCB via, note that it can also be modeled as a Pi Lumped Element Model. The pi-model circuit approximates the behavior of short transmission lines (lumped RLC circuits). But the exact values for the RLC parameters of a via depend on the size and geometry of the via. To determine the exact values, you need to use a 3D field solver, such as the Synopsys Raphael product to extract exact parameter values.

Equivalent RLCs for GHz boards require a 3D EM analysis for accurate modeling, such as Raphael provides to include the magnetic flux in the conductor at DC or for modeling edge effects accurately. If Raphael can produce the RLGC table model, you can simulate with the model in the PrimeSim Continuum solution and then compare the result with field solvers to evaluate the difference.

Note that modeling IC vias is more complex as you need to consider stress effects (since process-induced mechanical stress can cause electromigration and formation of voids or cracks in vias or metal lines), thermal effects (since vias have much higher thermal conductivity than the dielectric materials (ILD), and modeling via plug resistance, etc.

Coax Models

The following examples are for standard coax. These are obtained from commonly available tables. (The parameter fr1 is the frequency at which a specific amount of attenuation, at1, occurs for a specified length of coax, atlen.) Synopsys simulators accept dB (decibel) and ft. (foot) units.

Example:

```
.model rg9/u      u      LEVEL=3      plev=2      elev=3
+                Zk=51      vrel=.66
+                fr1=100meg at1=2.1db  atlen=100ft
*
.model rg9b/u     u      LEVEL=3      plev=2      elev=3
+                Zk=50      vrel=.66
+                fr1=100meg at1=2.1db  atlen=100ft
*
.model rg11/u     u      LEVEL=3      plev=2      elev=3
+                Zk=75      vrel=.78
+                fr1=100meg at1=1.5db  atlen=100ft
*
.model rg11a/u    u      LEVEL=3      plev=2      elev=3
+                Zk=75      vrel=.66
+                fr1=100meg at1=1.9db  atlen=100ft
*
.model rg54a/u    u      LEVEL=3      plev=2      elev=3
+                Zk=58      vrel=.66
+                fr1=100meg at1=3.1db  atlen=100ft
```

```
*
.model rg15/u u LEVEL=3 plev=2 elev=3
+ Zk=53.5 vrel=.66
+ frl=100meg atl=4.1db atlen=100ft
*
.model rg53/u u LEVEL=3 plev=2 elev=3
+ Zk=53.5 vrel=.66
+ frl=100meg atl=4.1db atlen=100ft
*
.model rg58a/u u LEVEL=3 plev=2 elev=3
+ Zk=50 vrel=.66
+ frl=100meg atl=5.3db atlen=100ft
*
.model rg58c/u u LEVEL=3 plev=2 elev=3
+ Zk=50 vrel=.66
+ frl=100meg atl=5.3db atlen=100ft
*
.model rg59b/u u LEVEL=3 plev=2 elev=3
+ Zk=75 vrel=.66
+ frl=100meg atl=3.75db atlen=100ft
*
.model rg62/u u LEVEL=3 plev=2 elev=3
+ Zk=93 vrel=.84
+ frl=100meg atl=3.1db atlen=100ft
*
.model rg62b/u u LEVEL=3 plev=2 elev=3
```

Twinlead Models

Example:

```
.model tw/sh u LEVEL=3 plev=3 elev=3
* Shielded TV type twinlead
+ Zk=300 vrel=.698
+ frl=57meg atl=1.7db atlen=100ft
*
.model tw/un u LEVEL=3 plev=3 elev=3
* Unshielded TV type twinlead
+ Zk=300 vrel=.733
+ frl=100meg atl=1.4db atlen=100ft
```

Two Coupled Microstrips

[Figure 71](#) shows two metal lines formed of the first aluminum layer of a modern CMOS process. The microstrip model assumes that the metal strips sit on top of a dielectric layer that covers the reference plane.

This example is based on demonstration netlist *strip2.sp*, which is available in directory `$<installdir>/demo/hspice/tline`:

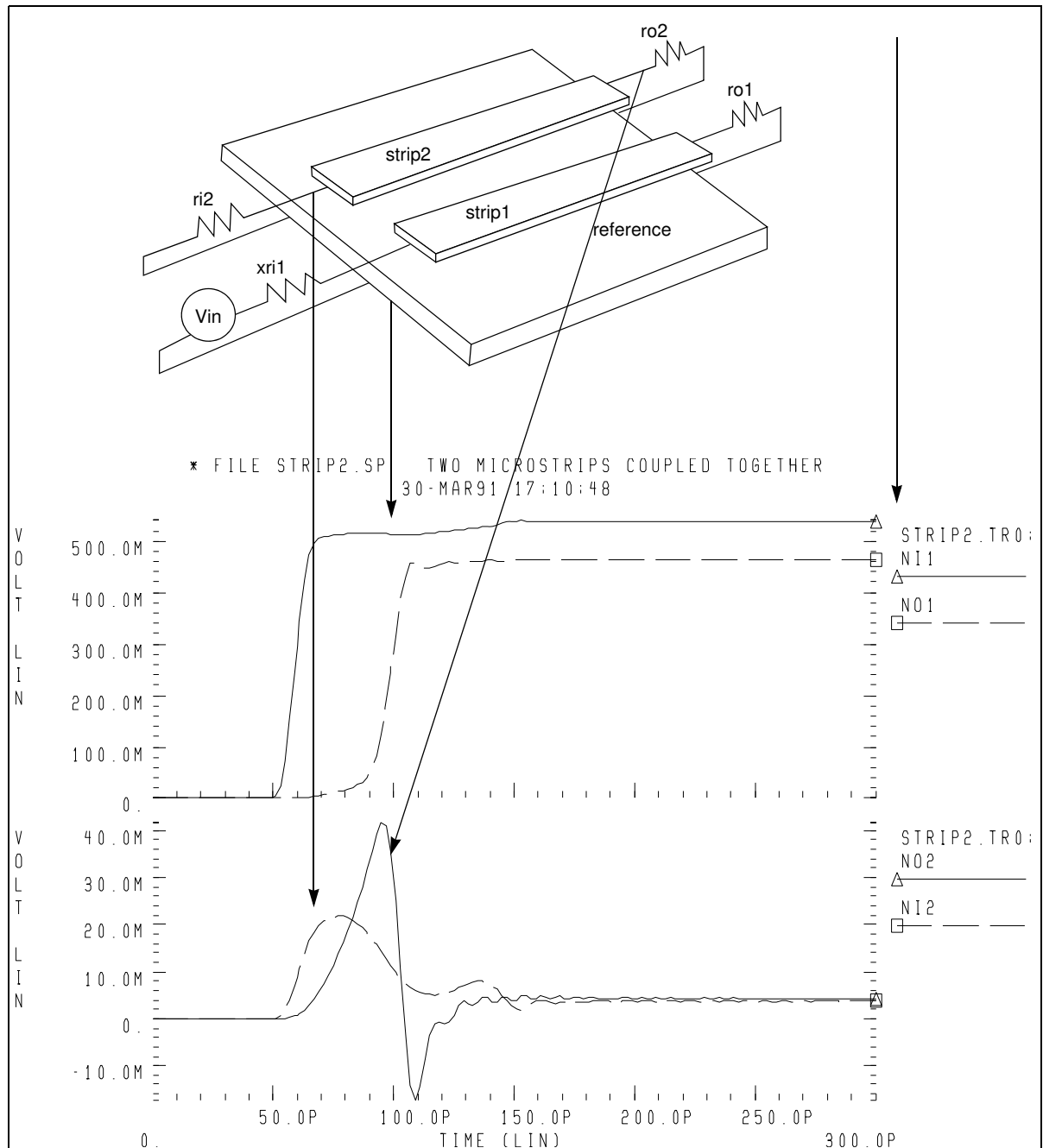
```
* file strip2.sp two microstrips coupled together
*.... the tests following use geometric/physical model
```

Appendix A: Lumped Transmission Line — U-Model

U-element Examples, Models, and Applications

```
.option post acct list
* ...signal source
.tran 1ps 500ps
.param rx=56
* excitation volatge + prefilter
v1 np1 0 pw1 0.0s 0v 50ps 0v 60ps 1v
xr11 np1 ni1 rcfilt rflt=rx tdflt=1ps
* ...circuit definition
ue1 ni1 ni2 0 no1 no2 noref u1 l=9.0m
ri2 ni2 0 rx
ro1 no1 noref rx
ro2 no2 noref rx
* ...model definition
.model u1 u level=3 plev=1 elev=1 nl=2
+ kd=3.5 xw=0.1u rho=17e-9 rhob=20e-9
+ wd=2.0u ht=2.0u th=0.8u sp=2.0u
+ llev=1 maxl=50
*
.end
*
```

Figure 71 Two Coupled Microstrips Geometrically Defined as LSI Metallization



Solving Ringing Problems with U-elements

Ringing oscillations at sharp signal edges may be produced by

- Oscillations due to the simulator
- Oscillations due to lossy approximation of a transmission line (U-element)
- Signal reflections due to impedance mismatch

The primary reason for using a circuit simulator to measure high speed transmission line effects is to calculate how much transient noise the system contains and to determine how to reduce it to acceptable values.

The following sections discuss these topics:

- [Oscillations Due to Simulation Errors](#)
- [Timestep Control Error](#)
- [Incorrect Number of Element Lumps](#)
- [Default Computation](#)
- [Using a Multi-Stage RC Filter to Prevent Ringing](#)
- [Signal Reflections Due to Impedance Mismatch](#)

Oscillations Due to Simulation Errors

The system noise results from the signal reflections in the circuit. It may be masked by noises from the simulator. Simulator noise must be eliminated in order to obtain reliable system noise estimates. The following sections describes ways to solve problems with simulator noise.

Timestep Control Error

The default method of integrating inductors and capacitors is trapezoidal integration. While this method gives excellent results for most simulations, it can lead to what is called trapezoidal ringing. This is numerical oscillations that look like circuit oscillations, but are actually timestep control failures. In particular, trapezoidal ringing can be caused by any discontinuous derivatives in the nonlinear capacitance models, or from the exponential charge expressions for diodes, BJTs, and JFETs.

Set the `.OPTION METHOD=GEAR` to change the integration method from trapezoidal. The GEAR method does not ring and, although it typically gives a slightly less accurate result, is still acceptable for transient noise analysis.

Incorrect Number of Element Lumps

Use the correct number of lumps in a lossy transmission line element. Too few lumps results in false ringing or inaccurate signal transmission, while too many lumps leads to an inordinately long simulation run.

Sometimes, as in verification tests, it is necessary to be able to specify the number of lumps in a transmission line element directly. The number of lumps in a lossy transmission line element can be directly specified, defaulted to an accuracy and limit based computation, or computed with altered accuracy and limit and risetime parameters.

Default Computation

In the default computation, `LUMPS=1` until a threshold of total delay versus risetime is reached:

```
TDeff = RISETIME/20
```

- `TDeff` = total end-to-end delay in the transmission line element
- `RISETIME` = the duration of the shortest signal ramp as given in the statement:
`.OPTION RISETIME = value`

At the threshold, two lumps are used. Above the threshold, the number of lumps is determined by:

```
number of lumps = minimum of 20 or [1+(TDeff/RISETIME)*20]
```

The upper limit of 20 is applied to enhance simulation speed.

If the standard accuracy-based computation does not provide enough lumps, or if it computes too many lumps for simulation efficiency, you can use one of several methods to change the number of lumps on one or more elements

- Specify `LUMPS=value` in the element statement.
- Specify `.MODEL MAXL=value` and `.MODEL WLUMP=value`.
- Specify a different `RISETIME=value` in an `.OPTION` statement.
- Specify `LUMPS=value` (direct specification)

Direct specification overrides the model and limit based computation, applying only to the element specified in the element statement as in the following example:

```
U35 n1 gnd n2 oref      model lumps=31 L=5m
```

The preceding example specifies 31 lumps for an element of length 5 mm.

Specify `MAXL` and `WLUMP` (altered accuracy and limit parameters). You can alter the default computation for all the elements that refer to a particular model by specifying the `MAXL` and `WLUMP` model parameters (which would otherwise default to 20). In the nondefault case the number of lumps, the threshold, and the upper limit all would be changed:

```
lumps = min{MAXL, [1+(TDeff/RISETIME)*WLUMP]}
Threshold: TDeff = RISETIME/WLUMP
Upper lim: MAXL
```

Specify a different `RISETIME` parameter in the `.OPTION` statement. You can change the threshold and number of lumps computed for all elements of all models, reduce or increase the `RISETIME` analysis parameter. Note that care is required if `RISETIME` is decreased, because the number of lumps may be limited by `MAXL` in some cases where it was not previously limited.

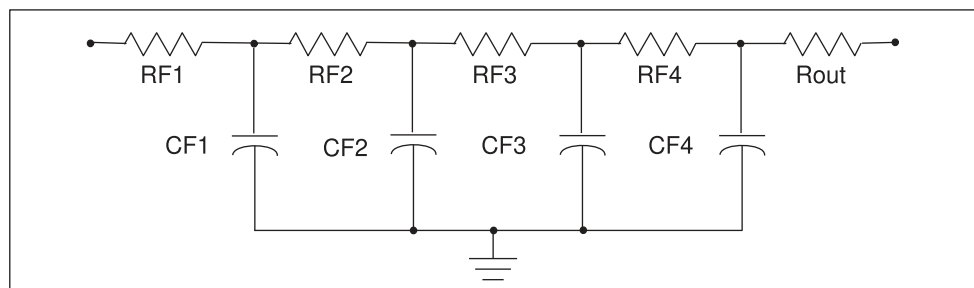
Using a Multi-Stage RC Filter to Prevent Ringing

Artificial sources such as pulse and piecewise linear sources often are used to simulate the action of real output buffer drivers. Since real buffers have a finite cutoff frequency, a multistage filter can be used to give the ideal voltage source reasonable impedance and bandwidth.

You can place a multistage RC filter, between the artificial source and any U-element to reduce the unrealistic source bandwidth and, consequently, the unrealistic ringing. To provide as much realism as possible, the interposed RC filter and the PWL source must be designed together to meet the following criteria

- Reduce the ringing to acceptable levels
- Preserve the realistic bandwidth of the source signal
- Provide a driver with any chosen impedance
- Provide accurately timed transient signals

Figure 72 Circuit Diagram of an RC Filter



Example:

```
.MACRO RCFNEW in out gnd_ref RFLT=50 TDFLT=100n
*
* Begin RCFILT.inc (RC filter) to smooth and match pulse
* sources to transmission lines. User specifies impedance
* (RFLT) and smoothing interval (TDFLT). TDFLT is usually
* specified at about .1*risetime of pulse source.
*
* cutoff freq, total time delay; and frequency dependent
* impedance and signal voltage at "out" node
*
* smoothing period = TDFLT.....(equivalent boxcar filter)
* delay= TDFLT
* fc=2/(pi*TDFLT).....(cutoff frequency)
* Zo~ RFLT*(.9 + .1/sqrt( 1 + (f/fc)^2 )
* V(out)/V(in)= [1 / sqrt( 1 + (f/fc)^2 )]^4
*
.PARAM TD1S='TDFLT/4.0'
RF1 in n1 '.00009*RFLT'
RF2 n1 n2 '.0009*RFLT'
RF3 n2 n3 '.009*RFLT'
RF4 n3 n4 '.09*RFLT'
Rout n4 out '.90*RFLT'
*
.PARAM CTD='TD1S/(.9*RFLT)'
CF1 n1 0 '10000*CTD'
CF2 n2 0 '1000*CTD'
CF3 n3 0 '100*CTD'
CF4 n4 0 '10*CTD'
*
.EOM
```

From the comments embedded in the macro, the output impedance varies from $RFLT$ in the DC limit to 3% less at FC and 10% less in the high frequency limit.

$$(rflt|_{dc} = 0.99999 \cdot rflt) \backslash$$

$$RFLT|_{FC} = 0.97 \cdot RFLT$$

Therefore, setting $RFLT$ to the desired driver impedance gives a reasonably good model for the corrected driver impedance. $TDFLT$ is generally set to 40% of the voltage source rise time.

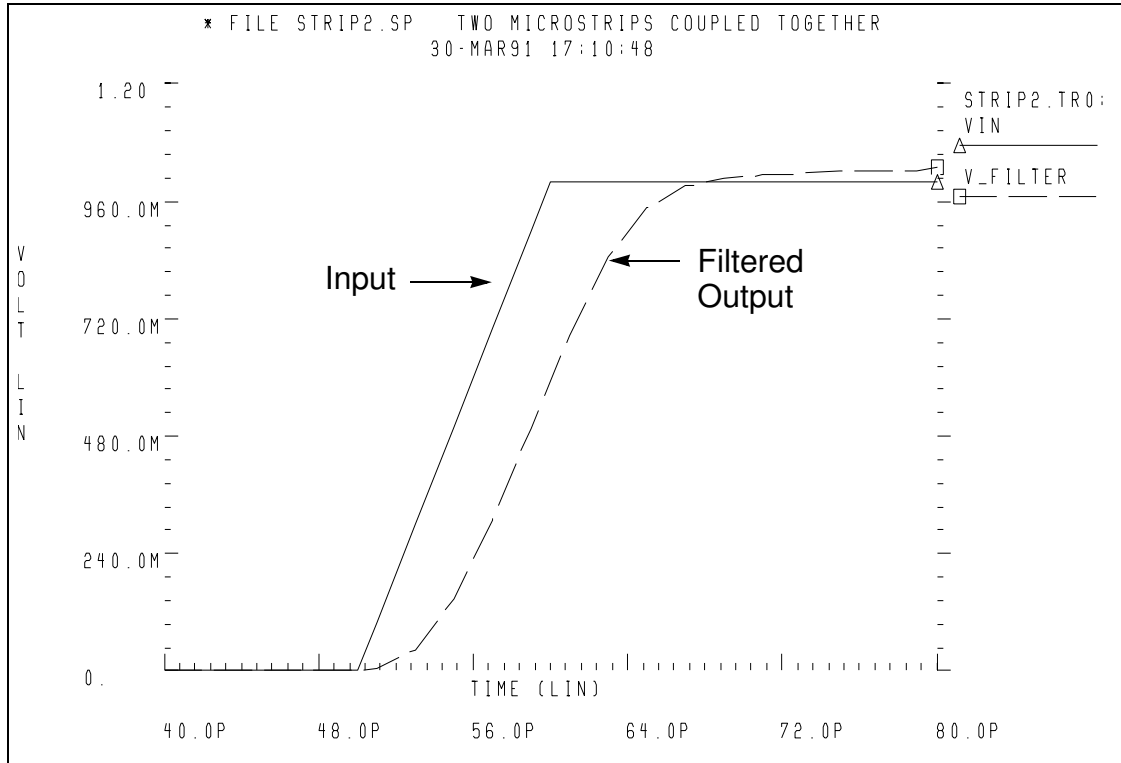
```
* excitation voltage + prefilter
V1 np1 0 PWL 0.0s 0v 50ps 0v 60ps 1v
xr1l NP1 NI1 RCFNEW rflt=rx tdflt=4ps
```

Note:

RCFNEW is an automatic include file named `$installdir/parts/behave/rcfilt.inc`, where `$installdir` is the installation directory.

Figure 73 shows the input and output voltages for the filter.

Figure 73 Multistage RC Filter Input and Output



Signal Reflections Due to Impedance Mismatch

The effect of impedance mismatch is demonstrated in the following example. This circuit has a 75 ohm driver, driving 3 inches of 8 mil wide PCB (middle layer), then driving 3 inches of 16 mil wide PCB.

Figure 74, and Figure 75, and Figure 76, show operational characteristics of such a circuit. The first steady value of impedance is 75 ohms, which is the impedance of the first transmission line section. The input impedance falls to 56 ohms after about 2.5ns when the negative reflection from the nx1 node reaches nil. This TDR displays one idiosyncrasy of the U-element. The high initial value of Z_{in} (TDR) is due to the fact that the input element of the U-element is inductive. The initial TDR spike can be reduced in amplitude and duration by simply using a U-element with a larger number of lumped elements.

Figure 74 Mathematical Transmission Line Structure

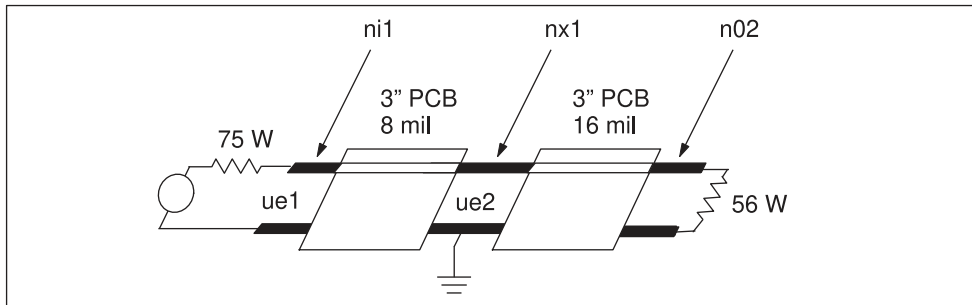


Figure 75 Waveforms in Mismatched Transmission Line Structure

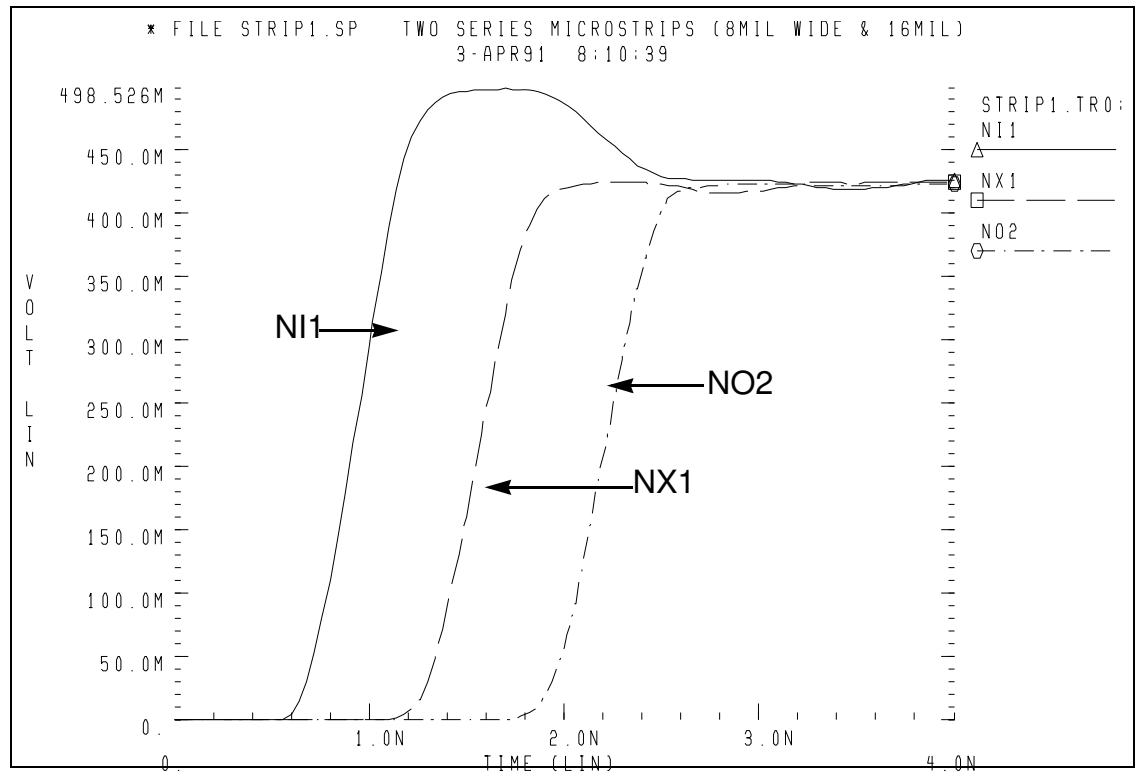
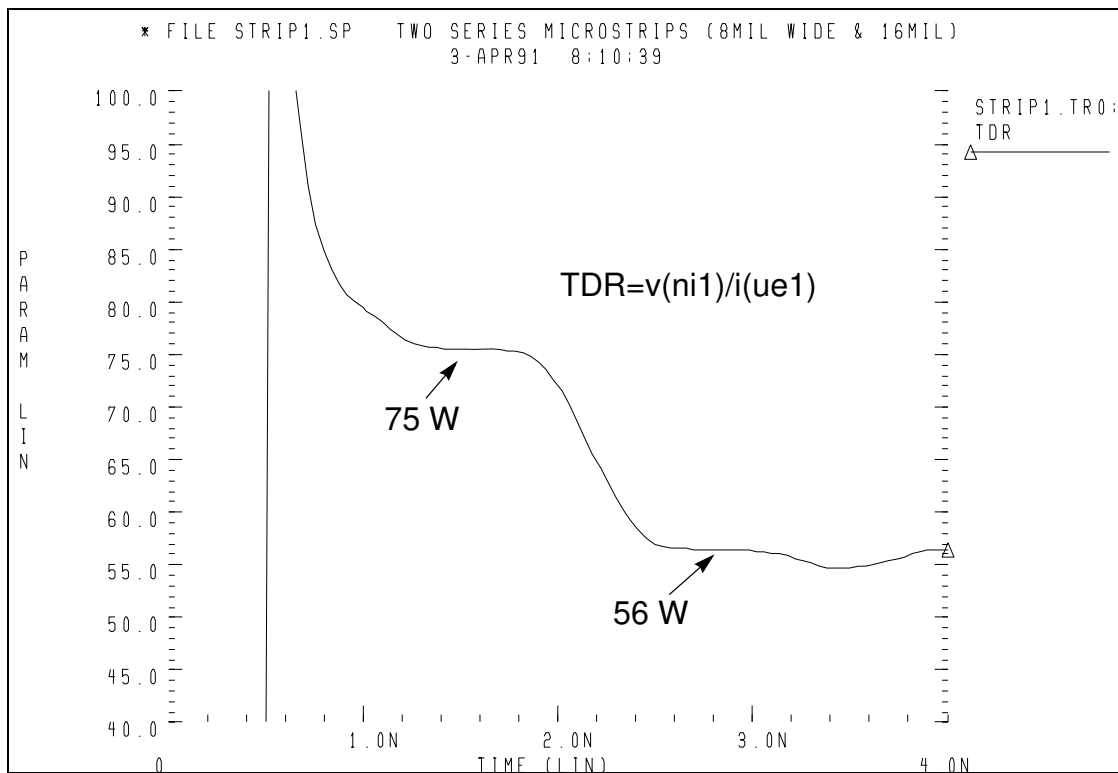


Figure 76 Impedance from TDR at Input



This example is based on demonstration netlist strip1.sp, which is available in directory
\$installdir/demo/hspice/tline:

```
* file strip1.sp two series microstrips (8mil wide & 16mil)
* drive 3 inches of 8mil middle layer pcb, series connected to
* 3 inches of 16mil wide middle layer pcb, 500ns risetime driver
*.... the tests following use geometric/physical model
.option post acct list
* ...signal source
.tran 20ps 4ns
.probe tdr=par('v(ni1)/i(ue1)')
.param r8mil=75 r16mil=56
* excitation volatge + prefilter
v1 np1 0 pwl 0.0s 0v 500ps 0v 1n 1v
xr1l np1 nil rcfilt rflt=r8mil tdflt=50ps
* ...circuit definition
ue1 ni1 0 nx1 0 u1 l=3000mil
ue2 nx1 0 no2 0 u2 l=3000mil
ro2 no2 0 r16mil
* ...model definition -- 8mil middle metal layer of a copper pcb
.model u1 u level=3 plev=1 elev=1 nl=1
+ th=1.3mil ht=10mil ts=32mil kd=4.5 dlev=0
+ wd=8mil xw=-2mil
```

Appendix A: Lumped Transmission Line — U-Model

U-element Examples, Models, and Applications

```
* ...model definition -- 16mil middle metal layer of a copper pcb
.model u2 u level=3 plev=1 elev=1 nl=1
+ th=1.3mil ht=10mil ts=32mil kd=4.5 dlev=0
+ wd=16mil xw=-2mil
*
.end
```